

File: cpu_map.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

cpu_map.h - CPU and pin mapping configuration file
Part of Grbl

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

/* The cpu_map.h files serve as a central pin mapping selection file for different
processor types or alternative pin layouts. This version of Grbl officially supports
only the Arduino Mega328p. */

```
#ifndef cpu_map_h
#define cpu_map_h
```

```
#ifdef CPU_MAP_ATMEGA328P // (Arduino Uno) Officially supported by Grbl.
```

```
  // Define serial port pins and interrupt vectors.
  #define SERIAL_RX    USART_RX_vect
  #define SERIAL_UDRE  USART_UDRE_vect
```

```
  // Define step pulse output pins. NOTE: All step bit pins must be on the same port.
  #define STEP_DDR      DDRD
  #define STEP_PORT     PORTD
  #define X_STEP_BIT    2 // Uno Digital Pin 2
  #define Y_STEP_BIT    3 // Uno Digital Pin 3
  #define Z_STEP_BIT    4 // Uno Digital Pin 4
  #define STEP_MASK     ((1<<X_STEP_BIT)|(1<<Y_STEP_BIT)|(1<<Z_STEP_BIT)) // All step bits
```

```
  // Define step direction output pins. NOTE: All direction pins must be on the same port.
  #define DIRECTION_DDR  DDRD
  #define DIRECTION_PORT PORTD
  #define X_DIRECTION_BIT 5 // Uno Digital Pin 5
  #define Y_DIRECTION_BIT 6 // Uno Digital Pin 6
```

```

#define Z_DIRECTION_BIT 7 // Uno Digital Pin 7
#define DIRECTION_MASK ((1<<X_DIRECTION_BIT)|(1<<Y_DIRECTION_BIT)|(1<<Z_DIRECTION_BIT)) // All direction

// Define stepper driver enable/disable output pin.
#define STEPPERS_DISABLE_DDR DDRB
#define STEPPERS_DISABLE_PORT PORTB
#define STEPPERS_DISABLE_BIT 0 // Uno Digital Pin 8
#define STEPPERS_DISABLE_MASK (1<<STEPPERS_DISABLE_BIT)

// Define homing/hard limit switch input pins and limit interrupt vectors.
// NOTE: All limit bit pins must be on the same port, but not on a port with other input pins (CONTROL).
#define LIMIT_DDR DDRB
#define LIMIT_PIN PINB
#define LIMIT_PORT PORTB
#define X_LIMIT_BIT 1 // Uno Digital Pin 9
#define Y_LIMIT_BIT 2 // Uno Digital Pin 10
#ifdef VARIABLE_SPINDLE // Z Limit pin and spindle enabled swapped to access hardware PWM on Pin 11.
    #define Z_LIMIT_BIT 4 // Uno Digital Pin 12
#else
    #define Z_LIMIT_BIT 3 // Uno Digital Pin 11
#endif
#if !defined(ENABLE_DUAL_AXIS)
    #define LIMIT_MASK ((1<<X_LIMIT_BIT)|(1<<Y_LIMIT_BIT)|(1<<Z_LIMIT_BIT)) // All limit bits
#endif
#define LIMIT_INT PCIE0 // Pin change interrupt enable pin
#define LIMIT_INT_vect PCINT0_vect
#define LIMIT_PCMSK PCMSK0 // Pin change interrupt register

// Define user-control controls (cycle start, reset, feed hold) input pins.
// NOTE: All CONTROLS pins must be on the same port and not on a port with other input pins (limits).
#define CONTROL_DDR DDRC
#define CONTROL_PIN PINC
#define CONTROL_PORT PORTC
#define CONTROL_RESET_BIT 0 // Uno Analog Pin 0
#define CONTROL_FEED_HOLD_BIT 1 // Uno Analog Pin 1
#define CONTROL_CYCLE_START_BIT 2 // Uno Analog Pin 2
#define CONTROL_SAFETY_DOOR_BIT 1 // Uno Analog Pin 1 NOTE: Safety door is shared with feed hold. Enabled by
#define CONTROL_INT PCIE1 // Pin change interrupt enable pin
#define CONTROL_INT_vect PCINT1_vect
#define CONTROL_PCMSK PCMSK1 // Pin change interrupt register
#define CONTROL_MASK ((1<<CONTROL_RESET_BIT)|(1<<CONTROL_FEED_HOLD_BIT)|(1<<CONTROL_CYCLE_START_BIT))
#define CONTROL_INVERT_MASK CONTROL_MASK // May be re-defined to only invert certain control pins.

// Define probe switch input pin.
#define PROBE_DDR DDRC
#define PROBE_PIN PINC
#define PROBE_PORT PORTC
#define PROBE_BIT 5 // Uno Analog Pin 5
#define PROBE_MASK (1<<PROBE_BIT)

#if !defined(ENABLE_DUAL_AXIS)

```

```

// Define flood and mist coolant enable output pins.
#define COOLANT_FLOOD_DDR DDRC
#define COOLANT_FLOOD_PORT PORTC
#define COOLANT_FLOOD_BIT 3 // Uno Analog Pin 3
#define COOLANT_MIST_DDR DDRC
#define COOLANT_MIST_PORT PORTC
#define COOLANT_MIST_BIT 4 // Uno Analog Pin 4

// Define spindle enable and spindle direction output pins.
#define SPINDLE_ENABLE_DDR DDRB
#define SPINDLE_ENABLE_PORT PORTB
// Z Limit pin and spindle PWM/enable pin swapped to access hardware PWM on Pin 11.
#ifndef VARIABLE_SPINDLE
    #ifdef USE_SPINDLE_DIR_AS_ENABLE_PIN
        // If enabled, spindle direction pin now used as spindle enable, while PWM remains on D11.
        #define SPINDLE_ENABLE_BIT 5 // Uno Digital Pin 13 (NOTE: D13 can't be pulled-high input due to LED.)
    #else
        #define SPINDLE_ENABLE_BIT 3 // Uno Digital Pin 11
    #endif
#else
    #define SPINDLE_ENABLE_BIT 4 // Uno Digital Pin 12
#endif
#ifdef USE_SPINDLE_DIR_AS_ENABLE_PIN
    #define SPINDLE_DIRECTION_DDR DDRB
    #define SPINDLE_DIRECTION_PORT PORTB
    #define SPINDLE_DIRECTION_BIT 5 // Uno Digital Pin 13 (NOTE: D13 can't be pulled-high input due to LED.)
#endif

// Variable spindle configuration below. Do not change unless you know what you are doing.
// NOTE: Only used when variable spindle is enabled.
#define SPINDLE_PWM_MAX_VALUE 255 // Don't change. 328p fast PWM mode fixes top value as 255.
#ifdef SPINDLE_PWM_MIN_VALUE
    #define SPINDLE_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define SPINDLE_PWM_OFF_VALUE 0
#define SPINDLE_PWM_RANGE (SPINDLE_PWM_MAX_VALUE-SPINDLE_PWM_MIN_VALUE)
#define SPINDLE_TCCRA_REGISTER TCCR2A
#define SPINDLE_TCCRB_REGISTER TCCR2B
#define SPINDLE_OCR_REGISTER OCR2A
#define SPINDLE_COMB_BIT COM2A1

// Prescaled, 8-bit Fast PWM mode.
#define SPINDLE_TCCRA_INIT_MASK ((1<<WGM20) | (1<<WGM21)) // Configures fast PWM mode.
// #define SPINDLE_TCCRB_INIT_MASK (1<<CS20) // Disable prescaler -> 62.5kHz
// #define SPINDLE_TCCRB_INIT_MASK (1<<CS21) // 1/8 prescaler -> 7.8kHz (Used in v0.9)
// #define SPINDLE_TCCRB_INIT_MASK ((1<<CS21) | (1<<CS20)) // 1/32 prescaler -> 1.96kHz
#define SPINDLE_TCCRB_INIT_MASK (1<<CS22) // 1/64 prescaler -> 0.98kHz (J-tech laser)

// NOTE: On the 328p, these must be the same as the SPINDLE_ENABLE settings.
#define SPINDLE_PWM_DDR DDRB

```

```

#define SPINDLE_PWM_PORT PORTB
#define SPINDLE_PWM_BIT 3 // Uno Digital Pin 11

#else

// Dual axis feature requires an independent step pulse pin to operate. The independent direction pin is not
// absolutely necessary but facilitates easy direction inverting with a Grbl $$ setting. These pins replace
// the spindle direction and optional coolant mist pins.

#ifdef DUAL_AXIS_CONFIG_PROTONEER_V3_51
// NOTE: Step pulse and direction pins may be on any port and output pin.
#define STEP_DDR_DUAL DDRC
#define STEP_PORT_DUAL PORTC
#define DUAL_STEP_BIT 4 // Uno Analog Pin 4
#define STEP_MASK_DUAL ((1<<DUAL_STEP_BIT))
#define DIRECTION_DDR_DUAL DDRC
#define DIRECTION_PORT_DUAL PORTC
#define DUAL_DIRECTION_BIT 3 // Uno Analog Pin 3
#define DIRECTION_MASK_DUAL ((1<<DUAL_DIRECTION_BIT))

// NOTE: Dual axis limit is shared with the z-axis limit pin by default. Pin used must be on the same port
// as other limit pins.
#define DUAL_LIMIT_BIT Z_LIMIT_BIT
#define LIMIT_MASK ((1<<X_LIMIT_BIT)|(1<<Y_LIMIT_BIT)|(1<<Z_LIMIT_BIT)|(1<<DUAL_LIMIT_BIT))

// Define coolant enable output pins.
// NOTE: Coolant flood moved from A3 to A4. Coolant mist not supported with dual axis feature on Arduino Uno.
#define COOLANT_FLOOD_DDR DDRB
#define COOLANT_FLOOD_PORT PORTB
#define COOLANT_FLOOD_BIT 5 // Uno Digital Pin 13

// Define spindle enable output pin.
// NOTE: Spindle enable moved from D12 to A3 (old coolant flood enable pin). Spindle direction pin is removed.
#define SPINDLE_ENABLE_DDR DDRB
#define SPINDLE_ENABLE_PORT PORTB
#ifdef VARIABLE_SPINDLE
// NOTE: USE_SPINDLE_DIR_AS_ENABLE_PIN not supported with dual axis feature.
#define SPINDLE_ENABLE_BIT 3 // Uno Digital Pin 11
#else
#define SPINDLE_ENABLE_BIT 4 // Uno Digital Pin 12
#endif

// Variable spindle configuration below. Do not change unless you know what you are doing.
// NOTE: Only used when variable spindle is enabled.
#define SPINDLE_PWM_MAX_VALUE 255 // Don't change. 328p fast PWM mode fixes top value as 255.
#ifdef SPINDLE_PWM_MIN_VALUE
#define SPINDLE_PWM_MIN_VALUE 1 // Must be greater than zero.
#endif
#define SPINDLE_PWM_OFF_VALUE 0
#define SPINDLE_PWM_RANGE (SPINDLE_PWM_MAX_VALUE-SPINDLE_PWM_MIN_VALUE)
#define SPINDLE_TCCRA_REGISTER TCCR2A

```

```

#define SPINDLE_TCCRB_REGISTER TCCR2B
#define SPINDLE_OCR_REGISTER OCR2A
#define SPINDLE_COMB_BIT COM2A1

// Prescaled, 8-bit Fast PWM mode.
#define SPINDLE_TCCRA_INIT_MASK ((1<<WGM20) | (1<<WGM21)) // Configures fast PWM mode.
// #define SPINDLE_TCCRB_INIT_MASK (1<<CS20) // Disable prescaler -> 62.5kHz
// #define SPINDLE_TCCRB_INIT_MASK (1<<CS21) // 1/8 prescaler -> 7.8kHz (Used in v0.9)
// #define SPINDLE_TCCRB_INIT_MASK ((1<<CS21) | (1<<CS20)) // 1/32 prescaler -> 1.96kHz
#define SPINDLE_TCCRB_INIT_MASK (1<<CS22) // 1/64 prescaler -> 0.98kHz (J-tech laser)

// NOTE: On the 328p, these must be the same as the SPINDLE_ENABLE settings.
#define SPINDLE_PWM_DDR DDRB
#define SPINDLE_PWM_PORT PORTB
#define SPINDLE_PWM_BIT 3 // Uno Digital Pin 11
#endif

// NOTE: Variable spindle not supported with this shield.
#ifndef DUAL_AXIS_CONFIG_CNC_SHIELD_CLONE
// NOTE: Step pulse and direction pins may be on any port and output pin.
#define STEP_DDR_DUAL DDRB
#define STEP_PORT_DUAL PORTB
#define DUAL_STEP_BIT 4 // Uno Digital Pin 12
#define STEP_MASK_DUAL ((1<<DUAL_STEP_BIT))
#define DIRECTION_DDR_DUAL DDRB
#define DIRECTION_PORT_DUAL PORTB
#define DUAL_DIRECTION_BIT 5 // Uno Digital Pin 13
#define DIRECTION_MASK_DUAL ((1<<DUAL_DIRECTION_BIT))

// NOTE: Dual axis limit is shared with the z-axis limit pin by default.
#define DUAL_LIMIT_BIT Z_LIMIT_BIT
#define LIMIT_MASK ((1<<X_LIMIT_BIT)|(1<<Y_LIMIT_BIT)|(1<<Z_LIMIT_BIT)|(1<<DUAL_LIMIT_BIT))

// Define coolant enable output pins.
// NOTE: Coolant flood moved from A3 to A4. Coolant mist not supported with dual axis feature on Arduino Uno.
#define COOLANT_FLOOD_DDR DDRC
#define COOLANT_FLOOD_PORT PORTC
#define COOLANT_FLOOD_BIT 4 // Uno Analog Pin 4

// Define spindle enable output pin.
// NOTE: Spindle enable moved from D12 to A3 (old coolant flood enable pin). Spindle direction pin is removed.
#define SPINDLE_ENABLE_DDR DDRC
#define SPINDLE_ENABLE_PORT PORTC
#define SPINDLE_ENABLE_BIT 3 // Uno Analog Pin 3
#endif

#endif

#endif

/*

```

```

#ifdef CPU_MAP_CUSTOM_PROC
    // For a custom pin map or different processor, copy and edit one of the available cpu
    // map files and modify it to your needs. Make sure the defined name is also changed in
    // the config.h file.
#endif
*/

#endif

```

File: defaults.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
defaults.h - defaults settings configuration file
Part of Grbl

```

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

/* The defaults.h file serves as a central default settings selector for different machine
types, from DIY CNC mills to CNC conversions of off-the-shelf machines. The settings
files listed here are supplied by users, so your results may vary. However, this should
give you a good starting point as you get to know your machine and tweak the settings for
your nefarious needs.
NOTE: Ensure one and only one of these DEFAULTS_XXX values is defined in config.h */

```

```

#ifndef defaults_h

```

```

#ifdef DEFAULTS_GENERIC
    // Grbl generic default settings. Should work across different machines.
#define DEFAULT_X_STEPS_PER_MM 250.0
#define DEFAULT_Y_STEPS_PER_MM 250.0
#define DEFAULT_Z_STEPS_PER_MM 250.0
#define DEFAULT_X_MAX_RATE 500.0 // mm/min
#define DEFAULT_Y_MAX_RATE 500.0 // mm/min
#define DEFAULT_Z_MAX_RATE 500.0 // mm/min

```

```

#define DEFAULT_X_ACCELERATION (10.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Y_ACCELERATION (10.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Z_ACCELERATION (10.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 200.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 200.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 200.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 1000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK 0
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 25 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.01 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 500.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
#endif

```

```

#ifdef DEFAULTS_SHERLINE_5400

```

```

// Description: Sherline 5400 mill with three NEMA 23 Keling KL23H256-21-8B 185 oz-in stepper motors,
// driven by three Pololu A4988 stepper drivers with a 30V, 6A power supply at 1.5A per winding.

```

```

#define MICROSTEPS 2
#define STEPS_PER_REV 200.0
#define MM_PER_REV (0.050*MM_PER_INCH) // 0.050 inch/rev leadscrew
#define DEFAULT_X_STEPS_PER_MM (STEPS_PER_REV*MICROSTEPS/MM_PER_REV)
#define DEFAULT_Y_STEPS_PER_MM (STEPS_PER_REV*MICROSTEPS/MM_PER_REV)
#define DEFAULT_Z_STEPS_PER_MM (STEPS_PER_REV*MICROSTEPS/MM_PER_REV)
#define DEFAULT_X_MAX_RATE 635.0 // mm/min (25 ipm)
#define DEFAULT_Y_MAX_RATE 635.0 // mm/min
#define DEFAULT_Z_MAX_RATE 635.0 // mm/min
#define DEFAULT_X_ACCELERATION (50.0*60*60) // 50*60*60 mm/min^2 = 50 mm/sec^2
#define DEFAULT_Y_ACCELERATION (50.0*60*60) // 50*60*60 mm/min^2 = 50 mm/sec^2
#define DEFAULT_Z_ACCELERATION (50.0*60*60) // 50*60*60 mm/min^2 = 50 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 225.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 125.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 170.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 2800.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10

```

```

#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<Y_AXIS)|(1<<Z_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 25 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.01 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // true
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 50.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 635.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
#endif

#ifdef DEFAULTS_POCKETNC_FR4
// Description: Pocket NC FR4 CNC mill.
#define DEFAULT_X_STEPS_PER_MM 800.0
#define DEFAULT_Y_STEPS_PER_MM 800.0
#define DEFAULT_Z_STEPS_PER_MM 800.0
#define DEFAULT_X_MAX_RATE 300.0 // mm/min
#define DEFAULT_Y_MAX_RATE 300.0 // mm/min
#define DEFAULT_Z_MAX_RATE 300.0 // mm/min
#define DEFAULT_X_ACCELERATION (30.0*60*60) // 15*60*60 mm/min^2 = 15 mm/sec^2
#define DEFAULT_Y_ACCELERATION (30.0*60*60) // 15*60*60 mm/min^2 = 15 mm/sec^2
#define DEFAULT_Z_ACCELERATION (30.0*60*60) // 15*60*60 mm/min^2 = 15 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 225.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 125.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 170.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 7000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<Y_AXIS)|(1<<Z_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 250 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 3 // WPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.01 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 1 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false

```

```

#define DEFAULT_HOMING_ENABLE 1 // false
#define DEFAULT_HOMING_DIR_MASK 1 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 100.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 300.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 3.0 // mm
#endif

#ifdef DEFAULTS_SHAPEOKO
// Description: Shapeoko CNC mill with three NEMA 17 stepper motors, driven by Synthetos
// grblShield with a 24V, 4.2A power supply.
#define MICROSTEPS_XY 8
#define STEP_REVS_XY 400
#define MM_PER_REV_XY (0.08*18*MM_PER_INCH) // 0.08 in belt pitch, 18 pulley teeth
#define MICROSTEPS_Z 2
#define STEP_REVS_Z 400
#define MM_PER_REV_Z 1.250 // 1.25 mm/rev leadscrew
#define DEFAULT_X_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Y_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Z_STEPS_PER_MM (MICROSTEPS_Z*STEP_REVS_Z/MM_PER_REV_Z)
#define DEFAULT_X_MAX_RATE 1000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 1000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 1000.0 // mm/min
#define DEFAULT_X_ACCELERATION (15.0*60*60) // 15*60*60 mm/min^2 = 15 mm/sec^2
#define DEFAULT_Y_ACCELERATION (15.0*60*60) // 15*60*60 mm/min^2 = 15 mm/sec^2
#define DEFAULT_Z_ACCELERATION (15.0*60*60) // 15*60*60 mm/min^2 = 15 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 200.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 200.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 200.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 10000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<Y_AXIS)|(1<<Z_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 255 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 250.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm

```

#endif

#ifdef DEFAULTS_SHAPEOKO_2

```
// Description: Shapeoko CNC mill with three NEMA 17 stepper motors, driven by Synthetos
// grblShield at 28V.
#define MICROSTEPS_XY 8
#define STEP_REVS_XY 200
#define MM_PER_REV_XY (2.0*20) // 2mm belt pitch, 20 pulley teeth
#define MICROSTEPS_Z 2
#define STEP_REVS_Z 200
#define MM_PER_REV_Z 1.250 // 1.25 mm/rev leadscrew
#define DEFAULT_X_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Y_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Z_STEPS_PER_MM (MICROSTEPS_Z*STEP_REVS_Z/MM_PER_REV_Z)
#define DEFAULT_X_MAX_RATE 5000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 5000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 500.0 // mm/min
#define DEFAULT_X_ACCELERATION (250.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_Y_ACCELERATION (250.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_Z_ACCELERATION (50.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 290.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 290.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 100.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 10000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<X_AXIS)|(1<<Z_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 255 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 250.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
```

#endif

#ifdef DEFAULTS_SHAPEOKO_3

```
// Description: Shapeoko CNC mill with three NEMA 23 stepper motors, driven by CarbideMotion
#define MICROSTEPS_XY 8
#define STEP_REVS_XY 200
```

```

#define MM_PER_REV_XY (2.0*20) // 2mm belt pitch, 20 pulley teeth
#define MICROSTEPS_Z 8
#define STEP_REVS_Z 200
#define MM_PER_REV_Z (2.0*20) // 2mm belt pitch, 20 pulley teeth
#define DEFAULT_X_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Y_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Z_STEPS_PER_MM (MICROSTEPS_Z*STEP_REVS_Z/MM_PER_REV_Z)
#define DEFAULT_X_MAX_RATE 5000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 5000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 5000.0 // mm/min
#define DEFAULT_X_ACCELERATION (400.0*60*60) // 400*60*60 mm/min^2 = 400 mm/sec^2
#define DEFAULT_Y_ACCELERATION (400.0*60*60) // 400*60*60 mm/min^2 = 400 mm/sec^2
#define DEFAULT_Z_ACCELERATION (400.0*60*60) // 400*60*60 mm/min^2 = 400 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 425.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 465.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 80.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 10000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<X_AXIS)|(1<<Z_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 255 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.01 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 100.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 1000.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 25 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 5.0 // mm
#endif

```

```

#ifdef DEFAULTS_X_CARVE_500MM

```

```

// Description: X-Carve 3D Carver CNC mill with three 200 step/rev motors driven by Synthetos
// grblShield at 24V.

```

```

#define MICROSTEPS_XY 8
#define STEP_REVS_XY 200
#define MM_PER_REV_XY (2.0*20) // 2mm belt pitch, 20 pulley teeth
#define MICROSTEPS_Z 2
#define STEP_REVS_Z 200
#define MM_PER_REV_Z 2.117 // ACME 3/8-12 Leadscrew
#define DEFAULT_X_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Y_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)

```

```

#define DEFAULT_Z_STEPS_PER_MM (MICROSTEPS_Z*STEP_REVS_Z/MM_PER_REV_Z)
#define DEFAULT_X_MAX_RATE 8000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 8000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 500.0 // mm/min
#define DEFAULT_X_ACCELERATION (500.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_Y_ACCELERATION (500.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_Z_ACCELERATION (50.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 290.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 290.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 100.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 10000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<X_AXIS)|(1<<Y_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 255 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 3 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 750.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
#endif

```

```

#ifdef DEFAULTS_X_CARVE_1000MM
// Description: X-Carve 3D Carver CNC mill with three 200 step/rev motors driven by Synthetos
// grblShield at 24V.
#define MICROSTEPS_XY 8
#define STEP_REVS_XY 200
#define MM_PER_REV_XY (2.0*20) // 2mm belt pitch, 20 pulley teeth
#define MICROSTEPS_Z 2
#define STEP_REVS_Z 200
#define MM_PER_REV_Z 2.117 // ACME 3/8-12 Leadscrew
#define DEFAULT_X_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Y_STEPS_PER_MM (MICROSTEPS_XY*STEP_REVS_XY/MM_PER_REV_XY)
#define DEFAULT_Z_STEPS_PER_MM (MICROSTEPS_Z*STEP_REVS_Z/MM_PER_REV_Z)
#define DEFAULT_X_MAX_RATE 8000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 8000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 500.0 // mm/min
#define DEFAULT_X_ACCELERATION (500.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_Y_ACCELERATION (500.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2

```

```

#define DEFAULT_Z_ACCELERATION (50.0*60*60) // 25*60*60 mm/min^2 = 25 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 740.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 790.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 100.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 10000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<X_AXIS)|(1<<Y_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 255 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 3 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 750.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
#endif

```

```

#ifdef DEFAULTS_BOBSCNC_E3
// Grbl settings for Bob's CNC E3 Machine
// https://www.bobscnc.com/products/e3-cnc-engraving-kit
#define DEFAULT_X_STEPS_PER_MM 80.0
#define DEFAULT_Y_STEPS_PER_MM 80.0
#define DEFAULT_Z_STEPS_PER_MM 2267.717
#define DEFAULT_X_MAX_RATE 10000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 10000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 500.0 // mm/min
#define DEFAULT_X_ACCELERATION (500.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Y_ACCELERATION (500.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Z_ACCELERATION (300.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 450.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 390.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 85.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 1000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 5
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK 0
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 25 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.01 // mm

```

```

#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 1 // true
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 1 // true
#define DEFAULT_SOFT_LIMIT_ENABLE 1 // true
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 1 // true
#define DEFAULT_HOMING_DIR_MASK 3 // move xy -dir, z dir
#define DEFAULT_HOMING_FEED_RATE 500.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 4000.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 5.0 // mm
#endif

```

```

#ifdef DEFAULTS_BOBSCNC_E4
// Grbl settings for Bob's CNC E4 Machine
// https://www.bobscnc.com/products/e4-cnc-router
#define DEFAULT_X_STEPS_PER_MM 80.0
#define DEFAULT_Y_STEPS_PER_MM 80.0
#define DEFAULT_Z_STEPS_PER_MM 2267.717
#define DEFAULT_X_MAX_RATE 10000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 10000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 500.0 // mm/min
#define DEFAULT_X_ACCELERATION (500.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Y_ACCELERATION (500.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Z_ACCELERATION (300.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 610.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 610.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 85.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 1000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 5
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK 0
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 25 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.01 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 1 // true
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 1 // true
#define DEFAULT_SOFT_LIMIT_ENABLE 1 // true
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 1 // true
#define DEFAULT_HOMING_DIR_MASK 3 // move xy -dir, z dir
#define DEFAULT_HOMING_FEED_RATE 500.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 4000.0 // mm/min

```

```

#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 5.0 // mm
#endif

#ifdef DEFAULTS_ZEN_TOOLWORKS_7x7
// Description: Zen Toolworks 7x7 mill with three Shinano SST43D2121 65oz-in NEMA 17 stepper motors.
// Leadscrew is different from some ZTW kits, where most are 1.25mm/rev rather than 8.0mm/rev here.
// Driven by 30V, 6A power supply and TI DRV8811 stepper motor drivers.
#define MICROSTEPS 8
#define STEPS_PER_REV 200.0
#define MM_PER_REV 8.0 // 8 mm/rev leadscrew
#define DEFAULT_X_STEPS_PER_MM (STEPS_PER_REV*MICROSTEPS/MM_PER_REV)
#define DEFAULT_Y_STEPS_PER_MM (STEPS_PER_REV*MICROSTEPS/MM_PER_REV)
#define DEFAULT_Z_STEPS_PER_MM (STEPS_PER_REV*MICROSTEPS/MM_PER_REV)
#define DEFAULT_X_MAX_RATE 6000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 6000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 6000.0 // mm/min
#define DEFAULT_X_ACCELERATION (600.0*60*60) // 600*60*60 mm/min^2 = 600 mm/sec^2
#define DEFAULT_Y_ACCELERATION (600.0*60*60) // 600*60*60 mm/min^2 = 600 mm/sec^2
#define DEFAULT_Z_ACCELERATION (600.0*60*60) // 600*60*60 mm/min^2 = 600 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 190.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 180.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 150.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 10000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK ((1<<Y_AXIS))
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 25 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 250.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
#endif

#ifdef DEFAULTS_OXCNC
// Grbl settings for OpenBuilds OX CNC Machine
// http://www.openbuilds.com/builds/openbuilds-ox-cnc-machine.341/
#define DEFAULT_X_STEPS_PER_MM 26.670

```

```

#define DEFAULT_Y_STEPS_PER_MM 26.670
#define DEFAULT_Z_STEPS_PER_MM 50
#define DEFAULT_X_MAX_RATE 500.0 // mm/min
#define DEFAULT_Y_MAX_RATE 500.0 // mm/min
#define DEFAULT_Z_MAX_RATE 500.0 // mm/min
#define DEFAULT_X_ACCELERATION (10.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Y_ACCELERATION (10.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Z_ACCELERATION (10.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 500.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 750.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 80.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 1000.0 // rpm
#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK 0
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 25 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.02 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 500.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
#endif

```

```

#ifdef DEFAULTS_SIMULATOR
// Settings only for Grbl Simulator (www.github.com/grbl/grbl-sim)
// Grbl generic default settings. Should work across different machines.
#define DEFAULT_X_STEPS_PER_MM 1000.0
#define DEFAULT_Y_STEPS_PER_MM 1000.0
#define DEFAULT_Z_STEPS_PER_MM 1000.0
#define DEFAULT_X_MAX_RATE 1000.0 // mm/min
#define DEFAULT_Y_MAX_RATE 1000.0 // mm/min
#define DEFAULT_Z_MAX_RATE 1000.0 // mm/min
#define DEFAULT_X_ACCELERATION (100.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Y_ACCELERATION (100.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_Z_ACCELERATION (100.0*60*60) // 10*60*60 mm/min^2 = 10 mm/sec^2
#define DEFAULT_X_MAX_TRAVEL 1000.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Y_MAX_TRAVEL 1000.0 // mm NOTE: Must be a positive value.
#define DEFAULT_Z_MAX_TRAVEL 1000.0 // mm NOTE: Must be a positive value.
#define DEFAULT_SPINDLE_RPM_MAX 1000.0 // rpm

```

```

#define DEFAULT_SPINDLE_RPM_MIN 0.0 // rpm
#define DEFAULT_STEP_PULSE_MICROSECONDS 10
#define DEFAULT_STEPPING_INVERT_MASK 0
#define DEFAULT_DIRECTION_INVERT_MASK 0
#define DEFAULT_STEPPER_IDLE_LOCK_TIME 25 // msec (0-254, 255 keeps steppers enabled)
#define DEFAULT_STATUS_REPORT_MASK 1 // MPos enabled
#define DEFAULT_JUNCTION_DEVIATION 0.01 // mm
#define DEFAULT_ARC_TOLERANCE 0.002 // mm
#define DEFAULT_REPORT_INCHES 0 // false
#define DEFAULT_INVERT_ST_ENABLE 0 // false
#define DEFAULT_INVERT_LIMIT_PINS 0 // false
#define DEFAULT_SOFT_LIMIT_ENABLE 0 // false
#define DEFAULT_HARD_LIMIT_ENABLE 0 // false
#define DEFAULT_INVERT_PROBE_PIN 0 // false
#define DEFAULT_LASER_MODE 0 // false
#define DEFAULT_HOMING_ENABLE 0 // false
#define DEFAULT_HOMING_DIR_MASK 0 // move positive dir
#define DEFAULT_HOMING_FEED_RATE 25.0 // mm/min
#define DEFAULT_HOMING_SEEK_RATE 500.0 // mm/min
#define DEFAULT_HOMING_DEBOUNCE_DELAY 250 // msec (0-65k)
#define DEFAULT_HOMING_PULLOFF 1.0 // mm
#endif

#endif

```

File: eeprom.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

// This file has been prepared for Doxygen automatic documentation generation.

```

/!* \file *****
*
* Atmel Corporation
*
* \li File:          eeprom.c
* \li Compiler:      IAR EWAAVR 3.10c
* \li Support mail:   avr@atmel.com
*
* \li Supported devices: All devices with split EEPROM erase/write
*                        capabilities can be used.
*                        The example is written for ATmega48.
*
* \li AppNote:        AVR103 - Using the EEPROM Programming Modes.
*
* \li Description:     Example on how to use the split EEPROM erase/write
*                        capabilities in e.g. ATmega48. All EEPROM
*                        programming modes are tested, i.e. Erase+Write,
*                        Erase-only and Write-only.
*
*                        $Revision: 1.6 $

```

```

*                               $Date: Friday, February 11, 2005 07:16:44 UTC $
*****/

#include <avr/io.h>
#include <avr/interrupt.h>

/* These EEPROM bits have different names on different devices. */
#ifndef EEPE
#define EEPE EWE //!< EEPROM program/write enable.
#define EEMPE EEMWE //!< EEPROM master program/write enable.
#endif

/* These two are unfortunately not defined in the device include files. */
#define EEP1 5 //!< EEPROM Programming Mode Bit 1.
#define EEP0 4 //!< EEPROM Programming Mode Bit 0.

/* Define to reduce code size. */
#define EEPROM_IGNORE_SELFPROG //!< Remove SPM flag polling.

/*! \brief Read byte from EEPROM.
*
* This function reads one byte from a given EEPROM address.
*
* \note The CPU is halted for 4 clock cycles during EEPROM read.
*
* \param addr EEPROM address to read from.
* \return The byte read from the EEPROM address.
*/
unsigned char eeprom_get_char( unsigned int addr )
{
    do {} while( EECR & (1<<EEPE) ); // Wait for completion of previous write.
    EEAR = addr; // Set EEPROM address register.
    EECR = (1<<EERE); // Start EEPROM read operation.
    return EEDR; // Return the byte read from EEPROM.
}

/*! \brief Write byte to EEPROM.
*
* This function writes one byte to a given EEPROM address.
* The differences between the existing byte and the new value is used
* to select the most efficient EEPROM programming mode.
*
* \note The CPU is halted for 2 clock cycles during EEPROM programming.
*
* \note When this function returns, the new EEPROM value is not available
* until the EEPROM programming time has passed. The EEPE bit in EECR
* should be polled to check whether the programming is finished.
*
* \note The EEPROM_GetChar() function checks the EEPE bit automatically.
*
* \param addr EEPROM address to write to.
* \param new_value New EEPROM value.

```

```

*/
void eeprom_put_char( unsigned int addr, unsigned char new_value )
{
    char old_value; // Old EEPROM value.
    char diff_mask; // Difference mask, i.e. old value XOR new value.

    cli(); // Ensure atomic operation for the write operation.

    do { while( EECR & (1<<EEPE) ); // Wait for completion of previous write.
    #ifndef EEPROM_IGNORE_SELFPROG
    do { while( SPMCSR & (1<<SELFPRGEN) ); // Wait for completion of SPM.
    #endif

    EEAR = addr; // Set EEPROM address register.
    EECR = (1<<EERE); // Start EEPROM read operation.
    old_value = EEDR; // Get old EEPROM value.
    diff_mask = old_value ^ new_value; // Get bit differences.

    // Check if any bits are changed to '1' in the new value.
    if( diff_mask & new_value ) {
        // Now we know that _some_ bits need to be erased to '1'.

        // Check if any bits in the new value are '0'.
        if( new_value != 0xff ) {
            // Now we know that some bits need to be programmed to '0' also.

            EEDR = new_value; // Set EEPROM data register.
            EECR = (1<<EEMPE) | // Set Master Write Enable bit...
                (0<<EEPM1) | (0<<EEPM0); // ...and Erase+Write mode.
            EECR |= (1<<EEPE); // Start Erase+Write operation.
        } else {
            // Now we know that all bits should be erased.

            EECR = (1<<EEMPE) | // Set Master Write Enable bit...
                (1<<EEPM0); // ...and Erase-only mode.
            EECR |= (1<<EEPE); // Start Erase-only operation.
        }
    } else {
        // Now we know that _no_ bits need to be erased to '1'.

        // Check if any bits are changed from '1' in the old value.
        if( diff_mask ) {
            // Now we know that _some_ bits need to be programmed to '0'.

            EEDR = new_value; // Set EEPROM data register.
            EECR = (1<<EEMPE) | // Set Master Write Enable bit...
                (1<<EEPM1); // ...and Write-only mode.
            EECR |= (1<<EEPE); // Start Write-only operation.
        }
    }
}

```

```
sei(); // Restore interrupt flag state.
```

```
}
```

```
// Extensions added as part of Grbl
```

```
void memcpy_to_eeprom_with_checksum(unsigned int destination, char *source, unsigned int size) {  
  unsigned char checksum = 0;  
  for(; size > 0; size--) {  
    checksum = (checksum << 1) || (checksum >> 7);  
    checksum += *source;  
    eeprom_put_char(destination++, *(source++));  
  }  
  eeprom_put_char(destination, checksum);  
}
```

```
int memcpy_from_eeprom_with_checksum(char *destination, unsigned int source, unsigned int size) {  
  unsigned char data, checksum = 0;  
  for(; size > 0; size--) {  
    data = eeprom_get_char(source++);  
    checksum = (checksum << 1) || (checksum >> 7);  
    checksum += data;  
    *(destination++) = data;  
  }  
  return(checksum == eeprom_get_char(source));  
}
```

```
// end of file
```

File: eeprom.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```
/*
```

```
  eeprom.h - EEPROM methods
```

```
  Part of Grbl
```

```
  Copyright (c) 2009-2011 Simen Svale Skogsrud
```

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License

along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

#ifndef eeprom_h

#define eeprom_h

unsigned char eeprom_get_char(unsigned int addr);

void eeprom_put_char(unsigned int addr, unsigned char new_value);

void memcpy_to_eeprom_with_checksum(unsigned int destination, char *source, unsigned int size);

int memcpy_from_eeprom_with_checksum(char *destination, unsigned int source, unsigned int size);

#endif

File: gcode.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

gcode.c - rs274/ngc parser.

Part of Grbl

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC

Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

#include "grbl.h"

// NOTE: Max line number is defined by the g-code standard to be 99999. It seems to be an
// arbitrary value, and some GUIs may require more. So we increased it based on a max safe
// value when converting a float (7.2 digit precision)s to an integer.

#define MAX_LINE_NUMBER 1000000

#define MAX_TOOL_NUMBER 255 // Limited by max unsigned 8-bit value

#define AXIS_COMMAND_NONE 0

#define AXIS_COMMAND_NON_MODAL 1

#define AXIS_COMMAND_MOTION_MODE 2

#define AXIS_COMMAND_TOOL_LENGTH_OFFSET 3 // *Undefined but required

```

// Declare gc extern struct
parser_state_t gc_state;
parser_block_t gc_block;

#define FAIL(status) return(status);

void gc_init()
{
    memset(&gc_state, 0, sizeof(parser_state_t));

    // Load default G54 coordinate system.
    if (!(settings_read_coord_data(gc_state.modal.coord_select,gc_state.coord_system))) {
        report_status_message(STATUS_SETTING_READ_FAIL);
    }
}

// Sets g-code parser position in mm. Input in steps. Called by the system abort and hard
// limit pull-off routines.
void gc_sync_position()
{
    system_convert_array_steps_to_mpos(gc_state.position,sys_position);
}

// Executes one line of 0-terminated G-Code. The line is assumed to contain only uppercase
// characters and signed floating point values (no whitespace). Comments and block delete
// characters have been removed. In this function, all units and positions are converted and
// exported to grbl's internal functions in terms of (mm, mm/min) and absolute machine
// coordinates, respectively.
uint8_t gc_execute_line(char *line)
{
    /* -----
    STEP 1: Initialize parser block struct and copy current g-code state modes. The parser
    updates these modes and commands as the block line is parser and will only be used and
    executed after successful error-checking. The parser block struct also contains a block
    values struct, word tracking variables, and a non-modal commands tracker for the new
    block. This struct contains all of the necessary information to execute the block. */

    memset(&gc_block, 0, sizeof(parser_block_t)); // Initialize the parser block struct.
    memcpy(&gc_block.modal,&gc_state.modal,sizeof(gc_modal_t)); // Copy current modes

    uint8_t axis_command = AXIS_COMMAND_NONE;
    uint8_t axis_0, axis_1, axis_linear;
    uint8_t coord_select = 0; // Tracks G10 P coordinate selection for execution

    // Initialize bitflag tracking variables for axis indices compatible operations.
    uint8_t axis_words = 0; // XYZ tracking
    uint8_t ijk_words = 0; // IJK tracking

```

```

// Initialize command and value words and parser flags variables.
uint16_t command_words = 0; // Tracks G and M command words. Also used for modal group violations.
uint16_t value_words = 0; // Tracks value words.
uint8_t gc_parser_flags = GC_PARSER_NONE;

// Determine if the line is a jogging motion or a normal g-code block.
if (line[0] == '$') { // NOTE: `$$` already parsed when passed to this function.
    // Set G1 and G94 enforced modes to ensure accurate error checks.
    gc_parser_flags |= GC_PARSER_JOG_MOTION;
    gc_block.modal.motion = MOTION_MODE_LINEAR;
    gc_block.modal.feed_rate = FEED_RATE_MODE_UNITS_PER_MIN;
#ifdef USE_LINE_NUMBERS
    gc_block.values.n = JOG_LINE_NUMBER; // Initialize default line number reported during jog.
#endif
}

/* -----
STEP 2: Import all g-code words in the block line. A g-code word is a letter followed by
a number, which can either be a 'G'/'M' command or sets/assigns a command value. Also,
perform initial error-checks for command word modal group violations, for any repeated
words, and for negative values set for the value words F, N, P, T, and S. */

uint8_t word_bit; // Bit-value for assigning tracking variables
uint8_t char_counter;
char letter;
float value;
uint8_t int_value = 0;
uint16_t mantissa = 0;
if (gc_parser_flags & GC_PARSER_JOG_MOTION) { char_counter = 3; } // Start parsing after `$$`
else { char_counter = 0; }

while (line[char_counter] != 0) { // Loop until no more g-code words in line.

    // Import the next g-code word, expecting a letter followed by a value. Otherwise, error out.
    letter = line[char_counter];
    if ((letter < 'A') || (letter > 'Z')) { FAIL(STATUS_EXPECTED_COMMAND_LETTER); } // [Expected word letter]
    char_counter++;
    if (!read_float(line, &char_counter, &value)) { FAIL(STATUS_BAD_NUMBER_FORMAT); } // [Expected word value]

    // Convert values to smaller uint8 significand and mantissa values for parsing this word.
    // NOTE: Mantissa is multiplied by 100 to catch non-integer command values. This is more
    // accurate than the NIST gcode requirement of x10 when used for commands, but not quite
    // accurate enough for value words that require integers to within 0.0001. This should be
    // a good enough compromise and catch most all non-integer errors. To make it compliant,
    // we would simply need to change the mantissa to int16, but this add compiled flash space.
    // Maybe update this later.
    int_value = trunc(value);
    mantissa = round(100*(value - int_value)); // Compute mantissa for Gxx.x commands.
    // NOTE: Rounding must be used to catch small floating point errors.

```

```
// Check if the g-code word is supported or errors due to modal group violations or has
// been repeated in the g-code block. If ok, update the command or record its value.
```

```
switch(letter) {
```

```
/* 'G' and 'M' Command Words: Parse commands and check for modal group violations.
```

```
NOTE: Modal group numbers are defined in Table 4 of NIST RS274-NGC v3, pg.20 */
```

```
case 'G':
```

```
// Determine 'G' command and its modal group
```

```
switch(int_value) {
```

```
case 10: case 28: case 30: case 92:
```

```
// Check for G10/28/30/92 being called with G0/1/2/3/38 on same block.
```

```
// * G43.1 is also an axis command but is not explicitly defined this way.
```

```
if (mantissa == 0) { // Ignore G28.1, G30.1, and G92.1
```

```
    if (axis_command) { FAIL(STATUS_GCODE_AXIS_COMMAND_CONFLICT); } // [Axis word/command conflict]
```

```
    axis_command = AXIS_COMMAND_NON_MODAL;
```

```
}
```

```
// No break. Continues to next line.
```

```
case 4: case 53:
```

```
word_bit = MODAL_GROUP_G0;
```

```
gc_block.non_modal_command = int_value;
```

```
if ((int_value == 28) || (int_value == 30) || (int_value == 92)) {
```

```
    if (!((mantissa == 0) || (mantissa == 10))) { FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); }
```

```
    gc_block.non_modal_command += mantissa;
```

```
    mantissa = 0; // Set to zero to indicate valid non-integer G command.
```

```
}
```

```
break;
```

```
case 0: case 1: case 2: case 3: case 38:
```

```
// Check for G0/1/2/3/38 being called with G10/28/30/92 on same block.
```

```
// * G43.1 is also an axis command but is not explicitly defined this way.
```

```
if (axis_command) { FAIL(STATUS_GCODE_AXIS_COMMAND_CONFLICT); } // [Axis word/command conflict]
```

```
axis_command = AXIS_COMMAND_MOTION_MODE;
```

```
// No break. Continues to next line.
```

```
case 80:
```

```
word_bit = MODAL_GROUP_G1;
```

```
gc_block.modal.motion = int_value;
```

```
if (int_value == 38){
```

```
    if (!((mantissa == 20) || (mantissa == 30) || (mantissa == 40) || (mantissa == 50))) {
```

```
        FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); // [Unsupported G38.x command]
```

```
    }
```

```
    gc_block.modal.motion += (mantissa/10)+100;
```

```
    mantissa = 0; // Set to zero to indicate valid non-integer G command.
```

```
}
```

```
break;
```

```
case 17: case 18: case 19:
```

```
word_bit = MODAL_GROUP_G2;
```

```
gc_block.modal.plane_select = int_value - 17;
```

```
break;
```

```
case 90: case 91:
```

```
if (mantissa == 0) {
```

```
    word_bit = MODAL_GROUP_G3;
```

```

    gc_block.modal.distance = int_value - 90;
} else {
    word_bit = MODAL_GROUP_G4;
    if ((mantissa != 10) || (int_value == 90)) { FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); } // [G90.1 not s
    mantissa = 0; // Set to zero to indicate valid non-integer G command.
    // Otherwise, arc IJK incremental mode is default. G91.1 does nothing.
}
break;
case 93: case 94:
    word_bit = MODAL_GROUP_G5;
    gc_block.modal.feed_rate = 94 - int_value;
    break;
case 20: case 21:
    word_bit = MODAL_GROUP_G6;
    gc_block.modal.units = 21 - int_value;
    break;
case 40:
    word_bit = MODAL_GROUP_G7;
    // NOTE: Not required since cutter radius compensation is always disabled. Only here
    // to support G40 commands that often appear in g-code program headers to setup defaults.
    // gc_block.modal.cutter_comp = CUTTER_COMP_DISABLE; // G40
    break;
case 43: case 49:
    word_bit = MODAL_GROUP_G8;
    // NOTE: The NIST g-code standard vaguely states that when a tool length offset is changed,
    // there cannot be any axis motion or coordinate offsets updated. Meaning G43, G43.1, and G49
    // all are explicit axis commands, regardless if they require axis words or not.
    if (axis_command) { FAIL(STATUS_GCODE_AXIS_COMMAND_CONFLICT); } // [Axis word/command conflict] }
    axis_command = AXIS_COMMAND_TOOL_LENGTH_OFFSET;
    if (int_value == 49) { // G49
        gc_block.modal.tool_length = TOOL_LENGTH_OFFSET_CANCEL;
    } else if (mantissa == 10) { // G43.1
        gc_block.modal.tool_length = TOOL_LENGTH_OFFSET_ENABLE_DYNAMIC;
    } else { FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); } // [Unsupported G43.x command]
    mantissa = 0; // Set to zero to indicate valid non-integer G command.
    break;
case 54: case 55: case 56: case 57: case 58: case 59:
    // NOTE: G59.x are not supported. (But their int_values would be 60, 61, and 62.)
    word_bit = MODAL_GROUP_G12;
    gc_block.modal.coord_select = int_value - 54; // Shift to array indexing.
    break;
case 61:
    word_bit = MODAL_GROUP_G13;
    if (mantissa != 0) { FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); } // [G61.1 not supported]
    // gc_block.modal.control = CONTROL_MODE_EXACT_PATH; // G61
    break;
default: FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); // [Unsupported G command]
}
if (mantissa > 0) { FAIL(STATUS_GCODE_COMMAND_VALUE_NOT_INTEGER); } // [Unsupported or invalid Gxx.x c
// Check for more than one command per modal group violations in the current block
// NOTE: Variable 'word_bit' is always assigned, if the command is valid.

```

```

if ( bit_istrue(command_words,bit(word_bit)) ) { FAIL(STATUS_GCODE_MODAL_GROUP_VIOLATION); }
command_words |= bit(word_bit);
break;

```

```

case 'M':

```

```

// Determine 'M' command and its modal group
if (mantissa > 0) { FAIL(STATUS_GCODE_COMMAND_VALUE_NOT_INTEGER); } // [No Mxx.x commands]
switch(int_value) {
  case 0: case 1: case 2: case 30:
    word_bit = MODAL_GROUP_M4;
    switch(int_value) {
      case 0: gc_block.modal.program_flow = PROGRAM_FLOW_PAUSED; break; // Program pause
      case 1: break; // Optional stop not supported. Ignore.
      default: gc_block.modal.program_flow = int_value; // Program end and reset
    }
    break;
  case 3: case 4: case 5:
    word_bit = MODAL_GROUP_M7;
    switch(int_value) {
      case 3: gc_block.modal.spindle = SPINDLE_ENABLE_CW; break;
      case 4: gc_block.modal.spindle = SPINDLE_ENABLE_CCW; break;
      case 5: gc_block.modal.spindle = SPINDLE_DISABLE; break;
    }
    break;
#ifdef ENABLE_M7
    case 7: case 8: case 9:
#else
    case 8: case 9:
#endif
    word_bit = MODAL_GROUP_M8;
    switch(int_value) {
      #ifdef ENABLE_M7
        case 7: gc_block.modal.coolant |= COOLANT_MIST_ENABLE; break;
      #endif
      case 8: gc_block.modal.coolant |= COOLANT_FLOOD_ENABLE; break;
      case 9: gc_block.modal.coolant = COOLANT_DISABLE; break; // M9 disables both M7 and M8.
    }
    break;
#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
    case 56:
      word_bit = MODAL_GROUP_M9;
      gc_block.modal.override = OVERRIDE_PARKING_MOTION;
      break;
#endif
    default: FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); // [Unsupported M command]
}

```

```

// Check for more than one command per modal group violations in the current block
// NOTE: Variable 'word_bit' is always assigned, if the command is valid.
if ( bit_istrue(command_words,bit(word_bit)) ) { FAIL(STATUS_GCODE_MODAL_GROUP_VIOLATION); }

```

```

command_words |= bit(word_bit);
break;

```

```

// NOTE: All remaining letters assign values.
default:

```

```

/* Non-Command Words: This initial parsing phase only checks for repeats of the remaining
   legal g-code words and stores their value. Error-checking is performed later since some
   words (I,J,K,L,P,R) have multiple connotations and/or depend on the issued commands. */
switch(letter){
  // case 'A': // Not supported
  // case 'B': // Not supported
  // case 'C': // Not supported
  // case 'D': // Not supported
  case 'F': word_bit = WORD_F; gc_block.values.f = value; break;
  // case 'H': // Not supported
  case 'I': word_bit = WORD_I; gc_block.values.ijk[X_AXIS] = value; ijk_words |= (1<<X_AXIS); break;
  case 'J': word_bit = WORD_J; gc_block.values.ijk[Y_AXIS] = value; ijk_words |= (1<<Y_AXIS); break;
  case 'K': word_bit = WORD_K; gc_block.values.ijk[Z_AXIS] = value; ijk_words |= (1<<Z_AXIS); break;
  case 'L': word_bit = WORD_L; gc_block.values.l = int_value; break;
  case 'N': word_bit = WORD_N; gc_block.values.n = trunc(value); break;
  case 'P': word_bit = WORD_P; gc_block.values.p = value; break;
  // NOTE: For certain commands, P value must be an integer, but none of these commands are supported.
  // case 'Q': // Not supported
  case 'R': word_bit = WORD_R; gc_block.values.r = value; break;
  case 'S': word_bit = WORD_S; gc_block.values.s = value; break;
  case 'T': word_bit = WORD_T;
  ■■■■■ if (value > MAX_TOOL_NUMBER) { FAIL(STATUS_GCODE_MAX_VALUE_EXCEEDED); }
    gc_block.values.t = int_value;
  ■■■■■ break;
  case 'X': word_bit = WORD_X; gc_block.values.xyz[X_AXIS] = value; axis_words |= (1<<X_AXIS); break;
  case 'Y': word_bit = WORD_Y; gc_block.values.xyz[Y_AXIS] = value; axis_words |= (1<<Y_AXIS); break;
  case 'Z': word_bit = WORD_Z; gc_block.values.xyz[Z_AXIS] = value; axis_words |= (1<<Z_AXIS); break;
  default: FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND);
}

```

```

// NOTE: Variable 'word_bit' is always assigned, if the non-command letter is valid.
if (bit_istrue(value_words,bit(word_bit))) { FAIL(STATUS_GCODE_WORD_REPEATED); } // [Word repeated]
// Check for invalid negative values for words F, N, P, T, and S.
// NOTE: Negative value check is done here simply for code-efficiency.
if ( bit(word_bit) & (bit(WORD_F)|bit(WORD_N)|bit(WORD_P)|bit(WORD_T)|bit(WORD_S)) ) {
  if (value < 0.0) { FAIL(STATUS_NEGATIVE_VALUE); } // [Word value cannot be negative]
}
value_words |= bit(word_bit); // Flag to indicate parameter assigned.

```

```

}
}
// Parsing complete!

```

```

/* -----

```

STEP 3: Error-check all commands and values passed in this block. This step ensures all of the commands are valid for execution and follows the NIST standard as closely as possible. If an error is found, all commands and values in this block are dumped and will not update the active system g-code modes. If the block is ok, the active system g-code modes will be updated based on the commands of this block, and signal for it to be executed.

Also, we have to pre-convert all of the values passed based on the modes set by the parsed block. There are a number of error-checks that require target information that can only be accurately calculated if we convert these values in conjunction with the error-checking. This relegates the next execution step as only updating the system g-code modes and performing the programmed actions in order. The execution step should not require any conversion calculations and would only require minimal checks necessary to execute.

*/

/* NOTE: At this point, the g-code block has been parsed and the block line can be freed.

NOTE: It's also possible, at some future point, to break up STEP 2, to allow piece-wise parsing of the block on a per-word basis, rather than the entire block. This could remove the need for maintaining a large string variable for the entire block and free up some memory. To do this, this would simply need to retain all of the data in STEP 1, such as the new block data struct, the modal group and value bitflag tracking variables, and axis array indices compatible variables. This data contains all of the information necessary to error-check the new g-code block when the EOL character is received. However, this would break Grbl's startup lines in how it currently works and would require some refactoring to make it compatible.

*/

// [0. Non-specific/common error-checks and miscellaneous setup]:

// Determine implicit axis command conditions. Axis words have been passed, but no explicit axis command has been sent. If so, set axis command to current motion mode.

```
if (axis_words) {  
  if (!axis_command) { axis_command = AXIS_COMMAND_MOTION_MODE; } // Assign implicit motion-mode  
}
```

// Check for valid line number N value.

```
if (bit_istrue(value_words,bit(WORD_N))) {  
  // Line number value cannot be less than zero (done) or greater than max line number.  
  if (gc_block.values.n > MAX_LINE_NUMBER) { FAIL(STATUS_GCODE_INVALID_LINE_NUMBER); } // [Exceeds max li  
}  
// bit_false(value_words,bit(WORD_N)); // NOTE: Single-meaning value word. Set at end of error-checking.
```

// Track for unused words at the end of error-checking.

// NOTE: Single-meaning value words are removed all at once at the end of error-checking, because they are always used when present. This was done to save a few bytes of flash. For clarity, the single-meaning value words may be removed as they are used. Also, axis words are treated in the same way. If there is an explicit/implicit axis command, XYZ words are always used and are removed at the end of error-checking.

// [1. Comments]: MSG's NOT SUPPORTED. Comment handling performed by protocol.

// [2. Set feed rate mode]: G93 F word missing with G1,G2/3 active, implicitly or explicitly. Feed rate is not defined after switching to G94 from G93.

```

// NOTE: For jogging, ignore prior feed rate mode. Enforce G94 and check for required F word.
if (gc_parser_flags & GC_PARSER_JOG_MOTION) {
    if (bit_isfalse(value_words,bit(WORD_F))) { FAIL(STATUS_GCODE_UNDEFINED_FEED_RATE); }
    if (gc_block.modal.units == UNITS_MODE_INCHES) { gc_block.values.f *= MM_PER_INCH; }
} else {
    if (gc_block.modal.feed_rate == FEED_RATE_MODE_INVERSE_TIME) { // = G93
        // NOTE: G38 can also operate in inverse time, but is undefined as an error. Missing F word check added here.
        if (axis_command == AXIS_COMMAND_MOTION_MODE) {
            if ((gc_block.modal.motion != MOTION_MODE_NONE) && (gc_block.modal.motion != MOTION_MODE_SEEK)) {
                if (bit_isfalse(value_words,bit(WORD_F))) { FAIL(STATUS_GCODE_UNDEFINED_FEED_RATE); } // [F word missing]
            }
        }
        // NOTE: It seems redundant to check for an F word to be passed after switching from G94 to G93. We would
        // accomplish the exact same thing if the feed rate value is always reset to zero and undefined after each
        // inverse time block, since the commands that use this value already perform undefined checks. This would
        // also allow other commands, following this switch, to execute and not error out needlessly. This code is
        // combined with the above feed rate mode and the below set feed rate error-checking.

        // [3. Set feed rate ]: F is negative (done.)
        // - In inverse time mode: Always implicitly zero the feed rate value before and after block completion.
        // NOTE: If in G93 mode or switched into it from G94, just keep F value as initialized zero or passed F word
        // value in the block. If no F word is passed with a motion command that requires a feed rate, this will error
        // out in the motion modes error-checking. However, if no F word is passed with NO motion command that requires
        // a feed rate, we simply move on and the state feed rate value gets updated to zero and remains undefined.
    } else { // = G94
        // - In units per mm mode: If F word passed, ensure value is in mm/min, otherwise push last state value.
        if (gc_state.modal.feed_rate == FEED_RATE_MODE_UNITS_PER_MIN) { // Last state is also G94
            if (bit_istrue(value_words,bit(WORD_F))) {
                if (gc_block.modal.units == UNITS_MODE_INCHES) { gc_block.values.f *= MM_PER_INCH; }
            } else {
                gc_block.values.f = gc_state.feed_rate; // Push last state feed rate
            }
        } // Else, switching to G94 from G93, so don't push last state feed rate. Its undefined or the passed F word value.
    }
}
// bit_false(value_words,bit(WORD_F)); // NOTE: Single-meaning value word. Set at end of error-checking.

// [4. Set spindle speed ]: S is negative (done.)
if (bit_isfalse(value_words,bit(WORD_S))) { gc_block.values.s = gc_state.spindle_speed; }
// bit_false(value_words,bit(WORD_S)); // NOTE: Single-meaning value word. Set at end of error-checking.

// [5. Select tool ]: NOT SUPPORTED. Only tracks value. T is negative (done.) Not an integer. Greater than max tool value
// bit_false(value_words,bit(WORD_T)); // NOTE: Single-meaning value word. Set at end of error-checking.

// [6. Change tool ]: N/A
// [7. Spindle control ]: N/A
// [8. Coolant control ]: N/A
// [9. Override control ]: Not supported except for a Grbl-only parking motion override control.
#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
    if (bit_istrue(command_words,bit(MODAL_GROUP_M9))) { // Already set as enabled in parser.
        if (bit_istrue(value_words,bit(WORD_P))) {

```

```

    if (gc_block.values.p == 0.0) { gc_block.modal.override = OVERRIDE_DISABLED; }
    bit_false(value_words,bit(WORD_P));
  }
}
#endif

// [10. Dwell ]: P value missing. P is negative (done.) NOTE: See below.
if (gc_block.non_modal_command == NON_MODAL_DWELL) {
  if (bit_isfalse(value_words,bit(WORD_P))) { FAIL(STATUS_GCODE_VALUE_WORD_MISSING); } // [P word missing]
  bit_false(value_words,bit(WORD_P));
}

// [11. Set active plane ]: N/A
switch (gc_block.modal.plane_select) {
case PLANE_SELECT_XY:
  axis_0 = X_AXIS;
  axis_1 = Y_AXIS;
  axis_linear = Z_AXIS;
  break;
case PLANE_SELECT_ZX:
  axis_0 = Z_AXIS;
  axis_1 = X_AXIS;
  axis_linear = Y_AXIS;
  break;
default: // case PLANE_SELECT_YZ:
  axis_0 = Y_AXIS;
  axis_1 = Z_AXIS;
  axis_linear = X_AXIS;
}

// [12. Set length units ]: N/A
// Pre-convert XYZ coordinate values to millimeters, if applicable.
uint8_t idx;
if (gc_block.modal.units == UNITS_MODE_INCHES) {
  for (idx=0; idx<N_AXIS; idx++) { // Axes indices are consistent, so loop may be used.
    if (bit_istrue(axis_words,bit(idx)) ) {
      gc_block.values.xyz[idx] *= MM_PER_INCH;
    }
  }
}

// [13. Cutter radius compensation ]: G41/42 NOT SUPPORTED. Error, if enabled while G53 is active.
// [G40 Errors]: G2/3 arc is programmed after a G40. The linear move after disabling is less than tool diameter.
// NOTE: Since cutter radius compensation is never enabled, these G40 errors don't apply. Grbl supports G40
// only for the purpose to not error when G40 is sent with a g-code program header to setup the default modes.

// [14. Cutter length compensation ]: G43 NOT SUPPORTED, but G43.1 and G49 are.
// [G43.1 Errors]: Motion command in same line.
// NOTE: Although not explicitly stated so, G43.1 should be applied to only one valid
// axis that is configured (in config.h). There should be an error if the configured axis
// is absent or if any of the other axis words are present.

```

```

if (axis_command == AXIS_COMMAND_TOOL_LENGTH_OFFSET ) { // Indicates called in block.
    if (gc_block.modal.tool_length == TOOL_LENGTH_OFFSET_ENABLE_DYNAMIC) {
        if (axis_words ^ (1<<TOOL_LENGTH_OFFSET_AXIS)) { FAIL(STATUS_GCODE_G43_DYNAMIC_AXIS_ERROR); }
    }
}

// [15. Coordinate system selection ]: *N/A. Error, if cutter radius comp is active.
// TODO: An EEPROM read of the coordinate data may require a buffer sync when the cycle
// is active. The read pauses the processor temporarily and may cause a rare crash. For
// future versions on processors with enough memory, all coordinate data should be stored
// in memory and written to EEPROM only when there is not a cycle active.
float block_coord_system[N_AXIS];
memcpy(block_coord_system,gc_state.coord_system,sizeof(gc_state.coord_system));
if ( bit_istrue(command_words,bit(MODAL_GROUP_G12)) ) { // Check if called in block
    if (gc_block.modal.coord_select > N_COORDINATE_SYSTEM) { FAIL(STATUS_GCODE_UNSUPPORTED_COORD_S
    if (gc_state.modal.coord_select != gc_block.modal.coord_select) {
        if (!(settings_read_coord_data(gc_block.modal.coord_select,block_coord_system))) { FAIL(STATUS_SETTING_READ_
    }
}

// [16. Set path control mode ]: N/A. Only G61. G61.1 and G64 NOT SUPPORTED.
// [17. Set distance mode ]: N/A. Only G91.1. G90.1 NOT SUPPORTED.
// [18. Set retract mode ]: NOT SUPPORTED.

// [19. Remaining non-modal actions ]: Check go to predefined position, set G10, or set axis offsets.
// NOTE: We need to separate the non-modal commands that are axis word-using (G10/G28/G30/G92), as these
// commands all treat axis words differently. G10 as absolute offsets or computes current position as
// the axis value, G92 similarly to G10 L20, and G28/30 as an intermediate target position that observes
// all the current coordinate system and G92 offsets.
switch (gc_block.non_modal_command) {
case NON_MODAL_SET_COORDINATE_DATA:
    // [G10 Errors]: L missing and is not 2 or 20. P word missing. (Negative P value done.)
    // [G10 L2 Errors]: R word NOT SUPPORTED. P value not 0 to nCoordSys(max 9). Axis words missing.
    // [G10 L20 Errors]: P must be 0 to nCoordSys(max 9). Axis words missing.
    if (!axis_words) { FAIL(STATUS_GCODE_NO_AXIS_WORDS) }; // [No axis words]
    if (bit_istrue(value_words,((1<<WORD_P)|(1<<WORD_L)))) { FAIL(STATUS_GCODE_VALUE_WORD_MISSING); } //
    coord_select = trunc(gc_block.values.p); // Convert p value to int.
    if (coord_select > N_COORDINATE_SYSTEM) { FAIL(STATUS_GCODE_UNSUPPORTED_COORD_SYS); } // [Greater
    if (gc_block.values.l != 20) {
        if (gc_block.values.l == 2) {
            if (bit_istrue(value_words,bit(WORD_R))) { FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); } // [G10 L2 R no
        } else { FAIL(STATUS_GCODE_UNSUPPORTED_COMMAND); } // [Unsupported L]
    }
    bit_false(value_words,(bit(WORD_L)|bit(WORD_P)));

    // Determine coordinate system to change and try to load from EEPROM.
    if (coord_select > 0) { coord_select--; } // Adjust P1-P6 index to EEPROM coordinate data indexing.
    else { coord_select = gc_block.modal.coord_select; } // Index P0 as the active coordinate system

    // NOTE: Store parameter data in IJK values. By rule, they are not in use with this command.
    if (!(settings_read_coord_data(coord_select,gc_block.values.ijk)) { FAIL(STATUS_SETTING_READ_FAIL); } // [EEPROM

```

```

// Pre-calculate the coordinate data changes.
for (idx=0; idx<N_AXIS; idx++) { // Axes indices are consistent, so loop may be used.
    // Update axes defined only in block. Always in machine coordinates. Can change non-active system.
    if (bit_istrue(axis_words,bit(idx)) ) {
        if (gc_block.values.l == 20) {
            // L20: Update coordinate system axis at current position (with modifiers) with programmed value
            // WPos = MPos - WCS - G92 - TLO -> WCS = MPos - G92 - TLO - WPos
            gc_block.values.ijk[idx] = gc_state.position[idx]-gc_state.coord_offset[idx]-gc_block.values.xyz[idx];
            if (idx == TOOL_LENGTH_OFFSET_AXIS) { gc_block.values.ijk[idx] -= gc_state.tool_length_offset; }
        } else {
            // L2: Update coordinate system axis to programmed value.
            gc_block.values.ijk[idx] = gc_block.values.xyz[idx];
        }
    } // Else, keep current stored value.
}
break;
case NON_MODAL_SET_COORDINATE_OFFSET:
    // [G92 Errors]: No axis words.
    if (!axis_words) { FAIL(STATUS_GCODE_NO_AXIS_WORDS); } // [No axis words]

    // Update axes defined only in block. Offsets current system to defined value. Does not update when
    // active coordinate system is selected, but is still active unless G92.1 disables it.
    for (idx=0; idx<N_AXIS; idx++) { // Axes indices are consistent, so loop may be used.
        if (bit_istrue(axis_words,bit(idx)) ) {
            // WPos = MPos - WCS - G92 - TLO -> G92 = MPos - WCS - TLO - WPos
            gc_block.values.xyz[idx] = gc_state.position[idx]-block_coord_system[idx]-gc_block.values.xyz[idx];
            if (idx == TOOL_LENGTH_OFFSET_AXIS) { gc_block.values.xyz[idx] -= gc_state.tool_length_offset; }
        } else {
            gc_block.values.xyz[idx] = gc_state.coord_offset[idx];
        }
    }
}
break;

```

default:

```

// At this point, the rest of the explicit axis commands treat the axis values as the traditional
// target position with the coordinate system offsets, G92 offsets, absolute override, and distance
// modes applied. This includes the motion mode commands. We can now pre-compute the target position.
// NOTE: Tool offsets may be appended to these conversions when/if this feature is added.
if (axis_command != AXIS_COMMAND_TOOL_LENGTH_OFFSET ) { // TLO block any axis command.
    if (axis_words) {
        for (idx=0; idx<N_AXIS; idx++) { // Axes indices are consistent, so loop may be used to save flash space.
            if ( bit_isfalse(axis_words,bit(idx)) ) {
                gc_block.values.xyz[idx] = gc_state.position[idx]; // No axis word in block. Keep same axis position.
            } else {
                // Update specified value according to distance mode or ignore if absolute override is active.
                // NOTE: G53 is never active with G28/30 since they are in the same modal group.
                if (gc_block.non_modal_command != NON_MODAL_ABSOLUTE_OVERRIDE) {
                    // Apply coordinate offsets based on distance mode.
                    if (gc_block.modal.distance == DISTANCE_MODE_ABSOLUTE) {

```



```

if (gc_block.modal.motion == MOTION_MODE_NONE) {
    // [G80 Errors]: Axis word are programmed while G80 is active.
    // NOTE: Even non-modal commands or TLO that use axis words will throw this strict error.
    if (axis_words) { FAIL(STATUS_GCODE_AXIS_WORDS_EXIST); } // [No axis words allowed]

    // Check remaining motion modes, if axis word are implicit (exist and not used by G10/28/30/92), or
    // was explicitly commanded in the g-code block.
} else if ( axis_command == AXIS_COMMAND_MOTION_MODE ) {

    if (gc_block.modal.motion == MOTION_MODE_SEEK) {
        // [G0 Errors]: Axis letter not configured or without real value (done.)
        // Axis words are optional. If missing, set axis command flag to ignore execution.
        if (!axis_words) { axis_command = AXIS_COMMAND_NONE; }

        // All remaining motion modes (all but G0 and G80), require a valid feed rate value. In units per mm mode,
        // the value must be positive. In inverse time mode, a positive value must be passed with each block.
    } else {
        // Check if feed rate is defined for the motion modes that require it.
        if (gc_block.values.f == 0.0) { FAIL(STATUS_GCODE_UNDEFINED_FEED_RATE); } // [Feed rate undefined]

        switch (gc_block.modal.motion) {
            case MOTION_MODE_LINEAR:
                // [G1 Errors]: Feed rate undefined. Axis letter not configured or without real value.
                // Axis words are optional. If missing, set axis command flag to ignore execution.
                if (!axis_words) { axis_command = AXIS_COMMAND_NONE; }
                break;
            case MOTION_MODE_CW_ARC:
                gc_parser_flags |= GC_PARSER_ARC_IS_CLOCKWISE; // No break intentional.
            case MOTION_MODE_CCW_ARC:
                // [G2/3 Errors All-Modes]: Feed rate undefined.
                // [G2/3 Radius-Mode Errors]: No axis words in selected plane. Target point is same as current.
                // [G2/3 Offset-Mode Errors]: No axis words and/or offsets in selected plane. The radius to the current
                // point and the radius to the target point differs more than 0.002mm (EMC def. 0.5mm OR 0.005mm and 0.1% radius)
                // [G2/3 Full-Circle-Mode Errors]: NOT SUPPORTED. Axis words exist. No offsets programmed. P must be an integer
                // NOTE: Both radius and offsets are required for arc tracing and are pre-computed with the error-checking.

                if (!axis_words) { FAIL(STATUS_GCODE_NO_AXIS_WORDS); } // [No axis words]
                if (!(axis_words & (bit(axis_0)|bit(axis_1)))) { FAIL(STATUS_GCODE_NO_AXIS_WORDS_IN_PLANE); } // [No axis words in plane]

                // Calculate the change in position along each selected axis
                float x,y;
                x = gc_block.values.xyz[axis_0]-gc_state.position[axis_0]; // Delta x between current position and target
                y = gc_block.values.xyz[axis_1]-gc_state.position[axis_1]; // Delta y between current position and target

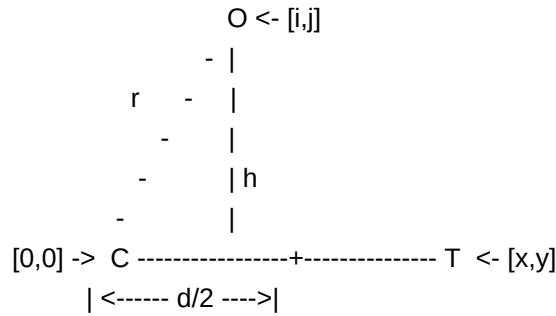
                if (value_words & bit(WORD_R)) { // Arc Radius Mode
                    bit_false(value_words,bit(WORD_R));
                    if (isequal_position_vector(gc_state.position, gc_block.values.xyz)) { FAIL(STATUS_GCODE_INVALID_TARGET); }

                    // Convert radius value to proper units.
                    if (gc_block.modal.units == UNITS_MODE_INCHES) { gc_block.values.r *= MM_PER_INCH; }
                    /* We need to calculate the center of the circle that has the designated radius and passes

```

through both the current position and the target position. This method calculates the following set of equations where $[x,y]$ is the vector from current to target position, $d ==$ magnitude of that vector, $h ==$ hypotenuse of the triangle formed by the radius of the circle, the distance to the center of the travel vector. A vector perpendicular to the travel vector $[-y,x]$ is scaled to the length of h $[-y/d*h, x/d*h]$ and added to the center of the travel vector $[x/2,y/2]$ to form the new point $[i,j]$ at $[x/2-y/d*h, y/2+x/d*h]$ which will be the center of our arc.

```
d^2 == x^2 + y^2
h^2 == r^2 - (d/2)^2
i == x/2 - y/d*h
j == y/2 + x/d*h
```



C - Current position
T - Target position
O - center of circle that pass through both C and T
d - distance from C to T
r - designated radius
h - distance from center of CT to O

Expanding the equations:

```
d -> sqrt(x^2 + y^2)
h -> sqrt(4 * r^2 - x^2 - y^2)/2
i -> (x - (y * sqrt(4 * r^2 - x^2 - y^2)) / sqrt(x^2 + y^2)) / 2
j -> (y + (x * sqrt(4 * r^2 - x^2 - y^2)) / sqrt(x^2 + y^2)) / 2
```

Which can be written:

```
i -> (x - (y * sqrt(4 * r^2 - x^2 - y^2))/sqrt(x^2 + y^2))/2
j -> (y + (x * sqrt(4 * r^2 - x^2 - y^2))/sqrt(x^2 + y^2))/2
```

Which we for size and speed reasons optimize to:

```
h_x2_div_d = sqrt(4 * r^2 - x^2 - y^2)/sqrt(x^2 + y^2)
i = (x - (y * h_x2_div_d))/2
j = (y + (x * h_x2_div_d))/2
*/
```

```
// First, use h_x2_div_d to compute 4*h^2 to check if it is negative or r is smaller
// than d. If so, the sqrt of a negative number is complex and error out.
float h_x2_div_d = 4.0 * gc_block.values.r*gc_block.values.r - x*x - y*y;
```

```
if (h_x2_div_d < 0) { FAIL(STATUS_GCODE_ARC_RADIUS_ERROR); } // [Arc radius error]
```

```
// Finish computing h_x2_div_d.
```

```
h_x2_div_d = -sqrt(h_x2_div_d)/hypot_f(x,y); // == -(h * 2 / d)
```

```
// Invert the sign of h_x2_div_d if the circle is counter clockwise (see sketch below)
```

```
if (gc_block.modal.motion == MOTION_MODE_CCW_ARC) { h_x2_div_d = -h_x2_div_d; }
```

```
/* The counter clockwise circle lies to the left of the target direction. When offset is positive,
the left hand circle will be generated - when it is negative the right hand circle is generated.
```

```
T <-- Target position
```

^

Clockwise circles with this center
will have > 180 deg of angular travel

Clockwise circles with this center will have
< 180 deg of angular travel, which is a good thing!

\ | /

center of arc when h_x2_div_d is positive -> x <---- | ----> x <- center of arc when h_x2_div_d is negative

|
|

```
C <-- Current position
```

```
*/
```

```
// Negative R is g-code-alese for "I want a circle with more than 180 degrees of travel" (go figure!),
```

```
// even though it is advised against ever generating such circles in a single line of g-code. By
```

```
// inverting the sign of h_x2_div_d the center of the circles is placed on the opposite side of the line of
```

```
// travel and thus we get the unadvisably long arcs as prescribed.
```

```
if (gc_block.values.r < 0) {
```

```
    h_x2_div_d = -h_x2_div_d;
```

```
    gc_block.values.r = -gc_block.values.r; // Finished with r. Set to positive for mc_arc
```

```
}
```

```
// Complete the operation by calculating the actual center of the arc
```

```
gc_block.values.ijk[axis_0] = 0.5*(x-(y*h_x2_div_d));
```

```
gc_block.values.ijk[axis_1] = 0.5*(y+(x*h_x2_div_d));
```

```
} else { // Arc Center Format Offset Mode
```

```
if (!(ijk_words & (bit(axis_0)|bit(axis_1)))) { FAIL(STATUS_GCODE_NO_OFFSETS_IN_PLANE); } // [No offsets in plane]
bit_false(value_words,(bit(WORD_I)|bit(WORD_J)|bit(WORD_K)));
```

```
// Convert IJK values to proper units.
```

```
if (gc_block.modal.units == UNITS_MODE_INCHES) {
```

```
    for (idx=0; idx<N_AXIS; idx++) { // Axes indices are consistent, so loop may be used to save flash space.
```

```
        if (ijk_words & bit(idx)) { gc_block.values.ijk[idx] *= MM_PER_INCH; }
```

```
    }
```

```
}
```

```
// Arc radius from center to target
```

```
x -= gc_block.values.ijk[axis_0]; // Delta x between circle center and target
```

```
y -= gc_block.values.ijk[axis_1]; // Delta y between circle center and target
```

```
float target_r = hypot_f(x,y);
```

```
// Compute arc radius for mc_arc. Defined from current location to center.
```

```

gc_block.values.r = hypot_f(gc_block.values.ijk[axis_0], gc_block.values.ijk[axis_1]);

// Compute difference between current location and target radii for final error-checks.
float delta_r = fabs(target_r-gc_block.values.r);
if (delta_r > 0.005) {
    if (delta_r > 0.5) { FAIL(STATUS_GCODE_INVALID_TARGET); } // [Arc definition error] > 0.5mm
    if (delta_r > (0.001*gc_block.values.r)) { FAIL(STATUS_GCODE_INVALID_TARGET); } // [Arc definition error] > 0.
}
}
break;
case MOTION_MODE_PROBE_TOWARD_NO_ERROR: case MOTION_MODE_PROBE_AWAY_NO_ERROR:
    gc_parser_flags |= GC_PARSER_PROBE_IS_NO_ERROR; // No break intentional.
case MOTION_MODE_PROBE_TOWARD: case MOTION_MODE_PROBE_AWAY:
    if ((gc_block.modal.motion == MOTION_MODE_PROBE_AWAY) ||
        (gc_block.modal.motion == MOTION_MODE_PROBE_AWAY_NO_ERROR)) { gc_parser_flags |= GC_PARSER_F
// [G38 Errors]: Target is same current. No axis words. Cutter compensation is enabled. Feed rate
// is undefined. Probe is triggered. NOTE: Probe check moved to probe cycle. Instead of returning
// an error, it issues an alarm to prevent further motion to the probe. It's also done there to
// allow the planner buffer to empty and move off the probe trigger before another probing cycle.
if (!axis_words) { FAIL(STATUS_GCODE_NO_AXIS_WORDS); } // [No axis words]
if (isequal_position_vector(gc_state.position, gc_block.values.xyz)) { FAIL(STATUS_GCODE_INVALID_TARGET); }
break;
}
}
}

// [21. Program flow ]: No error checks required.

// [0. Non-specific error-checks]: Complete unused value words check, i.e. IJK used when in arc
// radius mode, or axis words that aren't used in the block.
if (gc_parser_flags & GC_PARSER_JOG_MOTION) {
    // Jogging only uses the F feed rate and XYZ value words. N is valid, but S and T are invalid.
    bit_false(value_words, (bit(WORD_N)|bit(WORD_F)));
} else {
    bit_false(value_words, (bit(WORD_N)|bit(WORD_F)|bit(WORD_S)|bit(WORD_T))); // Remove single-meaning value word
}
if (axis_command) { bit_false(value_words, (bit(WORD_X)|bit(WORD_Y)|bit(WORD_Z))); } // Remove axis words.
if (value_words) { FAIL(STATUS_GCODE_UNUSED_WORDS); } // [Unused words]

/* -----
STEP 4: EXECUTE!!
Assumes that all error-checking has been completed and no failure modes exist. We just
need to update the state and execute the block according to the order-of-execution.
*/

// Initialize planner data struct for motion blocks.
plan_line_data_t plan_data;
plan_line_data_t *pl_data = &plan_data;
memset(pl_data, 0, sizeof(plan_line_data_t)); // Zero pl_data struct

// Intercept jog commands and complete error checking for valid jog commands and execute.

```

```

// NOTE: G-code parser state is not updated, except the position to ensure sequential jog
// targets are computed correctly. The final parser position after a jog is updated in
// protocol_execute_realtime() when jogging completes or is canceled.
if (gc_parser_flags & GC_PARSER_JOG_MOTION) {
    // Only distance and unit modal commands and G53 absolute override command are allowed.
    // NOTE: Feed rate word and axis word checks have already been performed in STEP 3.
    if (command_words & ~(bit(MODAL_GROUP_G3) | bit(MODAL_GROUP_G6) | bit(MODAL_GROUP_G0))) { FAIL(STATUS_OK); }
    if (!(gc_block.non_modal_command == NON_MODAL_ABSOLUTE_OVERRIDE || gc_block.non_modal_command == NON_MODAL_FEED_RATE_OVERRIDE)) { FAIL(STATUS_OK); }

    // Initialize planner data to current spindle and coolant modal state.
    pl_data->spindle_speed = gc_state.spindle_speed;
    plan_data.condition = (gc_state.modal.spindle | gc_state.modal.coolant);

    uint8_t status = jog_execute(&plan_data, &gc_block);
    if (status == STATUS_OK) { memcpy(gc_state.position, gc_block.values.xyz, sizeof(gc_block.values.xyz)); }
    return(status);
}

// If in laser mode, setup laser power based on current and past parser conditions.
if (bit_istrue(settings.flags,BITFLAG_LASER_MODE)) {
    if ( !(gc_block.modal.motion == MOTION_MODE_LINEAR) || (gc_block.modal.motion == MOTION_MODE_CW_ARC)
        || (gc_block.modal.motion == MOTION_MODE_CCW_ARC)) ) {
        gc_parser_flags |= GC_PARSER_LASER_DISABLE;
    }
}

// Any motion mode with axis words is allowed to be passed from a spindle speed update.
// NOTE: G1 and G0 without axis words sets axis_command to none. G28/30 are intentionally omitted.
// TODO: Check sync conditions for M3 enabled motions that don't enter the planner. (zero length).
if (axis_words && (axis_command == AXIS_COMMAND_MOTION_MODE)) {
    gc_parser_flags |= GC_PARSER_LASER_ISMOTION;
} else {
    // M3 constant power laser requires planner syncs to update the laser when changing between
    // a G1/2/3 motion mode state and vice versa when there is no motion in the line.
    if (gc_state.modal.spindle == SPINDLE_ENABLE_CW) {
        if ((gc_state.modal.motion == MOTION_MODE_LINEAR) || (gc_state.modal.motion == MOTION_MODE_CW_ARC)
            || (gc_state.modal.motion == MOTION_MODE_CCW_ARC)) {
            if (bit_istrue(gc_parser_flags,GC_PARSER_LASER_DISABLE)) {
                gc_parser_flags |= GC_PARSER_LASER_FORCE_SYNC; // Change from G1/2/3 motion mode.
            }
        }
    } else {
        // When changing to a G1 motion mode without axis words from a non-G1/2/3 motion mode.
        if (bit_isfalse(gc_parser_flags,GC_PARSER_LASER_DISABLE)) {
            gc_parser_flags |= GC_PARSER_LASER_FORCE_SYNC;
        }
    }
}

// [0. Non-specific/common error-checks and miscellaneous setup]:
// NOTE: If no line number is present, the value is zero.

```

```

gc_state.line_number = gc_block.values.n;
#ifdef USE_LINE_NUMBERS
    pl_data->line_number = gc_state.line_number; // Record data for planner use.
#endif

// [1. Comments feedback ]: NOT SUPPORTED

// [2. Set feed rate mode ]:
gc_state.modal.feed_rate = gc_block.modal.feed_rate;
if (gc_state.modal.feed_rate) { pl_data->condition |= PL_COND_FLAG_INVERSE_TIME; } // Set condition flag for planner

// [3. Set feed rate ]:
gc_state.feed_rate = gc_block.values.f; // Always copy this value. See feed rate error-checking.
pl_data->feed_rate = gc_state.feed_rate; // Record data for planner use.

// [4. Set spindle speed ]:
if ((gc_state.spindle_speed != gc_block.values.s) || bit_istrue(gc_parser_flags, GC_PARSER_LASER_FORCE_SYNC)) {
    if (gc_state.modal.spindle != SPINDLE_DISABLE) {
#ifdef VARIABLE_SPINDLE
        if (bit_isfalse(gc_parser_flags, GC_PARSER_LASER_ISMOTION)) {
            if (bit_istrue(gc_parser_flags, GC_PARSER_LASER_DISABLE)) {
                spindle_sync(gc_state.modal.spindle, 0.0);
            } else { spindle_sync(gc_state.modal.spindle, gc_block.values.s); }
        }
    }
    #else
        spindle_sync(gc_state.modal.spindle, 0.0);
    #endif
}
gc_state.spindle_speed = gc_block.values.s; // Update spindle speed state.
}
// NOTE: Pass zero spindle speed for all restricted laser motions.
if (bit_isfalse(gc_parser_flags, GC_PARSER_LASER_DISABLE)) {
    pl_data->spindle_speed = gc_state.spindle_speed; // Record data for planner use.
} // else { pl_data->spindle_speed = 0.0; } // Initialized as zero already.

// [5. Select tool ]: NOT SUPPORTED. Only tracks tool value.
gc_state.tool = gc_block.values.t;

// [6. Change tool ]: NOT SUPPORTED

// [7. Spindle control ]:
if (gc_state.modal.spindle != gc_block.modal.spindle) {
    // Update spindle control and apply spindle speed when enabling it in this block.
    // NOTE: All spindle state changes are synced, even in laser mode. Also, pl_data,
    // rather than gc_state, is used to manage laser state for non-laser motions.
    spindle_sync(gc_block.modal.spindle, pl_data->spindle_speed);
    gc_state.modal.spindle = gc_block.modal.spindle;
}
pl_data->condition |= gc_state.modal.spindle; // Set condition flag for planner use.

// [8. Coolant control ]:

```

```

if (gc_state.modal.coolant != gc_block.modal.coolant) {
    // NOTE: Coolant M-codes are modal. Only one command per line is allowed. But, multiple states
    // can exist at the same time, while coolant disable clears all states.
    coolant_sync(gc_block.modal.coolant);
    gc_state.modal.coolant = gc_block.modal.coolant;
}
pl_data->condition |= gc_state.modal.coolant; // Set condition flag for planner use.

// [9. Override control ]: NOT SUPPORTED. Always enabled. Except for a Grbl-only parking control.
#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
    if (gc_state.modal.override != gc_block.modal.override) {
        gc_state.modal.override = gc_block.modal.override;
        mc_override_ctrl_update(gc_state.modal.override);
    }
#endif

// [10. Dwell ]:
if (gc_block.non_modal_command == NON_MODAL_DWELL) { mc_dwell(gc_block.values.p); }

// [11. Set active plane ]:
gc_state.modal.plane_select = gc_block.modal.plane_select;

// [12. Set length units ]:
gc_state.modal.units = gc_block.modal.units;

// [13. Cutter radius compensation ]: G41/42 NOT SUPPORTED
// gc_state.modal.cutter_comp = gc_block.modal.cutter_comp; // NOTE: Not needed since always disabled.

// [14. Cutter length compensation ]: G43.1 and G49 supported. G43 NOT SUPPORTED.
// NOTE: If G43 were supported, its operation wouldn't be any different from G43.1 in terms
// of execution. The error-checking step would simply load the offset value into the correct
// axis of the block XYZ value array.
if (axis_command == AXIS_COMMAND_TOOL_LENGTH_OFFSET ) { // Indicates a change.
    gc_state.modal.tool_length = gc_block.modal.tool_length;
    if (gc_state.modal.tool_length == TOOL_LENGTH_OFFSET_CANCEL) { // G49
        gc_block.values.xyz[TOOL_LENGTH_OFFSET_AXIS] = 0.0;
    } // else G43.1
    if ( gc_state.tool_length_offset != gc_block.values.xyz[TOOL_LENGTH_OFFSET_AXIS] ) {
        gc_state.tool_length_offset = gc_block.values.xyz[TOOL_LENGTH_OFFSET_AXIS];
        system_flag_wco_change();
    }
}

// [15. Coordinate system selection ]:
if (gc_state.modal.coord_select != gc_block.modal.coord_select) {
    gc_state.modal.coord_select = gc_block.modal.coord_select;
    memcpy(gc_state.coord_system, block_coord_system, N_AXIS*sizeof(float));
    system_flag_wco_change();
}

// [16. Set path control mode ]: G61.1/G64 NOT SUPPORTED

```

```

// gc_state.modal.control = gc_block.modal.control; // NOTE: Always default.

// [17. Set distance mode ]:
gc_state.modal.distance = gc_block.modal.distance;

// [18. Set retract mode ]: NOT SUPPORTED

// [19. Go to predefined position, Set G10, or Set axis offsets ]:
switch(gc_block.non_modal_command) {
case NON_MODAL_SET_COORDINATE_DATA:
    settings_write_coord_data(coord_select,gc_block.values.ijk);
    // Update system coordinate system if currently active.
    if (gc_state.modal.coord_select == coord_select) {
        memcpy(gc_state.coord_system,gc_block.values.ijk,N_AXIS*sizeof(float));
        system_flag_wco_change();
    }
    break;
case NON_MODAL_GO_HOME_0: case NON_MODAL_GO_HOME_1:
    // Move to intermediate position before going home. Obeys current coordinate system and offsets
    // and absolute and incremental modes.
    pl_data->condition |= PL_COND_FLAG_RAPID_MOTION; // Set rapid motion condition flag.
    if (axis_command) { mc_line(gc_block.values.xyz, pl_data); }
    mc_line(gc_block.values.ijk, pl_data);
    memcpy(gc_state.position, gc_block.values.ijk, N_AXIS*sizeof(float));
    break;
case NON_MODAL_SET_HOME_0:
    settings_write_coord_data(SETTING_INDEX_G28,gc_state.position);
    break;
case NON_MODAL_SET_HOME_1:
    settings_write_coord_data(SETTING_INDEX_G30,gc_state.position);
    break;
case NON_MODAL_SET_COORDINATE_OFFSET:
    memcpy(gc_state.coord_offset,gc_block.values.xyz,sizeof(gc_block.values.xyz));
    system_flag_wco_change();
    break;
case NON_MODAL_RESET_COORDINATE_OFFSET:
    clear_vector(gc_state.coord_offset); // Disable G92 offsets by zeroing offset vector.
    system_flag_wco_change();
    break;
}

// [20. Motion modes ]:
// NOTE: Commands G10,G28,G30,G92 lock out and prevent axis words from use in motion modes.
// Enter motion modes only if there are axis words or a motion mode command word in the block.
gc_state.modal.motion = gc_block.modal.motion;
if (gc_state.modal.motion != MOTION_MODE_NONE) {
    if (axis_command == AXIS_COMMAND_MOTION_MODE) {
        uint8_t gc_update_pos = GC_UPDATE_POS_TARGET;
        if (gc_state.modal.motion == MOTION_MODE_LINEAR) {
            mc_line(gc_block.values.xyz, pl_data);

```

```

} else if (gc_state.modal.motion == MOTION_MODE_SEEK) {
    pl_data->condition |= PL_COND_FLAG_RAPID_MOTION; // Set rapid motion condition flag.
    mc_line(gc_block.values.xyz, pl_data);
} else if ((gc_state.modal.motion == MOTION_MODE_CW_ARC) || (gc_state.modal.motion == MOTION_MODE_CCW_ARC)) {
    mc_arc(gc_block.values.xyz, pl_data, gc_state.position, gc_block.values.ijk, gc_block.values.r,
        axis_0, axis_1, axis_linear, bit_istrue(gc_parser_flags, GC_PARSER_ARC_IS_CLOCKWISE));
} else {
    // NOTE: gc_block.values.xyz is returned from mc_probe_cycle with the updated position value. So
    // upon a successful probing cycle, the machine position and the returned value should be the same.
    #ifndef ALLOW_FEED_OVERRIDE_DURING_PROBE_CYCLES
        pl_data->condition |= PL_COND_FLAG_NO_FEED_OVERRIDE;
    #endif
    gc_update_pos = mc_probe_cycle(gc_block.values.xyz, pl_data, gc_parser_flags);
}

// As far as the parser is concerned, the position is now == target. In reality the
// motion control system might still be processing the action and the real tool position
// in any intermediate location.
if (gc_update_pos == GC_UPDATE_POS_TARGET) {
    memcpy(gc_state.position, gc_block.values.xyz, sizeof(gc_block.values.xyz)); // gc_state.position[] = gc_block.values.xyz[]
} else if (gc_update_pos == GC_UPDATE_POS_SYSTEM) {
    gc_sync_position(); // gc_state.position[] = sys_position
} // == GC_UPDATE_POS_NONE
}
}

// [21. Program flow]:
// M0,M1,M2,M30: Perform non-running program flow actions. During a program pause, the buffer may
// refill and can only be resumed by the cycle start run-time command.
gc_state.modal.program_flow = gc_block.modal.program_flow;
if (gc_state.modal.program_flow) {
    protocol_buffer_synchronize(); // Sync and finish all remaining buffered motions before moving on.
    if (gc_state.modal.program_flow == PROGRAM_FLOW_PAUSED) {
        if (sys.state != STATE_CHECK_MODE) {
            system_set_exec_state_flag(EXEC_FEED_HOLD); // Use feed hold for program pause.
            protocol_execute_realtime(); // Execute suspend.
        }
    }
} else { // == PROGRAM_FLOW_COMPLETED
    // Upon program complete, only a subset of g-codes reset to certain defaults, according to
    // LinuxCNC's program end descriptions and testing. Only modal groups [G-code 1,2,3,5,7,12]
    // and [M-code 7,8,9] reset to [G1,G17,G90,G94,G40,G54,M5,M9,M48]. The remaining modal groups
    // [G-code 4,6,8,10,13,14,15] and [M-code 4,5,6] and the modal words [F,S,T,H] do not reset.
    gc_state.modal.motion = MOTION_MODE_LINEAR;
    gc_state.modal.plane_select = PLANE_SELECT_XY;
    gc_state.modal.distance = DISTANCE_MODE_ABSOLUTE;
    gc_state.modal.feed_rate = FEED_RATE_MODE_UNITS_PER_MIN;
    // gc_state.modal.cutter_comp = CUTTER_COMP_DISABLE; // Not supported.
    gc_state.modal.coord_select = 0; // G54
    gc_state.modal.spindle = SPINDLE_DISABLE;
    gc_state.modal.coolant = COOLANT_DISABLE;
    #ifdef ENABLE_PARKING_OVERRIDE_CONTROL

```

```

#ifdef DEACTIVATE_PARKING_UPON_INIT
    gc_state.modal.override = OVERRIDE_DISABLED;
#else
    gc_state.modal.override = OVERRIDE_PARKING_MOTION;
#endif
#endif

#ifdef RESTORE_OVERRIDES_AFTER_PROGRAM_END
    sys.f_override = DEFAULT_FEED_OVERRIDE;
    sys.r_override = DEFAULT_RAPID_OVERRIDE;
    sys.spindle_speed_ovr = DEFAULT_SPINDLE_SPEED_OVERRIDE;
#endif

// Execute coordinate change and spindle/coolant stop.
if (sys.state != STATE_CHECK_MODE) {
    if (!(settings_read_coord_data(gc_state.modal.coord_select,gc_state.coord_system))) { FAIL(STATUS_SETTING_READ_BACK_ERROR); }
    system_flag_wco_change(); // Set to refresh immediately just in case something altered.
    spindle_set_state(SPINDLE_DISABLE,0.0);
    coolant_set_state(COOLANT_DISABLE);
}
report_feedback_message(MESSAGE_PROGRAM_END);
}
gc_state.modal.program_flow = PROGRAM_FLOW_RUNNING; // Reset program flow.
}

// TODO: % to denote start of program.

return(STATUS_OK);
}

```

/*
Not supported:

- Canned cycles
- Tool radius compensation
- A,B,C-axes
- Evaluation of expressions
- Variables
- Override control (TBD)
- Tool changes
- Switches

(*) Indicates optional parameter, enabled through config.h and re-compile
group 0 = {G92.2, G92.3} (Non modal: Cancel and re-enable G92 offsets)
group 1 = {G81 - G89} (Motion modes: Canned cycles)
group 4 = {M1} (Optional stop, ignored)
group 6 = {M6} (Tool change)
group 7 = {G41, G42} cutter radius compensation (G40 is supported)
group 8 = {G43} tool length offset (G43.1/G49 are supported)
group 8 = {M7*} enable mist coolant (* Compile-option)

group 9 = {M48, M49, M56*} enable/disable override switches (* Compile-option)
group 10 = {G98, G99} return mode canned cycles
group 13 = {G61.1, G64} path control mode (G61 is supported)
*/

File: gcode.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*
gcode.h - rs274/ngc parser.
Part of Grbl

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

#ifndef gcode_h
#define gcode_h

// Define modal group internal numbers for checking multiple command violations and tracking the
// type of command that is called in the block. A modal group is a group of g-code commands that are
// mutually exclusive, or cannot exist on the same line, because they each toggle a state or execute
// a unique motion. These are defined in the NIST RS274-NGC v3 g-code standard, available online,
// and are similar/identical to other g-code interpreters by manufacturers (Haas,Fanuc,Mazak,etc).
// NOTE: Modal group define values must be sequential and starting from zero.
#define MODAL_GROUP_G0 0 // [G4,G10,G28,G28.1,G30,G30.1,G53,G92,G92.1] Non-modal
#define MODAL_GROUP_G1 1 // [G0,G1,G2,G3,G38.2,G38.3,G38.4,G38.5,G80] Motion
#define MODAL_GROUP_G2 2 // [G17,G18,G19] Plane selection
#define MODAL_GROUP_G3 3 // [G90,G91] Distance mode
#define MODAL_GROUP_G4 4 // [G91.1] Arc IJK distance mode
#define MODAL_GROUP_G5 5 // [G93,G94] Feed rate mode
#define MODAL_GROUP_G6 6 // [G20,G21] Units
#define MODAL_GROUP_G7 7 // [G40] Cutter radius compensation mode. G41/42 NOT SUPPORTED.
#define MODAL_GROUP_G8 8 // [G43.1,G49] Tool length offset
#define MODAL_GROUP_G12 9 // [G54,G55,G56,G57,G58,G59] Coordinate system selection

```

#define MODAL_GROUP_G13 10 // [G61] Control mode

#define MODAL_GROUP_M4 11 // [M0,M1,M2,M30] Stopping
#define MODAL_GROUP_M7 12 // [M3,M4,M5] Spindle turning
#define MODAL_GROUP_M8 13 // [M7,M8,M9] Coolant control
#define MODAL_GROUP_M9 14 // [M56] Override control

// Define command actions for within execution-type modal groups (motion, stopping, non-modal). Used
// internally by the parser to know which command to execute.
// NOTE: Some macro values are assigned specific values to make g-code state reporting and parsing
// compile a little smaller. Necessary due to being completely out of flash on the 328p. Although not
// ideal, just be careful with values that state 'do not alter' and check both report.c and gcode.c
// to see how they are used, if you need to alter them.

// Modal Group G0: Non-modal actions
#define NON_MODAL_NO_ACTION 0 // (Default: Must be zero)
#define NON_MODAL_DWELL 4 // G4 (Do not alter value)
#define NON_MODAL_SET_COORDINATE_DATA 10 // G10 (Do not alter value)
#define NON_MODAL_GO_HOME_0 28 // G28 (Do not alter value)
#define NON_MODAL_SET_HOME_0 38 // G28.1 (Do not alter value)
#define NON_MODAL_GO_HOME_1 30 // G30 (Do not alter value)
#define NON_MODAL_SET_HOME_1 40 // G30.1 (Do not alter value)
#define NON_MODAL_ABSOLUTE_OVERRIDE 53 // G53 (Do not alter value)
#define NON_MODAL_SET_COORDINATE_OFFSET 92 // G92 (Do not alter value)
#define NON_MODAL_RESET_COORDINATE_OFFSET 102 //G92.1 (Do not alter value)

// Modal Group G1: Motion modes
#define MOTION_MODE_SEEK 0 // G0 (Default: Must be zero)
#define MOTION_MODE_LINEAR 1 // G1 (Do not alter value)
#define MOTION_MODE_CW_ARC 2 // G2 (Do not alter value)
#define MOTION_MODE_CCW_ARC 3 // G3 (Do not alter value)
#define MOTION_MODE_PROBE_TOWARD 140 // G38.2 (Do not alter value)
#define MOTION_MODE_PROBE_TOWARD_NO_ERROR 141 // G38.3 (Do not alter value)
#define MOTION_MODE_PROBE_AWAY 142 // G38.4 (Do not alter value)
#define MOTION_MODE_PROBE_AWAY_NO_ERROR 143 // G38.5 (Do not alter value)
#define MOTION_MODE_NONE 80 // G80 (Do not alter value)

// Modal Group G2: Plane select
#define PLANE_SELECT_XY 0 // G17 (Default: Must be zero)
#define PLANE_SELECT_ZX 1 // G18 (Do not alter value)
#define PLANE_SELECT_YZ 2 // G19 (Do not alter value)

// Modal Group G3: Distance mode
#define DISTANCE_MODE_ABSOLUTE 0 // G90 (Default: Must be zero)
#define DISTANCE_MODE_INCREMENTAL 1 // G91 (Do not alter value)

// Modal Group G4: Arc IJK distance mode
#define DISTANCE_ARC_MODE_INCREMENTAL 0 // G91.1 (Default: Must be zero)

// Modal Group M4: Program flow
#define PROGRAM_FLOW_RUNNING 0 // (Default: Must be zero)

```

```

#define PROGRAM_FLOW_PAUSED 3 // M0
#define PROGRAM_FLOW_OPTIONAL_STOP 1 // M1 NOTE: Not supported, but valid and ignored.
#define PROGRAM_FLOW_COMPLETED_M2 2 // M2 (Do not alter value)
#define PROGRAM_FLOW_COMPLETED_M30 30 // M30 (Do not alter value)

// Modal Group G5: Feed rate mode
#define FEED_RATE_MODE_UNITS_PER_MIN 0 // G94 (Default: Must be zero)
#define FEED_RATE_MODE_INVERSE_TIME 1 // G93 (Do not alter value)

// Modal Group G6: Units mode
#define UNITS_MODE_MM 0 // G21 (Default: Must be zero)
#define UNITS_MODE_INCHES 1 // G20 (Do not alter value)

// Modal Group G7: Cutter radius compensation mode
#define CUTTER_COMP_DISABLE 0 // G40 (Default: Must be zero)

// Modal Group G13: Control mode
#define CONTROL_MODE_EXACT_PATH 0 // G61 (Default: Must be zero)

// Modal Group M7: Spindle control
#define SPINDLE_DISABLE 0 // M5 (Default: Must be zero)
#define SPINDLE_ENABLE_CW PL_COND_FLAG_SPINDLE_CW // M3 (NOTE: Uses planner condition bit flag)
#define SPINDLE_ENABLE_CCW PL_COND_FLAG_SPINDLE_CCW // M4 (NOTE: Uses planner condition bit flag)

// Modal Group M8: Coolant control
#define COOLANT_DISABLE 0 // M9 (Default: Must be zero)
#define COOLANT_FLOOD_ENABLE PL_COND_FLAG_COOLANT_FLOOD // M8 (NOTE: Uses planner condition bit flag)
#define COOLANT_MIST_ENABLE PL_COND_FLAG_COOLANT_MIST // M7 (NOTE: Uses planner condition bit flag)

// Modal Group G8: Tool length offset
#define TOOL_LENGTH_OFFSET_CANCEL 0 // G49 (Default: Must be zero)
#define TOOL_LENGTH_OFFSET_ENABLE_DYNAMIC 1 // G43.1

// Modal Group M9: Override control
#ifndef DEACTIVATE_PARKING_UPON_INIT
    #define OVERRIDE_DISABLED 0 // (Default: Must be zero)
    #define OVERRIDE_PARKING_MOTION 1 // M56
#else
    #define OVERRIDE_PARKING_MOTION 0 // M56 (Default: Must be zero)
    #define OVERRIDE_DISABLED 1 // Parking disabled.
#endif

// Modal Group G12: Active work coordinate system
// N/A: Stores coordinate system value (54-59) to change to.

// Define parameter word mapping.
#define WORD_F 0
#define WORD_I 1
#define WORD_J 2
#define WORD_K 3
#define WORD_L 4

```

```

#define WORD_N 5
#define WORD_P 6
#define WORD_R 7
#define WORD_S 8
#define WORD_T 9
#define WORD_X 10
#define WORD_Y 11
#define WORD_Z 12

// Define g-code parser position updating flags
#define GC_UPDATE_POS_TARGET 0 // Must be zero
#define GC_UPDATE_POS_SYSTEM 1
#define GC_UPDATE_POS_NONE 2

// Define probe cycle exit states and assign proper position updating.
#define GC_PROBE_FOUND GC_UPDATE_POS_SYSTEM
#define GC_PROBE_ABORT GC_UPDATE_POS_NONE
#define GC_PROBE_FAIL_INIT GC_UPDATE_POS_NONE
#define GC_PROBE_FAIL_END GC_UPDATE_POS_TARGET
#ifdef SET_CHECK_MODE_PROBE_TO_START
#define GC_PROBE_CHECK_MODE GC_UPDATE_POS_NONE
#else
#define GC_PROBE_CHECK_MODE GC_UPDATE_POS_TARGET
#endif

// Define gcode parser flags for handling special cases.
#define GC_PARSER_NONE 0 // Must be zero.
#define GC_PARSER_JOG_MOTION bit(0)
#define GC_PARSER_CHECK_MANTISSA bit(1)
#define GC_PARSER_ARC_IS_CLOCKWISE bit(2)
#define GC_PARSER_PROBE_IS_AWAY bit(3)
#define GC_PARSER_PROBE_IS_NO_ERROR bit(4)
#define GC_PARSER_LASER_FORCE_SYNC bit(5)
#define GC_PARSER_LASER_DISABLE bit(6)
#define GC_PARSER_LASER_ISMOTION bit(7)

// NOTE: When this struct is zeroed, the above defines set the defaults for the system.
typedef struct {
    uint8_t motion; // {G0,G1,G2,G3,G38.2,G80}
    uint8_t feed_rate; // {G93,G94}
    uint8_t units; // {G20,G21}
    uint8_t distance; // {G90,G91}
    // uint8_t distance_arc; // {G91.1} NOTE: Don't track. Only default supported.
    uint8_t plane_select; // {G17,G18,G19}
    // uint8_t cutter_comp; // {G40} NOTE: Don't track. Only default supported.
    uint8_t tool_length; // {G43.1,G49}
    uint8_t coord_select; // {G54,G55,G56,G57,G58,G59}
    // uint8_t control; // {G61} NOTE: Don't track. Only default supported.
    uint8_t program_flow; // {M0,M1,M2,M30}
    uint8_t coolant; // {M7,M8,M9}

```

```

uint8_t spindle;    // {M3,M4,M5}
uint8_t override;   // {M56}
} gc_modal_t;

```

```

typedef struct {
    float f;        // Feed
    float ijk[3];   // I,J,K Axis arc offsets
    uint8_t l;      // G10 or canned cycles parameters
    int32_t n;      // Line number
    float p;        // G10 or dwell parameters
    // float q;      // G82 peck drilling
    float r;        // Arc radius
    float s;        // Spindle speed
    uint8_t t;      // Tool selection
    float xyz[3];   // X,Y,Z Translational axes
} gc_values_t;

```

```

typedef struct {
    gc_modal_t modal;

    float spindle_speed;    // RPM
    float feed_rate;        // Millimeters/min
    uint8_t tool;           // Tracks tool number. NOT USED.
    int32_t line_number;    // Last line number sent

    float position[N_AXIS]; // Where the interpreter considers the tool to be at this point in the code

    float coord_system[N_AXIS]; // Current work coordinate system (G54+). Stores offset from absolute machine
                                // position in mm. Loaded from EEPROM when called.
    float coord_offset[N_AXIS]; // Retains the G92 coordinate offset (work coordinates) relative to
                                // machine zero in mm. Non-persistent. Cleared upon reset and boot.
    float tool_length_offset;   // Tracks tool length offset value when enabled.
} parser_state_t;
extern parser_state_t gc_state;

```

```

typedef struct {
    uint8_t non_modal_command;
    gc_modal_t modal;
    gc_values_t values;
} parser_block_t;

```

```

// Initialize the parser
void gc_init();

```

```

// Execute one block of rs275/ngc/g-code
uint8_t gc_execute_line(char *line);

```

```

// Set g-code parser position. Input in steps.

```

```
void gc_sync_position();
```

```
#endif
```

File: grbl.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```
/*
```

```
grbl.h - main Grbl include file
```

```
Part of Grbl
```

Copyright (c) 2015-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```
*/
```

```
#ifndef grbl_h
```

```
#define grbl_h
```

```
// Grbl versioning system
```

```
#define GRBL_VERSION "1.1h"
```

```
#define GRBL_VERSION_BUILD "20190830"
```

```
// Define standard libraries used by Grbl.
```

```
#include <avr/io.h>
```

```
#include <avr/pgmspace.h>
```

```
#include <avr/interrupt.h>
```

```
#include <avr/wdt.h>
```

```
#include <util/delay.h>
```

```
#include <math.h>
```

```
#include <inttypes.h>
```

```
#include <string.h>
```

```
#include <stdlib.h>
```

```
#include <stdint.h>
```

```
#include <stdbool.h>
```

```
// Define the Grbl system include files. NOTE: Do not alter organization.
```

```
#include "config.h"
```

```
#include "nuts_bolts.h"
#include "settings.h"
#include "system.h"
#include "defaults.h"
#include "cpu_map.h"
#include "planner.h"
#include "coolant_control.h"
#include "eeprom.h"
#include "gcode.h"
#include "limits.h"
#include "motion_control.h"
#include "planner.h"
#include "print.h"
#include "probe.h"
#include "protocol.h"
#include "report.h"
#include "serial.h"
#include "spindle_control.h"
#include "stepper.h"
#include "jog.h"
```

```
// -----
// COMPILE-TIME ERROR CHECKING OF DEFINE VALUES:
```

```
#ifndef HOMING_CYCLE_0
  #error "Required HOMING_CYCLE_0 not defined."
#endif
```

```
#if defined(USE_SPINDLE_DIR_AS_ENABLE_PIN) && !defined(VARIABLE_SPINDLE)
  #error "USE_SPINDLE_DIR_AS_ENABLE_PIN may only be used with VARIABLE_SPINDLE enabled"
#endif
```

```
#if defined(USE_SPINDLE_DIR_AS_ENABLE_PIN) && !defined(CPU_MAP_ATMEGA328P)
  #error "USE_SPINDLE_DIR_AS_ENABLE_PIN may only be used with a 328p processor"
#endif
```

```
#if !defined(USE_SPINDLE_DIR_AS_ENABLE_PIN) && defined(SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED)
  #error "SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED may only be used with USE_SPINDLE_DIR_AS_ENABLE_PIN e
#endif
```

```
#if defined(PARKING_ENABLE)
  #if defined(HOMING_FORCE_SET_ORIGIN)
    #error "HOMING_FORCE_SET_ORIGIN is not supported with PARKING_ENABLE at this time."
  #endif
#endif
```

```
#if defined(ENABLE_PARKING_OVERRIDE_CONTROL)
  #if !defined(PARKING_ENABLE)
    #error "ENABLE_PARKING_OVERRIDE_CONTROL must be enabled with PARKING_ENABLE."
  #endif
#endif
```

```

#if defined(SPINDLE_PWM_MIN_VALUE)
  #if !(SPINDLE_PWM_MIN_VALUE > 0)
    #error "SPINDLE_PWM_MIN_VALUE must be greater than zero."
  #endif
#endif

#if (REPORT_WCO_REFRESH_BUSY_COUNT < REPORT_WCO_REFRESH_IDLE_COUNT)
  #error "WCO busy refresh is less than idle refresh."
#endif
#if (REPORT_OVR_REFRESH_BUSY_COUNT < REPORT_OVR_REFRESH_IDLE_COUNT)
  #error "Override busy refresh is less than idle refresh."
#endif
#if (REPORT_WCO_REFRESH_IDLE_COUNT < 2)
  #error "WCO refresh must be greater than one."
#endif
#if (REPORT_OVR_REFRESH_IDLE_COUNT < 1)
  #error "Override refresh must be greater than zero."
#endif

#if defined(ENABLE_DUAL_AXIS)
  #if !((DUAL_AXIS_SELECT == X_AXIS) || (DUAL_AXIS_SELECT == Y_AXIS))
    #error "Dual axis currently supports X or Y axes only."
  #endif
  #if defined(DUAL_AXIS_CONFIG_CNC_SHIELD_CLONE) && defined(VARIABLE_SPINDLE)
    #error "VARIABLE_SPINDLE not supported with DUAL_AXIS_CNC_SHIELD_CLONE."
  #endif
  #if defined(DUAL_AXIS_CONFIG_CNC_SHIELD_CLONE) && defined(DUAL_AXIS_CONFIG_PROTONEER_V3_51)
    #error "More than one dual axis configuration found. Select one."
  #endif
  #if !defined(DUAL_AXIS_CONFIG_CNC_SHIELD_CLONE) && !defined(DUAL_AXIS_CONFIG_PROTONEER_V3_51)
    #error "No supported dual axis configuration found. Select one."
  #endif
  #if defined(COREXY)
    #error "CORE XY not supported with dual axis feature."
  #endif
  #if defined(USE_SPINDLE_DIR_AS_ENABLE_PIN)
    #error "USE_SPINDLE_DIR_AS_ENABLE_PIN not supported with dual axis feature."
  #endif
  #if defined(ENABLE_M7)
    #error "ENABLE_M7 not supported with dual axis feature."
  #endif
#endif

// -----

#endif

```

File: jog.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

jog.h - Jogging methods

Part of Grbl

Copyright (c) 2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

#include "grbl.h"

// Sets up valid jog motion received from g-code parser, checks for soft-limits, and executes the jog.

uint8_t jog_execute(plan_line_data_t *pl_data, parser_block_t *gc_block)

{

 // Initialize planner data struct for jogging motions.

 // NOTE: Spindle and coolant are allowed to fully function with overrides during a jog.

 pl_data->feed_rate = gc_block->values.f;

 pl_data->condition |= PL_COND_FLAG_NO_FEED_OVERRIDE;

 #ifdef USE_LINE_NUMBERS

 pl_data->line_number = gc_block->values.n;

 #endif

 if (bit_istrue(settings.flags,BITFLAG_SOFT_LIMIT_ENABLE)) {

 if (system_check_travel_limits(gc_block->values.xyz)) { return(STATUS_TRAVEL_EXCEEDED); }

 }

 // Valid jog command. Plan, set state, and execute.

 mc_line(gc_block->values.xyz,pl_data);

 if (sys.state == STATE_IDLE) {

 if (plan_get_current_block() != NULL) { // Check if there is a block to execute.

 sys.state = STATE_JOG;

 st_prep_buffer();

 st_wake_up(); // NOTE: Manual start. No state machine required.

 }

 }

```
    return(STATUS_OK);
}
```

File: jog.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```
/*
jog.h - Jogging methods
Part of Grbl
```

Copyright (c) 2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```
*/
```

```
#ifndef jog_h
#define jog_h
```

```
#include "gcode.h"
```

```
// System motion line numbers must be zero.
#define JOG_LINE_NUMBER 0
```

```
// Sets up valid jog motion received from g-code parser, checks for soft-limits, and executes the jog.
uint8_t jog_execute(plan_line_data_t *pl_data, parser_block_t *gc_block);
```

```
#endif
```

File: limits.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```
/*
limits.c - code pertaining to limit-switches and performing the homing cycle
Part of Grbl
```

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#include "grbl.h"
```

```
// Homing axis search distance multiplier. Computed by this value times the cycle travel.
```

```
#ifndef HOMING_AXIS_SEARCH_SCALAR
```

```
#define HOMING_AXIS_SEARCH_SCALAR 1.5 // Must be > 1 to ensure limit switch will be engaged.
```

```
#endif
```

```
#ifndef HOMING_AXIS_LOCATE_SCALAR
```

```
#define HOMING_AXIS_LOCATE_SCALAR 5.0 // Must be > 1 to ensure limit switch is cleared.
```

```
#endif
```

```
#ifdef ENABLE_DUAL_AXIS
```

```
// Flags for dual axis async limit trigger check.
```

```
#define DUAL_AXIS_CHECK_DISABLE 0 // Must be zero
```

```
#define DUAL_AXIS_CHECK_ENABLE bit(0)
```

```
#define DUAL_AXIS_CHECK_TRIGGER_1 bit(1)
```

```
#define DUAL_AXIS_CHECK_TRIGGER_2 bit(2)
```

```
#endif
```

```
void limits_init()
```

```
{
```

```
LIMIT_DDR &= ~(LIMIT_MASK); // Set as input pins
```

```
#ifdef DISABLE_LIMIT_PIN_PULL_UP
```

```
LIMIT_PORT &= ~(LIMIT_MASK); // Normal low operation. Requires external pull-down.
```

```
#else
```

```
LIMIT_PORT |= (LIMIT_MASK); // Enable internal pull-up resistors. Normal high operation.
```

```
#endif
```

```
if (bit_istrue(settings.flags,BITFLAG_HARD_LIMIT_ENABLE)) {
```

```
LIMIT_PCMSK |= LIMIT_MASK; // Enable specific pins of the Pin Change Interrupt
```

```
PCICR |= (1 << LIMIT_INT); // Enable Pin Change Interrupt
```

```
} else {
```

```
limits_disable();
```

```
}
```

```

#ifdef ENABLE_SOFTWARE_DEBOUNCE
    MCUSR &= ~(1<<WDRF);
    WDTCSR |= (1<<WDCE) | (1<<WDE);
    WDTCSR = (1<<WDP0); // Set time-out at ~32msec.
#endif
}

```

// Disables hard limits.

```

void limits_disable()
{
    LIMIT_PCMSK &= ~LIMIT_MASK; // Disable specific pins of the Pin Change Interrupt
    PCICR &= ~(1 << LIMIT_INT); // Disable Pin Change Interrupt
}

```

// Returns limit state as a bit-wise uint8 variable. Each bit indicates an axis limit, where
 // triggered is 1 and not triggered is 0. Invert mask is applied. Axes are defined by their
 // number in bit position, i.e. Z_AXIS is (1<<2) or bit 2, and Y_AXIS is (1<<1) or bit 1.

```

uint8_t limits_get_state()
{
    uint8_t limit_state = 0;
    uint8_t pin = (LIMIT_PIN & LIMIT_MASK);
#ifdef INVERT_LIMIT_PIN_MASK
    pin ^= INVERT_LIMIT_PIN_MASK;
#endif
    if (bit_isfalse(settings.flags,BITFLAG_INVERT_LIMIT_PINS)) { pin ^= LIMIT_MASK; }
    if (pin) {
        uint8_t idx;
        for (idx=0; idx<N_AXIS; idx++) {
            if (pin & get_limit_pin_mask(idx)) { limit_state |= (1 << idx); }
        }
#ifdef ENABLE_DUAL_AXIS
        if (pin & (1<<DUAL_LIMIT_BIT)) { limit_state |= (1 << N_AXIS); }
#endif
    }
    return(limit_state);
}

```

// This is the Limit Pin Change Interrupt, which handles the hard limit feature. A bouncing
 // limit switch can cause a lot of problems, like false readings and multiple interrupt calls.
 // If a switch is triggered at all, something bad has happened and treat it as such, regardless
 // if a limit switch is being disengaged. It's impossible to reliably tell the state of a
 // bouncing pin because the Arduino microcontroller does not retain any state information when
 // detecting a pin change. If we poll the pins in the ISR, you can miss the correct reading if the
 // switch is bouncing.

// NOTE: Do not attach an e-stop to the limit pins, because this interrupt is disabled during
 // homing cycles and will not respond correctly. Upon user request or need, there may be a
 // special pinout for an e-stop, but it is generally recommended to just directly connect

```

// your e-stop switch to the Arduino reset pin, since it is the most correct way to do this.
#ifndef ENABLE_SOFTWARE_DEBOUNCE
ISR(LIMIT_INT_vect) // DEFAULT: Limit pin change interrupt process.
{
    // Ignore limit switches if already in an alarm state or in-process of executing an alarm.
    // When in the alarm state, Grbl should have been reset or will force a reset, so any pending
    // moves in the planner and serial buffers are all cleared and newly sent blocks will be
    // locked out until a homing cycle or a kill lock command. Allows the user to disable the hard
    // limit setting if their limits are constantly triggering after a reset and move their axes.
    if (sys.state != STATE_ALARM) {
        if (!(sys_rt_exec_alarm)) {
            #ifdef HARD_LIMIT_FORCE_STATE_CHECK
                // Check limit pin state.
                if (limits_get_state()) {
                    mc_reset(); // Initiate system kill.
                    system_set_exec_alarm(EXEC_ALARM_HARD_LIMIT); // Indicate hard limit critical event
                }
            #else
                mc_reset(); // Initiate system kill.
                system_set_exec_alarm(EXEC_ALARM_HARD_LIMIT); // Indicate hard limit critical event
            #endif
        }
    }
}

#else // OPTIONAL: Software debounce limit pin routine.
    // Upon limit pin change, enable watchdog timer to create a short delay.
    ISR(LIMIT_INT_vect) { if (!(WDTCR & (1<<WDIE))) { WDTCR |= (1<<WDIE); } }
    ISR(WDT_vect) // Watchdog timer ISR
    {
        WDTCR &= ~(1<<WDIE); // Disable watchdog timer.
        if (sys.state != STATE_ALARM) { // Ignore if already in alarm state.
            if (!(sys_rt_exec_alarm)) {
                // Check limit pin state.
                if (limits_get_state()) {
                    mc_reset(); // Initiate system kill.
                    system_set_exec_alarm(EXEC_ALARM_HARD_LIMIT); // Indicate hard limit critical event
                }
            }
        }
    }
}

#endif

// Homes the specified cycle axes, sets the machine position, and performs a pull-off motion after
// completing. Homing is a special motion case, which involves rapid uncontrolled stops to locate
// the trigger point of the limit switches. The rapid stops are handled by a system level axis lock
// mask, which prevents the stepper algorithm from executing step pulses. Homing motions typically
// circumvent the processes for executing motions in normal operation.
// NOTE: Only the abort realtime command can interrupt this process.
// TODO: Move limit pin-specific calls to a general function for portability.
void limits_go_home(uint8_t cycle_mask)
{

```

```
if (sys.abort) { return; } // Block if system reset has been issued.
```

```
// Initialize plan data struct for homing motion. Spindle and coolant are disabled.
```

```
plan_line_data_t plan_data;
```

```
plan_line_data_t *pl_data = &plan_data;
```

```
memset(pl_data,0,sizeof(plan_line_data_t));
```

```
pl_data->condition = (PL_COND_FLAG_SYSTEM_MOTION|PL_COND_FLAG_NO_FEED_OVERRIDE);
```

```
#ifdef USE_LINE_NUMBERS
```

```
    pl_data->line_number = HOMING_CYCLE_LINE_NUMBER;
```

```
#endif
```

```
// Initialize variables used for homing computations.
```

```
uint8_t n_cycle = (2*N_HOMING_LOCATE_CYCLE+1);
```

```
uint8_t step_pin[N_AXIS];
```

```
#ifdef ENABLE_DUAL_AXIS
```

```
    uint8_t step_pin_dual;
```

```
    uint8_t dual_axis_async_check;
```

```
    int32_t dual_trigger_position;
```

```
    #if (DUAL_AXIS_SELECT == X_AXIS)
```

```
        float fail_distance = (-DUAL_AXIS_HOMING_FAIL_AXIS_LENGTH_PERCENT/100.0)*settings.max_travel[Y_AXIS];
```

```
    #else
```

```
        float fail_distance = (-DUAL_AXIS_HOMING_FAIL_AXIS_LENGTH_PERCENT/100.0)*settings.max_travel[X_AXIS];
```

```
    #endif
```

```
    fail_distance = min(fail_distance, DUAL_AXIS_HOMING_FAIL_DISTANCE_MAX);
```

```
    fail_distance = max(fail_distance, DUAL_AXIS_HOMING_FAIL_DISTANCE_MIN);
```

```
    int32_t dual_fail_distance = trunc(fail_distance*settings.steps_per_mm[DUAL_AXIS_SELECT]);
```

```
    // int32_t dual_fail_distance = trunc((DUAL_AXIS_HOMING_TRIGGER_FAIL_DISTANCE)*settings.steps_per_mm[DUAL_AXIS_SELECT]);
```

```
#endif
```

```
float target[N_AXIS];
```

```
float max_travel = 0.0;
```

```
uint8_t idx;
```

```
for (idx=0; idx<N_AXIS; idx++) {
```

```
    // Initialize step pin masks
```

```
    step_pin[idx] = get_step_pin_mask(idx);
```

```
    #ifdef COREXY
```

```
        if ((idx==A_MOTOR)||((idx==B_MOTOR)) { step_pin[idx] = (get_step_pin_mask(X_AXIS)|get_step_pin_mask(Y_AXIS));
```

```
    #endif
```

```
    if (bit_istrue(cycle_mask,bit(idx))) {
```

```
        // Set target based on max_travel setting. Ensure homing switches engaged with search scalar.
```

```
        // NOTE: settings.max_travel[] is stored as a negative value.
```

```
        max_travel = max(max_travel,(-HOMING_AXIS_SEARCH_SCALAR)*settings.max_travel[idx]);
```

```
    }
```

```
}
```

```
#ifdef ENABLE_DUAL_AXIS
```

```
    step_pin_dual = (1<<DUAL_STEP_BIT);
```

```
#endif
```

```
// Set search mode with approach at seek rate to quickly engage the specified cycle_mask limit switches.
```

```
bool approach = true;
```

```
float homing_rate = settings.homing_seek_rate;
```

```

uint8_t limit_state, axislock, n_active_axis;
do {

    system_convert_array_steps_to_mpos(target,sys_position);

    // Initialize and declare variables needed for homing routine.
    axislock = 0;
    #ifdef ENABLE_DUAL_AXIS
        sys.homing_axis_lock_dual = 0;
        dual_trigger_position = 0;
        dual_axis_async_check = DUAL_AXIS_CHECK_DISABLE;
    #endif
    n_active_axis = 0;
    for (idx=0; idx<N_AXIS; idx++) {
        // Set target location for active axes and setup computation for homing rate.
        if (bit_istrue(cycle_mask,bit(idx))) {
            n_active_axis++;
            #ifdef COREXY
                if (idx == X_AXIS) {
                    int32_t axis_position = system_convert_corexy_to_y_axis_steps(sys_position);
                    sys_position[A_MOTOR] = axis_position;
                    sys_position[B_MOTOR] = -axis_position;
                } else if (idx == Y_AXIS) {
                    int32_t axis_position = system_convert_corexy_to_x_axis_steps(sys_position);
                    sys_position[A_MOTOR] = sys_position[B_MOTOR] = axis_position;
                } else {
                    sys_position[Z_AXIS] = 0;
                }
            #else
                sys_position[idx] = 0;
            #endif
            // Set target direction based on cycle mask and homing cycle approach state.
            // NOTE: This happens to compile smaller than any other implementation tried.
            if (bit_istrue(settings.homing_dir_mask,bit(idx))) {
                if (approach) { target[idx] = -max_travel; }
                else { target[idx] = max_travel; }
            } else {
                if (approach) { target[idx] = max_travel; }
                else { target[idx] = -max_travel; }
            }
            // Apply axislock to the step port pins active in this cycle.
            axislock |= step_pin[idx];
            #ifdef ENABLE_DUAL_AXIS
                if (idx == DUAL_AXIS_SELECT) { sys.homing_axis_lock_dual = step_pin_dual; }
            #endif
        }
    }

    homing_rate *= sqrt(n_active_axis); // [sqrt(N_AXIS)] Adjust so individual axes all move at homing rate.
    sys.homing_axis_lock = axislock;
}

```

```
// Perform homing cycle. Planner buffer should be empty, as required to initiate the homing cycle.
```

```
pl_data->feed_rate = homing_rate; // Set current homing rate.
```

```
plan_buffer_line(target, pl_data); // Bypass mc_line(). Directly plan homing motion.
```

```
sys.step_control = STEP_CONTROL_EXECUTE_SYS_MOTION; // Set to execute homing motion and clear existing flag
```

```
st_prep_buffer(); // Prep and fill segment buffer from newly planned block.
```

```
st_wake_up(); // Initiate motion
```

```
do {
```

```
  if (approach) {
```

```
    // Check limit state. Lock out cycle axes when they change.
```

```
    limit_state = limits_get_state();
```

```
    for (idx=0; idx<N_AXIS; idx++) {
```

```
      if (axislock & step_pin[idx]) {
```

```
        if (limit_state & (1 << idx)) {
```

```
          #ifdef COREXY
```

```
            if (idx==Z_AXIS) { axislock &= ~(step_pin[Z_AXIS]); } 
```

```
            else { axislock &= ~(step_pin[A_MOTOR][step_pin[B_MOTOR]); } 
```

```
          #else
```

```
            axislock &= ~(step_pin[idx]);
```

```
          #ifdef ENABLE_DUAL_AXIS
```

```
            if (idx == DUAL_AXIS_SELECT) { dual_axis_async_check |= DUAL_AXIS_CHECK_TRIGGER_1; } 
```

```
          #endif
```

```
        #endif
```

```
      }
```

```
    }
```

```
  }
```

```
  sys.homing_axis_lock = axislock;
```

```
  #ifdef ENABLE_DUAL_AXIS
```

```
    if (sys.homing_axis_lock_dual) { // NOTE: Only true when homing dual axis.
```

```
      if (limit_state & (1 << N_AXIS)) {
```

```
        sys.homing_axis_lock_dual = 0;
```

```
        dual_axis_async_check |= DUAL_AXIS_CHECK_TRIGGER_2;
```

```
      }
```

```
    }
```

```
// When first dual axis limit triggers, record position and begin checking distance until other limit triggers. Bail upon fail
```

```
if (dual_axis_async_check) {
```

```
  if (dual_axis_async_check & DUAL_AXIS_CHECK_ENABLE) {
```

```
    if ((dual_axis_async_check & (DUAL_AXIS_CHECK_TRIGGER_1 | DUAL_AXIS_CHECK_TRIGGER_2)) == (DUAL_AXIS_CHECK_TRIGGER_1 | DUAL_AXIS_CHECK_TRIGGER_2)) {
```

```
      dual_axis_async_check = DUAL_AXIS_CHECK_DISABLE;
```

```
    } else {
```

```
      if (abs(dual_trigger_position - sys_position[DUAL_AXIS_SELECT]) > dual_fail_distance) {
```

```
        system_set_exec_alarm(EXEC_ALARM_HOMING_FAIL_DUAL_APPROACH);
```

```
        mc_reset();
```

```
        protocol_execute_realtime();
```

```
        return;
```

```
      }
```

```
    }
```

```
  } else {
```

```
    dual_axis_async_check |= DUAL_AXIS_CHECK_ENABLE;
```

```

        dual_trigger_position = sys_position[DUAL_AXIS_SELECT];
    }
}
#endif
}

st_prep_buffer(); // Check and prep segment buffer. NOTE: Should take no longer than 200us.

// Exit routines: No time to run protocol_execute_realtime() in this loop.
if (sys_rt_exec_state & (EXEC_SAFETY_DOOR | EXEC_RESET | EXEC_CYCLE_STOP)) {
    uint8_t rt_exec = sys_rt_exec_state;
    // Homing failure condition: Reset issued during cycle.
    if (rt_exec & EXEC_RESET) { system_set_exec_alarm(EXEC_ALARM_HOMING_FAIL_RESET); }
    // Homing failure condition: Safety door was opened.
    if (rt_exec & EXEC_SAFETY_DOOR) { system_set_exec_alarm(EXEC_ALARM_HOMING_FAIL_DOOR); }
    // Homing failure condition: Limit switch still engaged after pull-off motion
    if (!approach && (limits_get_state() & cycle_mask)) { system_set_exec_alarm(EXEC_ALARM_HOMING_FAIL_PULLOFF); }
    // Homing failure condition: Limit switch not found during approach.
    if (approach && (rt_exec & EXEC_CYCLE_STOP)) { system_set_exec_alarm(EXEC_ALARM_HOMING_FAIL_APPROACH); }
    if (sys_rt_exec_alarm) {
        mc_reset(); // Stop motors, if they are running.
        protocol_execute_realtime();
        return;
    } else {
        // Pull-off motion complete. Disable CYCLE_STOP from executing.
        system_clear_exec_state_flag(EXEC_CYCLE_STOP);
        break;
    }
}

#ifdef ENABLE_DUAL_AXIS
} while ((STEP_MASK & axislock) || (sys.homing_axis_lock_dual));
#else
} while (STEP_MASK & axislock);
#endif

st_reset(); // Immediately force kill steppers and reset step segment buffer.
delay_ms(settings.homing_debounce_delay); // Delay to allow transient dynamics to dissipate.

// Reverse direction and reset homing rate for locate cycle(s).
approach = !approach;

// After first cycle, homing enters locating phase. Shorten search to pull-off distance.
if (approach) {
    max_travel = settings.homing_pulloff * HOMING_AXIS_LOCATE_SCALAR;
    homing_rate = settings.homing_feed_rate;
} else {
    max_travel = settings.homing_pulloff;
    homing_rate = settings.homing_seek_rate;
}

```

```

} while (n_cycle-- > 0);

// The active cycle axes should now be homed and machine limits have been located. By
// default, Grbl defines machine space as all negative, as do most CNCs. Since limit switches
// can be on either side of an axes, check and set axes machine zero appropriately. Also,
// set up pull-off maneuver from axes limit switches that have been homed. This provides
// some initial clearance off the switches and should also help prevent them from falsely
// triggering when hard limits are enabled or when more than one axes shares a limit pin.
int32_t set_axis_position;
// Set machine positions for homed limit switches. Don't update non-homed axes.
for (idx=0; idx<N_AXIS; idx++) {
  // NOTE: settings.max_travel[] is stored as a negative value.
  if (cycle_mask & bit(idx)) {
    #ifdef HOMING_FORCE_SET_ORIGIN
      set_axis_position = 0;
    #else
      if ( bit_istrue(settings.homing_dir_mask,bit(idx)) ) {
        set_axis_position = lround((settings.max_travel[idx]+settings.homing_pulloff)*settings.steps_per_mm[idx]);
      } else {
        set_axis_position = lround(-settings.homing_pulloff*settings.steps_per_mm[idx]);
      }
    #endif

    #ifdef COREXY
      if (idx==X_AXIS) {
        int32_t off_axis_position = system_convert_corexy_to_y_axis_steps(sys_position);
        sys_position[A_MOTOR] = set_axis_position + off_axis_position;
        sys_position[B_MOTOR] = set_axis_position - off_axis_position;
      } else if (idx==Y_AXIS) {
        int32_t off_axis_position = system_convert_corexy_to_x_axis_steps(sys_position);
        sys_position[A_MOTOR] = off_axis_position + set_axis_position;
        sys_position[B_MOTOR] = off_axis_position - set_axis_position;
      } else {
        sys_position[idx] = set_axis_position;
      }
    #else
      sys_position[idx] = set_axis_position;
    #endif
  }
}

sys.step_control = STEP_CONTROL_NORMAL_OP; // Return step control to normal operation.
}

```

```

// Performs a soft limit check. Called from mc_line() only. Assumes the machine has been homed,
// the workspace volume is in all negative space, and the system is in normal operation.
// NOTE: Used by jogging to limit travel within soft-limit volume.
void limits_soft_check(float *target)
{
  if (system_check_travel_limits(target)) {

```

```

sys.soft_limit = true;
// Force feed hold if cycle is active. All buffered blocks are guaranteed to be within
// workspace volume so just come to a controlled stop so position is not lost. When complete
// enter alarm mode.
if (sys.state == STATE_CYCLE) {
    system_set_exec_state_flag(EXEC_FEED_HOLD);
    do {
        protocol_execute_realtime();
        if (sys.abort) { return; }
    } while ( sys.state != STATE_IDLE );
}
mc_reset(); // Issue system reset and ensure spindle and coolant are shutdown.
system_set_exec_alarm(EXEC_ALARM_SOFT_LIMIT); // Indicate soft limit critical event
protocol_execute_realtime(); // Execute to enter critical event loop and system abort
return;
}
}

```

File: limits.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
limits.h - code pertaining to limit-switches and performing the homing cycle
Part of Grbl

```

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

```

```

#ifndef limits_h
#define limits_h

```

```

// Initialize the limits module
void limits_init();

```

```

// Disables hard limits.
void limits_disable();

// Returns limit state as a bit-wise uint8 variable.
uint8_t limits_get_state();

// Perform one portion of the homing cycle based on the input settings.
void limits_go_home(uint8_t cycle_mask);

// Check for soft limit violations
void limits_soft_check(float *target);

#endif

```

File: main.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
main.c - An embedded CNC Controller with rs274/ngc (g-code) support
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

#include "grbl.h"

// Declare system global variable structure
system_t sys;
int32_t sys_position[N_AXIS]; // Real-time machine (aka home) position vector in steps.
int32_t sys_probe_position[N_AXIS]; // Last probe position in machine coordinates and steps.
volatile uint8_t sys_probe_state; // Probing state value. Used to coordinate the probing cycle with stepper ISR.
volatile uint8_t sys_rt_exec_state; // Global realtime executor bitflag variable for state management. See EXEC bitmasks.
volatile uint8_t sys_rt_exec_alarm; // Global realtime executor bitflag variable for setting various alarms.
volatile uint8_t sys_rt_exec_motion_override; // Global realtime executor bitflag variable for motion-based overrides.

```

```

volatile uint8_t sys_rt_exec_accessory_override; // Global realtime executor bitflag variable for spindle/coolant overrides.
#ifdef DEBUG
    volatile uint8_t sys_rt_exec_debug;
#endif

int main(void)
{
    // Initialize system upon power-up.
    serial_init(); // Setup serial baud rate and interrupts
    settings_init(); // Load Grbl settings from EEPROM
    stepper_init(); // Configure stepper pins and interrupt timers
    system_init(); // Configure pinout pins and pin-change interrupt

    memset(sys_position,0,sizeof(sys_position)); // Clear machine position.
    sei(); // Enable interrupts

    // Initialize system state.
#ifdef FORCE_INITIALIZATION_ALARM
    // Force Grbl into an ALARM state upon a power-cycle or hard reset.
    sys.state = STATE_ALARM;
#else
    sys.state = STATE_IDLE;
#endif

    // Check for power-up and set system alarm if homing is enabled to force homing cycle
    // by setting Grbl's alarm state. Alarm locks out all g-code commands, including the
    // startup scripts, but allows access to settings and internal commands. Only a homing
    // cycle '$H' or kill alarm locks '$X' will disable the alarm.
    // NOTE: The startup script will run after successful completion of the homing cycle, but
    // not after disabling the alarm locks. Prevents motion startup blocks from crashing into
    // things uncontrollably. Very bad.
#ifdef HOMING_INIT_LOCK
    if (bit_istrue(settings.flags,BITFLAG_HOMING_ENABLE)) { sys.state = STATE_ALARM; }
#endif

    // Grbl initialization loop upon power-up or a system abort. For the latter, all processes
    // will return to this loop to be cleanly re-initialized.
    for(;;) {

        // Reset system variables.
        uint8_t prior_state = sys.state;
        memset(&sys, 0, sizeof(system_t)); // Clear system struct variable.
        sys.state = prior_state;
        sys.f_override = DEFAULT_FEED_OVERRIDE; // Set to 100%
        sys.r_override = DEFAULT_RAPID_OVERRIDE; // Set to 100%
        sys.spindle_speed_ovr = DEFAULT_SPINDLE_SPEED_OVERRIDE; // Set to 100%
        ■■memset(sys_probe_position,0,sizeof(sys_probe_position)); // Clear probe position.
        sys_probe_state = 0;
        sys_rt_exec_state = 0;
        sys_rt_exec_alarm = 0;
    }
}

```

```

sys_rt_exec_motion_override = 0;
sys_rt_exec_accessory_override = 0;

// Reset Grbl primary systems.
serial_reset_read_buffer(); // Clear serial read buffer
gc_init(); // Set g-code parser to default state
spindle_init();
coolant_init();
limits_init();
probe_init();
plan_reset(); // Clear block buffer and planner variables
st_reset(); // Clear stepper subsystem variables.

// Sync cleared gcode and planner positions to current system position.
plan_sync_position();
gc_sync_position();

// Print welcome message. Indicates an initialization has occurred at power-up or with a reset.
report_init_message();

// Start Grbl main loop. Processes program inputs and executes them.
protocol_main_loop();

}
return 0; /* Never reached */
}

```

File: motion_control.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
motion_control.c - high level interface for issuing motion commands
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#include "grbl.h"
```

```
// Execute linear motion in absolute millimeter coordinates. Feed rate given in millimeters/second  
// unless invert_feed_rate is true. Then the feed_rate means that the motion should be completed in  
// (1 minute)/feed_rate time.
```

```
// NOTE: This is the primary gateway to the grbl planner. All line motions, including arc line  
// segments, must pass through this routine before being passed to the planner. The separation of  
// mc_line and plan_buffer_line is done primarily to place non-planner-type functions from being  
// in the planner and to let backlash compensation or canned cycle integration simple and direct.  
void mc_line(float *target, plan_line_data_t *pl_data)
```

```
{  
  // If enabled, check for soft limit violations. Placed here all line motions are picked up  
  // from everywhere in Grbl.  
  if (bit_istrue(settings.flags,BITFLAG_SOFT_LIMIT_ENABLE)) {  
    // NOTE: Block jog state. Jogging is a special case and soft limits are handled independently.  
    if (sys.state != STATE_JOG) { limits_soft_check(target); }  
  }  
}
```

```
// If in check gcode mode, prevent motion by blocking planner. Soft limits still work.  
if (sys.state == STATE_CHECK_MODE) { return; }
```

```
// NOTE: Backlash compensation may be installed here. It will need direction info to track when  
// to insert a backlash line motion(s) before the intended line motion and will require its own  
// plan_check_full_buffer() and check for system abort loop. Also for position reporting  
// backlash steps will need to be also tracked, which will need to be kept at a system level.  
// There are likely some other things that will need to be tracked as well. However, we feel  
// that backlash compensation should NOT be handled by Grbl itself, because there are a myriad  
// of ways to implement it and can be effective or ineffective for different CNC machines. This  
// would be better handled by the interface as a post-processor task, where the original g-code  
// is translated and inserts backlash motions that best suits the machine.  
// NOTE: Perhaps as a middle-ground, all that needs to be sent is a flag or special command that  
// indicates to Grbl what is a backlash compensation motion, so that Grbl executes the move but  
// doesn't update the machine position values. Since the position values used by the g-code  
// parser and planner are separate from the system machine positions, this is doable.
```

```
// If the buffer is full: good! That means we are well ahead of the robot.  
// Remain in this loop until there is room in the buffer.
```

```
do {  
  protocol_execute_realtime(); // Check for any run-time commands  
  if (sys.abort) { return; } // Bail, if system abort.  
  if ( plan_check_full_buffer() ) { protocol_auto_cycle_start(); } // Auto-cycle start when buffer is full.  
  else { break; }  
} while (1);
```

```
// Plan and queue motion into planner buffer
```

```
if (plan_buffer_line(target, pl_data) == PLAN_EMPTY_BLOCK) {  
  if (bit_istrue(settings.flags,BITFLAG_LASER_MODE)) {  
    // Correctly set spindle state, if there is a coincident position passed. Forces a buffer
```

```

// sync while in M3 laser mode only.
if (pl_data->condition & PL_COND_FLAG_SPINDLE_CW) {
    spindle_sync(PL_COND_FLAG_SPINDLE_CW, pl_data->spindle_speed);
}
}
}
}

// Execute an arc in offset mode format. position == current xyz, target == target xyz,
// offset == offset from current xyz, axis_X defines circle plane in tool space, axis_linear is
// the direction of helical travel, radius == circle radius, isclockwise boolean. Used
// for vector transformation direction.
// The arc is approximated by generating a huge number of tiny, linear segments. The chordal tolerance
// of each segment is configured in settings.arc_tolerance, which is defined to be the maximum normal
// distance from segment to the circle when the end points both lie on the circle.
void mc_arc(float *target, plan_line_data_t *pl_data, float *position, float *offset, float radius,
    uint8_t axis_0, uint8_t axis_1, uint8_t axis_linear, uint8_t is_clockwise_arc)
{
    float center_axis0 = position[axis_0] + offset[axis_0];
    float center_axis1 = position[axis_1] + offset[axis_1];
    float r_axis0 = -offset[axis_0]; // Radius vector from center to current location
    float r_axis1 = -offset[axis_1];
    float rt_axis0 = target[axis_0] - center_axis0;
    float rt_axis1 = target[axis_1] - center_axis1;

    // CCW angle between position and target from circle center. Only one atan2() trig computation required.
    float angular_travel = atan2(r_axis0*rt_axis1-r_axis1*rt_axis0, r_axis0*rt_axis0+r_axis1*rt_axis1);
    if (is_clockwise_arc) { // Correct atan2 output per direction
        if (angular_travel >= -ARC_ANGULAR_TRAVEL_EPSILON) { angular_travel -= 2*M_PI; }
    } else {
        if (angular_travel <= ARC_ANGULAR_TRAVEL_EPSILON) { angular_travel += 2*M_PI; }
    }

    // NOTE: Segment end points are on the arc, which can lead to the arc diameter being smaller by up to
    // (2x) settings.arc_tolerance. For 99% of users, this is just fine. If a different arc segment fit
    // is desired, i.e. least-squares, midpoint on arc, just change the mm_per_arc_segment calculation.
    // For the intended uses of Grbl, this value shouldn't exceed 2000 for the strictest of cases.
    uint16_t segments = floor(fabs(0.5*angular_travel*radius)/
        sqrt(settings.arc_tolerance*(2*radius - settings.arc_tolerance)) );

    if (segments) {
        // Multiply inverse feed_rate to compensate for the fact that this movement is approximated
        // by a number of discrete segments. The inverse feed_rate should be correct for the sum of
        // all segments.
        if (pl_data->condition & PL_COND_FLAG_INVERSE_TIME) {
            pl_data->feed_rate *= segments;
            bit_false(pl_data->condition, PL_COND_FLAG_INVERSE_TIME); // Force as feed absolute mode over arc segments.
        }

        float theta_per_segment = angular_travel/segments;

```

```
float linear_per_segment = (target[axis_linear] - position[axis_linear])/segments;
```

```
/* Vector rotation by transformation matrix: r is the original vector, r_T is the rotated vector,  
and phi is the angle of rotation. Solution approach by Jens Geisler.
```

```
    r_T = [cos(phi) -sin(phi);  
          sin(phi) cos(phi)] * r ;
```

For arc generation, the center of the circle is the axis of rotation and the radius vector is defined from the circle center to the initial position. Each line segment is formed by successive vector rotations. Single precision values can accumulate error greater than tool precision in rare cases. So, exact arc path correction is implemented. This approach avoids the problem of too many very expensive trig operations [sin(),cos(),tan()] which can take 100-200 usec each to compute.

Small angle approximation may be used to reduce computation overhead further. A third-order approximation (second order sin() has too much error) holds for most, if not, all CNC applications. Note that this approximation will begin to accumulate a numerical drift error when theta_per_segment is greater than ~0.25 rad(14 deg) AND the approximation is successively used without correction several dozen times. This scenario is extremely unlikely, since segment lengths and theta_per_segment are automatically generated and scaled by the arc tolerance setting. Only a very large arc tolerance setting, unrealistic for CNC applications, would cause this numerical drift error. However, it is best to set N_ARC_CORRECTION from a low of ~4 to a high of ~20 or so to avoid trig operations while keeping arc generation accurate.

This approximation also allows mc_arc to immediately insert a line segment into the planner without the initial overhead of computing cos() or sin(). By the time the arc needs to be applied a correction, the planner should have caught up to the lag caused by the initial mc_arc overhead. This is important when there are successive arc motions.

```
*/
```

```
// Computes: cos_T = 1 - theta_per_segment^2/2, sin_T = theta_per_segment - theta_per_segment^3/6) in ~52usec
```

```
float cos_T = 2.0 - theta_per_segment*theta_per_segment;
```

```
float sin_T = theta_per_segment*0.16666667*(cos_T + 4.0);
```

```
cos_T *= 0.5;
```

```
float sin_Ti;
```

```
float cos_Ti;
```

```
float r_axisi;
```

```
uint16_t i;
```

```
uint8_t count = 0;
```

```
for (i = 1; i<segments; i++) { // Increment (segments-1).
```

```
    if (count < N_ARC_CORRECTION) {
```

```
        // Apply vector rotation matrix. ~40 usec
```

```
        r_axisi = r_axis0*sin_T + r_axis1*cos_T;
```

```
        r_axis0 = r_axis0*cos_T - r_axis1*sin_T;
```

```
        r_axis1 = r_axisi;
```

```
        count++;
```

```
    } else {
```

```
        // Arc correction to radius vector. Computed only every N_ARC_CORRECTION increments. ~375 usec
```

```
        // Compute exact location by applying transformation matrix from initial radius vector(=-offset).
```

```
        cos_Ti = cos(i*theta_per_segment);
```

```
        sin_Ti = sin(i*theta_per_segment);
```

```

    r_axis0 = -offset[axis_0]*cos_Ti + offset[axis_1]*sin_Ti;
    r_axis1 = -offset[axis_0]*sin_Ti - offset[axis_1]*cos_Ti;
    count = 0;
}

// Update arc_target location
position[axis_0] = center_axis0 + r_axis0;
position[axis_1] = center_axis1 + r_axis1;
position[axis_linear] += linear_per_segment;

mc_line(position, pl_data);

// Bail mid-circle on system abort. Runtime command check already performed by mc_line.
if (sys.abort) { return; }
}
}
// Ensure last segment arrives at target location.
mc_line(target, pl_data);
}

// Execute dwell in seconds.
void mc_dwell(float seconds)
{
    if (sys.state == STATE_CHECK_MODE) { return; }
    protocol_buffer_synchronize();
    delay_sec(seconds, DELAY_MODE_DWELL);
}

// Perform homing cycle to locate and set machine zero. Only '$H' executes this command.
// NOTE: There should be no motions in the buffer and Grbl must be in an idle state before
// executing the homing cycle. This prevents incorrect buffered plans after homing.
void mc_homing_cycle(uint8_t cycle_mask)
{
    // Check and abort homing cycle, if hard limits are already enabled. Helps prevent problems
    // with machines with limits wired on both ends of travel to one limit pin.
    // TODO: Move the pin-specific LIMIT_PIN call to limits.c as a function.
#ifdef LIMITS_TWO_SWITCHES_ON_AXES
    if (limits_get_state()) {
        mc_reset(); // Issue system reset and ensure spindle and coolant are shutdown.
        system_set_exec_alarm(EXEC_ALARM_HARD_LIMIT);
        return;
    }
#endif

    limits_disable(); // Disable hard limits pin change register for cycle duration

    // -----
    // Perform homing routine. NOTE: Special motion case. Only system reset works.

```

```

#ifdef HOMING_SINGLE_AXIS_COMMANDS
    if (cycle_mask) { limits_go_home(cycle_mask); } // Perform homing cycle based on mask.
    else
#endif
{
    // Search to engage all axes limit switches at faster homing seek rate.
    limits_go_home(HOMING_CYCLE_0); // Homing cycle 0
#ifdef HOMING_CYCLE_1
    limits_go_home(HOMING_CYCLE_1); // Homing cycle 1
#endif
#ifdef HOMING_CYCLE_2
    limits_go_home(HOMING_CYCLE_2); // Homing cycle 2
#endif
}

protocol_execute_realtime(); // Check for reset and set system abort.
if (sys.abort) { return; } // Did not complete. Alarm state set by mc_alarm.

// Homing cycle complete! Setup system for normal operation.
// -----

// Sync gcode parser and planner positions to homed position.
gc_sync_position();
plan_sync_position();

// If hard limits feature enabled, re-enable hard limits pin change register after homing cycle.
limits_init();
}

// Perform tool length probe cycle. Requires probe switch.
// NOTE: Upon probe failure, the program will be stopped and placed into ALARM state.
uint8_t mc_probe_cycle(float *target, plan_line_data_t *pl_data, uint8_t parser_flags)
{
    // TODO: Need to update this cycle so it obeys a non-auto cycle start.
    if (sys.state == STATE_CHECK_MODE) { return(GC_PROBE_CHECK_MODE); }

    // Finish all queued commands and empty planner buffer before starting probe cycle.
    protocol_buffer_synchronize();
    if (sys.abort) { return(GC_PROBE_ABORT); } // Return if system reset has been issued.

    // Initialize probing control variables
    uint8_t is_probe_away = bit_istrue(parser_flags, GC_PARSER_PROBE_IS_AWAY);
    uint8_t is_no_error = bit_istrue(parser_flags, GC_PARSER_PROBE_IS_NO_ERROR);
    sys.probe_succeeded = false; // Re-initialize probe history before beginning cycle.
    probe_configure_invert_mask(is_probe_away);

    // After syncing, check if probe is already triggered. If so, halt and issue alarm.
    // NOTE: This probe initialization error applies to all probing cycles.
    if ( probe_get_state() ) { // Check probe pin state.
        system_set_exec_alarm(EXEC_ALARM_PROBE_FAIL_INITIAL);
    }
}

```

```

    protocol_execute_realtime();
    probe_configure_invert_mask(false); // Re-initialize invert mask before returning.
    return(GC_PROBE_FAIL_INIT); // Nothing else to do but bail.
}

// Setup and queue probing motion. Auto cycle-start should not start the cycle.
mc_line(target, pl_data);

// Activate the probing state monitor in the stepper module.
sys_probe_state = PROBE_ACTIVE;

// Perform probing cycle. Wait here until probe is triggered or motion completes.
system_set_exec_state_flag(EXEC_CYCLE_START);
do {
    protocol_execute_realtime();
    if (sys.abort) { return(GC_PROBE_ABORT); } // Check for system abort
} while (sys.state != STATE_IDLE);

// Probing cycle complete!

// Set state variables and error out, if the probe failed and cycle with error is enabled.
if (sys_probe_state == PROBE_ACTIVE) {
    if (is_no_error) { memcpy(sys_probe_position, sys_position, sizeof(sys_position)); }
    else { system_set_exec_alarm(EXEC_ALARM_PROBE_FAIL_CONTACT); }
} else {
    sys.probe_succeeded = true; // Indicate to system the probing cycle completed successfully.
}
sys_probe_state = PROBE_OFF; // Ensure probe state monitor is disabled.
probe_configure_invert_mask(false); // Re-initialize invert mask.
protocol_execute_realtime(); // Check and execute run-time commands

// Reset the stepper and planner buffers to remove the remainder of the probe motion.
st_reset(); // Reset step segment buffer.
plan_reset(); // Reset planner buffer. Zero planner positions. Ensure probing motion is cleared.
plan_sync_position(); // Sync planner position to current machine position.

#ifdef MESSAGE_PROBE_COORDINATES
    // All done! Output the probe position as message.
    report_probe_parameters();
#endif

if (sys.probe_succeeded) { return(GC_PROBE_FOUND); } // Successful probe cycle.
else { return(GC_PROBE_FAIL_END); } // Failed to trigger probe within travel. With or without error.
}

// Plans and executes the single special motion case for parking. Independent of main planner buffer.
// NOTE: Uses the always free planner ring buffer head to store motion parameters for execution.
#ifdef PARKING_ENABLE
void mc_parking_motion(float *parking_target, plan_line_data_t *pl_data)
{

```

```

if (sys.abort) { return; } // Block during abort.

uint8_t plan_status = plan_buffer_line(parking_target, pl_data);

if (plan_status) {
    bit_true(sys.step_control, STEP_CONTROL_EXECUTE_SYS_MOTION);
    bit_false(sys.step_control, STEP_CONTROL_END_MOTION); // Allow parking motion to execute, if feed hold is active.
    st_parking_setup_buffer(); // Setup step segment buffer for special parking motion case
    st_prep_buffer();
    st_wake_up();
    do {
        protocol_exec_rt_system();
        if (sys.abort) { return; }
    } while (sys.step_control & STEP_CONTROL_EXECUTE_SYS_MOTION);
    st_parking_restore_buffer(); // Restore step segment buffer to normal run state.
} else {
    bit_false(sys.step_control, STEP_CONTROL_EXECUTE_SYS_MOTION);
    protocol_exec_rt_system();
}

}
#endif

#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
void mc_override_ctrl_update(uint8_t override_state)
{
    // Finish all queued commands before altering override control state
    protocol_buffer_synchronize();
    if (sys.abort) { return; }
    sys.override_ctrl = override_state;
}
#endif

// Method to ready the system to reset by setting the realtime reset command and killing any
// active processes in the system. This also checks if a system reset is issued while Grbl
// is in a motion state. If so, kills the steppers and sets the system alarm to flag position
// lost, since there was an abrupt uncontrolled deceleration. Called at an interrupt level by
// realtime abort command and hard limits. So, keep to a minimum.
void mc_reset()
{
    // Only this function can set the system reset. Helps prevent multiple kill calls.
    if (bit_isfalse(sys_rt_exec_state, EXEC_RESET)) {
        system_set_exec_state_flag(EXEC_RESET);

        // Kill spindle and coolant.
        spindle_stop();
        coolant_stop();

        // Kill steppers only if in any motion state, i.e. cycle, actively holding, or homing.

```

```

// NOTE: If steppers are kept enabled via the step idle delay setting, this also keeps
// the steppers enabled by avoiding the go_idle call altogether, unless the motion state is
// violated, by which, all bets are off.
if ((sys.state & (STATE_CYCLE | STATE_HOMING | STATE_JOG)) ||
    (sys.step_control & (STEP_CONTROL_EXECUTE_HOLD | STEP_CONTROL_EXECUTE_SYS_MOTION))) {
    if (sys.state == STATE_HOMING) {
        if (!sys_rt_exec_alarm) {system_set_exec_alarm(EXEC_ALARM_HOMING_FAIL_RESET); }
    } else { system_set_exec_alarm(EXEC_ALARM_ABORT_CYCLE); }
    st_go_idle(); // Force kill steppers. Position has likely been lost.
}
}
}

```

File: motion_control.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
motion_control.h - high level interface for issuing motion commands
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

```

```

#ifndef motion_control_h
#define motion_control_h

```

```

// System motion commands must have a line number of zero.
#define HOMING_CYCLE_LINE_NUMBER 0
#define PARKING_MOTION_LINE_NUMBER 0

```

```

#define HOMING_CYCLE_ALL 0 // Must be zero.
#define HOMING_CYCLE_X bit(X_AXIS)
#define HOMING_CYCLE_Y bit(Y_AXIS)
#define HOMING_CYCLE_Z bit(Z_AXIS)

```

```

// Execute linear motion in absolute millimeter coordinates. Feed rate given in millimeters/second
// unless invert_feed_rate is true. Then the feed_rate means that the motion should be completed in
// (1 minute)/feed_rate time.
void mc_line(float *target, plan_line_data_t *pl_data);

// Execute an arc in offset mode format. position == current xyz, target == target xyz,
// offset == offset from current xyz, axis_XXX defines circle plane in tool space, axis_linear is
// the direction of helical travel, radius == circle radius, is_clockwise_arc boolean. Used
// for vector transformation direction.
void mc_arc(float *target, plan_line_data_t *pl_data, float *position, float *offset, float radius,
  uint8_t axis_0, uint8_t axis_1, uint8_t axis_linear, uint8_t is_clockwise_arc);

// Dwell for a specific number of seconds
void mc_dwell(float seconds);

// Perform homing cycle to locate machine zero. Requires limit switches.
void mc_homing_cycle(uint8_t cycle_mask);

// Perform tool length probe cycle. Requires probe switch.
uint8_t mc_probe_cycle(float *target, plan_line_data_t *pl_data, uint8_t parser_flags);

// Handles updating the override control state.
void mc_override_ctrl_update(uint8_t override_state);

// Plans and executes the single special motion case for parking. Independent of main planner buffer.
void mc_parking_motion(float *parking_target, plan_line_data_t *pl_data);

// Performs system reset. If in motion state, kills all motion and sets system alarm.
void mc_reset();

#endif

```

File: nuts_bolts.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

nuts_bolts.c - Shared functions
Part of Grbl

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#include "grbl.h"
```

```
#define MAX_INT_DIGITS 8 // Maximum number of digits in int32 (and float)
```

```
// Extracts a floating point value from a string. The following code is based loosely on  
// the avr-libc strtod() function by Michael Stumpf and Dmitry Xmelkov and many freely  
// available conversion method examples, but has been highly optimized for Grbl. For known  
// CNC applications, the typical decimal value is expected to be in the range of E0 to E-4.  
// Scientific notation is officially not supported by g-code, and the 'E' character may  
// be a g-code word on some CNC systems. So, 'E' notation will not be recognized.  
// NOTE: Thanks to Radu-Eosif Mihailescu for identifying the issues with using strtod().  
uint8_t read_float(char *line, uint8_t *char_counter, float *float_ptr)
```

```
{  
  char *ptr = line + *char_counter;  
  unsigned char c;
```

```
  // Grab first character and increment pointer. No spaces assumed in line.  
  c = *ptr++;
```

```
  // Capture initial positive/minus character
```

```
  bool isnegative = false;
```

```
  if (c == '-') {  
    isnegative = true;  
    c = *ptr++;  
  } else if (c == '+') {  
    c = *ptr++;  
  }
```

```
  // Extract number into fast integer. Track decimal in terms of exponent value.
```

```
  uint32_t intval = 0;
```

```
  int8_t exp = 0;
```

```
  uint8_t ndigit = 0;
```

```
  bool isdecimal = false;
```

```
  while(1) {  
    c -= '0';  
    if (c <= 9) {  
      ndigit++;  
      if (ndigit <= MAX_INT_DIGITS) {  
        if (isdecimal) { exp--; }  
        intval = (((intval << 2) + intval) << 1) + c; // intval*10 + c
```

```

    } else {
        if (!(isdecimal)) { exp++; } // Drop overflow digits
    }
} else if (c == ('.'-'0') & 0xff) && !(isdecimal)) {
    isdecimal = true;
} else {
    break;
}
c = *ptr++;
}

// Return if no digits have been read.
if (!Indigit) { return(false); };

// Convert integer into floating point.
float fval;
fval = (float)intval;

// Apply decimal. Should perform no more than two floating point multiplications for the
// expected range of E0 to E-4.
if (fval != 0) {
    while (exp <= -2) {
        fval *= 0.01;
        exp += 2;
    }
    if (exp < 0) {
        fval *= 0.1;
    } else if (exp > 0) {
        do {
            fval *= 10.0;
        } while (--exp > 0);
    }
}

// Assign floating point value with correct sign.
if (isnegative) {
    *float_ptr = -fval;
} else {
    *float_ptr = fval;
}

*char_counter = ptr - line - 1; // Set char_counter to next statement

return(true);
}

// Non-blocking delay function used for general operation and suspend features.
void delay_sec(float seconds, uint8_t mode)
{
    uint16_t i = ceil(1000/DWELL_TIME_STEP*seconds);

```

```

while (i-- > 0) {
    if (sys.abort) { return; }
    if (mode == DELAY_MODE_DWELL) {
        protocol_execute_realtime();
    } else { // DELAY_MODE_SYS_SUSPEND
        // Execute rt_system() only to avoid nesting suspend loops.
        protocol_exec_rt_system();
        if (sys.suspend & SUSPEND_RESTART_RETRACT) { return; } // Bail, if safety door reopens.
    }
    _delay_ms(DWELL_TIME_STEP); // Delay DWELL_TIME_STEP increment
}
}

```

```

// Delays variable defined milliseconds. Compiler compatibility fix for _delay_ms(),
// which only accepts constants in future compiler releases.
void delay_ms(uint16_t ms)
{
    while ( ms-- ) { _delay_ms(1); }
}

```

```

// Delays variable defined microseconds. Compiler compatibility fix for _delay_us(),
// which only accepts constants in future compiler releases. Written to perform more
// efficiently with larger delays, as the counter adds parasitic time in each iteration.
void delay_us(uint32_t us)
{
    while (us) {
        if (us < 10) {
            _delay_us(1);
            us--;
        } else if (us < 100) {
            _delay_us(10);
            us -= 10;
        } else if (us < 1000) {
            _delay_us(100);
            us -= 100;
        } else {
            _delay_ms(1);
            us -= 1000;
        }
    }
}
}

```

```

// Simple hypotenuse computation function.
float hypot_f(float x, float y) { return(sqrt(x*x + y*y)); }

```

```

float convert_delta_vector_to_unit_vector(float *vector)
{

```

```

uint8_t idx;
float magnitude = 0.0;
for (idx=0; idx<N_AXIS; idx++) {
    if (vector[idx] != 0.0) {
        magnitude += vector[idx]*vector[idx];
    }
}
magnitude = sqrt(magnitude);
float inv_magnitude = 1.0/magnitude;
for (idx=0; idx<N_AXIS; idx++) { vector[idx] *= inv_magnitude; }
return(magnitude);
}

float limit_value_by_axis_maximum(float *max_value, float *unit_vec)
{
    uint8_t idx;
    float limit_value = SOME_LARGE_VALUE;
    for (idx=0; idx<N_AXIS; idx++) {
        if (unit_vec[idx] != 0) { // Avoid divide by zero.
            limit_value = min(limit_value,fabs(max_value[idx]/unit_vec[idx]));
        }
    }
    return(limit_value);
}

```

File: nuts_bolts.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
nuts_bolts.h - Header file for shared definitions, variables, and functions
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

```

```

#ifndef nuts_bolts_h
#define nuts_bolts_h

#define false 0
#define true 1

#define SOME_LARGE_VALUE 1.0E+38

// Axis array index values. Must start with 0 and be continuous.
#define N_AXIS 3 // Number of axes
#define X_AXIS 0 // Axis indexing value.
#define Y_AXIS 1
#define Z_AXIS 2
// #define A_AXIS 3

// CoreXY motor assignments. DO NOT ALTER.
// NOTE: If the A and B motor axis bindings are changed, this effects the CoreXY equations.
#ifdef COREXY
#define A_MOTOR X_AXIS // Must be X_AXIS
#define B_MOTOR Y_AXIS // Must be Y_AXIS
#endif

// Conversions
#define MM_PER_INCH (25.40)
#define INCH_PER_MM (0.0393701)
#define TICKS_PER_MICROSECOND (F_CPU/1000000)

#define DELAY_MODE_DWELL 0
#define DELAY_MODE_SYS_SUSPEND 1

// Useful macros
#define clear_vector(a) memset(a, 0, sizeof(a))
#define clear_vector_float(a) memset(a, 0.0, sizeof(float)*N_AXIS)
// #define clear_vector_long(a) memset(a, 0.0, sizeof(long)*N_AXIS)
#define max(a,b) (((a) > (b)) ? (a) : (b))
#define min(a,b) (((a) < (b)) ? (a) : (b))
#define isequal_position_vector(a,b) !(memcmp(a, b, sizeof(float)*N_AXIS))

// Bit field and masking macros
#define bit(n) (1 << n)
#define bit_true(x,mask) (x) |= (mask)
#define bit_false(x,mask) (x) &= ~(mask)
#define bit_istrue(x,mask) ((x & mask) != 0)
#define bit_isfalse(x,mask) ((x & mask) == 0)

// Read a floating point value from a string. Line points to the input buffer, char_counter
// is the indexer pointing to the current character of the line, while float_ptr is
// a pointer to the result variable. Returns true when it succeeds
uint8_t read_float(char *line, uint8_t *char_counter, float *float_ptr);

```

```

// Non-blocking delay function used for general operation and suspend features.
void delay_sec(float seconds, uint8_t mode);

// Delays variable-defined milliseconds. Compiler compatibility fix for _delay_ms().
void delay_ms(uint16_t ms);

// Delays variable-defined microseconds. Compiler compatibility fix for _delay_us().
void delay_us(uint32_t us);

// Computes hypotenuse, avoiding avr-gcc's bloated version and the extra error checking.
float hypot_f(float x, float y);

float convert_delta_vector_to_unit_vector(float *vector);
float limit_value_by_axis_maximum(float *max_value, float *unit_vec);

#endif

```

File: planner.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
planner.c - buffers movement commands and manages the acceleration profile plan
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud
 Copyright (c) 2011 Jens Geisler

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

```

```

#include "grbl.h"

```

```

static plan_block_t block_buffer[BLOCK_BUFFER_SIZE]; // A ring buffer for motion instructions
static uint8_t block_buffer_tail; // Index of the block to process now
static uint8_t block_buffer_head; // Index of the next block to be pushed
static uint8_t next_buffer_head; // Index of the next buffer head

```

```

static uint8_t block_buffer_planned; // Index of the optimally planned block

// Define planner variables
typedef struct {
    int32_t position[N_AXIS]; // The planner position of the tool in absolute steps. Kept separate
                             // from g-code position for movements requiring multiple line motions,
                             // i.e. arcs, canned cycles, and backlash compensation.
    float previous_unit_vec[N_AXIS]; // Unit vector of previous path line segment
    float previous_nominal_speed; // Nominal speed of previous path line segment
} planner_t;
static planner_t pl;

```

// Returns the index of the next block in the ring buffer. Also called by stepper segment buffer.

```

uint8_t plan_next_block_index(uint8_t block_index)
{
    block_index++;
    if (block_index == BLOCK_BUFFER_SIZE) { block_index = 0; }
    return(block_index);
}

```

// Returns the index of the previous block in the ring buffer

```

static uint8_t plan_prev_block_index(uint8_t block_index)
{
    if (block_index == 0) { block_index = BLOCK_BUFFER_SIZE; }
    block_index--;
    return(block_index);
}

```

```

/*          PLANNER SPEED DEFINITION
          +-----+ <- current->nominal_speed
          /       \
current->entry_speed -> +       \
          |       + <- next->entry_speed (aka exit speed)
          +-----+
          time -->

```

Recalculates the motion plan according to the following basic guidelines:

1. Go over every feasible block sequentially in reverse order and calculate the junction speeds (i.e. current->entry_speed) such that:
 - a. No junction speed exceeds the pre-computed maximum junction speed limit or nominal speeds of neighboring blocks.
 - b. A block entry speed cannot exceed one reverse-computed from its exit speed (next->entry_speed) with a maximum allowable deceleration over the block travel distance.
 - c. The last (or newest appended) block is planned from a complete stop (an exit speed of zero).
2. Go over every block in chronological (forward) order and dial down junction speed values if
 - a. The exit speed exceeds the one forward-computed from its entry speed with the maximum allowable acceleration over the block travel distance.

When these stages are complete, the planner will have maximized the velocity profiles throughout the all of the planner blocks, where every block is operating at its maximum allowable acceleration limits. In other words, for all of the blocks in the planner, the plan is optimal and no further speed improvements are possible. If a new block is added to the buffer, the plan is recomputed according to the said guidelines for a new optimal plan.

To increase computational efficiency of these guidelines, a set of planner block pointers have been created to indicate stop-compute points for when the planner guidelines cannot logically make any further changes or improvements to the plan when in normal operation and new blocks are streamed and added to the planner buffer. For example, if a subset of sequential blocks in the planner have been planned and are bracketed by junction velocities at their maximums (or by the first planner block as well), no new block added to the planner buffer will alter the velocity profiles within them. So we no longer have to compute them. Or, if a set of sequential blocks from the first block in the planner (or a optimal stop-compute point) are all accelerating, they are all optimal and can not be altered by a new block added to the planner buffer, as this will only further increase the plan speed to chronological blocks until a maximum junction velocity is reached. However, if the operational conditions of the plan changes from infrequently used feed holds or feedrate overrides, the stop-compute pointers will be reset and the entire plan is recomputed as stated in the general guidelines.

Planner buffer index mapping:

- `block_buffer_tail`: Points to the beginning of the planner buffer. First to be executed or being executed.
- `block_buffer_head`: Points to the buffer block after the last block in the buffer. Used to indicate whether the buffer is full or empty. As described for standard ring buffers, this block is always empty.
- `next_buffer_head`: Points to next planner buffer block after the buffer head block. When equal to the buffer tail, this indicates the buffer is full.
- `block_buffer_planned`: Points to the first buffer block after the last optimally planned block for normal streaming operating conditions. Use for planning optimizations by avoiding recomputing parts of the planner buffer that don't change with the addition of a new block, as describe above. In addition, this block can never be less than `block_buffer_tail` and will always be pushed forward and maintain this requirement when encountered by the `plan_discard_current_block()` routine during a cycle.

NOTE: Since the planner only computes on what's in the planner buffer, some motions with lots of short line segments, like G2/3 arcs or complex curves, may seem to move slow. This is because there simply isn't enough combined distance traveled in the entire buffer to accelerate up to the nominal speed and then decelerate to a complete stop at the end of the buffer, as stated by the guidelines. If this happens and becomes an annoyance, there are a few simple solutions: (1) Maximize the machine acceleration. The planner will be able to compute higher velocity profiles within the same combined distance. (2) Maximize line motion(s) distance per block to a desired tolerance. The more combined distance the planner has to use, the faster it can go. (3) Maximize the planner buffer size. This also will increase the combined distance for the planner to compute over. It also increases the number of computations the planner has to perform to compute an optimal plan, so select carefully. The Arduino 328p memory is already maxed out, but future ARM versions should have enough memory and speed for look-ahead blocks numbering up to a hundred or more.

```
*/
static void planner_recalculate()
{
    // Initialize block index to the last block in the planner buffer.
    uint8_t block_index = plan_prev_block_index(block_buffer_head);

    // Bail. Can't do anything with one only one plan-able block.
```

```

if (block_index == block_buffer_planned) { return; }

// Reverse Pass: Coarsely maximize all possible deceleration curves back-planning from the last
// block in buffer. Cease planning when the last optimal planned or tail pointer is reached.
// NOTE: Forward pass will later refine and correct the reverse pass to create an optimal plan.
float entry_speed_sqr;
plan_block_t *next;
plan_block_t *current = &block_buffer[block_index];

// Calculate maximum entry speed for last block in buffer, where the exit speed is always zero.
current->entry_speed_sqr = min( current->max_entry_speed_sqr, 2*current->acceleration*current->millimeters);

block_index = plan_prev_block_index(block_index);
if (block_index == block_buffer_planned) { // Only two plannable blocks in buffer. Reverse pass complete.
    // Check if the first block is the tail. If so, notify stepper to update its current parameters.
    if (block_index == block_buffer_tail) { st_update_plan_block_parameters(); }
} else { // Three or more plan-able blocks
    while (block_index != block_buffer_planned) {
        next = current;
        current = &block_buffer[block_index];
        block_index = plan_prev_block_index(block_index);

        // Check if next block is the tail block(=planned block). If so, update current stepper parameters.
        if (block_index == block_buffer_tail) { st_update_plan_block_parameters(); }

        // Compute maximum entry speed decelerating over the current block from its exit speed.
        if (current->entry_speed_sqr != current->max_entry_speed_sqr) {
            entry_speed_sqr = next->entry_speed_sqr + 2*current->acceleration*current->millimeters;
            if (entry_speed_sqr < current->max_entry_speed_sqr) {
                current->entry_speed_sqr = entry_speed_sqr;
            } else {
                current->entry_speed_sqr = current->max_entry_speed_sqr;
            }
        }
    }
}

// Forward Pass: Forward plan the acceleration curve from the planned pointer onward.
// Also scans for optimal plan breakpoints and appropriately updates the planned pointer.
next = &block_buffer[block_buffer_planned]; // Begin at buffer planned pointer
block_index = plan_next_block_index(block_buffer_planned);
while (block_index != block_buffer_head) {
    current = next;
    next = &block_buffer[block_index];

    // Any acceleration detected in the forward pass automatically moves the optimal planned
    // pointer forward, since everything before this is all optimal. In other words, nothing
    // can improve the plan from the buffer tail to the planned pointer by logic.
    if (current->entry_speed_sqr < next->entry_speed_sqr) {
        entry_speed_sqr = current->entry_speed_sqr + 2*current->acceleration*current->millimeters;
        // If true, current block is full-acceleration and we can move the planned pointer forward.
    }
}

```

```

    if (entry_speed_sqr < next->entry_speed_sqr) {
        next->entry_speed_sqr = entry_speed_sqr; // Always <= max_entry_speed_sqr. Backward pass sets this.
        block_buffer_planned = block_index; // Set optimal plan pointer.
    }
}

```

```

// Any block set at its maximum entry speed also creates an optimal plan up to this
// point in the buffer. When the plan is bracketed by either the beginning of the
// buffer and a maximum entry speed or two maximum entry speeds, every block in between
// cannot logically be further improved. Hence, we don't have to recompute them anymore.
if (next->entry_speed_sqr == next->max_entry_speed_sqr) { block_buffer_planned = block_index; }
block_index = plan_next_block_index( block_index );
}
}

```

```

void plan_reset()
{
    memset(&pl, 0, sizeof(planner_t)); // Clear planner struct
    plan_reset_buffer();
}

```

```

void plan_reset_buffer()
{
    block_buffer_tail = 0;
    block_buffer_head = 0; // Empty = tail
    next_buffer_head = 1; // plan_next_block_index(block_buffer_head)
    block_buffer_planned = 0; // = block_buffer_tail;
}

```

```

void plan_discard_current_block()
{
    if (block_buffer_head != block_buffer_tail) { // Discard non-empty buffer.
        uint8_t block_index = plan_next_block_index( block_buffer_tail );
        // Push block_buffer_planned pointer, if encountered.
        if (block_buffer_tail == block_buffer_planned) { block_buffer_planned = block_index; }
        block_buffer_tail = block_index;
    }
}

```

```

// Returns address of planner buffer block used by system motions. Called by segment generator.
plan_block_t *plan_get_system_motion_block()
{
    return(&block_buffer[block_buffer_head]);
}

```

```

// Returns address of first planner block, if available. Called by various main program functions.

```

```

plan_block_t *plan_get_current_block()
{
    if (block_buffer_head == block_buffer_tail) { return(NULL); } // Buffer empty
    return(&block_buffer[block_buffer_tail]);
}

```

```

float plan_get_exec_block_exit_speed_sqr()
{
    uint8_t block_index = plan_next_block_index(block_buffer_tail);
    if (block_index == block_buffer_head) { return( 0.0 ); }
    return( block_buffer[block_index].entry_speed_sqr );
}

```

```

// Returns the availability status of the block ring buffer. True, if full.
uint8_t plan_check_full_buffer()

```

```

{
    if (block_buffer_tail == next_buffer_head) { return(true); }
    return(false);
}

```

```

// Computes and returns block nominal speed based on running condition and override values.

```

```

// NOTE: All system motion commands, such as homing/parking, are not subject to overrides.

```

```

float plan_compute_profile_nominal_speed(plan_block_t *block)

```

```

{
    float nominal_speed = block->programmed_rate;
    if (block->condition & PL_COND_FLAG_RAPID_MOTION) { nominal_speed *= (0.01*sys.r_override); }
    else {
        if (!(block->condition & PL_COND_FLAG_NO_FEED_OVERRIDE)) { nominal_speed *= (0.01*sys.f_override); }
        if (nominal_speed > block->rapid_rate) { nominal_speed = block->rapid_rate; }
    }
    if (nominal_speed > MINIMUM_FEED_RATE) { return(nominal_speed); }
    return(MINIMUM_FEED_RATE);
}

```

```

// Computes and updates the max entry speed (sqr) of the block, based on the minimum of the junction's

```

```

// previous and current nominal speeds and max junction speed.

```

```

static void plan_compute_profile_parameters(plan_block_t *block, float nominal_speed, float prev_nominal_speed)

```

```

{
    // Compute the junction maximum entry based on the minimum of the junction speed and neighboring nominal speeds.
    if (nominal_speed > prev_nominal_speed) { block->max_entry_speed_sqr = prev_nominal_speed*prev_nominal_speed; }
    else { block->max_entry_speed_sqr = nominal_speed*nominal_speed; }
    if (block->max_entry_speed_sqr > block->max_junction_speed_sqr) { block->max_entry_speed_sqr = block->max_junction_speed_sqr; }
}

```

```

// Re-calculates buffered motions profile parameters upon a motion-based override change.

```

```

void plan_update_velocity_profile_parameters()

```

```

{
    uint8_t block_index = block_buffer_tail;
    plan_block_t *block;
    float nominal_speed;
    float prev_nominal_speed = SOME_LARGE_VALUE; // Set high for first block nominal speed calculation.
    while (block_index != block_buffer_head) {
        block = &block_buffer[block_index];
        nominal_speed = plan_compute_profile_nominal_speed(block);
        plan_compute_profile_parameters(block, nominal_speed, prev_nominal_speed);
        prev_nominal_speed = nominal_speed;
        block_index = plan_next_block_index(block_index);
    }
    pl.previous_nominal_speed = prev_nominal_speed; // Update prev nominal speed for next incoming block.
}

```

/* Add a new linear movement to the buffer. target[N_AXIS] is the signed, absolute target position in millimeters. Feed rate specifies the speed of the motion. If feed rate is inverted, the feed rate is taken to mean "frequency" and would complete the operation in 1/feed_rate minutes. All position data passed to the planner must be in terms of machine position to keep the planner independent of any coordinate system changes and offsets, which are handled by the g-code parser. NOTE: Assumes buffer is available. Buffer checks are handled at a higher level by motion_control. In other words, the buffer head is never equal to the buffer tail. Also the feed rate input value is used in three ways: as a normal feed rate if invert_feed_rate is false, as inverse time if invert_feed_rate is true, or as seek/rapids rate if the feed_rate value is negative (and invert_feed_rate always false).

The system motion condition tells the planner to plan a motion in the always unused block buffer head. It avoids changing the planner state and preserves the buffer to ensure subsequent gcode motions are still planned correctly, while the stepper module only points to the block buffer head to execute the special system motion. */

```

uint8_t plan_buffer_line(float *target, plan_line_data_t *pl_data)
{
    // Prepare and initialize new block. Copy relevant pl_data for block execution.
    plan_block_t *block = &block_buffer[block_buffer_head];
    memset(block, 0, sizeof(plan_block_t)); // Zero all block values.
    block->condition = pl_data->condition;
#ifdef VARIABLE_SPINDLE
    block->spindle_speed = pl_data->spindle_speed;
#endif
#ifdef USE_LINE_NUMBERS
    block->line_number = pl_data->line_number;
#endif

    // Compute and store initial move distance data.
    int32_t target_steps[N_AXIS], position_steps[N_AXIS];
    float unit_vec[N_AXIS], delta_mm;
    uint8_t idx;

    // Copy position data based on type of motion being planned.
    if (block->condition & PL_COND_FLAG_SYSTEM_MOTION) {
#ifdef COREXY

```

```

    position_steps[X_AXIS] = system_convert_corexy_to_x_axis_steps(sys_position);
    position_steps[Y_AXIS] = system_convert_corexy_to_y_axis_steps(sys_position);
    position_steps[Z_AXIS] = sys_position[Z_AXIS];
#else
    memcpy(position_steps, sys_position, sizeof(sys_position));
#endif
} else { memcpy(position_steps, pl.position, sizeof(pl.position)); }

#ifdef COREXY
    target_steps[A_MOTOR] = lround(target[A_MOTOR]*settings.steps_per_mm[A_MOTOR]);
    target_steps[B_MOTOR] = lround(target[B_MOTOR]*settings.steps_per_mm[B_MOTOR]);
    block->steps[A_MOTOR] = labs((target_steps[X_AXIS]-position_steps[X_AXIS]) + (target_steps[Y_AXIS]-position_steps[Y_AXIS]));
    block->steps[B_MOTOR] = labs((target_steps[X_AXIS]-position_steps[X_AXIS]) - (target_steps[Y_AXIS]-position_steps[Y_AXIS]));
#endif

for (idx=0; idx<N_AXIS; idx++) {
    // Calculate target position in absolute steps, number of steps for each axis, and determine max step events.
    // Also, compute individual axes distance for move and prep unit vector calculations.
    // NOTE: Computes true distance from converted step values.
    #ifdef COREXY
        if ( !(idx == A_MOTOR) && !(idx == B_MOTOR) ) {
            target_steps[idx] = lround(target[idx]*settings.steps_per_mm[idx]);
            block->steps[idx] = labs(target_steps[idx]-position_steps[idx]);
        }
        block->step_event_count = max(block->step_event_count, block->steps[idx]);
        if (idx == A_MOTOR) {
            delta_mm = (target_steps[X_AXIS]-position_steps[X_AXIS] + target_steps[Y_AXIS]-position_steps[Y_AXIS])/settings.steps_per_mm[A_MOTOR];
        } else if (idx == B_MOTOR) {
            delta_mm = (target_steps[X_AXIS]-position_steps[X_AXIS] - target_steps[Y_AXIS]+position_steps[Y_AXIS])/settings.steps_per_mm[B_MOTOR];
        } else {
            delta_mm = (target_steps[idx] - position_steps[idx])/settings.steps_per_mm[idx];
        }
    #else
        target_steps[idx] = lround(target[idx]*settings.steps_per_mm[idx]);
        block->steps[idx] = labs(target_steps[idx]-position_steps[idx]);
        block->step_event_count = max(block->step_event_count, block->steps[idx]);
        delta_mm = (target_steps[idx] - position_steps[idx])/settings.steps_per_mm[idx];
    #endif
    unit_vec[idx] = delta_mm; // Store unit vector numerator

    // Set direction bits. Bit enabled always means direction is negative.
    if (delta_mm < 0.0 ) { block->direction_bits |= get_direction_pin_mask(idx); }
}

// Bail if this is a zero-length block. Highly unlikely to occur.
if (block->step_event_count == 0) { return(PLAN_EMPTY_BLOCK); }

// Calculate the unit vector of the line move and the block maximum feed rate and acceleration scaled
// down such that no individual axes maximum values are exceeded with respect to the line direction.
// NOTE: This calculation assumes all axes are orthogonal (Cartesian) and works with ABC-axes,
// if they are also orthogonal/independent. Operates on the absolute value of the unit vector.

```

```

block->millimeters = convert_delta_vector_to_unit_vector(unit_vec);
block->acceleration = limit_value_by_axis_maximum(settings.acceleration, unit_vec);
block->rapid_rate = limit_value_by_axis_maximum(settings.max_rate, unit_vec);

// Store programmed rate.
if (block->condition & PL_COND_FLAG_RAPID_MOTION) { block->programmed_rate = block->rapid_rate; }
else {
    block->programmed_rate = pl_data->feed_rate;
    if (block->condition & PL_COND_FLAG_INVERSE_TIME) { block->programmed_rate *= block->millimeters; }
}

// TODO: Need to check this method handling zero junction speeds when starting from rest.
if ((block_buffer_head == block_buffer_tail) || (block->condition & PL_COND_FLAG_SYSTEM_MOTION)) {

    // Initialize block entry speed as zero. Assume it will be starting from rest. Planner will correct this later.
    // If system motion, the system motion block always is assumed to start from rest and end at a complete stop.
    block->entry_speed_sqr = 0.0;
    block->max_junction_speed_sqr = 0.0; // Starting from rest. Enforce start from zero velocity.

} else {
    // Compute maximum allowable entry speed at junction by centripetal acceleration approximation.
    // Let a circle be tangent to both previous and current path line segments, where the junction
    // deviation is defined as the distance from the junction to the closest edge of the circle,
    // colinear with the circle center. The circular segment joining the two paths represents the
    // path of centripetal acceleration. Solve for max velocity based on max acceleration about the
    // radius of the circle, defined indirectly by junction deviation. This may be also viewed as
    // path width or max_jerk in the previous Grbl version. This approach does not actually deviate
    // from path, but used as a robust way to compute cornering speeds, as it takes into account the
    // nonlinearities of both the junction angle and junction velocity.
    //
    // NOTE: If the junction deviation value is finite, Grbl executes the motions in an exact path
    // mode (G61). If the junction deviation value is zero, Grbl will execute the motion in an exact
    // stop mode (G61.1) manner. In the future, if continuous mode (G64) is desired, the math here
    // is exactly the same. Instead of motioning all the way to junction point, the machine will
    // just follow the arc circle defined here. The Arduino doesn't have the CPU cycles to perform
    // a continuous mode path, but ARM-based microcontrollers most certainly do.
    //
    // NOTE: The max junction speed is a fixed value, since machine acceleration limits cannot be
    // changed dynamically during operation nor can the line move geometry. This must be kept in
    // memory in the event of a feedrate override changing the nominal speeds of blocks, which can
    // change the overall maximum entry speed conditions of all blocks.

    float junction_unit_vec[N_AXIS];
    float junction_cos_theta = 0.0;
    for (idx=0; idx<N_AXIS; idx++) {
        junction_cos_theta -= pl.previous_unit_vec[idx]*unit_vec[idx];
        junction_unit_vec[idx] = unit_vec[idx]-pl.previous_unit_vec[idx];
    }

    // NOTE: Computed without any expensive trig, sin() or acos(), by trig half angle identity of cos(theta).
    if (junction_cos_theta > 0.999999) {

```

```

// For a 0 degree acute junction, just set minimum junction speed.
block->max_junction_speed_sqr = MINIMUM_JUNCTION_SPEED*MINIMUM_JUNCTION_SPEED;
} else {
    if (junction_cos_theta < -0.999999) {
        // Junction is a straight line or 180 degrees. Junction speed is infinite.
        block->max_junction_speed_sqr = SOME_LARGE_VALUE;
    } else {
        convert_delta_vector_to_unit_vector(junction_unit_vec);
        float junction_acceleration = limit_value_by_axis_maximum(settings.acceleration, junction_unit_vec);
        float sin_theta_d2 = sqrt(0.5*(1.0-junction_cos_theta)); // Trig half angle identity. Always positive.
        block->max_junction_speed_sqr = max( MINIMUM_JUNCTION_SPEED*MINIMUM_JUNCTION_SPEED,
            (junction_acceleration * settings.junction_deviation * sin_theta_d2)/(1.0-sin_theta_d2) );
    }
}
}

// Block system motion from updating this data to ensure next g-code motion is computed correctly.
if (!(block->condition & PL_COND_FLAG_SYSTEM_MOTION)) {
    float nominal_speed = plan_compute_profile_nominal_speed(block);
    plan_compute_profile_parameters(block, nominal_speed, pl.previous_nominal_speed);
    pl.previous_nominal_speed = nominal_speed;

    // Update previous path unit_vector and planner position.
    memcpy(pl.previous_unit_vec, unit_vec, sizeof(unit_vec)); // pl.previous_unit_vec[] = unit_vec[]
    memcpy(pl.position, target_steps, sizeof(target_steps)); // pl.position[] = target_steps[]

    // New block is all set. Update buffer head and next buffer head indices.
    block_buffer_head = next_buffer_head;
    next_buffer_head = plan_next_block_index(block_buffer_head);

    // Finish up by recalculating the plan with the new block.
    planner_recalculate();
}
return(PLAN_OK);
}

```

```

// Reset the planner position vectors. Called by the system abort/initialization routine.
void plan_sync_position()
{
    // TODO: For motor configurations not in the same coordinate frame as the machine position,
    // this function needs to be updated to accomodate the difference.
    uint8_t idx;
    for (idx=0; idx<N_AXIS; idx++) {
        #ifdef COREXY
            if (idx==X_AXIS) {
                pl.position[X_AXIS] = system_convert_corexy_to_x_axis_steps(sys_position);
            } else if (idx==Y_AXIS) {
                pl.position[Y_AXIS] = system_convert_corexy_to_y_axis_steps(sys_position);
            } else {
                pl.position[idx] = sys_position[idx];
            }
        #endif
    }
}

```

```

    }
    #else
    pl.position[idx] = sys_position[idx];
    #endif
}
}

// Returns the number of available blocks are in the planner buffer.
uint8_t plan_get_block_buffer_available()
{
    if (block_buffer_head >= block_buffer_tail) { return((BLOCK_BUFFER_SIZE-1)-(block_buffer_head-block_buffer_tail)); }
    return((block_buffer_tail-block_buffer_head-1));
}

// Returns the number of active blocks are in the planner buffer.
// NOTE: Deprecated. Not used unless classic status reports are enabled in config.h
uint8_t plan_get_block_buffer_count()
{
    if (block_buffer_head >= block_buffer_tail) { return(block_buffer_head-block_buffer_tail); }
    return(BLOCK_BUFFER_SIZE - (block_buffer_tail-block_buffer_head));
}

// Re-initialize buffer plan with a partially completed block, assumed to exist at the buffer tail.
// Called after a steppers have come to a complete stop for a feed hold and the cycle is stopped.
void plan_cycle_reinitialize()
{
    // Re-plan from a complete stop. Reset planner entry speeds and buffer planned pointer.
    st_update_plan_block_parameters();
    block_buffer_planned = block_buffer_tail;
    planner_recalculate();
}

```

File: planner.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
planner.h - buffers movement commands and manages the acceleration profile plan
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#ifndef planner_h
#define planner_h
```

```
// The number of linear motions that can be in the plan at any give time
```

```
#ifndef BLOCK_BUFFER_SIZE
#ifdef USE_LINE_NUMBERS
#define BLOCK_BUFFER_SIZE 15
#else
#define BLOCK_BUFFER_SIZE 16
#endif
#endif
```

```
// Returned status message from planner.
```

```
#define PLAN_OK true
#define PLAN_EMPTY_BLOCK false
```

```
// Define planner data condition flags. Used to denote running conditions of a block.
```

```
#define PL_COND_FLAG_RAPID_MOTION    bit(0)
#define PL_COND_FLAG_SYSTEM_MOTION  bit(1) // Single motion. Circumvents planner state. Used by home/park.
#define PL_COND_FLAG_NO_FEED_OVERRIDE bit(2) // Motion does not honor feed override.
#define PL_COND_FLAG_INVERSE_TIME    bit(3) // Interprets feed rate value as inverse time when set.
#define PL_COND_FLAG_SPINDLE_CW      bit(4)
#define PL_COND_FLAG_SPINDLE_CCW     bit(5)
#define PL_COND_FLAG_COOLANT_FLOOD   bit(6)
#define PL_COND_FLAG_COOLANT_MIST    bit(7)
#define PL_COND_MOTION_MASK (PL_COND_FLAG_RAPID_MOTION|PL_COND_FLAG_SYSTEM_MOTION|PL_COND_FLAG_NO_FEED_OVERRIDE)
#define PL_COND_SPINDLE_MASK (PL_COND_FLAG_SPINDLE_CW|PL_COND_FLAG_SPINDLE_CCW)
#define PL_COND_ACCESSORY_MASK (PL_COND_FLAG_SPINDLE_CW|PL_COND_FLAG_SPINDLE_CCW|PL_COND_FLAG_COOLANT_FLOOD|PL_COND_FLAG_COOLANT_MIST)
```

```
// This struct stores a linear movement of a g-code block motion with its critical "nominal" values
```

```
// are as specified in the source g-code.
```

```
typedef struct {
    // Fields used by the bresenham algorithm for tracing the line
    // NOTE: Used by stepper algorithm to execute the block correctly. Do not alter these values.
    uint32_t steps[N_AXIS]; // Step count along each axis
    uint32_t step_event_count; // The maximum step axis count and number of steps required to complete this block.
    uint8_t direction_bits; // The direction bit set for this block (refers to *_DIRECTION_BIT in config.h)
```

```
// Block condition data to ensure correct execution depending on states and overrides.
```

```

uint8_t condition;    // Block bitflag variable defining block run conditions. Copied from pl_line_data.
#ifdef USE_LINE_NUMBERS
    int32_t line_number; // Block line number for real-time reporting. Copied from pl_line_data.
#endif

// Fields used by the motion planner to manage acceleration. Some of these values may be updated
// by the stepper module during execution of special motion cases for replanning purposes.
float entry_speed_sqr;    // The current planned entry speed at block junction in (mm/min)^2
float max_entry_speed_sqr; // Maximum allowable entry speed based on the minimum of junction limit and
    // neighboring nominal speeds with overrides in (mm/min)^2
float acceleration;    // Axis-limit adjusted line acceleration in (mm/min^2). Does not change.
float millimeters;    // The remaining distance for this block to be executed in (mm).
    // NOTE: This value may be altered by stepper algorithm during execution.

// Stored rate limiting data used by planner when changes occur.
float max_junction_speed_sqr; // Junction entry speed limit based on direction vectors in (mm/min)^2
float rapid_rate;    // Axis-limit adjusted maximum rate for this block direction in (mm/min)
float programmed_rate;    // Programmed rate of this block (mm/min).

#ifdef VARIABLE_SPINDLE
    // Stored spindle speed data used by spindle overrides and resuming methods.
    float spindle_speed; // Block spindle speed. Copied from pl_line_data.
#endif
} plan_block_t;

// Planner data prototype. Must be used when passing new motions to the planner.
typedef struct {
    float feed_rate;    // Desired feed rate for line motion. Value is ignored, if rapid motion.
    float spindle_speed; // Desired spindle speed through line motion.
    uint8_t condition;    // Bitflag variable to indicate planner conditions. See defines above.
#ifdef USE_LINE_NUMBERS
    int32_t line_number; // Desired line number to report when executing.
#endif
} plan_line_data_t;

// Initialize and reset the motion plan subsystem
void plan_reset(); // Reset all
void plan_reset_buffer(); // Reset buffer only.

// Add a new linear movement to the buffer. target[N_AXIS] is the signed, absolute target position
// in millimeters. Feed rate specifies the speed of the motion. If feed rate is inverted, the feed
// rate is taken to mean "frequency" and would complete the operation in 1/feed_rate minutes.
uint8_t plan_buffer_line(float *target, plan_line_data_t *pl_data);

// Called when the current block is no longer needed. Discards the block and makes the memory
// available for new blocks.
void plan_discard_current_block();

// Gets the planner block for the special system motion cases. (Parking/Homing)

```

```

plan_block_t *plan_get_system_motion_block();

// Gets the current block. Returns NULL if buffer empty
plan_block_t *plan_get_current_block();

// Called periodically by step segment buffer. Mostly used internally by planner.
uint8_t plan_next_block_index(uint8_t block_index);

// Called by step segment buffer when computing executing block velocity profile.
float plan_get_exec_block_exit_speed_sqr();

// Called by main program during planner calculations and step segment buffer during initialization.
float plan_compute_profile_nominal_speed(plan_block_t *block);

// Re-calculates buffered motions profile parameters upon a motion-based override change.
void plan_update_velocity_profile_parameters();

// Reset the planner position vector (in steps)
void plan_sync_position();

// Reinitialize plan with a partially completed block
void plan_cycle_reinitialize();

// Returns the number of available blocks are in the planner buffer.
uint8_t plan_get_block_buffer_available();

// Returns the number of active blocks are in the planner buffer.
// NOTE: Deprecated. Not used unless classic status reports are enabled in config.h
uint8_t plan_get_block_buffer_count();

// Returns the status of the block ring buffer. True, if buffer is full.
uint8_t plan_check_full_buffer();

void plan_get_planner_mpos(float *target);

#endif

```

File: print.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
print.c - Functions for formatting output strings
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify

it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#include "grbl.h"
```

```
void printString(const char *s)
{
  while (*s)
    serial_write(*s++);
}
```

```
// Print a string stored in PGM-memory
void printPgmString(const char *s)
{
  char c;
  while ((c = pgm_read_byte_near(s++)))
    serial_write(c);
}
```

```
// void printIntegerInBase(unsigned long n, unsigned long base)
// {
//   unsigned char buf[8 * sizeof(long)]; // Assumes 8-bit chars.
//   unsigned long i = 0;
//   if (n == 0) {
//     serial_write('0');
//     return;
//   }
//   while (n > 0) {
//     buf[i++] = n % base;
//     n /= base;
//   }
//   for (; i > 0; i--)
//     serial_write(buf[i - 1] < 10 ?
//       '0' + buf[i - 1] :
//       'A' + buf[i - 1] - 10);
// }
```

```
// }
```

```
// Prints an uint8 variable in base 10.
```

```
void print_uint8_base10(uint8_t n)
{
    uint8_t digit_a = 0;
    uint8_t digit_b = 0;
    if (n >= 100) { // 100-255
        digit_a = '0' + n % 10;
        n /= 10;
    }
    if (n >= 10) { // 10-99
        digit_b = '0' + n % 10;
        n /= 10;
    }
    serial_write('0' + n);
    if (digit_b) { serial_write(digit_b); }
    if (digit_a) { serial_write(digit_a); }
}
```

```
// Prints an uint8 variable in base 2 with desired number of desired digits.
```

```
void print_uint8_base2_ndigit(uint8_t n, uint8_t digits) {
    unsigned char buf[digits];
    uint8_t i = 0;

    for (; i < digits; i++) {
        buf[i] = n % 2;
        n /= 2;
    }

    for (; i > 0; i--)
        serial_write('0' + buf[i - 1]);
}
```

```
void print_uint32_base10(uint32_t n)
```

```
{
    if (n == 0) {
        serial_write('0');
        return;
    }

    unsigned char buf[10];
    uint8_t i = 0;

    while (n > 0) {
        buf[i++] = n % 10;
        n /= 10;
    }
}
```

```

for (; i > 0; i--)
    serial_write('0' + buff[i-1]);
}

```

```

void printInteger(long n)
{
    if (n < 0) {
        serial_write('-');
        print_uint32_base10(-n);
    } else {
        print_uint32_base10(n);
    }
}

```

// Convert float to string by immediately converting to a long integer, which contains
 // more digits than a float. Number of decimal places, which are tracked by a counter,
 // may be set by the user. The integer is then efficiently converted to a string.
 // NOTE: AVR '%' and '/' integer operations are very efficient. Bitshifting speed-up
 // techniques are actually just slightly slower. Found this out the hard way.

```

void printFloat(float n, uint8_t decimal_places)

```

```

{
    if (n < 0) {
        serial_write('-');
        n = -n;
    }
}

```

```

    uint8_t decimals = decimal_places;

```

```

    while (decimals >= 2) { // Quickly convert values expected to be E0 to E-4.

```

```

        n *= 100;
        decimals -= 2;
    }

```

```

    if (decimals) { n *= 10; }

```

```

    n += 0.5; // Add rounding factor. Ensures carryover through entire value.

```

```

    // Generate digits backwards and store in string.

```

```

    unsigned char buff[13];

```

```

    uint8_t i = 0;

```

```

    uint32_t a = (long)n;

```

```

    while(a > 0) {
        buff[i++] = (a % 10) + '0'; // Get digit
        a /= 10;
    }

```

```

    while (i < decimal_places) {
        buff[i++] = '0'; // Fill in zeros to decimal point for (n < 1)
    }

```

```

    if (i == decimal_places) { // Fill in leading zero, if needed.

```

```

        buff[i++] = '0';
    }
}

```

```

// Print the generated string.
for (; i > 0; i--) {
    if (i == decimal_places) { serial_write('.'); } // Insert decimal point in right place.
    serial_write(buff[i-1]);
}
}

// Floating value printing handlers for special variables types used in Grbl and are defined
// in the config.h.
// - CoordValue: Handles all position or coordinate values in inches or mm reporting.
// - RateValue: Handles feed rate and current velocity in inches or mm reporting.
void printFloat_CoordValue(float n) {
    if (bit_istrue(settings.flags,BITFLAG_REPORT_INCHES)) {
        printFloat(n*INCH_PER_MM,N_DECIMAL_COORDVALUE_INCH);
    } else {
        printFloat(n,N_DECIMAL_COORDVALUE_MM);
    }
}

void printFloat_RateValue(float n) {
    if (bit_istrue(settings.flags,BITFLAG_REPORT_INCHES)) {
        printFloat(n*INCH_PER_MM,N_DECIMAL_RATEVALUE_INCH);
    } else {
        printFloat(n,N_DECIMAL_RATEVALUE_MM);
    }
}

// Debug tool to print free memory in bytes at the called point.
// NOTE: Keep commented unless using. Part of this function always gets compiled in.
// void printFreeMemory()
// {
//     extern int __heap_start, *__brkval;
//     uint16_t free; // Up to 64k values.
//     free = (int) &free - (__brkval == 0 ? (int) &__heap_start : (int) __brkval);
//     printInteger((int32_t)free);
//     printString(" ");
// }

```

File: print.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
print.h - Functions for formatting output strings
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#ifndef print_h
#define print_h
```

```
void printString(const char *s);
```

```
void printPgmString(const char *s);
```

```
void printInteger(long n);
```

```
void print_uint32_base10(uint32_t n);
```

```
// Prints an uint8 variable in base 10.
```

```
void print_uint8_base10(uint8_t n);
```

```
// Prints an uint8 variable in base 2 with desired number of desired digits.
```

```
void print_uint8_base2_ndigit(uint8_t n, uint8_t digits);
```

```
void printFloat(float n, uint8_t decimal_places);
```

```
// Floating value printing handlers for special variables types used in Grbl.
```

```
// - CoordValue: Handles all position or coordinate values in inches or mm reporting.
```

```
// - RateValue: Handles feed rate and current velocity in inches or mm reporting.
```

```
void printFloat_CoordValue(float n);
```

```
void printFloat_RateValue(float n);
```

```
// Debug tool to print free memory in bytes at the called point. Not used otherwise.
```

```
void printFreeMemory();
```

```
#endif
```

File: probe.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

probe.c - code pertaining to probing methods

Part of Grbl

Copyright (c) 2014-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

#include "grbl.h"

// Inverts the probe pin state depending on user settings and probing cycle mode.
uint8_t probe_invert_mask;

// Probe pin initialization routine.

void probe_init()

{

PROBE_DDR &= ~(PROBE_MASK); // Configure as input pins

#ifdef DISABLE_PROBE_PIN_PULL_UP

PROBE_PORT &= ~(PROBE_MASK); // Normal low operation. Requires external pull-down.

#else

PROBE_PORT |= PROBE_MASK; // Enable internal pull-up resistors. Normal high operation.

#endif

probe_configure_invert_mask(false); // Initialize invert mask.

}

// Called by probe_init() and the mc_probe() routines. Sets up the probe pin invert mask to appropriately set the pin logic according to setting for normal-high/normal-low operation and the probing cycle modes for toward-workpiece/away-from-workpiece.

void probe_configure_invert_mask(uint8_t is_probe_away)

{

probe_invert_mask = 0; // Initialize as zero.

if (bit_isfalse(settings.flags,BITFLAG_INVERT_PROBE_PIN)) { probe_invert_mask ^= PROBE_MASK; }

if (is_probe_away) { probe_invert_mask ^= PROBE_MASK; }

```
}
```

```
// Returns the probe pin state. Triggered = true. Called by gcode parser and probe state monitor.  
uint8_t probe_get_state() { return((PROBE_PIN & PROBE_MASK) ^ probe_invert_mask); }
```

```
// Monitors probe pin state and records the system position when detected. Called by the  
// stepper ISR per ISR tick.
```

```
// NOTE: This function must be extremely efficient as to not bog down the stepper ISR.
```

```
void probe_state_monitor()  
{  
  if (probe_get_state()) {  
    sys_probe_state = PROBE_OFF;  
    memcpy(sys_probe_position, sys_position, sizeof(sys_position));  
    bit_true(sys_rt_exec_state, EXEC_MOTION_CANCEL);  
  }  
}
```

File: probe.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```
/*
```

```
probe.h - code pertaining to probing methods
```

```
Part of Grbl
```

Copyright (c) 2014-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```
*/
```

```
#ifndef probe_h
```

```
#define probe_h
```

```
// Values that define the probing state machine.
```

```
#define PROBE_OFF 0 // Probing disabled or not in use. (Must be zero.)
```

```
#define PROBE_ACTIVE 1 // Actively watching the input pin.
```

```

// Probe pin initialization routine.
void probe_init();

// Called by probe_init() and the mc_probe() routines. Sets up the probe pin invert mask to
// appropriately set the pin logic according to setting for normal-high/normal-low operation
// and the probing cycle modes for toward-workpiece/away-from-workpiece.
void probe_configure_invert_mask(uint8_t is_probe_away);

// Returns probe pin state. Triggered = true. Called by gcode parser and probe state monitor.
uint8_t probe_get_state();

// Monitors probe pin state and records the system position when detected. Called by the
// stepper ISR per ISR tick.
void probe_state_monitor();

#endif

```

File: protocol.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
  protocol.c - controls Grbl execution protocol and procedures
  Part of Grbl

  Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
  Copyright (c) 2009-2011 Simen Svale Skogsrud

  Grbl is free software: you can redistribute it and/or modify
  it under the terms of the GNU General Public License as published by
  the Free Software Foundation, either version 3 of the License, or
  (at your option) any later version.

  Grbl is distributed in the hope that it will be useful,
  but WITHOUT ANY WARRANTY; without even the implied warranty of
  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
  GNU General Public License for more details.

  You should have received a copy of the GNU General Public License
  along with Grbl. If not, see <http://www.gnu.org/licenses/>.
*/

#include "grbl.h"

// Define line flags. Includes comment type tracking and line overflow detection.
#define LINE_FLAG_OVERFLOW bit(0)
#define LINE_FLAG_COMMENT_PARENTHESES bit(1)
#define LINE_FLAG_COMMENT_SEMICOLON bit(2)

```

```
static char line[LINE_BUFFER_SIZE]; // Line to be executed. Zero-terminated.
```

```
static void protocol_exec_rt_suspend();
```

```
/*
  GRBL PRIMARY LOOP:
*/
void protocol_main_loop()
{
  // Perform some machine checks to make sure everything is good to go.
#ifdef CHECK_LIMITS_AT_INIT
  if (bit_istrue(settings.flags, BITFLAG_HARD_LIMIT_ENABLE)) {
    if (limits_get_state()) {
      sys.state = STATE_ALARM; // Ensure alarm state is active.
      report_feedback_message(MESSAGE_CHECK_LIMITS);
    }
  }
#endif
  // Check for and report alarm state after a reset, error, or an initial power up.
  // NOTE: Sleep mode disables the stepper drivers and position can't be guaranteed.
  // Re-initialize the sleep state as an ALARM mode to ensure user homes or acknowledges.
  if (sys.state & (STATE_ALARM | STATE_SLEEP)) {
    report_feedback_message(MESSAGE_ALARM_LOCK);
    sys.state = STATE_ALARM; // Ensure alarm state is set.
  } else {
    // Check if the safety door is open.
    sys.state = STATE_IDLE;
    if (system_check_safety_door_ajar()) {
      bit_true(sys_rt_exec_state, EXEC_SAFETY_DOOR);
      protocol_execute_realtime(); // Enter safety door mode. Should return as IDLE state.
    }
    // All systems go!
    system_execute_startup(line); // Execute startup script.
  }

  // -----
  // Primary loop! Upon a system abort, this exits back to main() to reset the system.
  // This is also where Grbl idles while waiting for something to do.
  // -----

  uint8_t line_flags = 0;
  uint8_t char_counter = 0;
  uint8_t c;
  for (;;) {

    // Process one line of incoming serial data, as the data becomes available. Performs an
    // initial filtering by removing spaces and comments and capitalizing all letters.
    while((c = serial_read()) != SERIAL_NO_DATA) {
      if ((c == '\n') || (c == '\r')) { // End of line reached
```

```

protocol_execute_realtime(); // Runtime command check point.
if (sys.abort) { return; } // Bail to calling function upon system abort

line[char_counter] = 0; // Set string termination character.
#ifdef REPORT_ECHO_LINE_RECEIVED
    report_echo_line_received(line);
#endif

// Direct and execute one line of formatted input, and report status of execution.
if (line_flags & LINE_FLAG_OVERFLOW) {
    // Report line overflow error.
    report_status_message(STATUS_OVERFLOW);
} else if (line[0] == 0) {
    // Empty or comment line. For syncing purposes.
    report_status_message(STATUS_OK);
} else if (line[0] == '$') {
    // Grbl '$' system command
    report_status_message(system_execute_line(line));
} else if (sys.state & (STATE_ALARM | STATE_JOG)) {
    // Everything else is gcode. Block if in alarm or jog mode.
    report_status_message(STATUS_SYSTEM_GC_LOCK);
} else {
    // Parse and execute g-code block.
    report_status_message(gc_execute_line(line));
}

// Reset tracking data for next line.
line_flags = 0;
char_counter = 0;

} else {

    if (line_flags) {
        // Throw away all (except EOL) comment characters and overflow characters.
        if (c == ')') {
            // End of ')' comment. Resume line allowed.
            if (line_flags & LINE_FLAG_COMMENT_PARENTHESSES) { line_flags &= ~(LINE_FLAG_COMMENT_PARENTHESSES); }
        }
    } else {
        if (c <= ' ') {
            // Throw away whitespace and control characters
        } else if (c == '/') {
            // Block delete NOT SUPPORTED. Ignore character.
            // NOTE: If supported, would simply need to check the system if block delete is enabled.
        } else if (c == '(') {
            // Enable comments flag and ignore all characters until ')' or EOL.
            // NOTE: This doesn't follow the NIST definition exactly, but is good enough for now.
            // In the future, we could simply remove the items within the comments, but retain the
            // comment control characters, so that the g-code parser can error-check it.
            line_flags |= LINE_FLAG_COMMENT_PARENTHESSES;
        } else if (c == ';') {

```

```

    // NOTE: ';' comment to EOL is a LinuxCNC definition. Not NIST.
    line_flags |= LINE_FLAG_COMMENT_SEMICOLON;
// TODO: Install '%' feature
// } else if (c == '%') {
    // Program start-end percent sign NOT SUPPORTED.
    // NOTE: This maybe installed to tell Grbl when a program is running vs manual input,
    // where, during a program, the system auto-cycle start will continue to execute
    // everything until the next '%' sign. This will help fix resuming issues with certain
    // functions that empty the planner buffer to execute its task on-time.
} else if (char_counter >= (LINE_BUFFER_SIZE-1)) {
    // Detect line buffer overflow and set flag.
    line_flags |= LINE_FLAG_OVERFLOW;
} else if (c >= 'a' && c <= 'z') { // Upcase lowercase
    line[char_counter++] = c-'a'+'A';
} else {
    line[char_counter++] = c;
}
}

}

// If there are no more characters in the serial read buffer to be processed and executed,
// this indicates that g-code streaming has either filled the planner buffer or has
// completed. In either case, auto-cycle start, if enabled, any queued moves.
protocol_auto_cycle_start();

protocol_execute_realtime(); // Runtime command check point.
if (sys.abort) { return; } // Bail to main() program loop to reset system.
}

return; /* Never reached */
}

// Block until all buffered steps are executed or in a cycle state. Works with feed hold
// during a synchronize call, if it should happen. Also, waits for clean cycle end.
void protocol_buffer_synchronize()
{
    // If system is queued, ensure cycle resumes if the auto start flag is present.
    protocol_auto_cycle_start();
    do {
        protocol_execute_realtime(); // Check and execute run-time commands
        if (sys.abort) { return; } // Check for system abort
    } while (plan_get_current_block() || (sys.state == STATE_CYCLE));
}

// Auto-cycle start triggers when there is a motion ready to execute and if the main program is not
// actively parsing commands.
// NOTE: This function is called from the main loop, buffer sync, and mc_line() only and executes

```

```

// when one of these conditions exist respectively: There are no more blocks sent (i.e. streaming
// is finished, single commands), a command that needs to wait for the motions in the buffer to
// execute calls a buffer sync, or the planner buffer is full and ready to go.
void protocol_auto_cycle_start()
{
    if (plan_get_current_block() != NULL) { // Check if there are any blocks in the buffer.
        system_set_exec_state_flag(EXEC_CYCLE_START); // If so, execute them!
    }
}

// This function is the general interface to Grbl's real-time command execution system. It is called
// from various check points in the main program, primarily where there may be a while loop waiting
// for a buffer to clear space or any point where the execution time from the last check point may
// be more than a fraction of a second. This is a way to execute realtime commands asynchronously
// (aka multitasking) with grbl's g-code parsing and planning functions. This function also serves
// as an interface for the interrupts to set the system realtime flags, where only the main program
// handles them, removing the need to define more computationally-expensive volatile variables. This
// also provides a controlled way to execute certain tasks without having two or more instances of
// the same task, such as the planner recalculating the buffer upon a feedhold or overrides.
// NOTE: The sys_rt_exec_state variable flags are set by any process, step or serial interrupts, pinouts,
// limit switches, or the main program.
void protocol_execute_realtime()
{
    protocol_exec_rt_system();
    if (sys.suspend) { protocol_exec_rt_suspend(); }
}

// Executes run-time commands, when required. This function primarily operates as Grbl's state
// machine and controls the various real-time features Grbl has to offer.
// NOTE: Do not alter this unless you know exactly what you are doing!
void protocol_exec_rt_system()
{
    uint8_t rt_exec; // Temp variable to avoid calling volatile multiple times.
    rt_exec = sys_rt_exec_alarm; // Copy volatile sys_rt_exec_alarm.
    if (rt_exec) { // Enter only if any bit flag is true
        // System alarm. Everything has shutdown by something that has gone severely wrong. Report
        // the source of the error to the user. If critical, Grbl disables by entering an infinite
        // loop until system reset/abort.
        sys.state = STATE_ALARM; // Set system alarm state
        report_alarm_message(rt_exec);
        // Halt everything upon a critical event flag. Currently hard and soft limits flag this.
        if ((rt_exec == EXEC_ALARM_HARD_LIMIT) || (rt_exec == EXEC_ALARM_SOFT_LIMIT)) {
            report_feedback_message(MESSAGE_CRITICAL_EVENT);
            system_clear_exec_state_flag(EXEC_RESET); // Disable any existing reset
            do {
                // Block everything, except reset and status reports, until user issues reset or power
                // cycles. Hard limits typically occur while unattended or not paying attention. Gives
                // the user and a GUI time to do what is needed before resetting, like killing the
                // incoming stream. The same could be said about soft limits. While the position is not

```

```

    // lost, continued streaming could cause a serious crash if by chance it gets executed.
} while (bit_isspace(sys_rt_exec_state, EXEC_RESET));
}
system_clear_exec_alarm(); // Clear alarm
}

rt_exec = sys_rt_exec_state; // Copy volatile sys_rt_exec_state.
if (rt_exec) {

    // Execute system abort.
    if (rt_exec & EXEC_RESET) {
        sys.abort = true; // Only place this is set true.
        return; // Nothing else to do but exit.
    }

    // Execute and serial print status
    if (rt_exec & EXEC_STATUS_REPORT) {
        report_realtime_status();
        system_clear_exec_state_flag(EXEC_STATUS_REPORT);
    }

    // NOTE: Once hold is initiated, the system immediately enters a suspend state to block all
    // main program processes until either reset or resumed. This ensures a hold completes safely.
    if (rt_exec & (EXEC_MOTION_CANCEL | EXEC_FEED_HOLD | EXEC_SAFETY_DOOR | EXEC_SLEEP)) {

        // State check for allowable states for hold methods.
        if (!(sys.state & (STATE_ALARM | STATE_CHECK_MODE))) {

            // If in CYCLE or JOG states, immediately initiate a motion HOLD.
            if (sys.state & (STATE_CYCLE | STATE_JOG)) {
                if (!(sys.suspend & (SUSPEND_MOTION_CANCEL | SUSPEND_JOG_CANCEL))) { // Block, if already holding.
                    st_update_plan_block_parameters(); // Notify stepper module to recompute for hold deceleration.
                    sys.step_control = STEP_CONTROL_EXECUTE_HOLD; // Initiate suspend state with active flag.
                    if (sys.state == STATE_JOG) { // Jog cancelled upon any hold event, except for sleeping.
                        if (!(rt_exec & EXEC_SLEEP)) { sys.suspend |= SUSPEND_JOG_CANCEL; }
                    }
                }
            }
        }

        // If IDLE, Grbl is not in motion. Simply indicate suspend state and hold is complete.
        if (sys.state == STATE_IDLE) { sys.suspend = SUSPEND_HOLD_COMPLETE; }

        // Execute and flag a motion cancel with deceleration and return to idle. Used primarily by probing cycle
        // to halt and cancel the remainder of the motion.
        if (rt_exec & EXEC_MOTION_CANCEL) {
            // MOTION_CANCEL only occurs during a CYCLE, but a HOLD and SAFETY_DOOR may be initiated beforehand
            // to hold the CYCLE. Motion cancel is valid for a single planner block motion only, while jog cancel
            // will handle and clear multiple planner block motions.
            if (!(sys.state & STATE_JOG)) { sys.suspend |= SUSPEND_MOTION_CANCEL; } // NOTE: State is STATE_CYCLE.
        }

        // Execute a feed hold with deceleration, if required. Then, suspend system.

```

```

if (rt_exec & EXEC_FEED_HOLD) {
    // Block SAFETY_DOOR, JOG, and SLEEP states from changing to HOLD state.
    if (!(sys.state & (STATE_SAFETY_DOOR | STATE_JOG | STATE_SLEEP))) { sys.state = STATE_HOLD; }
}

// Execute a safety door stop with a feed hold and disable spindle/coolant.
// NOTE: Safety door differs from feed holds by stopping everything no matter state, disables powered
// devices (spindle/coolant), and blocks resuming until switch is re-engaged.
if (rt_exec & EXEC_SAFETY_DOOR) {
    report_feedback_message(MESSAGE_SAFETY_DOOR_AJAR);
    // If jogging, block safety door methods until jog cancel is complete. Just flag that it happened.
    if (!(sys.suspend & SUSPEND_JOG_CANCEL)) {
        // Check if the safety re-opened during a restore parking motion only. Ignore if
        // already retracting, parked or in sleep state.
        if (sys.state == STATE_SAFETY_DOOR) {
            if (sys.suspend & SUSPEND_INITIATE_RESTORE) { // Actively restoring
                #ifdef PARKING_ENABLE
                    // Set hold and reset appropriate control flags to restart parking sequence.
                    if (sys.step_control & STEP_CONTROL_EXECUTE_SYS_MOTION) {
                        st_update_plan_block_parameters(); // Notify stepper module to recompute for hold deceleration.
                        sys.step_control = (STEP_CONTROL_EXECUTE_HOLD | STEP_CONTROL_EXECUTE_SYS_MOTION);
                        sys.suspend &= ~(SUSPEND_HOLD_COMPLETE);
                    } // else NO_MOTION is active.
                #endif
                sys.suspend &= ~(SUSPEND_RETRACT_COMPLETE | SUSPEND_INITIATE_RESTORE | SUSPEND_RESTORE);
                sys.suspend |= SUSPEND_RESTART_RETRACT;
            }
        }
        if (sys.state != STATE_SLEEP) { sys.state = STATE_SAFETY_DOOR; }
    }
    // NOTE: This flag doesn't change when the door closes, unlike sys.state. Ensures any parking motions
    // are executed if the door switch closes and the state returns to HOLD.
    sys.suspend |= SUSPEND_SAFETY_DOOR_AJAR;
}

}

if (rt_exec & EXEC_SLEEP) {
    if (sys.state == STATE_ALARM) { sys.suspend |= (SUSPEND_RETRACT_COMPLETE | SUSPEND_HOLD_COMPLETE);
    sys.state = STATE_SLEEP;
}

}

system_clear_exec_state_flag((EXEC_MOTION_CANCEL | EXEC_FEED_HOLD | EXEC_SAFETY_DOOR | EXEC_SLEEP));

// Execute a cycle start by starting the stepper interrupt to begin executing the blocks in queue.
if (rt_exec & EXEC_CYCLE_START) {
    // Block if called at same time as the hold commands: feed hold, motion cancel, and safety door.
    // Ensures auto-cycle-start doesn't resume a hold without an explicit user-input.
    if (!(rt_exec & (EXEC_FEED_HOLD | EXEC_MOTION_CANCEL | EXEC_SAFETY_DOOR))) {
        // Resume door state when parking motion has retracted and door has been closed.
    }
}

```

```

if ((sys.state == STATE_SAFETY_DOOR) && !(sys.suspend & SUSPEND_SAFETY_DOOR_AJAR)) {
    if (sys.suspend & SUSPEND_RESTORE_COMPLETE) {
        sys.state = STATE_IDLE; // Set to IDLE to immediately resume the cycle.
    } else if (sys.suspend & SUSPEND_RETRACT_COMPLETE) {
        // Flag to re-energize powered components and restore original position, if disabled by SAFETY_DOOR.
        // NOTE: For a safety door to resume, the switch must be closed, as indicated by HOLD state, and
        // the retraction execution is complete, which implies the initial feed hold is not active. To
        // restore normal operation, the restore procedures must be initiated by the following flag. Once,
        // they are complete, it will call CYCLE_START automatically to resume and exit the suspend.
        sys.suspend |= SUSPEND_INITIATE_RESTORE;
    }
}
// Cycle start only when IDLE or when a hold is complete and ready to resume.
if ((sys.state == STATE_IDLE) || ((sys.state & STATE_HOLD) && (sys.suspend & SUSPEND_HOLD_COMPLETE))) {
    if (sys.state == STATE_HOLD && sys.spindle_stop_ovr) {
        sys.spindle_stop_ovr |= SPINDLE_STOP_OVR_RESTORE_CYCLE; // Set to restore in suspend routine and cycle s
    } else {
        // Start cycle only if queued motions exist in planner buffer and the motion is not canceled.
        sys.step_control = STEP_CONTROL_NORMAL_OP; // Restore step control to normal operation
        if (plan_get_current_block() && bit_isfalse(sys.suspend, SUSPEND_MOTION_CANCEL)) {
            sys.suspend = SUSPEND_DISABLE; // Break suspend state.
            sys.state = STATE_CYCLE;
            st_prep_buffer(); // Initialize step segment buffer before beginning cycle.
            st_wake_up();
        } else { // Otherwise, do nothing. Set and resume IDLE state.
            sys.suspend = SUSPEND_DISABLE; // Break suspend state.
            sys.state = STATE_IDLE;
        }
    }
}
}
}
system_clear_exec_state_flag(EXEC_CYCLE_START);
}

if (rt_exec & EXEC_CYCLE_STOP) {
    // Reinitializes the cycle plan and stepper system after a feed hold for a resume. Called by
    // realtime command execution in the main program, ensuring that the planner re-plans safely.
    // NOTE: Bresenham algorithm variables are still maintained through both the planner and stepper
    // cycle reinitializations. The stepper path should continue exactly as if nothing has happened.
    // NOTE: EXEC_CYCLE_STOP is set by the stepper subsystem when a cycle or feed hold completes.
    if ((sys.state & (STATE_HOLD|STATE_SAFETY_DOOR|STATE_SLEEP)) && !(sys.soft_limit) && !(sys.suspend & SUSPEND_HOLD_COMPLETE)) {
        // Hold complete. Set to indicate ready to resume. Remain in HOLD or DOOR states until user
        // has issued a resume command or reset.
        plan_cycle_reinitialize();
        if (sys.step_control & STEP_CONTROL_EXECUTE_HOLD) { sys.suspend |= SUSPEND_HOLD_COMPLETE; }
        bit_false(sys.step_control, (STEP_CONTROL_EXECUTE_HOLD | STEP_CONTROL_EXECUTE_SYS_MOTION));
    } else {
        // Motion complete. Includes CYCLE/JOG/HOMING states and jog cancel/motion cancel/soft limit events.
        // NOTE: Motion and jog cancel both immediately return to idle after the hold completes.
        if (sys.suspend & SUSPEND_JOG_CANCEL) { // For jog cancel, flush buffers and sync positions.
            sys.step_control = STEP_CONTROL_NORMAL_OP;
        }
    }
}

```

```

    plan_reset();
    st_reset();
    gc_sync_position();
    plan_sync_position();
}
if (sys.suspend & SUSPEND_SAFETY_DOOR_AJAR) { // Only occurs when safety door opens during jog.
    sys.suspend &= ~(SUSPEND_JOG_CANCEL);
    sys.suspend |= SUSPEND_HOLD_COMPLETE;
    sys.state = STATE_SAFETY_DOOR;
} else {
    sys.suspend = SUSPEND_DISABLE;
    sys.state = STATE_IDLE;
}
}
system_clear_exec_state_flag(EXEC_CYCLE_STOP);
}
}

// Execute overrides.
rt_exec = sys_rt_exec_motion_override; // Copy volatile sys_rt_exec_motion_override
if (rt_exec) {
    system_clear_exec_motion_overrides(); // Clear all motion override flags.

    uint8_t new_f_override = sys.f_override;
    if (rt_exec & EXEC_FEED_OVR_RESET) { new_f_override = DEFAULT_FEED_OVERRIDE; }
    if (rt_exec & EXEC_FEED_OVR_COARSE_PLUS) { new_f_override += FEED_OVERRIDE_COARSE_INCREMENT; }
    if (rt_exec & EXEC_FEED_OVR_COARSE_MINUS) { new_f_override -= FEED_OVERRIDE_COARSE_INCREMENT; }
    if (rt_exec & EXEC_FEED_OVR_FINE_PLUS) { new_f_override += FEED_OVERRIDE_FINE_INCREMENT; }
    if (rt_exec & EXEC_FEED_OVR_FINE_MINUS) { new_f_override -= FEED_OVERRIDE_FINE_INCREMENT; }
    new_f_override = min(new_f_override, MAX_FEED_RATE_OVERRIDE);
    new_f_override = max(new_f_override, MIN_FEED_RATE_OVERRIDE);

    uint8_t new_r_override = sys.r_override;
    if (rt_exec & EXEC_RAPID_OVR_RESET) { new_r_override = DEFAULT_RAPID_OVERRIDE; }
    if (rt_exec & EXEC_RAPID_OVR_MEDIUM) { new_r_override = RAPID_OVERRIDE_MEDIUM; }
    if (rt_exec & EXEC_RAPID_OVR_LOW) { new_r_override = RAPID_OVERRIDE_LOW; }

    if ((new_f_override != sys.f_override) || (new_r_override != sys.r_override)) {
        sys.f_override = new_f_override;
        sys.r_override = new_r_override;
        sys.report_ovr_counter = 0; // Set to report change immediately
        plan_update_velocity_profile_parameters();
        plan_cycle_reinitialize();
    }
}

rt_exec = sys_rt_exec_accessory_override;
if (rt_exec) {
    system_clear_exec_accessory_overrides(); // Clear all accessory override flags.

    // NOTE: Unlike motion overrides, spindle overrides do not require a planner reinitialization.

```

```

uint8_t last_s_override = sys.spindle_speed_ovr;
if (rt_exec & EXEC_SPINDLE_OVR_RESET) { last_s_override = DEFAULT_SPINDLE_SPEED_OVERRIDE; }
if (rt_exec & EXEC_SPINDLE_OVR_COARSE_PLUS) { last_s_override += SPINDLE_OVERRIDE_COARSE_INCREMENT; }
if (rt_exec & EXEC_SPINDLE_OVR_COARSE_MINUS) { last_s_override -= SPINDLE_OVERRIDE_COARSE_INCREMENT; }
if (rt_exec & EXEC_SPINDLE_OVR_FINE_PLUS) { last_s_override += SPINDLE_OVERRIDE_FINE_INCREMENT; }
if (rt_exec & EXEC_SPINDLE_OVR_FINE_MINUS) { last_s_override -= SPINDLE_OVERRIDE_FINE_INCREMENT; }
last_s_override = min(last_s_override, MAX_SPINDLE_SPEED_OVERRIDE);
last_s_override = max(last_s_override, MIN_SPINDLE_SPEED_OVERRIDE);

if (last_s_override != sys.spindle_speed_ovr) {
    sys.spindle_speed_ovr = last_s_override;
    // NOTE: Spindle speed overrides during HOLD state are taken care of by suspend function.
    if (sys.state == STATE_IDLE) { spindle_set_state(gc_state.modal.spindle, gc_state.spindle_speed); }
    else { bit_true(sys.step_control, STEP_CONTROL_UPDATE_SPINDLE_PWM); }
    sys.report_ovr_counter = 0; // Set to report change immediately
}

if (rt_exec & EXEC_SPINDLE_OVR_STOP) {
    // Spindle stop override allowed only while in HOLD state.
    // NOTE: Report counters are set in spindle_set_state() when spindle stop is executed.
    if (sys.state == STATE_HOLD) {
        if (!(sys.spindle_stop_ovr)) { sys.spindle_stop_ovr = SPINDLE_STOP_OVR_INITIATE; }
        else if (sys.spindle_stop_ovr & SPINDLE_STOP_OVR_ENABLED) { sys.spindle_stop_ovr |= SPINDLE_STOP_OVR_F
    }
}

// NOTE: Since coolant state always performs a planner sync whenever it changes, the current
// run state can be determined by checking the parser state.
// NOTE: Coolant overrides only operate during IDLE, CYCLE, HOLD, and JOG states. Ignored otherwise.
if (rt_exec & (EXEC_COOLANT_FLOOD_OVR_TOGGLE | EXEC_COOLANT_MIST_OVR_TOGGLE)) {
    if ((sys.state == STATE_IDLE) || (sys.state & (STATE_CYCLE | STATE_HOLD | STATE_JOG))) {
        uint8_t coolant_state = gc_state.modal.coolant;
        #ifdef ENABLE_M7
        if (rt_exec & EXEC_COOLANT_MIST_OVR_TOGGLE) {
            if (coolant_state & COOLANT_MIST_ENABLE) { bit_false(coolant_state, COOLANT_MIST_ENABLE); }
            else { coolant_state |= COOLANT_MIST_ENABLE; }
        }
        if (rt_exec & EXEC_COOLANT_FLOOD_OVR_TOGGLE) {
            if (coolant_state & COOLANT_FLOOD_ENABLE) { bit_false(coolant_state, COOLANT_FLOOD_ENABLE); }
            else { coolant_state |= COOLANT_FLOOD_ENABLE; }
        }
        #else
        if (coolant_state & COOLANT_FLOOD_ENABLE) { bit_false(coolant_state, COOLANT_FLOOD_ENABLE); }
        else { coolant_state |= COOLANT_FLOOD_ENABLE; }
        #endif
        coolant_set_state(coolant_state); // Report counter set in coolant_set_state().
        gc_state.modal.coolant = coolant_state;
    }
}
}
}

```

```

#ifdef DEBUG
    if (sys_rt_exec_debug) {
        report_realtime_debug();
        sys_rt_exec_debug = 0;
    }
#endif

// Reload step segment buffer
if (sys.state & (STATE_CYCLE | STATE_HOLD | STATE_SAFETY_DOOR | STATE_HOMING | STATE_SLEEP | STATE_
    st_prep_buffer());
}

}

// Handles Grbl system suspend procedures, such as feed hold, safety door, and parking motion.
// The system will enter this loop, create local variables for suspend tasks, and return to
// whatever function that invoked the suspend, such that Grbl resumes normal operation.
// This function is written in a way to promote custom parking motions. Simply use this as a
// template
static void protocol_exec_rt_suspend()
{
#ifdef PARKING_ENABLE
    // Declare and initialize parking local variables
    float restore_target[N_AXIS];
    float parking_target[N_AXIS];
    float retract_waypoint = PARKING_PULLOUT_INCREMENT;
    plan_line_data_t plan_data;
    plan_line_data_t *pl_data = &plan_data;
    memset(pl_data, 0, sizeof(plan_line_data_t));
    pl_data->condition = (PL_COND_FLAG_SYSTEM_MOTION | PL_COND_FLAG_NO_FEED_OVERRIDE);
#ifdef USE_LINE_NUMBERS
    pl_data->line_number = PARKING_MOTION_LINE_NUMBER;
#endif
#endif

plan_block_t *block = plan_get_current_block();
uint8_t restore_condition;
#ifdef VARIABLE_SPINDLE
    float restore_spindle_speed;
    if (block == NULL) {
        restore_condition = (gc_state.modal.spindle | gc_state.modal.coolant);
        restore_spindle_speed = gc_state.spindle_speed;
    } else {
        restore_condition = (block->condition & PL_COND_SPINDLE_MASK) | coolant_get_state();
        restore_spindle_speed = block->spindle_speed;
    }
#endif
#ifdef DISABLE_LASER_DURING_HOLD
    if (bit_istrue(settings.flags, BITFLAG_LASER_MODE)) {
        system_set_exec_accessory_override_flag(EXEC_SPINDLE_OVR_STOP);
    }
}

```

```

#endif
#else
    if (block == NULL) { restore_condition = (gc_state.modal.spindle | gc_state.modal.coolant); }
    else { restore_condition = (block->condition & PL_COND_SPINDLE_MASK) | coolant_get_state(); }
#endif

while (sys.suspend) {

    if (sys.abort) { return; }

    // Block until initial hold is complete and the machine has stopped motion.
    if (sys.suspend & SUSPEND_HOLD_COMPLETE) {

        // Parking manager. Handles de/re-energizing, switch state checks, and parking motions for
        // the safety door and sleep states.
        if (sys.state & (STATE_SAFETY_DOOR | STATE_SLEEP)) {

            // Handles retraction motions and de-energizing.
            if (bit_istrue(sys.suspend,SUSPEND_RETRACT_COMPLETE)) {

                // Ensure any prior spindle stop override is disabled at start of safety door routine.
                sys.spindle_stop_ovr = SPINDLE_STOP_OVR_DISABLED;

                #ifndef PARKING_ENABLE

                spindle_set_state(SPINDLE_DISABLE,0.0); // De-energize
                coolant_set_state(COOLANT_DISABLE); // De-energize

            #else
            ■■■■■■
            // Get current position and store restore location and spindle retract waypoint.
            system_convert_array_steps_to_mpos(parking_target,sys_position);
            if (bit_istrue(sys.suspend,SUSPEND_RESTART_RETRACT)) {
                memcpy(restore_target,parking_target,sizeof(parking_target));
                retract_waypoint += restore_target[PARKING_AXIS];
                retract_waypoint = min(retract_waypoint,PARKING_TARGET);
            }

            // Execute slow pull-out parking retract motion. Parking requires homing enabled, the
            // current location not exceeding the parking target location, and laser mode disabled.
            // NOTE: State is will remain DOOR, until the de-energizing and retract is complete.
            #ifdef ENABLE_PARKING_OVERRIDE_CONTROL
            if ((bit_istrue(settings.flags,BITFLAG_HOMING_ENABLE)) &&
                (parking_target[PARKING_AXIS] < PARKING_TARGET) &&
                bit_istrue(settings.flags,BITFLAG_LASER_MODE) &&
                (sys.override_ctrl == OVERRIDE_PARKING_MOTION)) {

            #else
            if ((bit_istrue(settings.flags,BITFLAG_HOMING_ENABLE)) &&
                (parking_target[PARKING_AXIS] < PARKING_TARGET) &&
                bit_istrue(settings.flags,BITFLAG_LASER_MODE)) {

            #endif

```

```

// Retract spindle by pullout distance. Ensure retraction motion moves away from
// the workpiece and waypoint motion doesn't exceed the parking target location.
if (parking_target[PARKING_AXIS] < retract_waypoint) {
    parking_target[PARKING_AXIS] = retract_waypoint;
    pl_data->feed_rate = PARKING_PULLOUT_RATE;
    pl_data->condition |= (restore_condition & PL_COND_ACCESSORY_MASK); // Retain accessory state
    pl_data->spindle_speed = restore_spindle_speed;
    mc_parking_motion(parking_target, pl_data);
}

// NOTE: Clear accessory state after retract and after an aborted restore motion.
pl_data->condition = (PL_COND_FLAG_SYSTEM_MOTION|PL_COND_FLAG_NO_FEED_OVERRIDE);
pl_data->spindle_speed = 0.0;
spindle_set_state(SPINdle_DISABLE,0.0); // De-energize
coolant_set_state(COOLANT_DISABLE); // De-energize

// Execute fast parking retract motion to parking target location.
if (parking_target[PARKING_AXIS] < PARKING_TARGET) {
    parking_target[PARKING_AXIS] = PARKING_TARGET;
    pl_data->feed_rate = PARKING_RATE;
    mc_parking_motion(parking_target, pl_data);
}

} else {

    // Parking motion not possible. Just disable the spindle and coolant.
    // NOTE: Laser mode does not start a parking motion to ensure the laser stops immediately.
    spindle_set_state(SPINdle_DISABLE,0.0); // De-energize
    coolant_set_state(COOLANT_DISABLE); // De-energize

}

#endif

sys.suspend &= ~(SUSPEND_RESTART_RETRACT);
sys.suspend |= SUSPEND_RETRACT_COMPLETE;

} else {

if (sys.state == STATE_SLEEP) {
    report_feedback_message(MESSAGE_SLEEP_MODE);
    // Spindle and coolant should already be stopped, but do it again just to be sure.
    spindle_set_state(SPINdle_DISABLE,0.0); // De-energize
    coolant_set_state(COOLANT_DISABLE); // De-energize
    st_go_idle(); // Disable steppers
    while (!(sys.abort)) { protocol_exec_rt_system(); } // Do nothing until reset.
    return; // Abort received. Return to re-initialize.
}

// Allows resuming from parking/safety door. Actively checks if safety door is closed and ready to resume.

```

```

if (sys.state == STATE_SAFETY_DOOR) {
    if (!(system_check_safety_door_ajar())) {
        sys.suspend &= ~(SUSPEND_SAFETY_DOOR_AJAR); // Reset door ajar flag to denote ready to resume.
    }
}

// Handles parking restore and safety door resume.
if (sys.suspend & SUSPEND_INITIATE_RESTORE) {

#ifdef PARKING_ENABLE
    // Execute fast restore motion to the pull-out position. Parking requires homing enabled.
    // NOTE: State is will remain DOOR, until the de-energizing and retract is complete.
#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
    if (((settings.flags & (BITFLAG_HOMING_ENABLE|BITFLAG_LASER_MODE)) == BITFLAG_HOMING_ENABLE)
        (sys.override_ctrl == OVERRIDE_PARKING_MOTION)) {
    #else
    if ((settings.flags & (BITFLAG_HOMING_ENABLE|BITFLAG_LASER_MODE)) == BITFLAG_HOMING_ENABLE) {
    #endif
        // Check to ensure the motion doesn't move below pull-out position.
        if (parking_target[PARKING_AXIS] <= PARKING_TARGET) {
            parking_target[PARKING_AXIS] = retract_waypoint;
            pl_data->feed_rate = PARKING_RATE;
            mc_parking_motion(parking_target, pl_data);
        }
    }
#endif

    // Delayed Tasks: Restart spindle and coolant, delay to power-up, then resume cycle.
    if (gc_state.modal.spindle != SPINDLE_DISABLE) {
        // Block if safety door re-opened during prior restore actions.
        if (bit_isspace(sys.suspend,SUSPEND_RESTART_RETRACT)) {
            if (bit_istrue(settings.flags,BITFLAG_LASER_MODE)) {
                // When in laser mode, ignore spindle spin-up delay. Set to turn on laser when cycle starts.
                bit_true(sys.step_control, STEP_CONTROL_UPDATE_SPINDLE_PWM);
            } else {
                spindle_set_state((restore_condition & (PL_COND_FLAG_SPINDLE_CW | PL_COND_FLAG_SPINDLE_CCW)),
                delay_sec(SAFETY_DOOR_SPINDLE_DELAY, DELAY_MODE_SYS_SUSPEND);
            }
        }
    }

    if (gc_state.modal.coolant != COOLANT_DISABLE) {
        // Block if safety door re-opened during prior restore actions.
        if (bit_isspace(sys.suspend,SUSPEND_RESTART_RETRACT)) {
            // NOTE: Laser mode will honor this delay. An exhaust system is often controlled by this pin.
            coolant_set_state((restore_condition & (PL_COND_FLAG_COOLANT_FLOOD | PL_COND_FLAG_COOLANT_M
            delay_sec(SAFETY_DOOR_COOLANT_DELAY, DELAY_MODE_SYS_SUSPEND);
        }
    }

#ifdef PARKING_ENABLE
    // Execute slow plunge motion from pull-out position to resume position.

```

```

#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
if (((settings.flags & (BITFLAG_HOMING_ENABLE|BITFLAG_LASER_MODE)) == BITFLAG_HOMING_ENABLE)
    (sys.override_ctrl == OVERRIDE_PARKING_MOTION)) {
#else
if ((settings.flags & (BITFLAG_HOMING_ENABLE|BITFLAG_LASER_MODE)) == BITFLAG_HOMING_ENABLE) {
#endif
    // Block if safety door re-opened during prior restore actions.
    if (bit_istrue(sys.suspend,SUSPEND_RESTART_RETRACT)) {
        // Regardless if the retract parking motion was a valid/safe motion or not, the
        // restore parking motion should logically be valid, either by returning to the
        // original position through valid machine space or by not moving at all.
        pl_data->feed_rate = PARKING_PULLOUT_RATE;
        pl_data->condition |= (restore_condition & PL_COND_ACCESSORY_MASK); // Restore accessory state
        pl_data->spindle_speed = restore_spindle_speed;
        mc_parking_motion(restore_target, pl_data);
    }
}
#endif

if (bit_istrue(sys.suspend,SUSPEND_RESTART_RETRACT)) {
    sys.suspend |= SUSPEND_RESTORE_COMPLETE;
    system_set_exec_state_flag(EXEC_CYCLE_START); // Set to resume program.
}
}

}

} else {

    // Feed hold manager. Controls spindle stop override states.
    // NOTE: Hold ensured as completed by condition check at the beginning of suspend routine.
    if (sys.spindle_stop_ovr) {
        // Handles beginning of spindle stop
        if (sys.spindle_stop_ovr & SPINDLE_STOP_OVR_INITIATE) {
            if (gc_state.modal.spindle != SPINDLE_DISABLE) {
                spindle_set_state(SPINDLE_DISABLE,0.0); // De-energize
                sys.spindle_stop_ovr = SPINDLE_STOP_OVR_ENABLED; // Set stop override state to enabled, if de-energized.
            } else {
                sys.spindle_stop_ovr = SPINDLE_STOP_OVR_DISABLED; // Clear stop override state
            }
        }
        // Handles restoring of spindle state
    } else if (sys.spindle_stop_ovr & (SPINDLE_STOP_OVR_RESTORE | SPINDLE_STOP_OVR_RESTORE_CYCLE)) {
        if (gc_state.modal.spindle != SPINDLE_DISABLE) {
            report_feedback_message(MESSAGE_SPINDLE_RESTORE);
            if (bit_istrue(settings.flags,BITFLAG_LASER_MODE)) {
                // When in laser mode, ignore spindle spin-up delay. Set to turn on laser when cycle starts.
                bit_true(sys.step_control, STEP_CONTROL_UPDATE_SPINDLE_PWM);
            } else {
                spindle_set_state((restore_condition & (PL_COND_FLAG_SPINDLE_CW | PL_COND_FLAG_SPINDLE_CCW)),
            }
        }
    }
}

```

```

    }
    if (sys.spindle_stop_ovr & SPINDLE_STOP_OVR_RESTORE_CYCLE) {
        system_set_exec_state_flag(EXEC_CYCLE_START); // Set to resume program.
    }
    sys.spindle_stop_ovr = SPINDLE_STOP_OVR_DISABLED; // Clear stop override state
}
} else {
    // Handles spindle state during hold. NOTE: Spindle speed overrides may be altered during hold state.
    // NOTE: STEP_CONTROL_UPDATE_SPINDLE_PWM is automatically reset upon resume in step generator.
    if (bit_istrue(sys.step_control, STEP_CONTROL_UPDATE_SPINDLE_PWM)) {
        spindle_set_state((restore_condition & (PL_COND_FLAG_SPINDLE_CW | PL_COND_FLAG_SPINDLE_CCW)), re
        bit_false(sys.step_control, STEP_CONTROL_UPDATE_SPINDLE_PWM);
    }
}

}
}

protocol_exec_rt_system();

}
}

```

File: protocol.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
protocol.h - controls Grbl execution protocol and procedures
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

#ifdef protocol_h
#define protocol_h

```

```

// Line buffer size from the serial input stream to be executed.
// NOTE: Not a problem except for extreme cases, but the line buffer size can be too small
// and g-code blocks can get truncated. Officially, the g-code standards support up to 256
// characters. In future versions, this will be increased, when we know how much extra
// memory space we can invest into here or we re-write the g-code parser not to have this
// buffer.
#ifndef LINE_BUFFER_SIZE
#define LINE_BUFFER_SIZE 80
#endif

// Starts Grbl main loop. It handles all incoming characters from the serial port and executes
// them as they complete. It is also responsible for finishing the initialization procedures.
void protocol_main_loop();

// Checks and executes a realtime command at various stop points in main program
void protocol_execute_realtime();
void protocol_exec_rt_system();

// Executes the auto cycle feature, if enabled.
void protocol_auto_cycle_start();

// Block until all buffered steps are executed
void protocol_buffer_synchronize();

#endif

```

File: report.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
report.c - reporting and messaging methods
Part of Grbl

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <http://www.gnu.org/licenses/>.
*/

```

```
/*
```

This file functions as the primary feedback interface for Grbl. Any outgoing data, such as the protocol status messages, feedback messages, and status reports, are stored here. For the most part, these functions primarily are called from protocol.c methods. If a different style feedback is desired (i.e. JSON), then a user can change these following methods to accomodate their needs.

```
*/
```

```
#include "grbl.h"
```

```
// Internal report utilities to reduce flash with repetitive tasks turned into functions.
```

```
void report_util_setting_prefix(uint8_t n) { serial_write('$'); print_uint8_base10(n); serial_write('='); }
```

```
static void report_util_line_feed() { printPgmString(PSTR("\r\n")); }
```

```
static void report_util_feedback_line_feed() { serial_write(']'); report_util_line_feed(); }
```

```
static void report_util_gcode_modes_G() { printPgmString(PSTR(" G")); }
```

```
static void report_util_gcode_modes_M() { printPgmString(PSTR(" M")); }
```

```
// static void report_util_comment_line_feed() { serial_write(';'); report_util_line_feed(); }
```

```
static void report_util_axis_values(float *axis_value) {
```

```
    uint8_t idx;
```

```
    for (idx=0; idx<N_AXIS; idx++) {
```

```
        printFloat_CoordValue(axis_value[idx]);
```

```
        if (idx < (N_AXIS-1)) { serial_write(','); }
```

```
    }
```

```
}
```

```
/*
```

```
static void report_util_setting_string(uint8_t n) {
```

```
    serial_write(' ');
```

```
    serial_write('(');
```

```
    switch(n) {
```

```
        case 0: printPgmString(PSTR("stp pulse")); break;
```

```
        case 1: printPgmString(PSTR("idl delay")); break;
```

```
        case 2: printPgmString(PSTR("stp inv")); break;
```

```
        case 3: printPgmString(PSTR("dir inv")); break;
```

```
        case 4: printPgmString(PSTR("stp en inv")); break;
```

```
        case 5: printPgmString(PSTR("lim inv")); break;
```

```
        case 6: printPgmString(PSTR("prb inv")); break;
```

```
        case 10: printPgmString(PSTR("rpt")); break;
```

```
        case 11: printPgmString(PSTR("jnc dev")); break;
```

```
        case 12: printPgmString(PSTR("arc tol")); break;
```

```
        case 13: printPgmString(PSTR("rpt inch")); break;
```

```
        case 20: printPgmString(PSTR("sft lim")); break;
```

```
        case 21: printPgmString(PSTR("hrd lim")); break;
```

```
        case 22: printPgmString(PSTR("hm cyc")); break;
```

```
        case 23: printPgmString(PSTR("hm dir inv")); break;
```

```
        case 24: printPgmString(PSTR("hm feed")); break;
```

```
        case 25: printPgmString(PSTR("hm seek")); break;
```

```
        case 26: printPgmString(PSTR("hm delay")); break;
```

```
        case 27: printPgmString(PSTR("hm pulloff")); break;
```

```

case 30: printPgmString(PSTR("rpm max")); break;
case 31: printPgmString(PSTR("rpm min")); break;
case 32: printPgmString(PSTR("laser")); break;
default:
    n -= AXIS_SETTINGS_START_VAL;
    uint8_t idx = 0;
    while (n >= AXIS_SETTINGS_INCREMENT) {
        n -= AXIS_SETTINGS_INCREMENT;
        idx++;
    }
    serial_write(n+'x');
    switch (idx) {
        case 0: printPgmString(PSTR(":stp/mm")); break;
        case 1: printPgmString(PSTR(":mm/min")); break;
        case 2: printPgmString(PSTR(":mm/s^2")); break;
        case 3: printPgmString(PSTR(":mm max")); break;
    }
    break;
}
report_util_comment_line_feed();
}
*/

static void report_util_uint8_setting(uint8_t n, int val) {
    report_util_setting_prefix(n);
    print_uint8_base10(val);
    report_util_line_feed(); // report_util_setting_string(n);
}

static void report_util_float_setting(uint8_t n, float val, uint8_t n_decimal) {
    report_util_setting_prefix(n);
    printFloat(val, n_decimal);
    report_util_line_feed(); // report_util_setting_string(n);
}

```

// Handles the primary confirmation protocol response for streaming interfaces and human-feedback.
 // For every incoming line, this method responds with an 'ok' for a successful command or an
 // 'error:' to indicate some error event with the line or some critical system error during
 // operation. Errors events can originate from the g-code parser, settings module, or asynchronously
 // from a critical error, such as a triggered hard limit. Interface should always monitor for these
 // responses.

```

void report_status_message(uint8_t status_code)
{
    switch(status_code) {
        case STATUS_OK: // STATUS_OK
            printPgmString(PSTR("ok\r\n")); break;
        default:
            printPgmString(PSTR("error:"));
            print_uint8_base10(status_code);
            report_util_line_feed();
    }
}

```

```
}
```

```
// Prints alarm messages.
```

```
void report_alarm_message(uint8_t alarm_code)
```

```
{
```

```
    printPgmString(PSTR("ALARM:"));
```

```
    print_uint8_base10(alarm_code);
```

```
    report_util_line_feed();
```

```
    delay_ms(500); // Force delay to ensure message clears serial write buffer.
```

```
}
```

```
// Prints feedback messages. This serves as a centralized method to provide additional
```

```
// user feedback for things that are not of the status/alarm message protocol. These are
```

```
// messages such as setup warnings, switch toggling, and how to exit alarms.
```

```
// NOTE: For interfaces, messages are always placed within brackets. And if silent mode
```

```
// is installed, the message number codes are less than zero.
```

```
void report_feedback_message(uint8_t message_code)
```

```
{
```

```
    printPgmString(PSTR("[MSG:"));
```

```
    switch(message_code) {
```

```
        case MESSAGE_CRITICAL_EVENT:
```

```
            printPgmString(PSTR("Reset to continue")); break;
```

```
        case MESSAGE_ALARM_LOCK:
```

```
            printPgmString(PSTR("$H'$X' to unlock")); break;
```

```
        case MESSAGE_ALARM_UNLOCK:
```

```
            printPgmString(PSTR("Caution: Unlocked")); break;
```

```
        case MESSAGE_ENABLED:
```

```
            printPgmString(PSTR("Enabled")); break;
```

```
        case MESSAGE_DISABLED:
```

```
            printPgmString(PSTR("Disabled")); break;
```

```
        case MESSAGE_SAFETY_DOOR_AJAR:
```

```
            printPgmString(PSTR("Check Door")); break;
```

```
        case MESSAGE_CHECK_LIMITS:
```

```
            printPgmString(PSTR("Check Limits")); break;
```

```
        case MESSAGE_PROGRAM_END:
```

```
            printPgmString(PSTR("Pgm End")); break;
```

```
        case MESSAGE_RESTORE_DEFAULTS:
```

```
            printPgmString(PSTR("Restoring defaults")); break;
```

```
        case MESSAGE_SPINDLE_RESTORE:
```

```
            printPgmString(PSTR("Restoring spindle")); break;
```

```
        case MESSAGE_SLEEP_MODE:
```

```
            printPgmString(PSTR("Sleeping")); break;
```

```
    }
```

```
    report_util_feedback_line_feed();
```

```
}
```

```
// Welcome message
```

```
void report_init_message()
```

```
{
```

```
    printPgmString(PSTR("\r\nGrbl " GRBL_VERSION " ['$' for help]\r\n"));
```

```

}

// Grbl help message
void report_grbl_help() {
  printPgmString(PSTR("[HLP:$$ $# $G $I $N $x=val $Nx=line $J=line $SLP $C $X $H ~ ! ? ctrl-x]\n\n"));
}

// Grbl global settings print out.
// NOTE: The numbering scheme here must correlate to storing in settings.c
void report_grbl_settings() {
  // Print Grbl settings.
  report_util_uint8_setting(0,settings.pulse_microseconds);
  report_util_uint8_setting(1,settings.stepper_idle_lock_time);
  report_util_uint8_setting(2,settings.step_invert_mask);
  report_util_uint8_setting(3,settings.dir_invert_mask);
  report_util_uint8_setting(4,bit_istrue(settings.flags,BITFLAG_INVERT_ST_ENABLE));
  report_util_uint8_setting(5,bit_istrue(settings.flags,BITFLAG_INVERT_LIMIT_PINS));
  report_util_uint8_setting(6,bit_istrue(settings.flags,BITFLAG_INVERT_PROBE_PIN));
  report_util_uint8_setting(10,settings.status_report_mask);
  report_util_float_setting(11,settings.junction_deviation,N_DECIMAL_SETTINGVALUE);
  report_util_float_setting(12,settings.arc_tolerance,N_DECIMAL_SETTINGVALUE);
  report_util_uint8_setting(13,bit_istrue(settings.flags,BITFLAG_REPORT_INCHES));
  report_util_uint8_setting(20,bit_istrue(settings.flags,BITFLAG_SOFT_LIMIT_ENABLE));
  report_util_uint8_setting(21,bit_istrue(settings.flags,BITFLAG_HARD_LIMIT_ENABLE));
  report_util_uint8_setting(22,bit_istrue(settings.flags,BITFLAG_HOMING_ENABLE));
  report_util_uint8_setting(23,settings.homing_dir_mask);
  report_util_float_setting(24,settings.homing_feed_rate,N_DECIMAL_SETTINGVALUE);
  report_util_float_setting(25,settings.homing_seek_rate,N_DECIMAL_SETTINGVALUE);
  report_util_uint8_setting(26,settings.homing_debounce_delay);
  report_util_float_setting(27,settings.homing_pulloff,N_DECIMAL_SETTINGVALUE);
  report_util_float_setting(30,settings.rpm_max,N_DECIMAL_RPMVALUE);
  report_util_float_setting(31,settings.rpm_min,N_DECIMAL_RPMVALUE);
#ifdef VARIABLE_SPINDLE
  report_util_uint8_setting(32,bit_istrue(settings.flags,BITFLAG_LASER_MODE));
#else
  report_util_uint8_setting(32,0);
#endif
  // Print axis settings
  uint8_t idx, set_idx;
  uint8_t val = AXIS_SETTINGS_START_VAL;
  for (set_idx=0; set_idx<AXIS_N_SETTINGS; set_idx++) {
    for (idx=0; idx<N_AXIS; idx++) {
      switch (set_idx) {
        case 0: report_util_float_setting(val+idx,settings.steps_per_mm[idx],N_DECIMAL_SETTINGVALUE); break;
        case 1: report_util_float_setting(val+idx,settings.max_rate[idx],N_DECIMAL_SETTINGVALUE); break;
        case 2: report_util_float_setting(val+idx,settings.acceleration[idx]/(60*60),N_DECIMAL_SETTINGVALUE); break;
        case 3: report_util_float_setting(val+idx,-settings.max_travel[idx],N_DECIMAL_SETTINGVALUE); break;
      }
    }
    val += AXIS_SETTINGS_INCREMENT;
  }
}

```

```
}  
}
```

```
// Prints current probe parameters. Upon a probe command, these parameters are updated upon a  
// successful probe or upon a failed probe with the G38.3 without errors command (if supported).  
// These values are retained until Grbl is power-cycled, whereby they will be re-zeroed.
```

```
void report_probe_parameters()  
{  
  // Report in terms of machine position.  
  printPgmString(PSTR("[PRB:"]);  
  float print_position[N_AXIS];  
  system_convert_array_steps_to_mpos(print_position,sys_probe_position);  
  report_util_axis_values(print_position);  
  serial_write(':');  
  print_uint8_base10(sys.probe_succeeded);  
  report_util_feedback_line_feed();  
}
```

```
// Prints Grbl NGC parameters (coordinate offsets, probing)
```

```
void report_ngc_parameters()  
{  
  float coord_data[N_AXIS];  
  uint8_t coord_select;  
  for (coord_select = 0; coord_select <= SETTING_INDEX_NCOORD; coord_select++) {  
    if (!(settings_read_coord_data(coord_select,coord_data))) {  
      report_status_message(STATUS_SETTING_READ_FAIL);  
      return;  
    }  
    printPgmString(PSTR("[G"));  
    switch (coord_select) {  
      case 6: printPgmString(PSTR("28")); break;  
      case 7: printPgmString(PSTR("30")); break;  
      default: print_uint8_base10(coord_select+54); break; // G54-G59  
    }  
    serial_write(':');  
    report_util_axis_values(coord_data);  
    report_util_feedback_line_feed();  
  }  
  printPgmString(PSTR("[G92:")); // Print G92,G92.1 which are not persistent in memory  
  report_util_axis_values(gc_state.coord_offset);  
  report_util_feedback_line_feed();  
  printPgmString(PSTR("[TLO:")); // Print tool length offset value  
  printFloat_CoordValue(gc_state.tool_length_offset);  
  report_util_feedback_line_feed();  
  report_probe_parameters(); // Print probe parameters. Not persistent in memory.  
}
```

```
// Print current gcode parser mode state
```

```

void report_gcode_modes()
{
    printPgmString(PSTR("[GC:G"));
    if (gc_state.modal.motion >= MOTION_MODE_PROBE_TOWARD) {
        printPgmString(PSTR("38."));
        print_uint8_base10(gc_state.modal.motion - (MOTION_MODE_PROBE_TOWARD-2));
    } else {
        print_uint8_base10(gc_state.modal.motion);
    }

    report_util_gcode_modes_G();
    print_uint8_base10(gc_state.modal.coord_select+54);

    report_util_gcode_modes_G();
    print_uint8_base10(gc_state.modal.plane_select+17);

    report_util_gcode_modes_G();
    print_uint8_base10(21-gc_state.modal.units);

    report_util_gcode_modes_G();
    print_uint8_base10(gc_state.modal.distance+90);

    report_util_gcode_modes_G();
    print_uint8_base10(94-gc_state.modal.feed_rate);

    if (gc_state.modal.program_flow) {
        report_util_gcode_modes_M();
        switch (gc_state.modal.program_flow) {
            case PROGRAM_FLOW_PAUSED : serial_write('0'); break;
            // case PROGRAM_FLOW_OPTIONAL_STOP : serial_write('1'); break; // M1 is ignored and not supported.
            case PROGRAM_FLOW_COMPLETED_M2 :
            case PROGRAM_FLOW_COMPLETED_M30 :
                print_uint8_base10(gc_state.modal.program_flow);
                break;
        }
    }

    report_util_gcode_modes_M();
    switch (gc_state.modal.spindle) {
        case SPINDLE_ENABLE_CW : serial_write('3'); break;
        case SPINDLE_ENABLE_CCW : serial_write('4'); break;
        case SPINDLE_DISABLE : serial_write('5'); break;
    }

#ifdef ENABLE_M7
    if (gc_state.modal.coolant) { // Note: Multiple coolant states may be active at the same time.
        if (gc_state.modal.coolant & PL_COND_FLAG_COOLANT_MIST) { report_util_gcode_modes_M(); serial_write('7'); }
        if (gc_state.modal.coolant & PL_COND_FLAG_COOLANT_FLOOD) { report_util_gcode_modes_M(); serial_write('8'); }
    } else { report_util_gcode_modes_M(); serial_write('9'); }
#else
    report_util_gcode_modes_M();
#endif
}

```

```

    if (gc_state.modal.coolant) { serial_write('8'); }
    else { serial_write('9'); }
#endif

#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
    if (sys.override_ctrl == OVERRIDE_PARKING_MOTION) {
        report_util_gcode_modes_M();
        print_uint8_base10(56);
    }
#endif

    printPgmString(PSTR(" T"));
    print_uint8_base10(gc_state.tool);

    printPgmString(PSTR(" F"));
    printFloat_RateValue(gc_state.feed_rate);

#ifdef VARIABLE_SPINDLE
    printPgmString(PSTR(" S"));
    printFloat(gc_state.spindle_speed,N_DECIMAL_RPMVALUE);
#endif

    report_util_feedback_line_feed();
}

// Prints specified startup line
void report_startup_line(uint8_t n, char *line)
{
    printPgmString(PSTR("$N"));
    print_uint8_base10(n);
    serial_write('=');
    printString(line);
    report_util_line_feed();
}

void report_execute_startup_message(char *line, uint8_t status_code)
{
    serial_write('>');
    printString(line);
    serial_write(':');
    report_status_message(status_code);
}

// Prints build info line
void report_build_info(char *line)
{
    printPgmString(PSTR("[VER:" GRBL_VERSION "." GRBL_VERSION_BUILD ":");
    printString(line);
    report_util_feedback_line_feed();
    printPgmString(PSTR("[OPT:"); // Generate compile-time build option list
#ifdef VARIABLE_SPINDLE

```

```
    serial_write('V');
#endif
#ifdef USE_LINE_NUMBERS
    serial_write('N');
#endif
#ifdef ENABLE_M7
    serial_write('M');
#endif
#ifdef COREXY
    serial_write('C');
#endif
#ifdef PARKING_ENABLE
    serial_write('P');
#endif
#ifdef HOMING_FORCE_SET_ORIGIN
    serial_write('Z');
#endif
#ifdef HOMING_SINGLE_AXIS_COMMANDS
    serial_write('H');
#endif
#ifdef LIMITS_TWO_SWITCHES_ON_AXES
    serial_write('T');
#endif
#ifdef ALLOW_FEED_OVERRIDE_DURING_PROBE_CYCLES
    serial_write('A');
#endif
#ifdef USE_SPINDLE_DIR_AS_ENABLE_PIN
    serial_write('D');
#endif
#ifdef SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED
    serial_write('0');
#endif
#ifdef ENABLE_SOFTWARE_DEBOUNCE
    serial_write('S');
#endif
#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
    serial_write('R');
#endif
#ifdef HOMING_INIT_LOCK
    serial_write('L');
#endif
#ifdef ENABLE_SAFETY_DOOR_INPUT_PIN
    serial_write('+');
#endif
#ifdef ENABLE_RESTORE_EEPROM_WIPE_ALL // NOTE: Shown when disabled.
    serial_write('*');
#endif
#ifdef ENABLE_RESTORE_EEPROM_DEFAULT_SETTINGS // NOTE: Shown when disabled.
    serial_write('$');
#endif
#ifdef ENABLE_RESTORE_EEPROM_CLEAR_PARAMETERS // NOTE: Shown when disabled.
```

```

    serial_write('#');
#endif
#ifndef ENABLE_BUILD_INFO_WRITE_COMMAND // NOTE: Shown when disabled.
    serial_write('I');
#endif
#ifndef FORCE_BUFFER_SYNC_DURING_EEPROM_WRITE // NOTE: Shown when disabled.
    serial_write('E');
#endif
#ifndef FORCE_BUFFER_SYNC_DURING_WCO_CHANGE // NOTE: Shown when disabled.
    serial_write('W');
#endif
#ifdef ENABLE_DUAL_AXIS
    serial_write('2');
#endif
// NOTE: Compiled values, like override increments/max/min values, may be added at some point later.
serial_write(',');
print_uint8_base10(BLOCK_BUFFER_SIZE-1);
serial_write(',');
print_uint8_base10(RX_BUFFER_SIZE);

report_util_feedback_line_feed();
}

```

// Prints the character string line Grbl has received from the user, which has been pre-parsed,
// and has been sent into protocol_execute_line() routine to be executed by Grbl.

```

void report_echo_line_received(char *line)
{
    printPgmString(PSTR("[echo: ")); printString(line);
    report_util_feedback_line_feed();
}

```

// Prints real-time data. This function grabs a real-time snapshot of the stepper subprogram
// and the actual location of the CNC machine. Users may change the following function to their
// specific needs, but the desired real-time data report must be as short as possible. This is
// requires as it minimizes the computational overhead and allows grbl to keep running smoothly,
// especially during g-code programs with fast, short line segments and high frequency reports (5-20Hz).

```

void report_realtime_status()
{
    uint8_t idx;
    int32_t current_position[N_AXIS]; // Copy current state of the system position variable
    memcpy(current_position, sys_position, sizeof(sys_position));
    float print_position[N_AXIS];
    system_convert_array_steps_to_mpos(print_position, current_position);

    // Report current machine state and sub-states
    serial_write('<');
    switch (sys.state) {
        case STATE_IDLE: printPgmString(PSTR("Idle")); break;
        case STATE_CYCLE: printPgmString(PSTR("Run")); break;
    }
}

```

```

case STATE_HOLD:
    if (!(sys.suspend & SUSPEND_JOG_CANCEL)) {
        printPgmString(PSTR("Hold:"));
        if (sys.suspend & SUSPEND_HOLD_COMPLETE) { serial_write('0'); } // Ready to resume
        else { serial_write('1'); } // Actively holding
        break;
    } // Continues to print jog state during jog cancel.
case STATE_JOG: printPgmString(PSTR("Jog")); break;
case STATE_HOMING: printPgmString(PSTR("Home")); break;
case STATE_ALARM: printPgmString(PSTR("Alarm")); break;
case STATE_CHECK_MODE: printPgmString(PSTR("Check")); break;
case STATE_SAFETY_DOOR:
    printPgmString(PSTR("Door:"));
    if (sys.suspend & SUSPEND_INITIATE_RESTORE) {
        serial_write('3'); // Restoring
    } else {
        if (sys.suspend & SUSPEND_RETRACT_COMPLETE) {
            if (sys.suspend & SUSPEND_SAFETY_DOOR_AJAR) {
                serial_write('1'); // Door ajar
            } else {
                serial_write('0');
            } // Door closed and ready to resume
        } else {
            serial_write('2'); // Retracting
        }
    }
    break;
case STATE_SLEEP: printPgmString(PSTR("Sleep")); break;
}

```

```

float wco[N_AXIS];
if (bit_isspace(settings.status_report_mask,BITFLAG_RT_STATUS_POSITION_TYPE) ||
    (sys.report_wco_counter == 0) ) {
    for (idx=0; idx< N_AXIS; idx++) {
        // Apply work coordinate offsets and tool length offset to current position.
        wco[idx] = gc_state.coord_system[idx]+gc_state.coord_offset[idx];
        if (idx == TOOL_LENGTH_OFFSET_AXIS) { wco[idx] += gc_state.tool_length_offset; }
        if (bit_isspace(settings.status_report_mask,BITFLAG_RT_STATUS_POSITION_TYPE)) {
            print_position[idx] -= wco[idx];
        }
    }
}

```

```

// Report machine position
if (bit_istrue(settings.status_report_mask,BITFLAG_RT_STATUS_POSITION_TYPE)) {
    printPgmString(PSTR("|MPos:"));
} else {
    printPgmString(PSTR("|WPos:"));
}
report_util_axis_values(print_position);

```

```

// Returns planner and serial read buffer states.
#ifdef REPORT_FIELD_BUFFER_STATE
    if (bit_istrue(settings.status_report_mask,BITFLAG_RT_STATUS_BUFFER_STATE)) {
        printPgmString(PSTR("|Bf:"));
        print_uint8_base10(plan_get_block_buffer_available());
        serial_write(',');
        print_uint8_base10(serial_get_rx_buffer_available());
    }
#endif

#ifdef USE_LINE_NUMBERS
#ifdef REPORT_FIELD_LINE_NUMBERS
    // Report current line number
    plan_block_t * cur_block = plan_get_current_block();
    if (cur_block != NULL) {
        uint32_t ln = cur_block->line_number;
        if (ln > 0) {
            printPgmString(PSTR("|Ln:"));
            printInteger(ln);
        }
    }
#endif
#endif

// Report realtime feed speed
#ifdef REPORT_FIELD_CURRENT_FEED_SPEED
#ifdef VARIABLE_SPINDLE
    printPgmString(PSTR("|FS:"));
    printFloat_RateValue(st_get_realtime_rate());
    serial_write(',');
    printFloat(sys.spindle_speed,N_DECIMAL_RPMVALUE);
#else
    printPgmString(PSTR("|F:"));
    printFloat_RateValue(st_get_realtime_rate());
#endif
#endif

#ifdef REPORT_FIELD_PIN_STATE
    uint8_t lim_pin_state = limits_get_state();
    uint8_t ctrl_pin_state = system_control_get_state();
    uint8_t prb_pin_state = probe_get_state();
    if (lim_pin_state | ctrl_pin_state | prb_pin_state) {
        printPgmString(PSTR("|Pn:"));
        if (prb_pin_state) { serial_write('P'); }
        if (lim_pin_state) {
#ifdef ENABLE_DUAL_AXIS
            #if (DUAL_AXIS_SELECT == X_AXIS)
                if (bit_istrue(lim_pin_state,(bit(X_AXIS)|bit(N_AXIS)))) { serial_write('X'); }
                if (bit_istrue(lim_pin_state,bit(Y_AXIS))) { serial_write('Y'); }
            #endif
            #if (DUAL_AXIS_SELECT == Y_AXIS)

```

```

        if (bit_istrue(lim_pin_state,bit(X_AXIS))) { serial_write('X'); }
        if (bit_istrue(lim_pin_state,(bit(Y_AXIS)|bit(N_AXIS)))) { serial_write('Y'); }
    #endif
    if (bit_istrue(lim_pin_state,bit(Z_AXIS))) { serial_write('Z'); }
    #else
        if (bit_istrue(lim_pin_state,bit(X_AXIS))) { serial_write('X'); }
        if (bit_istrue(lim_pin_state,bit(Y_AXIS))) { serial_write('Y'); }
        if (bit_istrue(lim_pin_state,bit(Z_AXIS))) { serial_write('Z'); }
    #endif
}
if (ctrl_pin_state) {
    #ifdef ENABLE_SAFETY_DOOR_INPUT_PIN
        if (bit_istrue(ctrl_pin_state,CONTROL_PIN_INDEX_SAFETY_DOOR)) { serial_write('D'); }
    #endif
    if (bit_istrue(ctrl_pin_state,CONTROL_PIN_INDEX_RESET)) { serial_write('R'); }
    if (bit_istrue(ctrl_pin_state,CONTROL_PIN_INDEX_FEED_HOLD)) { serial_write('H'); }
    if (bit_istrue(ctrl_pin_state,CONTROL_PIN_INDEX_CYCLE_START)) { serial_write('S'); }
}
}
#endif

#ifdef REPORT_FIELD_WORK_COORD_OFFSET
    if (sys.report_wco_counter > 0) { sys.report_wco_counter--; }
    else {
        if (sys.state & (STATE_HOMING | STATE_CYCLE | STATE_HOLD | STATE_JOG | STATE_SAFETY_DOOR)) {
            sys.report_wco_counter = (REPORT_WCO_REFRESH_BUSY_COUNT-1); // Reset counter for slow refresh
        } else { sys.report_wco_counter = (REPORT_WCO_REFRESH_IDLE_COUNT-1); }
        if (sys.report_ovr_counter == 0) { sys.report_ovr_counter = 1; } // Set override on next report.
        printPgmString(PSTR("|WCO:"));
        report_util_axis_values(wco);
    }
#endif

#ifdef REPORT_FIELD_OVERRIDES
    if (sys.report_ovr_counter > 0) { sys.report_ovr_counter--; }
    else {
        if (sys.state & (STATE_HOMING | STATE_CYCLE | STATE_HOLD | STATE_JOG | STATE_SAFETY_DOOR)) {
            sys.report_ovr_counter = (REPORT_OVR_REFRESH_BUSY_COUNT-1); // Reset counter for slow refresh
        } else { sys.report_ovr_counter = (REPORT_OVR_REFRESH_IDLE_COUNT-1); }
        printPgmString(PSTR("|Ov:"));
        print_uint8_base10(sys.f_override);
        serial_write(',');
        print_uint8_base10(sys.r_override);
        serial_write(',');
        print_uint8_base10(sys.spindle_speed_ovr);

        uint8_t sp_state = spindle_get_state();
        uint8_t cl_state = coolant_get_state();
        if (sp_state || cl_state) {
            printPgmString(PSTR("|A:"));
            if (sp_state) { // != SPINDLE_STATE_DISABLE

```

```

#ifdef VARIABLE_SPINDLE
#ifdef USE_SPINDLE_DIR_AS_ENABLE_PIN
    serial_write('S'); // CW
#else
    if (sp_state == SPINDLE_STATE_CW) { serial_write('S'); } // CW
    else { serial_write('C'); } // CCW
#endif
#else
    if (sp_state & SPINDLE_STATE_CW) { serial_write('S'); } // CW
    else { serial_write('C'); } // CCW
#endif
}
if (cl_state & COOLANT_STATE_FLOOD) { serial_write('F'); }
#ifdef ENABLE_M7
    if (cl_state & COOLANT_STATE_MIST) { serial_write('M'); }
#endif
}
}
#endif

serial_write('>');
report_util_line_feed();
}

#ifdef DEBUG
void report_realtime_debug()
{

}
#endif

```

File: report.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
report.h - reporting and messaging methods
Part of Grbl

```

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the

GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

#ifndef report_h

#define report_h

// Define Grbl status codes. Valid values (0-255)

#define STATUS_OK 0

#define STATUS_EXPECTED_COMMAND_LETTER 1

#define STATUS_BAD_NUMBER_FORMAT 2

#define STATUS_INVALID_STATEMENT 3

#define STATUS_NEGATIVE_VALUE 4

#define STATUS_SETTING_DISABLED 5

#define STATUS_SETTING_STEP_PULSE_MIN 6

#define STATUS_SETTING_READ_FAIL 7

#define STATUS_IDLE_ERROR 8

#define STATUS_SYSTEM_GC_LOCK 9

#define STATUS_SOFT_LIMIT_ERROR 10

#define STATUS_OVERFLOW 11

#define STATUS_MAX_STEP_RATE_EXCEEDED 12

#define STATUS_CHECK_DOOR 13

#define STATUS_LINE_LENGTH_EXCEEDED 14

#define STATUS_TRAVEL_EXCEEDED 15

#define STATUS_INVALID_JOG_COMMAND 16

#define STATUS_SETTING_DISABLED_LASER 17

#define STATUS_GCODE_UNSUPPORTED_COMMAND 20

#define STATUS_GCODE_MODAL_GROUP_VIOLATION 21

#define STATUS_GCODE_UNDEFINED_FEED_RATE 22

#define STATUS_GCODE_COMMAND_VALUE_NOT_INTEGER 23

#define STATUS_GCODE_AXIS_COMMAND_CONFLICT 24

#define STATUS_GCODE_WORD_REPEATED 25

#define STATUS_GCODE_NO_AXIS_WORDS 26

#define STATUS_GCODE_INVALID_LINE_NUMBER 27

#define STATUS_GCODE_VALUE_WORD_MISSING 28

#define STATUS_GCODE_UNSUPPORTED_COORD_SYS 29

#define STATUS_GCODE_G53_INVALID_MOTION_MODE 30

#define STATUS_GCODE_AXIS_WORDS_EXIST 31

#define STATUS_GCODE_NO_AXIS_WORDS_IN_PLANE 32

#define STATUS_GCODE_INVALID_TARGET 33

#define STATUS_GCODE_ARC_RADIUS_ERROR 34

#define STATUS_GCODE_NO_OFFSETS_IN_PLANE 35

#define STATUS_GCODE_UNUSED_WORDS 36

#define STATUS_GCODE_G43_DYNAMIC_AXIS_ERROR 37

#define STATUS_GCODE_MAX_VALUE_EXCEEDED 38

// Define Grbl alarm codes. Valid values (1-255). 0 is reserved.

#define ALARM_HARD_LIMIT_ERROR EXEC_ALARM_HARD_LIMIT

#define ALARM_SOFT_LIMIT_ERROR EXEC_ALARM_SOFT_LIMIT

```
#define ALARM_ABORT_CYCLE      EXEC_ALARM_ABORT_CYCLE
#define ALARM_PROBE_FAIL_INITIAL EXEC_ALARM_PROBE_FAIL_INITIAL
#define ALARM_PROBE_FAIL_CONTACT EXEC_ALARM_PROBE_FAIL_CONTACT
#define ALARM_HOMING_FAIL_RESET EXEC_ALARM_HOMING_FAIL_RESET
#define ALARM_HOMING_FAIL_DOOR EXEC_ALARM_HOMING_FAIL_DOOR
#define ALARM_HOMING_FAIL_PULLOFF EXEC_ALARM_HOMING_FAIL_PULLOFF
#define ALARM_HOMING_FAIL_APPROACH EXEC_ALARM_HOMING_FAIL_APPROACH
```

```
// Define Grbl feedback message codes. Valid values (0-255).
```

```
#define MESSAGE_CRITICAL_EVENT 1
#define MESSAGE_ALARM_LOCK 2
#define MESSAGE_ALARM_UNLOCK 3
#define MESSAGE_ENABLED 4
#define MESSAGE_DISABLED 5
#define MESSAGE_SAFETY_DOOR_AJAR 6
#define MESSAGE_CHECK_LIMITS 7
#define MESSAGE_PROGRAM_END 8
#define MESSAGE_RESTORE_DEFAULTS 9
#define MESSAGE_SPINDLE_RESTORE 10
#define MESSAGE_SLEEP_MODE 11
```

```
// Prints system status messages.
```

```
void report_status_message(uint8_t status_code);
```

```
// Prints system alarm messages.
```

```
void report_alarm_message(uint8_t alarm_code);
```

```
// Prints miscellaneous feedback messages.
```

```
void report_feedback_message(uint8_t message_code);
```

```
// Prints welcome message
```

```
void report_init_message();
```

```
// Prints Grbl help and current global settings
```

```
void report_grbl_help();
```

```
// Prints Grbl global settings
```

```
void report_grbl_settings();
```

```
// Prints an echo of the pre-parsed line received right before execution.
```

```
void report_echo_line_received(char *line);
```

```
// Prints realtime status report
```

```
void report_realtime_status();
```

```
// Prints recorded probe position
```

```
void report_probe_parameters();
```

```
// Prints Grbl NGC parameters (coordinate offsets, probe)
```

```
void report_ngc_parameters();
```

```

// Prints current g-code parser mode state
void report_gcode_modes();

// Prints startup line when requested and executed.
void report_startup_line(uint8_t n, char *line);
void report_execute_startup_message(char *line, uint8_t status_code);

// Prints build info and user info
void report_build_info(char *line);

#ifdef DEBUG
    void report_realtime_debug();
#endif

#endif

```

File: serial.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
serial.c - Low level functions for sending and recieving bytes via the serial port
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

#include "grbl.h"

#define RX_RING_BUFFER (RX_BUFFER_SIZE+1)
#define TX_RING_BUFFER (TX_BUFFER_SIZE+1)

uint8_t serial_rx_buffer[RX_RING_BUFFER];
uint8_t serial_rx_buffer_head = 0;
volatile uint8_t serial_rx_buffer_tail = 0;

```

```
uint8_t serial_tx_buffer[TX_RING_BUFFER];
uint8_t serial_tx_buffer_head = 0;
volatile uint8_t serial_tx_buffer_tail = 0;
```

// Returns the number of bytes available in the RX serial buffer.

```
uint8_t serial_get_rx_buffer_available()
{
    uint8_t rtail = serial_rx_buffer_tail; // Copy to limit multiple calls to volatile
    if (serial_rx_buffer_head >= rtail) { return(RX_BUFFER_SIZE - (serial_rx_buffer_head-rtail)); }
    return((rtail-serial_rx_buffer_head-1));
}
```

// Returns the number of bytes used in the RX serial buffer.

// NOTE: Deprecated. Not used unless classic status reports are enabled in config.h.

```
uint8_t serial_get_rx_buffer_count()
{
    uint8_t rtail = serial_rx_buffer_tail; // Copy to limit multiple calls to volatile
    if (serial_rx_buffer_head >= rtail) { return(serial_rx_buffer_head-rtail); }
    return (RX_BUFFER_SIZE - (rtail-serial_rx_buffer_head));
}
```

// Returns the number of bytes used in the TX serial buffer.

// NOTE: Not used except for debugging and ensuring no TX bottlenecks.

```
uint8_t serial_get_tx_buffer_count()
{
    uint8_t ttail = serial_tx_buffer_tail; // Copy to limit multiple calls to volatile
    if (serial_tx_buffer_head >= ttail) { return(serial_tx_buffer_head-ttail); }
    return (TX_RING_BUFFER - (ttail-serial_tx_buffer_head));
}
```

void serial_init()

```
{
    // Set baud rate
    #if BAUD_RATE < 57600
        uint16_t UBRR0_value = ((F_CPU / (8L * BAUD_RATE)) - 1)/2 ;
        UCSR0A &= ~(1 << U2X0); // baud doubler off - Only needed on Uno XXX
    #else
        uint16_t UBRR0_value = ((F_CPU / (4L * BAUD_RATE)) - 1)/2;
        UCSR0A |= (1 << U2X0); // baud doubler on for high baud rates, i.e. 115200
    #endif
    UBRR0H = UBRR0_value >> 8;
    UBRR0L = UBRR0_value;

    // enable rx, tx, and interrupt on complete reception of a byte
    UCSR0B |= (1<<RXEN0 | 1<<TXEN0 | 1<<RXCIE0);

    // defaults to 8-bit, no parity, 1 stop bit
```

```
}
```

```
// Writes one byte to the TX serial buffer. Called by main program.
```

```
void serial_write(uint8_t data) {
```

```
    // Calculate next head
```

```
    uint8_t next_head = serial_tx_buffer_head + 1;
```

```
    if (next_head == TX_RING_BUFFER) { next_head = 0; }
```

```
    // Wait until there is space in the buffer
```

```
    while (next_head == serial_tx_buffer_tail) {
```

```
        // TODO: Restructure st_prep_buffer() calls to be executed here during a long print.
```

```
        if (sys_rt_exec_state & EXEC_RESET) { return; } // Only check for abort to avoid an endless loop.
```

```
    }
```

```
    // Store data and advance head
```

```
    serial_tx_buffer[serial_tx_buffer_head] = data;
```

```
    serial_tx_buffer_head = next_head;
```

```
    // Enable Data Register Empty Interrupt to make sure tx-streaming is running
```

```
    UCSR0B |= (1 << UDRIE0);
```

```
}
```

```
// Data Register Empty Interrupt handler
```

```
ISR(SERIAL_UDRE)
```

```
{
```

```
    uint8_t tail = serial_tx_buffer_tail; // Temporary serial_tx_buffer_tail (to optimize for volatile)
```

```
    // Send a byte from the buffer
```

```
    UDR0 = serial_tx_buffer[tail];
```

```
    // Update tail position
```

```
    tail++;
```

```
    if (tail == TX_RING_BUFFER) { tail = 0; }
```

```
    serial_tx_buffer_tail = tail;
```

```
    // Turn off Data Register Empty Interrupt to stop tx-streaming if this concludes the transfer
```

```
    if (tail == serial_tx_buffer_head) { UCSR0B &= ~(1 << UDRIE0); }
```

```
}
```

```
// Fetches the first byte in the serial read buffer. Called by main program.
```

```
uint8_t serial_read()
```

```
{
```

```
    uint8_t tail = serial_rx_buffer_tail; // Temporary serial_rx_buffer_tail (to optimize for volatile)
```

```
    if (serial_rx_buffer_head == tail) {
```

```
        return SERIAL_NO_DATA;
```

```
    } else {
```

```
        uint8_t data = serial_rx_buffer[tail];
```

```

tail++;
if (tail == RX_RING_BUFFER) { tail = 0; }
serial_rx_buffer_tail = tail;

return data;
}
}

```

ISR(SERIAL_RX)

```

{
uint8_t data = UDR0;
uint8_t next_head;

// Pick off realtime command characters directly from the serial stream. These characters are
// not passed into the main buffer, but these set system state flag bits for realtime execution.
switch (data) {
case CMD_RESET:      mc_reset(); break; // Call motion control reset routine.
case CMD_STATUS_REPORT: system_set_exec_state_flag(EXEC_STATUS_REPORT); break; // Set as true
case CMD_CYCLE_START:  system_set_exec_state_flag(EXEC_CYCLE_START); break; // Set as true
case CMD_FEED_HOLD:    system_set_exec_state_flag(EXEC_FEED_HOLD); break; // Set as true
default :
if (data > 0x7F) { // Real-time control characters are extended ACSII only.
switch(data) {
case CMD_SAFETY_DOOR:  system_set_exec_state_flag(EXEC_SAFETY_DOOR); break; // Set as true
case CMD_JOG_CANCEL:
if (sys.state & STATE_JOG) { // Block all other states from invoking motion cancel.
system_set_exec_state_flag(EXEC_MOTION_CANCEL);
}
break;
#ifdef DEBUG
case CMD_DEBUG_REPORT: {uint8_t sreg = SREG; cli(); bit_true(sys_rt_exec_debug,EXEC_DEBUG_REPORT);
#endif
case CMD_FEED_OVR_RESET: system_set_exec_motion_override_flag(EXEC_FEED_OVR_RESET); break;
case CMD_FEED_OVR_COARSE_PLUS: system_set_exec_motion_override_flag(EXEC_FEED_OVR_COARSE_P
case CMD_FEED_OVR_COARSE_MINUS: system_set_exec_motion_override_flag(EXEC_FEED_OVR_COARSE_
case CMD_FEED_OVR_FINE_PLUS: system_set_exec_motion_override_flag(EXEC_FEED_OVR_FINE_PLUS); br
case CMD_FEED_OVR_FINE_MINUS: system_set_exec_motion_override_flag(EXEC_FEED_OVR_FINE_MINUS);
case CMD_RAPID_OVR_RESET: system_set_exec_motion_override_flag(EXEC_RAPID_OVR_RESET); break;
case CMD_RAPID_OVR_MEDIUM: system_set_exec_motion_override_flag(EXEC_RAPID_OVR_MEDIUM); break;
case CMD_RAPID_OVR_LOW: system_set_exec_motion_override_flag(EXEC_RAPID_OVR_LOW); break;
case CMD_SPINDLE_OVR_RESET: system_set_exec_accessory_override_flag(EXEC_SPINDLE_OVR_RESET); b
case CMD_SPINDLE_OVR_COARSE_PLUS: system_set_exec_accessory_override_flag(EXEC_SPINDLE_OVR_C
case CMD_SPINDLE_OVR_COARSE_MINUS: system_set_exec_accessory_override_flag(EXEC_SPINDLE_OVR_
case CMD_SPINDLE_OVR_FINE_PLUS: system_set_exec_accessory_override_flag(EXEC_SPINDLE_OVR_FINE_
case CMD_SPINDLE_OVR_FINE_MINUS: system_set_exec_accessory_override_flag(EXEC_SPINDLE_OVR_FINE
case CMD_SPINDLE_OVR_STOP: system_set_exec_accessory_override_flag(EXEC_SPINDLE_OVR_STOP); brea
case CMD_COOLANT_FLOOD_OVR_TOGGLE: system_set_exec_accessory_override_flag(EXEC_COOLANT_FLO
#ifdef ENABLE_M7
case CMD_COOLANT_MIST_OVR_TOGGLE: system_set_exec_accessory_override_flag(EXEC_COOLANT_MIST

```

```

    #endif
}
// Throw away any unfound extended-ASCII character by not passing it to the serial buffer.
} else { // Write character to buffer
    next_head = serial_rx_buffer_head + 1;
    if (next_head == RX_RING_BUFFER) { next_head = 0; }

    // Write data to buffer unless it is full.
    if (next_head != serial_rx_buffer_tail) {
        serial_rx_buffer[serial_rx_buffer_head] = data;
        serial_rx_buffer_head = next_head;
    }
}
}
}

void serial_reset_read_buffer()
{
    serial_rx_buffer_tail = serial_rx_buffer_head;
}

```

File: serial.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
serial.c - Low level functions for sending and receiving bytes via the serial port
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svalle Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 GNU General Public License for more details.

You should have received a copy of the GNU General Public License
 along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```

*/

#ifndef serial_h
#define serial_h

```

```

#ifndef RX_BUFFER_SIZE
#define RX_BUFFER_SIZE 128
#endif
#ifndef TX_BUFFER_SIZE
#ifdef USE_LINE_NUMBERS
#define TX_BUFFER_SIZE 112
#else
#define TX_BUFFER_SIZE 104
#endif
#endif

#define SERIAL_NO_DATA 0xff

void serial_init();

// Writes one byte to the TX serial buffer. Called by main program.
void serial_write(uint8_t data);

// Fetches the first byte in the serial read buffer. Called by main program.
uint8_t serial_read();

// Reset and empty data in read buffer. Used by e-stop and reset.
void serial_reset_read_buffer();

// Returns the number of bytes available in the RX serial buffer.
uint8_t serial_get_rx_buffer_available();

// Returns the number of bytes used in the RX serial buffer.
// NOTE: Deprecated. Not used unless classic status reports are enabled in config.h.
uint8_t serial_get_rx_buffer_count();

// Returns the number of bytes used in the TX serial buffer.
// NOTE: Not used except for debugging and ensuring no TX bottlenecks.
uint8_t serial_get_tx_buffer_count();

#endif

```

File: settings.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
settings.c - eeprom configuration handling
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#include "grbl.h"
```

```
settings_t settings;
```

```
const __flash settings_t defaults = {\n  .pulse_microseconds = DEFAULT_STEP_PULSE_MICROSECONDS,\n  .stepper_idle_lock_time = DEFAULT_STEPPER_IDLE_LOCK_TIME,\n  .step_invert_mask = DEFAULT_STEPPING_INVERT_MASK,\n  .dir_invert_mask = DEFAULT_DIRECTION_INVERT_MASK,\n  .status_report_mask = DEFAULT_STATUS_REPORT_MASK,\n  .junction_deviation = DEFAULT_JUNCTION_DEVIATION,\n  .arc_tolerance = DEFAULT_ARC_TOLERANCE,\n  .rpm_max = DEFAULT_SPINDLE_RPM_MAX,\n  .rpm_min = DEFAULT_SPINDLE_RPM_MIN,\n  .homing_dir_mask = DEFAULT_HOMING_DIR_MASK,\n  .homing_feed_rate = DEFAULT_HOMING_FEED_RATE,\n  .homing_seek_rate = DEFAULT_HOMING_SEEK_RATE,\n  .homing_debounce_delay = DEFAULT_HOMING_DEBOUNCE_DELAY,\n  .homing_pulloff = DEFAULT_HOMING_PULLOFF,\n  .flags = (DEFAULT_REPORT_INCHES << BIT_REPORT_INCHES) | \\\n    (DEFAULT_LASER_MODE << BIT_LASER_MODE) | \\\n    (DEFAULT_INVERT_ST_ENABLE << BIT_INVERT_ST_ENABLE) | \\\n    (DEFAULT_HARD_LIMIT_ENABLE << BIT_HARD_LIMIT_ENABLE) | \\\n    (DEFAULT_HOMING_ENABLE << BIT_HOMING_ENABLE) | \\\n    (DEFAULT_SOFT_LIMIT_ENABLE << BIT_SOFT_LIMIT_ENABLE) | \\\n    (DEFAULT_INVERT_LIMIT_PINS << BIT_INVERT_LIMIT_PINS) | \\\n    (DEFAULT_INVERT_PROBE_PIN << BIT_INVERT_PROBE_PIN),\n  .steps_per_mm[X_AXIS] = DEFAULT_X_STEPS_PER_MM,\n  .steps_per_mm[Y_AXIS] = DEFAULT_Y_STEPS_PER_MM,\n  .steps_per_mm[Z_AXIS] = DEFAULT_Z_STEPS_PER_MM,\n  .max_rate[X_AXIS] = DEFAULT_X_MAX_RATE,\n  .max_rate[Y_AXIS] = DEFAULT_Y_MAX_RATE,\n  .max_rate[Z_AXIS] = DEFAULT_Z_MAX_RATE,\n  .acceleration[X_AXIS] = DEFAULT_X_ACCELERATION,\n  .acceleration[Y_AXIS] = DEFAULT_Y_ACCELERATION,\n  .acceleration[Z_AXIS] = DEFAULT_Z_ACCELERATION,\n  .max_travel[X_AXIS] = (-DEFAULT_X_MAX_TRAVEL),
```

```
.max_travel[Y_AXIS] = (-DEFAULT_Y_MAX_TRAVEL),  
.max_travel[Z_AXIS] = (-DEFAULT_Z_MAX_TRAVEL));
```

```
// Method to store startup lines into EEPROM
```

```
void settings_store_startup_line(uint8_t n, char *line)
```

```
{  
  #ifdef FORCE_BUFFER_SYNC_DURING_EEPROM_WRITE  
    protocol_buffer_synchronize(); // A startup line may contain a motion and be executing.  
  #endif  
  uint32_t addr = n*(LINE_BUFFER_SIZE+1)+EEPROM_ADDR_STARTUP_BLOCK;  
  memcpy_to_eeprom_with_checksum(addr,(char*)line, LINE_BUFFER_SIZE);  
}
```

```
// Method to store build info into EEPROM
```

```
// NOTE: This function can only be called in IDLE state.
```

```
void settings_store_build_info(char *line)
```

```
{  
  // Build info can only be stored when state is IDLE.  
  memcpy_to_eeprom_with_checksum(EEPROM_ADDR_BUILD_INFO,(char*)line, LINE_BUFFER_SIZE);  
}
```

```
// Method to store coord data parameters into EEPROM
```

```
void settings_write_coord_data(uint8_t coord_select, float *coord_data)
```

```
{  
  #ifdef FORCE_BUFFER_SYNC_DURING_EEPROM_WRITE  
    protocol_buffer_synchronize();  
  #endif  
  uint32_t addr = coord_select*(sizeof(float)*N_AXIS+1) + EEPROM_ADDR_PARAMETERS;  
  memcpy_to_eeprom_with_checksum(addr,(char*)coord_data, sizeof(float)*N_AXIS);  
}
```

```
// Method to store Grbl global settings struct and version number into EEPROM
```

```
// NOTE: This function can only be called in IDLE state.
```

```
void write_global_settings()
```

```
{  
  eeprom_put_char(0, SETTINGS_VERSION);  
  memcpy_to_eeprom_with_checksum(EEPROM_ADDR_GLOBAL, (char*)&settings, sizeof(settings_t));  
}
```

```
// Method to restore EEPROM-saved Grbl global settings back to defaults.
```

```
void settings_restore(uint8_t restore_flag) {
```

```
  if (restore_flag & SETTINGS_RESTORE_DEFAULTS) {  
    settings = defaults;  
    write_global_settings();  
  }
```

```

if (restore_flag & SETTINGS_RESTORE_PARAMETERS) {
    uint8_t idx;
    float coord_data[N_AXIS];
    memset(&coord_data, 0, sizeof(coord_data));
    for (idx=0; idx <= SETTING_INDEX_NCOORD; idx++) { settings_write_coord_data(idx, coord_data); }
}

```

```

if (restore_flag & SETTINGS_RESTORE_STARTUP_LINES) {
    #if N_STARTUP_LINE > 0
        eeprom_put_char(EEPROM_ADDR_STARTUP_BLOCK, 0);
        eeprom_put_char(EEPROM_ADDR_STARTUP_BLOCK+1, 0); // Checksum
    #endif
    #if N_STARTUP_LINE > 1
        eeprom_put_char(EEPROM_ADDR_STARTUP_BLOCK+(LINE_BUFFER_SIZE+1), 0);
        eeprom_put_char(EEPROM_ADDR_STARTUP_BLOCK+(LINE_BUFFER_SIZE+2), 0); // Checksum
    #endif
}

```

```

if (restore_flag & SETTINGS_RESTORE_BUILD_INFO) {
    eeprom_put_char(EEPROM_ADDR_BUILD_INFO, 0);
    eeprom_put_char(EEPROM_ADDR_BUILD_INFO+1, 0); // Checksum
}
}

```

// Reads startup line from EEPROM. Updated pointed line string data.

```

uint8_t settings_read_startup_line(uint8_t n, char *line)
{
    uint32_t addr = n*(LINE_BUFFER_SIZE+1)+EEPROM_ADDR_STARTUP_BLOCK;
    if (!(memcpy_from_eeprom_with_checksum((char*)line, addr, LINE_BUFFER_SIZE))) {
        // Reset line with default value
        line[0] = 0; // Empty line
        settings_store_startup_line(n, line);
        return(false);
    }
    return(true);
}

```

// Reads startup line from EEPROM. Updated pointed line string data.

```

uint8_t settings_read_build_info(char *line)
{
    if (!(memcpy_from_eeprom_with_checksum((char*)line, EEPROM_ADDR_BUILD_INFO, LINE_BUFFER_SIZE))) {
        // Reset line with default value
        line[0] = 0; // Empty line
        settings_store_build_info(line);
        return(false);
    }
    return(true);
}

```

// Read selected coordinate data from EEPROM. Updates pointed coord_data value.

```
uint8_t settings_read_coord_data(uint8_t coord_select, float *coord_data)
{
    uint32_t addr = coord_select*(sizeof(float)*N_AXIS+1) + EEPROM_ADDR_PARAMETERS;
    if (!(memcpy_from_eeprom_with_checksum((char*)coord_data, addr, sizeof(float)*N_AXIS))) {
        // Reset with default zero vector
        clear_vector_float(coord_data);
        settings_write_coord_data(coord_select, coord_data);
        return(false);
    }
    return(true);
}
```

// Reads Grbl global settings struct from EEPROM.

```
uint8_t read_global_settings() {
    // Check version-byte of eeprom
    uint8_t version = eeprom_get_char(0);
    if (version == SETTINGS_VERSION) {
        // Read settings-record and check checksum
        if (!(memcpy_from_eeprom_with_checksum((char*)&settings, EEPROM_ADDR_GLOBAL, sizeof(settings_t)))) {
            return(false);
        }
    } else {
        return(false);
    }
    return(true);
}
```

// A helper method to set settings from command line

```
uint8_t settings_store_global_setting(uint8_t parameter, float value) {
    if (value < 0.0) { return(STATUS_NEGATIVE_VALUE); }
    if (parameter >= AXIS_SETTINGS_START_VAL) {
        // Store axis configuration. Axis numbering sequence set by AXIS_SETTING defines.
        // NOTE: Ensure the setting index corresponds to the report.c settings printout.
        parameter -= AXIS_SETTINGS_START_VAL;
        uint8_t set_idx = 0;
        while (set_idx < AXIS_N_SETTINGS) {
            if (parameter < N_AXIS) {
                // Valid axis setting found.
                switch (set_idx) {
                    case 0:
                        #ifdef MAX_STEP_RATE_HZ
                            if (value*settings.max_rate[parameter] > (MAX_STEP_RATE_HZ*60.0)) { return(STATUS_MAX_STEP_RATE_EXCEEDED); }
                        #endif
                        settings.steps_per_mm[parameter] = value;
                        break;
                    case 1:
                        #ifdef MAX_STEP_RATE_HZ
```

```

        if (value*settings.steps_per_mm[parameter] > (MAX_STEP_RATE_HZ*60.0)) { return(STATUS_MAX_STEP_RATE_EXCEEDED); }
    #endif
    settings.max_rate[parameter] = value;
    break;
    case 2: settings.acceleration[parameter] = value*60*60; break; // Convert to mm/min^2 for grbl internal use.
    case 3: settings.max_travel[parameter] = -value; break; // Store as negative for grbl internal use.
}
break; // Exit while-loop after setting has been configured and proceed to the EEPROM write call.
} else {
    set_idx++;
    // If axis index greater than N_AXIS or setting index greater than number of axis settings, error out.
    if ((parameter < AXIS_SETTINGS_INCREMENT) || (set_idx == AXIS_N_SETTINGS)) { return(STATUS_INVALID_SETTING_INDEX); }
    parameter -= AXIS_SETTINGS_INCREMENT;
}
}
} else {
    // Store non-axis Grbl settings
    uint8_t int_value = trunc(value);
    switch(parameter) {
        case 0:
            if (int_value < 3) { return(STATUS_SETTING_STEP_PULSE_MIN); }
            settings.pulse_microseconds = int_value; break;
        case 1: settings.stepper_idle_lock_time = int_value; break;
        case 2:
            settings.step_invert_mask = int_value;
            st_generate_step_dir_invert_masks(); // Regenerate step and direction port invert masks.
            break;
        case 3:
            settings.dir_invert_mask = int_value;
            st_generate_step_dir_invert_masks(); // Regenerate step and direction port invert masks.
            break;
        case 4: // Reset to ensure change. Immediate re-init may cause problems.
            if (int_value) { settings.flags |= BITFLAG_INVERT_ST_ENABLE; }
            else { settings.flags &= ~BITFLAG_INVERT_ST_ENABLE; }
            break;
        case 5: // Reset to ensure change. Immediate re-init may cause problems.
            if (int_value) { settings.flags |= BITFLAG_INVERT_LIMIT_PINS; }
            else { settings.flags &= ~BITFLAG_INVERT_LIMIT_PINS; }
            break;
        case 6: // Reset to ensure change. Immediate re-init may cause problems.
            if (int_value) { settings.flags |= BITFLAG_INVERT_PROBE_PIN; }
            else { settings.flags &= ~BITFLAG_INVERT_PROBE_PIN; }
            probe_configure_invert_mask(false);
            break;
        case 10: settings.status_report_mask = int_value; break;
        case 11: settings.junction_deviation = value; break;
        case 12: settings.arc_tolerance = value; break;
        case 13:
            if (int_value) { settings.flags |= BITFLAG_REPORT_INCHES; }
            else { settings.flags &= ~BITFLAG_REPORT_INCHES; }
            system_flag_wco_change(); // Make sure WCO is immediately updated.
    }
}
}

```

```

    break;
case 20:
    if (int_value) {
        if (bit_isfalse(settings.flags, BITFLAG_HOMING_ENABLE)) { return(STATUS_SOFT_LIMIT_ERROR); }
        settings.flags |= BITFLAG_SOFT_LIMIT_ENABLE;
    } else { settings.flags &= ~BITFLAG_SOFT_LIMIT_ENABLE; }
    break;
case 21:
    if (int_value) { settings.flags |= BITFLAG_HARD_LIMIT_ENABLE; }
    else { settings.flags &= ~BITFLAG_HARD_LIMIT_ENABLE; }
    limits_init(); // Re-init to immediately change. NOTE: Nice to have but could be problematic later.
    break;
case 22:
    if (int_value) { settings.flags |= BITFLAG_HOMING_ENABLE; }
    else {
        settings.flags &= ~BITFLAG_HOMING_ENABLE;
        settings.flags &= ~BITFLAG_SOFT_LIMIT_ENABLE; // Force disable soft-limits.
    }
    break;
case 23: settings.homing_dir_mask = int_value; break;
case 24: settings.homing_feed_rate = value; break;
case 25: settings.homing_seek_rate = value; break;
case 26: settings.homing_debounce_delay = int_value; break;
case 27: settings.homing_pulloff = value; break;
case 30: settings.rpm_max = value; spindle_init(); break; // Re-initialize spindle rpm calibration
case 31: settings.rpm_min = value; spindle_init(); break; // Re-initialize spindle rpm calibration
case 32:
    #ifdef VARIABLE_SPINDLE
        if (int_value) { settings.flags |= BITFLAG_LASER_MODE; }
        else { settings.flags &= ~BITFLAG_LASER_MODE; }
    #else
        return(STATUS_SETTING_DISABLED_LASER);
    #endif
    break;
default:
    return(STATUS_INVALID_STATEMENT);
}
}
write_global_settings();
return(STATUS_OK);
}

```

// Initialize the config subsystem

```

void settings_init() {
    if(!read_global_settings()) {
        report_status_message(STATUS_SETTING_READ_FAIL);
        settings_restore(SETTINGS_RESTORE_ALL); // Force restore all EEPROM data.
        report_grbl_settings();
    }
}

```

// Returns step pin mask according to Grbl internal axis indexing.

```
uint8_t get_step_pin_mask(uint8_t axis_idx)
{
  if ( axis_idx == X_AXIS ) { return((1<<X_STEP_BIT)); }
  if ( axis_idx == Y_AXIS ) { return((1<<Y_STEP_BIT)); }
  return((1<<Z_STEP_BIT));
}
```

// Returns direction pin mask according to Grbl internal axis indexing.

```
uint8_t get_direction_pin_mask(uint8_t axis_idx)
{
  if ( axis_idx == X_AXIS ) { return((1<<X_DIRECTION_BIT)); }
  if ( axis_idx == Y_AXIS ) { return((1<<Y_DIRECTION_BIT)); }
  return((1<<Z_DIRECTION_BIT));
}
```

// Returns limit pin mask according to Grbl internal axis indexing.

```
uint8_t get_limit_pin_mask(uint8_t axis_idx)
{
  if ( axis_idx == X_AXIS ) { return((1<<X_LIMIT_BIT)); }
  if ( axis_idx == Y_AXIS ) { return((1<<Y_LIMIT_BIT)); }
  return((1<<Z_LIMIT_BIT));
}
```

File: settings.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

settings.h - eeprom configuration handling

Part of Grbl

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC

Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify

it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,

but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License

along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#ifndef settings_h
#define settings_h
```

```
#include "grbl.h"
```

```
// Version of the EEPROM data. Will be used to migrate existing data from older versions of Grbl
// when firmware is upgraded. Always stored in byte 0 of eeprom
#define SETTINGS_VERSION 10 // NOTE: Check settings_reset() when moving to next version.
```

```
// Define bit flag masks for the boolean settings in settings.flag.
```

```
#define BIT_REPORT_INCHES    0
#define BIT_LASER_MODE      1
#define BIT_INVERT_ST_ENABLE 2
#define BIT_HARD_LIMIT_ENABLE 3
#define BIT_HOMING_ENABLE   4
#define BIT_SOFT_LIMIT_ENABLE 5
#define BIT_INVERT_LIMIT_PINS 6
#define BIT_INVERT_PROBE_PIN 7
```

```
#define BITFLAG_REPORT_INCHES    bit(BIT_REPORT_INCHES)
#define BITFLAG_LASER_MODE      bit(BIT_LASER_MODE)
#define BITFLAG_INVERT_ST_ENABLE bit(BIT_INVERT_ST_ENABLE)
#define BITFLAG_HARD_LIMIT_ENABLE bit(BIT_HARD_LIMIT_ENABLE)
#define BITFLAG_HOMING_ENABLE   bit(BIT_HOMING_ENABLE)
#define BITFLAG_SOFT_LIMIT_ENABLE bit(BIT_SOFT_LIMIT_ENABLE)
#define BITFLAG_INVERT_LIMIT_PINS bit(BIT_INVERT_LIMIT_PINS)
#define BITFLAG_INVERT_PROBE_PIN bit(BIT_INVERT_PROBE_PIN)
```

```
// Define status reporting boolean enable bit flags in settings.status_report_mask
```

```
#define BITFLAG_RT_STATUS_POSITION_TYPE    bit(0)
#define BITFLAG_RT_STATUS_BUFFER_STATE    bit(1)
```

```
// Define settings restore bitflags.
```

```
#define SETTINGS_RESTORE_DEFAULTS bit(0)
#define SETTINGS_RESTORE_PARAMETERS bit(1)
#define SETTINGS_RESTORE_STARTUP_LINES bit(2)
#define SETTINGS_RESTORE_BUILD_INFO bit(3)
#ifndef SETTINGS_RESTORE_ALL
  #define SETTINGS_RESTORE_ALL 0xFF // All bitflags
#endif
```

```
// Define EEPROM memory address location values for Grbl settings and parameters
```

```
// NOTE: The Atmega328p has 1KB EEPROM. The upper half is reserved for parameters and
// the startup script. The lower half contains the global settings and space for future
// developments.
```

```
#define EEPROM_ADDR_GLOBAL    1U
#define EEPROM_ADDR_PARAMETERS 512U
```

```

#define EEPROM_ADDR_STARTUP_BLOCK 768U
#define EEPROM_ADDR_BUILD_INFO 942U

// Define EEPROM address indexing for coordinate parameters
#define N_COORDINATE_SYSTEM 6 // Number of supported work coordinate systems (from index 1)
#define SETTING_INDEX_NCOORD N_COORDINATE_SYSTEM+1 // Total number of system stored (from index 0)
// NOTE: Work coordinate indices are (0=G54, 1=G55, ... , 6=G59)
#define SETTING_INDEX_G28 N_COORDINATE_SYSTEM // Home position 1
#define SETTING_INDEX_G30 N_COORDINATE_SYSTEM+1 // Home position 2
// #define SETTING_INDEX_G92 N_COORDINATE_SYSTEM+2 // Coordinate offset (G92.2,G92.3 not supported)

// Define Grbl axis settings numbering scheme. Starts at START_VAL, every INCREMENT, over N_SETTINGS.
#define AXIS_N_SETTINGS 4
#define AXIS_SETTINGS_START_VAL 100 // NOTE: Reserving settings values >= 100 for axis settings. Up to 255.
#define AXIS_SETTINGS_INCREMENT 10 // Must be greater than the number of axis settings

// Global persistent settings (Stored from byte EEPROM_ADDR_GLOBAL onwards)
typedef struct {
  // Axis settings
  float steps_per_mm[N_AXIS];
  float max_rate[N_AXIS];
  float acceleration[N_AXIS];
  float max_travel[N_AXIS];

  // Remaining Grbl settings
  uint8_t pulse_microseconds;
  uint8_t step_invert_mask;
  uint8_t dir_invert_mask;
  uint8_t stepper_idle_lock_time; // If max value 255, steppers do not disable.
  uint8_t status_report_mask; // Mask to indicate desired report data.
  float junction_deviation;
  float arc_tolerance;

  float rpm_max;
  float rpm_min;

  uint8_t flags; // Contains default boolean settings

  uint8_t homing_dir_mask;
  float homing_feed_rate;
  float homing_seek_rate;
  uint16_t homing_debounce_delay;
  float homing_pulloff;
} settings_t;
extern settings_t settings;

// Initialize the configuration subsystem (load settings from EEPROM)
void settings_init();

// Helper function to clear and restore EEPROM defaults
void settings_restore(uint8_t restore_flag);

```

```

// A helper method to set new settings from command line
uint8_t settings_store_global_setting(uint8_t parameter, float value);

// Stores the protocol line variable as a startup line in EEPROM
void settings_store_startup_line(uint8_t n, char *line);

// Reads an EEPROM startup line to the protocol line variable
uint8_t settings_read_startup_line(uint8_t n, char *line);

// Stores build info user-defined string
void settings_store_build_info(char *line);

// Reads build info user-defined string
uint8_t settings_read_build_info(char *line);

// Writes selected coordinate data to EEPROM
void settings_write_coord_data(uint8_t coord_select, float *coord_data);

// Reads selected coordinate data from EEPROM
uint8_t settings_read_coord_data(uint8_t coord_select, float *coord_data);

// Returns the step pin mask according to Grbl's internal axis numbering
uint8_t get_step_pin_mask(uint8_t i);

// Returns the direction pin mask according to Grbl's internal axis numbering
uint8_t get_direction_pin_mask(uint8_t i);

// Returns the limit pin mask according to Grbl's internal axis numbering
uint8_t get_limit_pin_mask(uint8_t i);

#endif

```

File: spindle_control.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
spindle_control.c - spindle control methods
Part of Grbl

```

Copyright (c) 2012-2017 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#include "grbl.h"
```

```
#ifndef VARIABLE_SPINDLE
  static float pwm_gradient; // Precalulated value to speed up rpm to PWM conversions.
#endif
```

```
void spindle_init()
{
  #ifdef VARIABLE_SPINDLE
    // Configure variable spindle PWM and enable pin, if required. On the Uno, PWM and enable are
    // combined unless configured otherwise.
    SPINDLE_PWM_DDR |= (1<<SPINDLE_PWM_BIT); // Configure as PWM output pin.
    SPINDLE_TCCRA_REGISTER = SPINDLE_TCCRA_INIT_MASK; // Configure PWM output compare timer
    SPINDLE_TCCRB_REGISTER = SPINDLE_TCCRB_INIT_MASK;
    #ifdef USE_SPINDLE_DIR_AS_ENABLE_PIN
      SPINDLE_ENABLE_DDR |= (1<<SPINDLE_ENABLE_BIT); // Configure as output pin.
    #else
      #ifndef ENABLE_DUAL_AXIS
        SPINDLE_DIRECTION_DDR |= (1<<SPINDLE_DIRECTION_BIT); // Configure as output pin.
      #endif
    #endif
    pwm_gradient = SPINDLE_PWM_RANGE/(settings.rpm_max-settings.rpm_min);
  #else
    SPINDLE_ENABLE_DDR |= (1<<SPINDLE_ENABLE_BIT); // Configure as output pin.
    #ifndef ENABLE_DUAL_AXIS
      SPINDLE_DIRECTION_DDR |= (1<<SPINDLE_DIRECTION_BIT); // Configure as output pin.
    #endif
  #endif

  spindle_stop();
}
```

```
uint8_t spindle_get_state()
{
  #ifdef VARIABLE_SPINDLE
    #ifdef USE_SPINDLE_DIR_AS_ENABLE_PIN
      // No spindle direction output pin.
    #endif
    #ifdef INVERT_SPINDLE_ENABLE_PIN
      if (bit_isfalse(SPINDLE_ENABLE_PORT,(1<<SPINDLE_ENABLE_BIT))) { return(SPINDLE_STATE_CW); }
    #endif
  #endif
}
```

```

    #else
        if (bit_istrue(SPINDLE_ENABLE_PORT,(1<<SPINDLE_ENABLE_BIT))) { return(SPINDLE_STATE_CW); }
    #endif
#else
    if (SPINDLE_TCCRA_REGISTER & (1<<SPINDLE_COMB_BIT)) { // Check if PWM is enabled.
        #ifdef ENABLE_DUAL_AXIS
            return(SPINDLE_STATE_CW);
        #else
            if (SPINDLE_DIRECTION_PORT & (1<<SPINDLE_DIRECTION_BIT)) { return(SPINDLE_STATE_CCW); }
            else { return(SPINDLE_STATE_CW); }
        #endif
    }
#endif
#else
    #ifdef INVERT_SPINDLE_ENABLE_PIN
        if (bit_isfalse(SPINDLE_ENABLE_PORT,(1<<SPINDLE_ENABLE_BIT))) {
    #else
        if (bit_istrue(SPINDLE_ENABLE_PORT,(1<<SPINDLE_ENABLE_BIT))) {
    #endif
        #ifdef ENABLE_DUAL_AXIS
            return(SPINDLE_STATE_CW);
        #else
            if (SPINDLE_DIRECTION_PORT & (1<<SPINDLE_DIRECTION_BIT)) { return(SPINDLE_STATE_CCW); }
            else { return(SPINDLE_STATE_CW); }
        #endif
    }
#endif
return(SPINDLE_STATE_DISABLE);
}

```

// Disables the spindle and sets PWM output to zero when PWM variable spindle speed is enabled.

// Called by various main program and ISR routines. Keep routine small, fast, and efficient.

// Called by spindle_init(), spindle_set_speed(), spindle_set_state(), and mc_reset().

void spindle_stop()

```

{
    #ifdef VARIABLE_SPINDLE
        SPINDLE_TCCRA_REGISTER &= ~(1<<SPINDLE_COMB_BIT); // Disable PWM. Output voltage is zero.
    #ifdef USE_SPINDLE_DIR_AS_ENABLE_PIN
        #ifdef INVERT_SPINDLE_ENABLE_PIN
            SPINDLE_ENABLE_PORT |= (1<<SPINDLE_ENABLE_BIT); // Set pin to high
        #else
            SPINDLE_ENABLE_PORT &= ~(1<<SPINDLE_ENABLE_BIT); // Set pin to low
        #endif
    #endif
#else
    #ifdef INVERT_SPINDLE_ENABLE_PIN
        SPINDLE_ENABLE_PORT |= (1<<SPINDLE_ENABLE_BIT); // Set pin to high
    #else
        SPINDLE_ENABLE_PORT &= ~(1<<SPINDLE_ENABLE_BIT); // Set pin to low
    #endif
}

```

```

#endif
}

#ifdef VARIABLE_SPINDLE
// Sets spindle speed PWM output and enable pin, if configured. Called by spindle_set_state()
// and stepper ISR. Keep routine small and efficient.
void spindle_set_speed(uint8_t pwm_value)
{
    SPINDLE_OCR_REGISTER = pwm_value; // Set PWM output level.
#ifdef SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED
    if (pwm_value == SPINDLE_PWM_OFF_VALUE) {
        spindle_stop();
    } else {
        SPINDLE_TCCRA_REGISTER |= (1<<SPINDLE_COMB_BIT); // Ensure PWM output is enabled.
#ifdef INVERT_SPINDLE_ENABLE_PIN
        SPINDLE_ENABLE_PORT &= ~(1<<SPINDLE_ENABLE_BIT);
#else
        SPINDLE_ENABLE_PORT |= (1<<SPINDLE_ENABLE_BIT);
#endif
    }
#else
    if (pwm_value == SPINDLE_PWM_OFF_VALUE) {
        SPINDLE_TCCRA_REGISTER &= ~(1<<SPINDLE_COMB_BIT); // Disable PWM. Output voltage is zero.
    } else {
        SPINDLE_TCCRA_REGISTER |= (1<<SPINDLE_COMB_BIT); // Ensure PWM output is enabled.
    }
#endif
}

#ifdef ENABLE_PIECEWISE_LINEAR_SPINDLE

// Called by spindle_set_state() and step segment generator. Keep routine small and efficient.
uint8_t spindle_compute_pwm_value(float rpm) // 328p PWM register is 8-bit.
{
    uint8_t pwm_value;
    rpm *= (0.010*sys.spindle_speed_ovr); // Scale by spindle speed override value.
    // Calculate PWM register value based on rpm max/min settings and programmed rpm.
    if ((settings.rpm_min >= settings.rpm_max) || (rpm >= RPM_MAX)) {
        rpm = RPM_MAX;
        pwm_value = SPINDLE_PWM_MAX_VALUE;
    } else if (rpm <= RPM_MIN) {
        if (rpm == 0.0) { // S0 disables spindle
            pwm_value = SPINDLE_PWM_OFF_VALUE;
        } else {
            rpm = RPM_MIN;
            pwm_value = SPINDLE_PWM_MIN_VALUE;
        }
    } else {
        // Compute intermediate PWM value with linear spindle speed model via piecewise linear fit model.

```

```

    #if (N_PIECES > 3)
        if (rpm > RPM_POINT34) {
            pwm_value = floor(RPM_LINE_A4*rpm - RPM_LINE_B4);
        } else
    #endif
    #if (N_PIECES > 2)
        if (rpm > RPM_POINT23) {
            pwm_value = floor(RPM_LINE_A3*rpm - RPM_LINE_B3);
        } else
    #endif
    #if (N_PIECES > 1)
        if (rpm > RPM_POINT12) {
            pwm_value = floor(RPM_LINE_A2*rpm - RPM_LINE_B2);
        } else
    #endif
    {
        pwm_value = floor(RPM_LINE_A1*rpm - RPM_LINE_B1);
    }
}
sys.spindle_speed = rpm;
return(pwm_value);
}

#else

// Called by spindle_set_state() and step segment generator. Keep routine small and efficient.
uint8_t spindle_compute_pwm_value(float rpm) // 328p PWM register is 8-bit.
{
    uint8_t pwm_value;
    rpm *= (0.010*sys.spindle_speed_ovr); // Scale by spindle speed override value.
    // Calculate PWM register value based on rpm max/min settings and programmed rpm.
    if ((settings.rpm_min >= settings.rpm_max) || (rpm >= settings.rpm_max)) {
        // No PWM range possible. Set simple on/off spindle control pin state.
        sys.spindle_speed = settings.rpm_max;
        pwm_value = SPINDLE_PWM_MAX_VALUE;
    } else if (rpm <= settings.rpm_min) {
        if (rpm == 0.0) { // S0 disables spindle
            sys.spindle_speed = 0.0;
            pwm_value = SPINDLE_PWM_OFF_VALUE;
        } else { // Set minimum PWM output
            sys.spindle_speed = settings.rpm_min;
            pwm_value = SPINDLE_PWM_MIN_VALUE;
        }
    } else {
        // Compute intermediate PWM value with linear spindle speed model.
        // NOTE: A nonlinear model could be installed here, if required, but keep it VERY light-weight.
        sys.spindle_speed = rpm;
        pwm_value = floor((rpm-settings.rpm_min)*pwm_gradient) + SPINDLE_PWM_MIN_VALUE;
    }
    return(pwm_value);
}

```

```
#endif
#endif
```

```
// Immediately sets spindle running state with direction and spindle rpm via PWM, if enabled.
// Called by g-code parser spindle_sync(), parking retract and restore, g-code program end,
// sleep, and spindle stop override.
```

```
#ifndef VARIABLE_SPINDLE
```

```
void spindle_set_state(uint8_t state, float rpm)
```

```
#else
```

```
void _spindle_set_state(uint8_t state)
```

```
#endif
```

```
{
```

```
if (sys.abort) { return; } // Block during abort.
```

```
if (state == SPINDLE_DISABLE) { // Halt or set spindle direction and rpm.
```

```
#ifndef VARIABLE_SPINDLE
```

```
sys.spindle_speed = 0.0;
```

```
#endif
```

```
spindle_stop();
```

```
} else {
```

```
#if !defined(USE_SPINDLE_DIR_AS_ENABLE_PIN) && !defined(ENABLE_DUAL_AXIS)
```

```
if (state == SPINDLE_ENABLE_CW) {
```

```
SPINDLE_DIRECTION_PORT &= ~(1<<SPINDLE_DIRECTION_BIT);
```

```
} else {
```

```
SPINDLE_DIRECTION_PORT |= (1<<SPINDLE_DIRECTION_BIT);
```

```
}
```

```
#endif
```

```
#ifndef VARIABLE_SPINDLE
```

```
// NOTE: Assumes all calls to this function is when Grbl is not moving or must remain off.
```

```
if (settings.flags & BITFLAG_LASER_MODE) {
```

```
if (state == SPINDLE_ENABLE_CCW) { rpm = 0.0; } // TODO: May need to be rpm_min*(100/MAX_SPINDLE_SPEED)
```

```
}
```

```
spindle_set_speed(spindle_compute_pwm_value(rpm));
```

```
#endif
```

```
#if (defined(USE_SPINDLE_DIR_AS_ENABLE_PIN) && \
```

```
!defined(SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED)) || !defined(VARIABLE_SPINDLE)
```

```
// NOTE: Without variable spindle, the enable bit should just turn on or off, regardless
```

```
// if the spindle speed value is zero, as its ignored anyhow.
```

```
#ifndef INVERT_SPINDLE_ENABLE_PIN
```

```
SPINDLE_ENABLE_PORT &= ~(1<<SPINDLE_ENABLE_BIT);
```

```
#else
```

```
SPINDLE_ENABLE_PORT |= (1<<SPINDLE_ENABLE_BIT);
```

```
#endif
```

```
#endif
```

```

}

sys.report_ovr_counter = 0; // Set to report change immediately
}

// G-code parser entry-point for setting spindle state. Forces a planner buffer sync and bails
// if an abort or check-mode is active.
#ifdef VARIABLE_SPINDLE
void spindle_sync(uint8_t state, float rpm)
{
    if (sys.state == STATE_CHECK_MODE) { return; }
    protocol_buffer_synchronize(); // Empty planner buffer to ensure spindle is set when programmed.
    spindle_set_state(state,rpm);
}
#else
void _spindle_sync(uint8_t state)
{
    if (sys.state == STATE_CHECK_MODE) { return; }
    protocol_buffer_synchronize(); // Empty planner buffer to ensure spindle is set when programmed.
    _spindle_set_state(state);
}
#endif

```

File: spindle_control.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
spindle_control.h - spindle control methods
Part of Grbl

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC
Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <http://www.gnu.org/licenses/>.

*/

#ifdef spindle_control_h

```

```

#define spindle_control_h

#define SPINDLE_NO_SYNC false
#define SPINDLE_FORCE_SYNC true

#define SPINDLE_STATE_DISABLE 0 // Must be zero.
#define SPINDLE_STATE_CW      bit(0)
#define SPINDLE_STATE_CCW     bit(1)

// Initializes spindle pins and hardware PWM, if enabled.
void spindle_init();

// Returns current spindle output state. Overrides may alter it from programmed states.
uint8_t spindle_get_state();

// Called by g-code parser when setting spindle state and requires a buffer sync.
// Immediately sets spindle running state with direction and spindle rpm via PWM, if enabled.
// Called by spindle_sync() after sync and parking motion/spindle stop override during restore.
#ifdef VARIABLE_SPINDLE

// Called by g-code parser when setting spindle state and requires a buffer sync.
void spindle_sync(uint8_t state, float rpm);

// Sets spindle running state with direction, enable, and spindle PWM.
void spindle_set_state(uint8_t state, float rpm);

// Sets spindle PWM quickly for stepper ISR. Also called by spindle_set_state().
// NOTE: 328p PWM register is 8-bit.
void spindle_set_speed(uint8_t pwm_value);

// Computes 328p-specific PWM register value for the given RPM for quick updating.
uint8_t spindle_compute_pwm_value(float rpm);

#else

// Called by g-code parser when setting spindle state and requires a buffer sync.
#define spindle_sync(state, rpm) _spindle_sync(state)
void _spindle_sync(uint8_t state);

// Sets spindle running state with direction and enable.
#define spindle_set_state(state, rpm) _spindle_set_state(state)
void _spindle_set_state(uint8_t state);

#endif

// Stop and start spindle routines. Called by all spindle routines and stepper ISR.
void spindle_stop();

#endif

```

File: stepper.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```
/*
stepper.c - stepper motor driver: executes motion plans using stepper motors
Part of Grbl

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <http://www.gnu.org/licenses/>.
*/

#include "grbl.h"

// Some useful constants.
#define DT_SEGMENT (1.0/(ACCELERATION_TICKS_PER_SECOND*60.0)) // min/segment
#define REQ_MM_INCREMENT_SCALAR 1.25
#define RAMP_ACCEL 0
#define RAMP_CRUISE 1
#define RAMP_DECEL 2
#define RAMP_DECEL_OVERRIDE 3

#define PREP_FLAG_RECALCULATE bit(0)
#define PREP_FLAG_HOLD_PARTIAL_BLOCK bit(1)
#define PREP_FLAG_PARKING bit(2)
#define PREP_FLAG_DECEL_OVERRIDE bit(3)

// Define Adaptive Multi-Axis Step-Smoothing(AMASS) levels and cutoff frequencies. The highest level
// frequency bin starts at 0Hz and ends at its cutoff frequency. The next lower level frequency bin
// starts at the next higher cutoff frequency, and so on. The cutoff frequencies for each level must
// be considered carefully against how much it over-drives the stepper ISR, the accuracy of the 16-bit
// timer, and the CPU overhead. Level 0 (no AMASS, normal operation) frequency bin starts at the
// Level 1 cutoff frequency and up to as fast as the CPU allows (over 30kHz in limited testing).
// NOTE: AMASS cutoff frequency multiplied by ISR overdrive factor must not exceed maximum step frequency.
// NOTE: Current settings are set to overdrive the ISR to no more than 16kHz, balancing CPU overhead
```

```

// and timer accuracy. Do not alter these settings unless you know what you are doing.
#ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
    #define MAX_AMASS_LEVEL 3
    // AMASS_LEVEL0: Normal operation. No AMASS. No upper cutoff frequency. Starts at LEVEL1 cutoff frequency.
    #define AMASS_LEVEL1 (F_CPU/8000) // Over-drives ISR (x2). Defined as F_CPU/(Cutoff frequency in Hz)
    #define AMASS_LEVEL2 (F_CPU/4000) // Over-drives ISR (x4)
    #define AMASS_LEVEL3 (F_CPU/2000) // Over-drives ISR (x8)

    #if MAX_AMASS_LEVEL <= 0
        error "AMASS must have 1 or more levels to operate correctly."
    #endif
#endif

// Stores the planner block Bresenham algorithm execution data for the segments in the segment
// buffer. Normally, this buffer is partially in-use, but, for the worst case scenario, it will
// never exceed the number of accessible stepper buffer segments (SEGMENT_BUFFER_SIZE-1).
// NOTE: This data is copied from the prepped planner blocks so that the planner blocks may be
// discarded when entirely consumed and completed by the segment buffer. Also, AMASS alters this
// data for its own use.
typedef struct {
    uint32_t steps[N_AXIS];
    uint32_t step_event_count;
    uint8_t direction_bits;
    #ifdef ENABLE_DUAL_AXIS
        uint8_t direction_bits_dual;
    #endif
    #ifdef VARIABLE_SPINDLE
        uint8_t is_pwm_rate_adjusted; // Tracks motions that require constant laser power/rate
    #endif
} st_block_t;
static st_block_t st_block_buffer[SEGMENT_BUFFER_SIZE-1];

// Primary stepper segment ring buffer. Contains small, short line segments for the stepper
// algorithm to execute, which are "checked-out" incrementally from the first block in the
// planner buffer. Once "checked-out", the steps in the segments buffer cannot be modified by
// the planner, where the remaining planner block steps still can.
typedef struct {
    uint16_t n_step; // Number of step events to be executed for this segment
    uint16_t cycles_per_tick; // Step distance traveled per ISR tick, aka step rate.
    uint8_t st_block_index; // Stepper block data index. Uses this information to execute this segment.
    #ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
        uint8_t amass_level; // Indicates AMASS level for the ISR to execute this segment
    #else
        uint8_t prescaler; // Without AMASS, a prescaler is required to adjust for slow timing.
    #endif
    #ifdef VARIABLE_SPINDLE
        uint8_t spindle_pwm;
    #endif
} segment_t;
static segment_t segment_buffer[SEGMENT_BUFFER_SIZE];

```

```

// Stepper ISR data struct. Contains the running data for the main stepper ISR.
typedef struct {
    // Used by the bresenham line algorithm
    uint32_t counter_x,    // Counter variables for the bresenham line tracer
        counter_y,
        counter_z;
#ifdef STEP_PULSE_DELAY
    uint8_t step_bits; // Stores out_bits output to complete the step pulse delay
#endif

    uint8_t execute_step; // Flags step execution for each interrupt.
    uint8_t step_pulse_time; // Step pulse reset time after step rise
    uint8_t step_outbits;    // The next stepping-bits to be output
    uint8_t dir_outbits;
#ifdef ENABLE_DUAL_AXIS
    uint8_t step_outbits_dual;
    uint8_t dir_outbits_dual;
#endif
#ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
    uint32_t steps[N_AXIS];
#endif

    uint16_t step_count; // Steps remaining in line segment motion
    uint8_t exec_block_index; // Tracks the current st_block index. Change indicates new block.
    st_block_t *exec_block; // Pointer to the block data for the segment being executed
    segment_t *exec_segment; // Pointer to the segment being executed
} stepper_t;
static stepper_t st;

// Step segment ring buffer indices
static volatile uint8_t segment_buffer_tail;
static uint8_t segment_buffer_head;
static uint8_t segment_next_head;

// Step and direction port invert masks.
static uint8_t step_port_invert_mask;
static uint8_t dir_port_invert_mask;
#ifdef ENABLE_DUAL_AXIS
    static uint8_t step_port_invert_mask_dual;
    static uint8_t dir_port_invert_mask_dual;
#endif

// Used to avoid ISR nesting of the "Stepper Driver Interrupt". Should never occur though.
static volatile uint8_t busy;

// Pointers for the step segment being prepped from the planner buffer. Accessed only by the
// main program. Pointers may be planning segments or planner blocks ahead of what being executed.
static plan_block_t *pl_block; // Pointer to the planner block being prepped
static st_block_t *st_prep_block; // Pointer to the stepper block data being prepped

```

```

// Segment preparation data struct. Contains all the necessary information to compute new segments
// based on the current executing planner block.
typedef struct {
    uint8_t st_block_index; // Index of stepper common data block being prepped
    uint8_t recalculate_flag;

    float dt_remainder;
    float steps_remaining;
    float step_per_mm;
    float req_mm_increment;

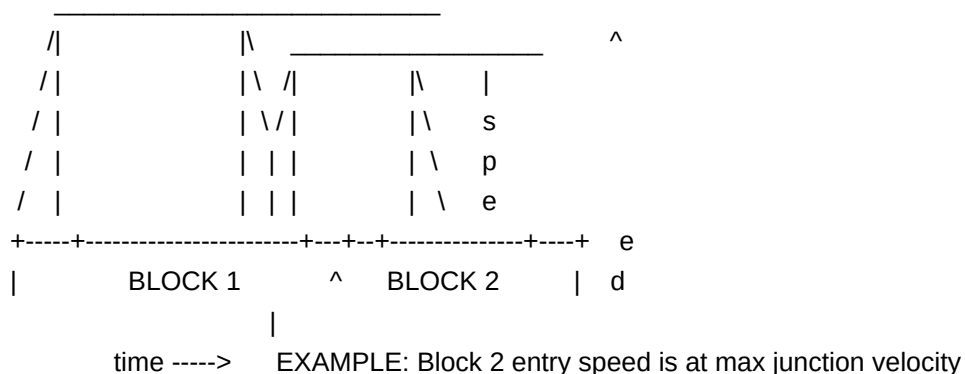
#ifdef PARKING_ENABLE
    uint8_t last_st_block_index;
    float last_steps_remaining;
    float last_step_per_mm;
    float last_dt_remainder;
#endif

    uint8_t ramp_type; // Current segment ramp state
    float mm_complete; // End of velocity profile from end of current planner block in (mm).
                        // NOTE: This value must coincide with a step(no mantissa) when converted.
    float current_speed; // Current speed at the end of the segment buffer (mm/min)
    float maximum_speed; // Maximum speed of executing block. Not always nominal speed. (mm/min)
    float exit_speed; // Exit speed of executing block (mm/min)
    float accelerate_until; // Acceleration ramp end measured from end of block (mm)
    float decelerate_after; // Deceleration ramp start measured from end of block (mm)

#ifdef VARIABLE_SPINDLE
    float inv_rate; // Used by PWM laser mode to speed up segment calculations.
    uint8_t current_spindle_pwm;
#endif
} st_prep_t;
static st_prep_t prep;

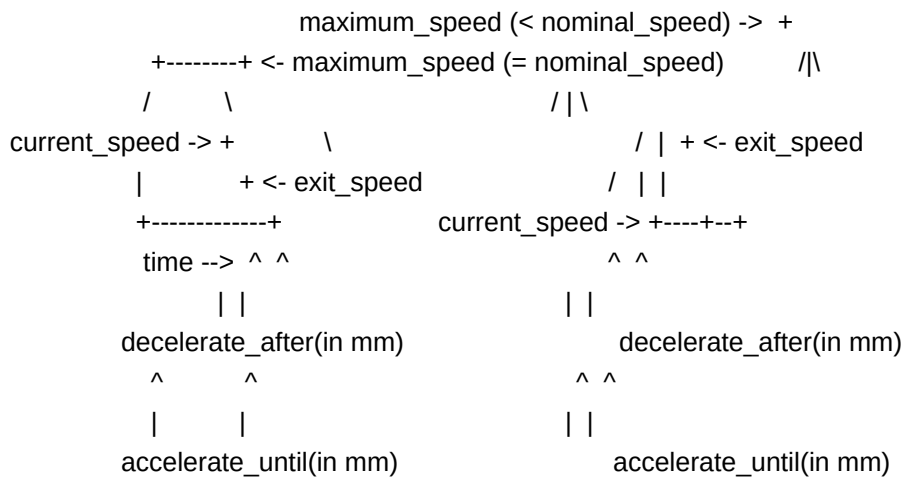
```

/* BLOCK VELOCITY PROFILE DEFINITION



The planner block buffer is planned assuming constant acceleration velocity profiles and are continuously joined at block junctions as shown above. However, the planner only actively computes the block entry speeds for an optimal velocity plan, but does not compute the block internal velocity profiles. These velocity profiles are computed ad-hoc as they are executed by the

stepper algorithm and consists of only 7 possible types of profiles: cruise-only, cruise-deceleration, acceleration-cruise, acceleration-only, deceleration-only, full-trapezoid, and triangle(no cruise).



The step segment buffer computes the executing block velocity profile and tracks the critical parameters for the stepper algorithm to accurately trace the profile. These critical parameters are shown and defined in the above illustration.

*/

// Stepper state initialization. Cycle should only start if the st.cycle_start flag is
// enabled. Startup init and limits call this function but shouldn't start the cycle.

void st_wake_up()

{

// Enable stepper drivers.

if (bit_istrue(settings.flags,BITFLAG_INVERT_ST_ENABLE)) { STEPPERS_DISABLE_PORT |= (1<<STEPPERS_DISABLE_BIT);
else { STEPPERS_DISABLE_PORT &= ~(1<<STEPPERS_DISABLE_BIT); }

// Initialize stepper output bits to ensure first ISR call does not step.

st.step_outbits = step_port_invert_mask;

// Initialize step pulse timing from settings. Here to ensure updating after re-writing.

#ifdef STEP_PULSE_DELAY

// Set total step pulse time after direction pin set. Ad hoc computation from oscilloscope.

st.step_pulse_time = -(((settings.pulse_microseconds+STEP_PULSE_DELAY-2)*TICKS_PER_MICROSECOND) >> 3);

// Set delay between direction pin write and step command.

OCR0A = -(((settings.pulse_microseconds)*TICKS_PER_MICROSECOND) >> 3);

#else // Normal operation

// Set step pulse time. Ad hoc computation from oscilloscope. Uses two's complement.

st.step_pulse_time = -(((settings.pulse_microseconds-2)*TICKS_PER_MICROSECOND) >> 3);

#endif

// Enable Stepper Driver Interrupt

TIMSK1 |= (1<<OCIE1A);

}

// Stepper shutdown

```

void st_go_idle()
{
    // Disable Stepper Driver Interrupt. Allow Stepper Port Reset Interrupt to finish, if active.
    TIMSK1 &= ~(1<<OCIE1A); // Disable Timer1 interrupt
    TCCR1B = (TCCR1B & ~((1<<CS12) | (1<<CS11))) | (1<<CS10); // Reset clock to no prescaling.
    busy = false;

    // Set stepper driver idle state, disabled or enabled, depending on settings and circumstances.
    bool pin_state = false; // Keep enabled.
    if (((settings.stepper_idle_lock_time != 0xff) || sys_rt_exec_alarm || sys.state == STATE_SLEEP) && sys.state != STATE_HALT)
        // Force stepper dwell to lock axes for a defined amount of time to ensure the axes come to a complete
        // stop and not drift from residual inertial forces at the end of the last movement.
        delay_ms(settings.stepper_idle_lock_time);
    pin_state = true; // Override. Disable steppers.
}
if (bit_istrue(settings.flags,BITFLAG_INVERT_ST_ENABLE)) { pin_state = !pin_state; } // Apply pin invert.
if (pin_state) { STEPPERS_DISABLE_PORT |= (1<<STEPPERS_DISABLE_BIT); }
else { STEPPERS_DISABLE_PORT &= ~(1<<STEPPERS_DISABLE_BIT); }
}

```

/* "The Stepper Driver Interrupt" - This timer interrupt is the workhorse of Grbl. Grbl employs the venerable Bresenham line algorithm to manage and exactly synchronize multi-axis moves. Unlike the popular DDA algorithm, the Bresenham algorithm is not susceptible to numerical round-off errors and only requires fast integer counters, meaning low computational overhead and maximizing the Arduino's capabilities. However, the downside of the Bresenham algorithm is, for certain multi-axis motions, the non-dominant axes may suffer from un-smooth step pulse trains, or aliasing, which can lead to strange audible noises or shaking. This is particularly noticeable or may cause motion issues at low step frequencies (0-5kHz), but is usually not a physical problem at higher frequencies, although audible.

To improve Bresenham multi-axis performance, Grbl uses what we call an Adaptive Multi-Axis Step Smoothing (AMASS) algorithm, which does what the name implies. At lower step frequencies, AMASS artificially increases the Bresenham resolution without effecting the algorithm's innate exactness. AMASS adapts its resolution levels automatically depending on the step frequency to be executed, meaning that for even lower step frequencies the step smoothing level increases. Algorithmically, AMASS is achieved by a simple bit-shifting of the Bresenham step count for each AMASS level. For example, for a Level 1 step smoothing, we bit shift the Bresenham step event count, effectively multiplying it by 2, while the axis step counts remain the same, and then double the stepper ISR frequency. In effect, we are allowing the non-dominant Bresenham axes step in the intermediate ISR tick, while the dominant axis is stepping every two ISR ticks, rather than every ISR tick in the traditional sense. At AMASS Level 2, we simply bit-shift again, so the non-dominant Bresenham axes can step within any of the four ISR ticks, the dominant axis steps every four ISR ticks, and quadruple the stepper ISR frequency. And so on. This, in effect, virtually eliminates multi-axis aliasing issues with the Bresenham algorithm and does not significantly alter Grbl's performance, but in fact, more efficiently utilizes unused CPU cycles overall throughout all configurations.

AMASS retains the Bresenham algorithm exactness by requiring that it always executes a full Bresenham step, regardless of AMASS Level. Meaning that for an AMASS Level 2, all four intermediate steps must be completed such that baseline Bresenham (Level 0) count is always retained. Similarly, AMASS Level 3 means all eight intermediate steps must be executed. Although the AMASS Levels are in reality arbitrary, where the baseline Bresenham counts can

be multiplied by any integer value, multiplication by powers of two are simply used to ease CPU overhead with bitshift integer operations.

This interrupt is simple and dumb by design. All the computational heavy-lifting, as in determining accelerations, is performed elsewhere. This interrupt pops pre-computed segments, defined as constant velocity over n number of steps, from the step segment buffer and then executes them by pulsing the stepper pins appropriately via the Bresenham algorithm. This ISR is supported by The Stepper Port Reset Interrupt which it uses to reset the stepper port after each pulse. The bresenham line tracer algorithm controls all stepper outputs simultaneously with these two interrupts.

NOTE: This interrupt must be as efficient as possible and complete before the next ISR tick, which for Grbl must be less than 33.3usec (@30kHz ISR rate). Oscilloscope measured time in ISR is 5usec typical and 25usec maximum, well below requirement.

NOTE: This ISR expects at least one step to be executed per segment.

*/

```
// TODO: Replace direct updating of the int32 position counters in the ISR somehow. Perhaps use smaller
// int8 variables and update position counters only when a segment completes. This can get complicated
// with probing and homing cycles that require true real-time positions.
```

```
ISR(TIMER1_COMPA_vect)
```

```
{
```

```
  if (busy) { return; } // The busy-flag is used to avoid reentering this interrupt
```

```
  // Set the direction pins a couple of nanoseconds before we step the steppers
```

```
  DIRECTION_PORT = (DIRECTION_PORT & ~DIRECTION_MASK) | (st.dir_outbits & DIRECTION_MASK);
```

```
  #ifdef ENABLE_DUAL_AXIS
```

```
    DIRECTION_PORT_DUAL = (DIRECTION_PORT_DUAL & ~DIRECTION_MASK_DUAL) | (st.dir_outbits_dual & DIREC
```

```
  #endif
```

```
  // Then pulse the stepping pins
```

```
  #ifdef STEP_PULSE_DELAY
```

```
    st.step_bits = (STEP_PORT & ~STEP_MASK) | st.step_outbits; // Store out_bits to prevent overwriting.
```

```
    #ifdef ENABLE_DUAL_AXIS
```

```
      st.step_bits_dual = (STEP_PORT_DUAL & ~STEP_MASK_DUAL) | st.step_outbits_dual;
```

```
    #endif
```

```
  #else // Normal operation
```

```
    STEP_PORT = (STEP_PORT & ~STEP_MASK) | st.step_outbits;
```

```
    #ifdef ENABLE_DUAL_AXIS
```

```
      STEP_PORT_DUAL = (STEP_PORT_DUAL & ~STEP_MASK_DUAL) | st.step_outbits_dual;
```

```
    #endif
```

```
  #endif
```

```
  // Enable step pulse reset timer so that The Stepper Port Reset Interrupt can reset the signal after
```

```
  // exactly settings.pulse_microseconds microseconds, independent of the main Timer1 prescaler.
```

```
  TCNT0 = st.step_pulse_time; // Reload Timer0 counter
```

```
  TCCR0B = (1<<CS01); // Begin Timer0. Full speed, 1/8 prescaler
```

```
  busy = true;
```

```
  sei(); // Re-enable interrupts to allow Stepper Port Reset Interrupt to fire on-time.
```

```
    // NOTE: The remaining code in this ISR will finish before returning to main program.
```

```
  // If there is no step segment, attempt to pop one from the stepper buffer
```

```

if (st.exec_segment == NULL) {
    // Anything in the buffer? If so, load and initialize next step segment.
    if (segment_buffer_head != segment_buffer_tail) {
        // Initialize new step segment and load number of steps to execute
        st.exec_segment = &segment_buffer[segment_buffer_tail];

        #ifndef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
            // With AMASS is disabled, set timer prescaler for segments with slow step frequencies (< 250Hz).
            TCCR1B = (TCCR1B & ~(0x07<<CS10)) | (st.exec_segment->prescaler<<CS10);
        #endif

        // Initialize step segment timing per step and load number of steps to execute.
        OCR1A = st.exec_segment->cycles_per_tick;
        st.step_count = st.exec_segment->n_step; // NOTE: Can sometimes be zero when moving slow.
        // If the new segment starts a new planner block, initialize stepper variables and counters.
        // NOTE: When the segment data index changes, this indicates a new planner block.
        if ( st.exec_block_index != st.exec_segment->st_block_index ) {
            st.exec_block_index = st.exec_segment->st_block_index;
            st.exec_block = &st_block_buffer[st.exec_block_index];

            // Initialize Bresenham line and distance counters
            st.counter_x = st.counter_y = st.counter_z = (st.exec_block->step_event_count >> 1);
        }
        st.dir_outbits = st.exec_block->direction_bits ^ dir_port_invert_mask;
        #ifdef ENABLE_DUAL_AXIS
            st.dir_outbits_dual = st.exec_block->direction_bits_dual ^ dir_port_invert_mask_dual;
        #endif

        #ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
            // With AMASS enabled, adjust Bresenham axis increment counters according to AMASS level.
            st.steps[X_AXIS] = st.exec_block->steps[X_AXIS] >> st.exec_segment->amass_level;
            st.steps[Y_AXIS] = st.exec_block->steps[Y_AXIS] >> st.exec_segment->amass_level;
            st.steps[Z_AXIS] = st.exec_block->steps[Z_AXIS] >> st.exec_segment->amass_level;
        #endif

        #ifdef VARIABLE_SPINDLE
            // Set real-time spindle output as segment is loaded, just prior to the first step.
            spindle_set_speed(st.exec_segment->spindle_pwm);
        #endif

    } else {
        // Segment buffer empty. Shutdown.
        st_go_idle();
        #ifdef VARIABLE_SPINDLE
            // Ensure pwm is set properly upon completion of rate-controlled motion.
            if (st.exec_block->is_pwm_rate_adjusted) { spindle_set_speed(SPINDLE_PWM_OFF_VALUE); }
        #endif
        system_set_exec_state_flag(EXEC_CYCLE_STOP); // Flag main program for cycle end
        return; // Nothing to do but exit.
    }
}

```

```

// Check probing state.
if (sys_probe_state == PROBE_ACTIVE) { probe_state_monitor(); }

// Reset step out bits.
st.step_outbits = 0;
#ifdef ENABLE_DUAL_AXIS
    st.step_outbits_dual = 0;
#endif

// Execute step displacement profile by Bresenham line algorithm
#ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
    st.counter_x += st.steps[X_AXIS];
#else
    st.counter_x += st.exec_block->steps[X_AXIS];
#endif
if (st.counter_x > st.exec_block->step_event_count) {
    st.step_outbits |= (1<<X_STEP_BIT);
    #if defined(ENABLE_DUAL_AXIS) && (DUAL_AXIS_SELECT == X_AXIS)
        st.step_outbits_dual = (1<<DUAL_STEP_BIT);
    #endif
    st.counter_x -= st.exec_block->step_event_count;
    if (st.exec_block->direction_bits & (1<<X_DIRECTION_BIT)) { sys_position[X_AXIS]--; }
    else { sys_position[X_AXIS]++; }
}
#ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
    st.counter_y += st.steps[Y_AXIS];
#else
    st.counter_y += st.exec_block->steps[Y_AXIS];
#endif
if (st.counter_y > st.exec_block->step_event_count) {
    st.step_outbits |= (1<<Y_STEP_BIT);
    #if defined(ENABLE_DUAL_AXIS) && (DUAL_AXIS_SELECT == Y_AXIS)
        st.step_outbits_dual = (1<<DUAL_STEP_BIT);
    #endif
    st.counter_y -= st.exec_block->step_event_count;
    if (st.exec_block->direction_bits & (1<<Y_DIRECTION_BIT)) { sys_position[Y_AXIS]--; }
    else { sys_position[Y_AXIS]++; }
}
#ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
    st.counter_z += st.steps[Z_AXIS];
#else
    st.counter_z += st.exec_block->steps[Z_AXIS];
#endif
if (st.counter_z > st.exec_block->step_event_count) {
    st.step_outbits |= (1<<Z_STEP_BIT);
    st.counter_z -= st.exec_block->step_event_count;
    if (st.exec_block->direction_bits & (1<<Z_DIRECTION_BIT)) { sys_position[Z_AXIS]--; }
    else { sys_position[Z_AXIS]++; }
}

```

```

// During a homing cycle, lock out and prevent desired axes from moving.
if (sys.state == STATE_HOMING) {
    st.step_outbits &= sys.homing_axis_lock;
#ifdef ENABLE_DUAL_AXIS
    st.step_outbits_dual &= sys.homing_axis_lock_dual;
#endif
}

st.step_count--; // Decrement step events count
if (st.step_count == 0) {
    // Segment is complete. Discard current segment and advance segment indexing.
    st.exec_segment = NULL;
    if ( ++segment_buffer_tail == SEGMENT_BUFFER_SIZE) { segment_buffer_tail = 0; }
}

st.step_outbits ^= step_port_invert_mask; // Apply step port invert mask
#ifdef ENABLE_DUAL_AXIS
    st.step_outbits_dual ^= step_port_invert_mask_dual;
#endif
busy = false;
}

```

/* The Stepper Port Reset Interrupt: Timer0 OVF interrupt handles the falling edge of the step pulse. This should always trigger before the next Timer1 COMPA interrupt and independently finish, if Timer1 is disabled after completing a move.

NOTE: Interrupt collisions between the serial and stepper interrupts can cause delays by a few microseconds, if they execute right before one another. Not a big deal, but can cause issues at high step rates if another high frequency asynchronous interrupt is added to Grbl.

*/

```

// This interrupt is enabled by ISR_TIMER1_COMPAREA when it sets the motor port bits to execute
// a step. This ISR resets the motor port after a short period (settings.pulse_microseconds)
// completing one step cycle.

```

```
ISR(TIMER0_OVF_vect)
```

```

{
    // Reset stepping pins (leave the direction pins)
    STEP_PORT = (STEP_PORT & ~STEP_MASK) | (step_port_invert_mask & STEP_MASK);
#ifdef ENABLE_DUAL_AXIS
    STEP_PORT_DUAL = (STEP_PORT_DUAL & ~STEP_MASK_DUAL) | (step_port_invert_mask_dual & STEP_MASK_DUAL);
#endif
    TCCR0B = 0; // Disable Timer0 to prevent re-entering this interrupt when it's not needed.
}

```

```
#ifdef STEP_PULSE_DELAY
```

```

// This interrupt is used only when STEP_PULSE_DELAY is enabled. Here, the step pulse is
// initiated after the STEP_PULSE_DELAY time period has elapsed. The ISR TIMER2_OVF interrupt
// will then trigger after the appropriate settings.pulse_microseconds, as in normal operation.
// The new timing between direction, step pulse, and step complete events are setup in the
// st_wake_up() routine.

```

```
ISR(TIMER0_COMPA_vect)
```

```

{
    STEP_PORT = st.step_bits; // Begin step pulse.
#ifdef ENABLE_DUAL_AXIS
    STEP_PORT_DUAL = st.step_bits_dual;
#endif
}
#endif

```

// Generates the step and direction port invert masks used in the Stepper Interrupt Driver.

```

void st_generate_step_dir_invert_masks()
{
    uint8_t idx;
    step_port_invert_mask = 0;
    dir_port_invert_mask = 0;
    for (idx=0; idx<N_AXIS; idx++) {
        if (bit_istrue(settings.step_invert_mask,bit(idx))) { step_port_invert_mask |= get_step_pin_mask(idx); }
        if (bit_istrue(settings.dir_invert_mask,bit(idx))) { dir_port_invert_mask |= get_direction_pin_mask(idx); }
    }
#ifdef ENABLE_DUAL_AXIS
    step_port_invert_mask_dual = 0;
    dir_port_invert_mask_dual = 0;
    // NOTE: Dual axis invert uses the N_AXIS bit to set step and direction invert pins.
    if (bit_istrue(settings.step_invert_mask,bit(N_AXIS))) { step_port_invert_mask_dual = (1<<DUAL_STEP_BIT); }
    if (bit_istrue(settings.dir_invert_mask,bit(N_AXIS))) { dir_port_invert_mask_dual = (1<<DUAL_DIRECTION_BIT); }
#endif
}

```

// Reset and clear stepper subsystem variables

```

void st_reset()
{
    // Initialize stepper driver idle state.
    st_go_idle();

    // Initialize stepper algorithm variables.
    memset(&prep, 0, sizeof(st_prep_t));
    memset(&st, 0, sizeof(stepper_t));
    st.exec_segment = NULL;
    pl_block = NULL; // Planner block pointer used by segment buffer
    segment_buffer_tail = 0;
    segment_buffer_head = 0; // empty = tail
    segment_next_head = 1;
    busy = false;

    st_generate_step_dir_invert_masks();
    st.dir_outbits = dir_port_invert_mask; // Initialize direction bits to default.

```

// Initialize step and direction port pins.

```

STEP_PORT = (STEP_PORT & ~STEP_MASK) | step_port_invert_mask;
DIRECTION_PORT = (DIRECTION_PORT & ~DIRECTION_MASK) | dir_port_invert_mask;

```

```

#ifdef ENABLE_DUAL_AXIS
    st.dir_outbits_dual = dir_port_invert_mask_dual;
    STEP_PORT_DUAL = (STEP_PORT_DUAL & ~STEP_MASK_DUAL) | step_port_invert_mask_dual;
    DIRECTION_PORT_DUAL = (DIRECTION_PORT_DUAL & ~DIRECTION_MASK_DUAL) | dir_port_invert_mask_dual;
#endif
}

// Initialize and start the stepper motor subsystem
void stepper_init()
{
    // Configure step and direction interface pins
    STEP_DDR |= STEP_MASK;
    STEPPERS_DISABLE_DDR |= 1<<STEPPERS_DISABLE_BIT;
    DIRECTION_DDR |= DIRECTION_MASK;

#ifdef ENABLE_DUAL_AXIS
    STEP_DDR_DUAL |= STEP_MASK_DUAL;
    DIRECTION_DDR_DUAL |= DIRECTION_MASK_DUAL;
#endif

    // Configure Timer 1: Stepper Driver Interrupt
    TCCR1B &= ~(1<<WGM13); // waveform generation = 0100 = CTC
    TCCR1B |= (1<<WGM12);
    TCCR1A &= ~((1<<WGM11) | (1<<WGM10));
    TCCR1A &= ~((1<<COM1A1) | (1<<COM1A0) | (1<<COM1B1) | (1<<COM1B0)); // Disconnect OC1 output
    // TCCR1B = (TCCR1B & ~((1<<CS12) | (1<<CS11))) | (1<<CS10); // Set in st_go_idle().
    // TIMSK1 &= ~(1<<OCIE1A); // Set in st_go_idle().

    // Configure Timer 0: Stepper Port Reset Interrupt
    TIMSK0 &= ~((1<<OCIE0B) | (1<<OCIE0A) | (1<<TOIE0)); // Disconnect OC0 outputs and OVF interrupt.
    TCCR0A = 0; // Normal operation
    TCCR0B = 0; // Disable Timer0 until needed
    TIMSK0 |= (1<<TOIE0); // Enable Timer0 overflow interrupt
#ifdef STEP_PULSE_DELAY
    TIMSK0 |= (1<<OCIE0A); // Enable Timer0 Compare Match A interrupt
#endif
}

// Called by planner_recalculate() when the executing block is updated by the new plan.
void st_update_plan_block_parameters()
{
    if (pl_block != NULL) { // Ignore if at start of a new block.
        prep.recalculate_flag |= PREP_FLAG_RECALCULATE;
        pl_block->entry_speed_sqr = prep.current_speed*prep.current_speed; // Update entry speed.
        pl_block = NULL; // Flag st_prep_segment() to load and check active velocity profile.
    }
}

```

```

// Increments the step segment buffer block data ring buffer.
static uint8_t st_next_block_index(uint8_t block_index)
{
    block_index++;
    if ( block_index == (SEGMENT_BUFFER_SIZE-1) ) { return(0); }
    return(block_index);
}

#ifdef PARKING_ENABLE
// Changes the run state of the step segment buffer to execute the special parking motion.
void st_parking_setup_buffer()
{
    // Store step execution data of partially completed block, if necessary.
    if (prep.recalculate_flag & PREP_FLAG_HOLD_PARTIAL_BLOCK) {
        prep.last_st_block_index = prep.st_block_index;
        prep.last_steps_remaining = prep.steps_remaining;
        prep.last_dt_remainder = prep.dt_remainder;
        prep.last_step_per_mm = prep.step_per_mm;
    }
    // Set flags to execute a parking motion
    prep.recalculate_flag |= PREP_FLAG_PARKING;
    prep.recalculate_flag &= ~(PREP_FLAG_RECALCULATE);
    pl_block = NULL; // Always reset parking motion to reload new block.
}

// Restores the step segment buffer to the normal run state after a parking motion.
void st_parking_restore_buffer()
{
    // Restore step execution data and flags of partially completed block, if necessary.
    if (prep.recalculate_flag & PREP_FLAG_HOLD_PARTIAL_BLOCK) {
        st_prep_block = &st_block_buffer[prep.last_st_block_index];
        prep.st_block_index = prep.last_st_block_index;
        prep.steps_remaining = prep.last_steps_remaining;
        prep.dt_remainder = prep.last_dt_remainder;
        prep.step_per_mm = prep.last_step_per_mm;
        prep.recalculate_flag = (PREP_FLAG_HOLD_PARTIAL_BLOCK | PREP_FLAG_RECALCULATE);
        prep.req_mm_increment = REQ_MM_INCREMENT_SCALAR/prep.step_per_mm; // Recompute this value.
    } else {
        prep.recalculate_flag = false;
    }
    pl_block = NULL; // Set to reload next block.
}
#endif

/* Prepares step segment buffer. Continuously called from main program.

```

The segment buffer is an intermediary buffer interface between the execution of steps

by the stepper algorithm and the velocity profiles generated by the planner. The stepper algorithm only executes steps within the segment buffer and is filled by the main program when steps are "checked-out" from the first block in the planner buffer. This keeps the step execution and planning optimization processes atomic and protected from each other. The number of steps "checked-out" from the planner buffer and the number of segments in the segment buffer is sized and computed such that no operation in the main program takes longer than the time it takes the stepper algorithm to empty it before refilling it.

Currently, the segment buffer conservatively holds roughly up to 40-50 msec of steps.

NOTE: Computation units are in steps, millimeters, and minutes.

```
*/
void st_prep_buffer()
{
    // Block step prep buffer, while in a suspend state and there is no suspend motion to execute.
    if (bit_istrue(sys.step_control, STEP_CONTROL_END_MOTION)) { return; }

    while (segment_buffer_tail != segment_next_head) { // Check if we need to fill the buffer.

        // Determine if we need to load a new planner block or if the block needs to be recomputed.
        if (pl_block == NULL) {

            // Query planner for a queued block
            if (sys.step_control & STEP_CONTROL_EXECUTE_SYS_MOTION) { pl_block = plan_get_system_motion_block(); }
            else { pl_block = plan_get_current_block(); }
            if (pl_block == NULL) { return; } // No planner blocks. Exit.

            // Check if we need to only recompute the velocity profile or load a new block.
            if (prep.recalculate_flag & PREP_FLAG_RECALCULATE) {

                #ifdef PARKING_ENABLE
                if (prep.recalculate_flag & PREP_FLAG_PARKING) { prep.recalculate_flag &= ~(PREP_FLAG_RECALCULATE); }
                else { prep.recalculate_flag = false; }
                #else
                prep.recalculate_flag = false;
                #endif

            } else {

                // Load the Bresenham stepping data for the block.
                prep.st_block_index = st_next_block_index(prep.st_block_index);

                // Prepare and copy Bresenham algorithm segment data from the new planner block, so that
                // when the segment buffer completes the planner block, it may be discarded when the
                // segment buffer finishes the prepped block, but the stepper ISR is still executing it.
                st_prep_block = &st_block_buffer[prep.st_block_index];
                st_prep_block->direction_bits = pl_block->direction_bits;
                #ifdef ENABLE_DUAL_AXIS
                if (DUAL_AXIS_SELECT == X_AXIS)
                    if (st_prep_block->direction_bits & (1<<X_DIRECTION_BIT)) {
                #elif DUAL_AXIS_SELECT == Y_AXIS
                    if (st_prep_block->direction_bits & (1<<Y_DIRECTION_BIT)) {
                #endif
            }
        }
    }
}
```

```

    st_prep_block->direction_bits_dual = (1<<DUAL_DIRECTION_BIT);
} else { st_prep_block->direction_bits_dual = 0; }
#endif
uint8_t idx;
#ifndef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
    for (idx=0; idx<N_AXIS; idx++) { st_prep_block->steps[idx] = (pl_block->steps[idx] << 1); }
    st_prep_block->step_event_count = (pl_block->step_event_count << 1);
#else
    // With AMASS enabled, simply bit-shift multiply all Bresenham data by the max AMASS
    // level, such that we never divide beyond the original data anywhere in the algorithm.
    // If the original data is divided, we can lose a step from integer roundoff.
    for (idx=0; idx<N_AXIS; idx++) { st_prep_block->steps[idx] = pl_block->steps[idx] << MAX_AMASS_LEVEL; }
    st_prep_block->step_event_count = pl_block->step_event_count << MAX_AMASS_LEVEL;
#endif

// Initialize segment buffer data for generating the segments.
prep.steps_remaining = (float)pl_block->step_event_count;
prep.step_per_mm = prep.steps_remaining/pl_block->millimeters;
prep.req_mm_increment = REQ_MM_INCREMENT_SCALAR/prep.step_per_mm;
prep.dt_remainder = 0.0; // Reset for new segment block

if ((sys.step_control & STEP_CONTROL_EXECUTE_HOLD) || (prep.recalculate_flag & PREP_FLAG_DECEL_OVERRIDE))
    // New block loaded mid-hold. Override planner block entry speed to enforce deceleration.
    prep.current_speed = prep.exit_speed;
    pl_block->entry_speed_sqr = prep.exit_speed*prep.exit_speed;
    prep.recalculate_flag &= ~(PREP_FLAG_DECEL_OVERRIDE);
} else {
    prep.current_speed = sqrt(pl_block->entry_speed_sqr);
}

#ifdef VARIABLE_SPINDLE
    // Setup laser mode variables. PWM rate adjusted motions will always complete a motion with the
    // spindle off.
    st_prep_block->is_pwm_rate_adjusted = false;
    if (settings.flags & BITFLAG_LASER_MODE) {
        if (pl_block->condition & PL_COND_FLAG_SPINDLE_CCW) {
            // Pre-compute inverse programmed rate to speed up PWM updating per step segment.
            prep.inv_rate = 1.0/pl_block->programmed_rate;
            st_prep_block->is_pwm_rate_adjusted = true;
        }
    }
}
#endif
}

```

■■■■/* -----

■■■■ Compute the velocity profile of a new planner block based on its entry and exit
■■■■ speeds, or recompute the profile of a partially-completed planner block if the
■■■■ planner has updated it. For a commanded forced-deceleration, such as from a feed
■■■■ hold, override the planner velocities and decelerate to the target exit speed.
■■■■*/
■■■■prep.mm_complete = 0.0; // Default velocity profile complete at 0.0mm from end of block.

```

float inv_2_accel = 0.5/pl_block->acceleration;
if (sys.step_control & STEP_CONTROL_EXECUTE_HOLD) { // [Forced Deceleration to Zero Velocity]
    // Compute velocity profile parameters for a feed hold in-progress. This profile overrides
    // the planner block profile, enforcing a deceleration to zero speed.
    prep.ramp_type = RAMP_DECEL;
    // Compute decelerate distance relative to end of block.
    float decel_dist = pl_block->millimeters - inv_2_accel*pl_block->entry_speed_sqr;
    if (decel_dist < 0.0) {
        // Deceleration through entire planner block. End of feed hold is not in this block.
        prep.exit_speed = sqrt(pl_block->entry_speed_sqr-2*pl_block->acceleration*pl_block->millimeters);
    } else {
        prep.mm_complete = decel_dist; // End of feed hold.
        prep.exit_speed = 0.0;
    }
} else { // [Normal Operation]
    // Compute or recompute velocity profile parameters of the prepped planner block.
    prep.ramp_type = RAMP_ACCEL; // Initialize as acceleration ramp.
    prep.accelerate_until = pl_block->millimeters;

    float exit_speed_sqr;
    float nominal_speed;
    if (sys.step_control & STEP_CONTROL_EXECUTE_SYS_MOTION) {
        prep.exit_speed = exit_speed_sqr = 0.0; // Enforce stop at end of system motion.
    } else {
        exit_speed_sqr = plan_get_exec_block_exit_speed_sqr();
        prep.exit_speed = sqrt(exit_speed_sqr);
    }

    nominal_speed = plan_compute_profile_nominal_speed(pl_block);
    float nominal_speed_sqr = nominal_speed*nominal_speed;
    float intersect_distance =
        0.5*(pl_block->millimeters+inv_2_accel*(pl_block->entry_speed_sqr-exit_speed_sqr));

    if (pl_block->entry_speed_sqr > nominal_speed_sqr) { // Only occurs during override reductions.
        prep.accelerate_until = pl_block->millimeters - inv_2_accel*(pl_block->entry_speed_sqr-nominal_speed_sqr);
        if (prep.accelerate_until <= 0.0) { // Deceleration-only.
            prep.ramp_type = RAMP_DECEL;
            // prep.decelerate_after = pl_block->millimeters;
            // prep.maximum_speed = prep.current_speed;

            // Compute override block exit speed since it doesn't match the planner exit speed.
            prep.exit_speed = sqrt(pl_block->entry_speed_sqr - 2*pl_block->acceleration*pl_block->millimeters);
            prep.recalculate_flag |= PREP_FLAG_DECEL_OVERRIDE; // Flag to load next block as deceleration override.

            // TODO: Determine correct handling of parameters in deceleration-only.
            // Can be tricky since entry speed will be current speed, as in feed holds.
            // Also, look into near-zero speed handling issues with this.
        }
    } else {
        // Decelerate to cruise or cruise-decelerate types. Guaranteed to intersect updated plan.
        prep.decelerate_after = inv_2_accel*(nominal_speed_sqr-exit_speed_sqr); // Should always be >= 0.0 due to planner
    }
}

```

```

        prep.maximum_speed = nominal_speed;
        prep.ramp_type = RAMP_DECEL_OVERRIDE;
    }
    #####} else if (intersect_distance > 0.0) {
    #####if (intersect_distance < pl_block->millimeters) { // Either trapezoid or triangle types
    #####// NOTE: For acceleration-cruise and cruise-only types, following calculation will be 0.0.
    #####prep.decelerate_after = inv_2_accel*(nominal_speed_sqr-exit_speed_sqr);
    #####if (prep.decelerate_after < intersect_distance) { // Trapezoid type
    #####prep.maximum_speed = nominal_speed;
    #####if (pl_block->entry_speed_sqr == nominal_speed_sqr) {
    #####// Cruise-deceleration or cruise-only type.
    #####prep.ramp_type = RAMP_CRUISE;
    #####} else {
    #####// Full-trapezoid or acceleration-cruise types
    #####prep.accelerate_until -= inv_2_accel*(nominal_speed_sqr-pl_block->entry_speed_sqr);
    #####}
    #####} else { // Triangle type
    #####prep.accelerate_until = intersect_distance;
    #####prep.decelerate_after = intersect_distance;
    #####prep.maximum_speed = sqrt(2.0*pl_block->acceleration*intersect_distance+exit_speed_sqr);
    #####}
    #####} else { // Deceleration-only type
        prep.ramp_type = RAMP_DECEL;
        // prep.decelerate_after = pl_block->millimeters;
        // prep.maximum_speed = prep.current_speed;
    #####}
    #####} else { // Acceleration-only type
    #####prep.accelerate_until = 0.0;
    #####// prep.decelerate_after = 0.0;
    #####prep.maximum_speed = prep.exit_speed;
    #####}
    #####}

#ifdef VARIABLE_SPINDLE
    bit_true(sys.step_control, STEP_CONTROL_UPDATE_SPINDLE_PWM); // Force update whenever updating block.
#endif
}

// Initialize new segment
segment_t *prep_segment = &segment_buffer[segment_buffer_head];

// Set new segment to point to the current segment data block.
prep_segment->st_block_index = prep.st_block_index;

/*-----
    Compute the average velocity of this new segment by determining the total distance
    traveled over the segment time DT_SEGMENT. The following code first attempts to create
    a full segment based on the current ramp conditions. If the segment time is incomplete
    when terminating at a ramp state change, the code will continue to loop through the
    progressing ramp states to fill the remaining segment execution time. However, if
    an incomplete segment terminates at the end of the velocity profile, the segment is

```

considered completed despite having a truncated execution time less than DT_SEGMENT.

The velocity profile is always assumed to progress through the ramp sequence: acceleration ramp, cruising state, and deceleration ramp. Each ramp's travel distance may range from zero to the length of the block. Velocity profiles can end either at the end of planner block (typical) or mid-block at the end of a forced deceleration, such as from a feed hold.

*/

```
float dt_max = DT_SEGMENT; // Maximum segment time
float dt = 0.0; // Initialize segment time
float time_var = dt_max; // Time worker variable
float mm_var; // mm-Distance worker variable
float speed_var; // Speed worker variable
float mm_remaining = pl_block->millimeters; // New segment distance from end of block.
float minimum_mm = mm_remaining-prep.req_mm_increment; // Guarantee at least one step.
if (minimum_mm < 0.0) { minimum_mm = 0.0; }

do {
  switch (prep.ramp_type) {
    case RAMP_DECEL_OVERRIDE:
      speed_var = pl_block->acceleration*time_var;
      if (prep.current_speed-prep.maximum_speed <= speed_var) {
        // Cruise or cruise-deceleration types only for deceleration override.
        mm_remaining = prep.accelerate_until;
        time_var = 2.0*(pl_block->millimeters-mm_remaining)/(prep.current_speed+prep.maximum_speed);
        prep.ramp_type = RAMP_CRUISE;
        prep.current_speed = prep.maximum_speed;
      } else { // Mid-deceleration override ramp.
        mm_remaining -= time_var*(prep.current_speed - 0.5*speed_var);
        prep.current_speed -= speed_var;
      }
      break;
    case RAMP_ACCEL:
      // NOTE: Acceleration ramp only computes during first do-while loop.
      speed_var = pl_block->acceleration*time_var;
      mm_remaining -= time_var*(prep.current_speed + 0.5*speed_var);
      if (mm_remaining < prep.accelerate_until) { // End of acceleration ramp.
        // Acceleration-cruise, acceleration-deceleration ramp junction, or end of block.
        mm_remaining = prep.accelerate_until; // NOTE: 0.0 at EOB
        time_var = 2.0*(pl_block->millimeters-mm_remaining)/(prep.current_speed+prep.maximum_speed);
        if (mm_remaining == prep.decelerate_after) { prep.ramp_type = RAMP_DECEL; }
        else { prep.ramp_type = RAMP_CRUISE; }
        prep.current_speed = prep.maximum_speed;
      } else { // Acceleration only.
        prep.current_speed += speed_var;
      }
      break;
    case RAMP_CRUISE:
      // NOTE: mm_var used to retain the last mm_remaining for incomplete segment time_var calculations.
      // NOTE: If maximum_speed*time_var value is too low, round-off can cause mm_var to not change. To
      // prevent this, simply enforce a minimum speed threshold in the planner.
      mm_var = mm_remaining - prep.maximum_speed*time_var;
```

```

if (mm_var < prep.decelerate_after) { // End of cruise.
    // Cruise-deceleration junction or end of block.
    time_var = (mm_remaining - prep.decelerate_after)/prep.maximum_speed;
    mm_remaining = prep.decelerate_after; // NOTE: 0.0 at EOB
    prep.ramp_type = RAMP_DECEL;
} else { // Cruising only.
    mm_remaining = mm_var;
}
break;
default: // case RAMP_DECEL:
    // NOTE: mm_var used as a misc worker variable to prevent errors when near zero speed.
    speed_var = pl_block->acceleration*time_var; // Used as delta speed (mm/min)
    if (prep.current_speed > speed_var) { // Check if at or below zero speed.
        // Compute distance from end of segment to end of block.
        mm_var = mm_remaining - time_var*(prep.current_speed - 0.5*speed_var); // (mm)
        if (mm_var > prep.mm_complete) { // Typical case. In deceleration ramp.
            mm_remaining = mm_var;
            prep.current_speed -= speed_var;
            break; // Segment complete. Exit switch-case statement. Continue do-while loop.
        }
    }
    // Otherwise, at end of block or end of forced-deceleration.
    time_var = 2.0*(mm_remaining-prep.mm_complete)/(prep.current_speed+prep.exit_speed);
    mm_remaining = prep.mm_complete;
    prep.current_speed = prep.exit_speed;
}
dt += time_var; // Add computed ramp time to total segment time.
if (dt < dt_max) { time_var = dt_max - dt; } // **Incomplete** At ramp junction.
else {
    if (mm_remaining > minimum_mm) { // Check for very slow segments with zero steps.
        // Increase segment time to ensure at least one step in segment. Override and loop
        // through distance calculations until minimum_mm or mm_complete.
        dt_max += DT_SEGMENT;
        time_var = dt_max - dt;
    } else {
        break; // **Complete** Exit loop. Segment execution time maxed.
    }
}
} while (mm_remaining > prep.mm_complete); // **Complete** Exit loop. Profile complete.

#ifdef VARIABLE_SPINDLE
/* -----
    Compute spindle speed PWM output for step segment
*/

if (st_prep_block->is_pwm_rate_adjusted || (sys.step_control & STEP_CONTROL_UPDATE_SPINDLE_PWM)) {
    if (pl_block->condition & (PL_COND_FLAG_SPINDLE_CW | PL_COND_FLAG_SPINDLE_CCW)) {
        float rpm = pl_block->spindle_speed;
        // NOTE: Feed and rapid overrides are independent of PWM value and do not alter laser power/rate.
        if (st_prep_block->is_pwm_rate_adjusted) { rpm *= (prep.current_speed * prep.inv_rate); }
        // If current_speed is zero, then may need to be rpm_min*(100/MAX_SPINDLE_SPEED_OVERRIDE)
    }
}

```

```

    // but this would be instantaneous only and during a motion. May not matter at all.
    prep.current_spindle_pwm = spindle_compute_pwm_value(rpm);
} else {
    sys.spindle_speed = 0.0;
    prep.current_spindle_pwm = SPINDLE_PWM_OFF_VALUE;
}
bit_false(sys.step_control, STEP_CONTROL_UPDATE_SPINDLE_PWM);
}
prep_segment->spindle_pwm = prep.current_spindle_pwm; // Reload segment PWM value

#endif

/* -----
Compute segment step rate, steps to execute, and apply necessary rate corrections.
NOTE: Steps are computed by direct scalar conversion of the millimeter distance
remaining in the block, rather than incrementally tallying the steps executed per
segment. This helps in removing floating point round-off issues of several additions.
However, since floats have only 7.2 significant digits, long moves with extremely
high step counts can exceed the precision of floats, which can lead to lost steps.
Fortunately, this scenario is highly unlikely and unrealistic in CNC machines
supported by Grbl (i.e. exceeding 10 meters axis travel at 200 step/mm).
*/
float step_dist_remaining = prep.step_per_mm*mm_remaining; // Convert mm_remaining to steps
float n_steps_remaining = ceil(step_dist_remaining); // Round-up current steps remaining
float last_n_steps_remaining = ceil(prepare.steps_remaining); // Round-up last steps remaining
prep_segment->n_step = last_n_steps_remaining-n_steps_remaining; // Compute number of steps to execute.

// Bail if we are at the end of a feed hold and don't have a step to execute.
if (prep_segment->n_step == 0) {
    if (sys.step_control & STEP_CONTROL_EXECUTE_HOLD) {
        // Less than one step to decelerate to zero speed, but already very close. AMASS
        // requires full steps to execute. So, just bail.
        bit_true(sys.step_control, STEP_CONTROL_END_MOTION);
#ifdef PARKING_ENABLE
        if (!(prep.recalculate_flag & PREP_FLAG_PARKING)) { prep.recalculate_flag |= PREP_FLAG_HOLD_PARTIAL_BLOCK; }
#endif
        return; // Segment not generated, but current step data still retained.
    }
}

// Compute segment step rate. Since steps are integers and mm distances traveled are not,
// the end of every segment can have a partial step of varying magnitudes that are not
// executed, because the stepper ISR requires whole steps due to the AMASS algorithm. To
// compensate, we track the time to execute the previous segment's partial step and simply
// apply it with the partial step distance to the current segment, so that it minutely
// adjusts the whole segment rate to keep step output exact. These rate adjustments are
// typically very small and do not adversely effect performance, but ensures that Grbl
// outputs the exact acceleration and velocity profiles as computed by the planner.
dt += prep.dt_remainder; // Apply previous segment partial step execute time
float inv_rate = dt/(last_n_steps_remaining - step_dist_remaining); // Compute adjusted step rate inverse

```

```

// Compute CPU cycles per step for the prepped segment.
uint32_t cycles = ceil( (TICKS_PER_MICROSECOND*1000000*60)*inv_rate ); // (cycles/step)

#ifdef ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING
// Compute step timing and multi-axis smoothing level.
// NOTE: AMASS overdrives the timer with each level, so only one prescaler is required.
if (cycles < AMASS_LEVEL1) { prep_segment->amass_level = 0; }
else {
    if (cycles < AMASS_LEVEL2) { prep_segment->amass_level = 1; }
    else if (cycles < AMASS_LEVEL3) { prep_segment->amass_level = 2; }
    else { prep_segment->amass_level = 3; }
    cycles >>= prep_segment->amass_level;
    prep_segment->n_step <=& prep_segment->amass_level;
}
if (cycles < (1UL << 16)) { prep_segment->cycles_per_tick = cycles; } // < 65536 (4.1ms @ 16MHz)
else { prep_segment->cycles_per_tick = 0xffff; } // Just set the slowest speed possible.
#else
// Compute step timing and timer prescaler for normal step generation.
if (cycles < (1UL << 16)) { // < 65536 (4.1ms @ 16MHz)
    prep_segment->prescaler = 1; // prescaler: 0
    prep_segment->cycles_per_tick = cycles;
} else if (cycles < (1UL << 19)) { // < 524288 (32.8ms@16MHz)
    prep_segment->prescaler = 2; // prescaler: 8
    prep_segment->cycles_per_tick = cycles >> 3;
} else {
    prep_segment->prescaler = 3; // prescaler: 64
    if (cycles < (1UL << 22)) { // < 4194304 (262ms@16MHz)
        prep_segment->cycles_per_tick = cycles >> 6;
    } else { // Just set the slowest speed possible. (Around 4 step/sec.)
        prep_segment->cycles_per_tick = 0xffff;
    }
}
}
#endif

// Segment complete! Increment segment buffer indices, so stepper ISR can immediately execute it.
segment_buffer_head = segment_next_head;
if ( ++segment_next_head == SEGMENT_BUFFER_SIZE ) { segment_next_head = 0; }

// Update the appropriate planner and segment data.
pl_block->millimeters = mm_remaining;
prep.steps_remaining = n_steps_remaining;
prep.dt_remainder = (n_steps_remaining - step_dist_remaining)*inv_rate;

// Check for exit conditions and flag to load next planner block.
if (mm_remaining == prep.mm_complete) {
    // End of planner block or forced-termination. No more distance to be executed.
    if (mm_remaining > 0.0) { // At end of forced-termination.
        // Reset prep parameters for resuming and then bail. Allow the stepper ISR to complete
        // the segment queue, where realtime protocol will set new state upon receiving the
        // cycle stop flag from the ISR. Prep_segment is blocked until then.
        bit_true(sys.step_control,STEP_CONTROL_END_MOTION);
    }
}

```

```

#ifdef PARKING_ENABLE
    if (!(prep.recalculate_flag & PREP_FLAG_PARKING)) { prep.recalculate_flag |= PREP_FLAG_HOLD_PARTIAL_BLOCK; }
#endif
    return; // Bail!
} else { // End of planner block
    // The planner block is complete. All steps are set to be executed in the segment buffer.
    if (sys.step_control & STEP_CONTROL_EXECUTE_SYS_MOTION) {
        bit_true(sys.step_control, STEP_CONTROL_END_MOTION);
        return;
    }
    pl_block = NULL; // Set pointer to indicate check and load next planner block.
    plan_discard_current_block();
}
}
}
}

```

```

// Called by realtime status reporting to fetch the current speed being executed. This value
// however is not exactly the current speed, but the speed computed in the last step segment
// in the segment buffer. It will always be behind by up to the number of segment blocks (-1)
// divided by the ACCELERATION TICKS PER SECOND in seconds.
float st_get_realtime_rate()
{
    if (sys.state & (STATE_CYCLE | STATE_HOMING | STATE_HOLD | STATE_JOG | STATE_SAFETY_DOOR)){
        return prep.current_speed;
    }
    return 0.0f;
}

```

File: stepper.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
stepper.h - stepper motor driver: executes motion plans of planner.c using the stepper motors
Part of Grbl

```

Copyright (c) 2011-2016 Sungeun K. Jeon for Gnea Research LLC
 Copyright (c) 2009-2011 Simen Svale Skogsrud

Grbl is free software: you can redistribute it and/or modify
 it under the terms of the GNU General Public License as published by
 the Free Software Foundation, either version 3 of the License, or
 (at your option) any later version.

Grbl is distributed in the hope that it will be useful,
 but WITHOUT ANY WARRANTY; without even the implied warranty of
 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the

GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#ifndef stepper_h
#define stepper_h
```

```
#ifndef SEGMENT_BUFFER_SIZE
#define SEGMENT_BUFFER_SIZE 6
#endif
```

```
// Initialize and setup the stepper motor subsystem
void stepper_init();
```

```
// Enable steppers, but cycle does not start unless called by motion control or realtime command.
void st_wake_up();
```

```
// Immediately disables steppers
void st_go_idle();
```

```
// Generate the step and direction port invert masks.
void st_generate_step_dir_invert_masks();
```

```
// Reset the stepper subsystem variables
void st_reset();
```

```
// Changes the run state of the step segment buffer to execute the special parking motion.
void st_parking_setup_buffer();
```

```
// Restores the step segment buffer to the normal run state after a parking motion.
void st_parking_restore_buffer();
```

```
// Reloads step segment buffer. Called continuously by realtime execution system.
void st_prep_buffer();
```

```
// Called by planner_recalculate() when the executing block is updated by the new plan.
void st_update_plan_block_parameters();
```

```
// Called by realtime status reporting if realtime rate reporting is enabled in config.h.
float st_get_realtime_rate();
```

```
#endif
```

File: system.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

system.c - Handles system level commands and real-time processes
Part of Grbl

Copyright (c) 2014-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#include "grbl.h"
```

```
void system_init()
```

```
{  
  CONTROL_DDR &= ~(CONTROL_MASK); // Configure as input pins  
#ifdef DISABLE_CONTROL_PIN_PULL_UP  
  CONTROL_PORT &= ~(CONTROL_MASK); // Normal low operation. Requires external pull-down.  
#else  
  CONTROL_PORT |= CONTROL_MASK; // Enable internal pull-up resistors. Normal high operation.  
#endif  
  CONTROL_PCMSK |= CONTROL_MASK; // Enable specific pins of the Pin Change Interrupt  
  PCICR |= (1 << CONTROL_INT); // Enable Pin Change Interrupt  
}
```

```
// Returns control pin state as a uint8 bitfield. Each bit indicates the input pin state, where  
// triggered is 1 and not triggered is 0. Invert mask is applied. Bitfield organization is  
// defined by the CONTROL_PIN_INDEX in the header file.
```

```
uint8_t system_control_get_state()  
{  
  uint8_t control_state = 0;  
  uint8_t pin = (CONTROL_PIN & CONTROL_MASK) ^ CONTROL_MASK;  
#ifdef INVERT_CONTROL_PIN_MASK  
  pin ^= INVERT_CONTROL_PIN_MASK;  
#endif  
  if (pin) {  
#ifdef ENABLE_SAFETY_DOOR_INPUT_PIN
```

```

    if (bit_istrue(pin,(1<<CONTROL_SAFETY_DOOR_BIT))) { control_state |= CONTROL_PIN_INDEX_SAFETY_DOOR; }
#else
    if (bit_istrue(pin,(1<<CONTROL_FEED_HOLD_BIT))) { control_state |= CONTROL_PIN_INDEX_FEED_HOLD; }
#endif
    if (bit_istrue(pin,(1<<CONTROL_RESET_BIT))) { control_state |= CONTROL_PIN_INDEX_RESET; }
    if (bit_istrue(pin,(1<<CONTROL_CYCLE_START_BIT))) { control_state |= CONTROL_PIN_INDEX_CYCLE_START; }
}
return(control_state);
}

```

// Pin change interrupt for pin-out commands, i.e. cycle start, feed hold, and reset. Sets
 // only the realtime command execute variable to have the main program execute these when
 // its ready. This works exactly like the character-based realtime commands when picked off
 // directly from the incoming serial data stream.

```

ISR(CONTROL_INT_vect)
{
    uint8_t pin = system_control_get_state();
    if (pin) {
        if (bit_istrue(pin,CONTROL_PIN_INDEX_RESET)) {
            mc_reset();
        }
        if (bit_istrue(pin,CONTROL_PIN_INDEX_CYCLE_START)) {
            bit_true(sys_rt_exec_state, EXEC_CYCLE_START);
        }
#ifdef ENABLE_SAFETY_DOOR_INPUT_PIN
        if (bit_istrue(pin,CONTROL_PIN_INDEX_FEED_HOLD)) {
            bit_true(sys_rt_exec_state, EXEC_FEED_HOLD);
        }
    #else
        if (bit_istrue(pin,CONTROL_PIN_INDEX_SAFETY_DOOR)) {
            bit_true(sys_rt_exec_state, EXEC_SAFETY_DOOR);
        }
    #endif
    }
}

```

// Returns if safety door is ajar(T) or closed(F), based on pin state.

```

uint8_t system_check_safety_door_ajar()
{
#ifdef ENABLE_SAFETY_DOOR_INPUT_PIN
    return(system_control_get_state() & CONTROL_PIN_INDEX_SAFETY_DOOR);
#else
    return(false); // Input pin not enabled, so just return that it's closed.
#endif
}

```

// Executes user startup script, if stored.

```

void system_execute_startup(char *line)
{

```

```

uint8_t n;
for (n=0; n < N_STARTUP_LINE; n++) {
    if (!(settings_read_startup_line(n, line))) {
        line[0] = 0;
        report_execute_startup_message(line, STATUS_SETTING_READ_FAIL);
    } else {
        if (line[0] != 0) {
            uint8_t status_code = gc_execute_line(line);
            report_execute_startup_message(line, status_code);
        }
    }
}
}
}

```

// Directs and executes one line of formatted input from protocol_process. While mostly
 // incoming streaming g-code blocks, this also executes Grbl internal commands, such as
 // settings, initiating the homing cycle, and toggling switch states. This differs from
 // the realtime command module by being susceptible to when Grbl is ready to execute the
 // next line during a cycle, so for switches like block delete, the switch only effects
 // the lines that are processed afterward, not necessarily real-time during a cycle,
 // since there are motions already stored in the buffer. However, this 'lag' should not
 // be an issue, since these commands are not typically used during a cycle.

```

uint8_t system_execute_line(char *line)
{
    uint8_t char_counter = 1;
    uint8_t helper_var = 0; // Helper variable
    float parameter, value;
    switch( line[char_counter] ) {
        case 0 : report_grbl_help(); break;
        case 'J' : // Jogging
            // Execute only if in IDLE or JOG states.
            if (sys.state != STATE_IDLE && sys.state != STATE_JOG) { return(STATUS_IDLE_ERROR); }
            if(line[2] != '=') { return(STATUS_INVALID_STATEMENT); }
            return(gc_execute_line(line)); // NOTE: $J= is ignored inside g-code parser and used to detect jog motions.
            break;
        case '$': case 'G': case 'C': case 'X':
            if ( line[2] != 0 ) { return(STATUS_INVALID_STATEMENT); }
            switch( line[1] ) {
                case '$' : // Prints Grbl settings
                    if ( sys.state & (STATE_CYCLE | STATE_HOLD) ) { return(STATUS_IDLE_ERROR); } // Block during cycle. Takes to
                    else { report_grbl_settings(); }
                    break;
                case 'G' : // Prints gcode parser state
                    // TODO: Move this to realtime commands for GUIs to request this data during suspend-state.
                    report_gcode_modes();
                    break;
                case 'C' : // Set check g-code mode [IDLE/CHECK]
                    // Perform reset when toggling off. Check g-code mode should only work if Grbl
                    // is idle and ready, regardless of alarm locks. This is mainly to keep things
                    // simple and consistent.

```

```

if ( sys.state == STATE_CHECK_MODE ) {
    mc_reset();
    report_feedback_message(MESSAGE_DISABLED);
} else {
    if (sys.state) { return(STATUS_IDLE_ERROR); } // Requires no alarm mode.
    sys.state = STATE_CHECK_MODE;
    report_feedback_message(MESSAGE_ENABLED);
}
break;
case 'X' : // Disable alarm lock [ALARM]
if (sys.state == STATE_ALARM) {
    // Block if safety door is ajar.
    if (system_check_safety_door_ajar()) { return(STATUS_CHECK_DOOR); }
    report_feedback_message(MESSAGE_ALARM_UNLOCK);
    sys.state = STATE_IDLE;
    // Don't run startup script. Prevents stored moves in startup from causing accidents.
} // Otherwise, no effect.
break;
}
break;
default :
// Block any system command that requires the state as IDLE/ALARM. (i.e. EEPROM, homing)
if ( !(sys.state == STATE_IDLE || sys.state == STATE_ALARM) ) { return(STATUS_IDLE_ERROR); }
switch( line[1] ) {
case '#' : // Print Grbl NGC parameters
    if ( line[2] != 0 ) { return(STATUS_INVALID_STATEMENT); }
    else { report_ngc_parameters(); }
    break;
case 'H' : // Perform homing cycle [IDLE/ALARM]
    if (bit_isfalse(settings.flags,BITFLAG_HOMING_ENABLE)) {return(STATUS_SETTING_DISABLED); }
    if (system_check_safety_door_ajar()) { return(STATUS_CHECK_DOOR); } // Block if safety door is ajar.
    sys.state = STATE_HOMING; // Set system state variable
    if (line[2] == 0) {
        mc_homing_cycle(HOMING_CYCLE_ALL);
#ifdef HOMING_SINGLE_AXIS_COMMANDS
    } else if (line[3] == 0) {
        switch (line[2]) {
            case 'X': mc_homing_cycle(HOMING_CYCLE_X); break;
            case 'Y': mc_homing_cycle(HOMING_CYCLE_Y); break;
            case 'Z': mc_homing_cycle(HOMING_CYCLE_Z); break;
            default: return(STATUS_INVALID_STATEMENT);
        }
    }
#endif
    } else { return(STATUS_INVALID_STATEMENT); }
    if (!sys.abort) { // Execute startup scripts after successful homing.
        sys.state = STATE_IDLE; // Set to IDLE when complete.
        st_go_idle(); // Set steppers to the settings idle state before returning.
        if (line[2] == 0) { system_execute_startup(line); }
    }
    break;
case 'S' : // Puts Grbl to sleep [IDLE/ALARM]

```

```

if ((line[2] != 'L') || (line[3] != 'P') || (line[4] != 0)) { return(STATUS_INVALID_STATEMENT); }
system_set_exec_state_flag(EXEC_SLEEP); // Set to execute sleep mode immediately
break;
case 'I' : // Print or store build info. [IDLE/ALARM]
if ( line[++char_counter] == 0 ) {
    settings_read_build_info(line);
    report_build_info(line);
#ifdef ENABLE_BUILD_INFO_WRITE_COMMAND
} else { // Store startup line [IDLE/ALARM]
    if(line[char_counter++] != '=') { return(STATUS_INVALID_STATEMENT); }
    helper_var = char_counter; // Set helper variable as counter to start of user info line.
    do {
        line[char_counter-helper_var] = line[char_counter];
    } while (line[char_counter++] != 0);
    settings_store_build_info(line);
#endif
}
break;
case 'R' : // Restore defaults [IDLE/ALARM]
if ((line[2] != 'S') || (line[3] != 'T') || (line[4] != '=') || (line[6] != 0)) { return(STATUS_INVALID_STATEMENT); }
switch (line[5]) {
    #ifdef ENABLE_RESTORE_EEPROM_DEFAULT_SETTINGS
    case '$': settings_restore(SETTINGS_RESTORE_DEFAULTS); break;
    #endif
    #ifdef ENABLE_RESTORE_EEPROM_CLEAR_PARAMETERS
    case '#': settings_restore(SETTINGS_RESTORE_PARAMETERS); break;
    #endif
    #ifdef ENABLE_RESTORE_EEPROM_WIPE_ALL
    case '*': settings_restore(SETTINGS_RESTORE_ALL); break;
    #endif
    default: return(STATUS_INVALID_STATEMENT);
}
report_feedback_message(MESSAGE_RESTORE_DEFAULTS);
mc_reset(); // Force reset to ensure settings are initialized correctly.
break;
case 'N' : // Startup lines. [IDLE/ALARM]
if ( line[++char_counter] == 0 ) { // Print startup lines
    for (helper_var=0; helper_var < N_STARTUP_LINE; helper_var++) {
        if (!(settings_read_startup_line(helper_var, line))) {
            report_status_message(STATUS_SETTING_READ_FAIL);
        } else {
            report_startup_line(helper_var,line);
        }
    }
}
break;
} else { // Store startup line [IDLE Only] Prevents motion during ALARM.
    if (sys.state != STATE_IDLE) { return(STATUS_IDLE_ERROR); } // Store only when idle.
    helper_var = true; // Set helper_var to flag storing method.
    // No break. Continues into default: to read remaining command characters.
}
default : // Storing setting methods [IDLE/ALARM]

```

```

    if(!read_float(line, &char_counter, &parameter)) { return(STATUS_BAD_NUMBER_FORMAT); }
    if(line[char_counter++] != '=') { return(STATUS_INVALID_STATEMENT); }
    if (helper_var) { // Store startup line
        // Prepare sending gcode block to gcode parser by shifting all characters
        helper_var = char_counter; // Set helper variable as counter to start of gcode block
        do {
            line[char_counter-helper_var] = line[char_counter];
        } while (line[char_counter++] != 0);
        // Execute gcode block to ensure block is valid.
        helper_var = gc_execute_line(line); // Set helper_var to returned status code.
        if (helper_var) { return(helper_var); }
        else {
            helper_var = trunc(parameter); // Set helper_var to int value of parameter
            settings_store_startup_line(helper_var,line);
        }
    } else { // Store global setting.
        if(!read_float(line, &char_counter, &value)) { return(STATUS_BAD_NUMBER_FORMAT); }
        if((line[char_counter] != 0) || (parameter > 255)) { return(STATUS_INVALID_STATEMENT); }
        return(settings_store_global_setting((uint8_t)parameter, value));
    }
}
}
return(STATUS_OK); // If '$' command makes it to here, then everything's ok.
}

```

```

void system_flag_wco_change()
{
#ifdef FORCE_BUFFER_SYNC_DURING_WCO_CHANGE
    protocol_buffer_synchronize();
#endif
    sys.report_wco_counter = 0;
}

```

// Returns machine position of axis 'idx'. Must be sent a 'step' array.
// NOTE: If motor steps and machine position are not in the same coordinate frame, this function
// serves as a central place to compute the transformation.

```

float system_convert_axis_steps_to_mpos(int32_t *steps, uint8_t idx)
{
    float pos;
#ifdef COREXY
    if (idx==X_AXIS) {
        pos = (float)system_convert_corexy_to_x_axis_steps(steps) / settings.steps_per_mm[idx];
    } else if (idx==Y_AXIS) {
        pos = (float)system_convert_corexy_to_y_axis_steps(steps) / settings.steps_per_mm[idx];
    } else {
        pos = steps[idx]/settings.steps_per_mm[idx];
    }
#else

```

```

    pos = steps[idx]/settings.steps_per_mm[idx];
#endif
return(pos);
}

```

```

void system_convert_array_steps_to_mpos(float *position, int32_t *steps)
{
    uint8_t idx;
    for (idx=0; idx<N_AXIS; idx++) {
        position[idx] = system_convert_axis_steps_to_mpos(steps, idx);
    }
    return;
}

```

// CoreXY calculation only. Returns x or y-axis "steps" based on CoreXY motor steps.

```

#ifdef COREXY
int32_t system_convert_corexy_to_x_axis_steps(int32_t *steps)
{
    return( (steps[A_MOTOR] + steps[B_MOTOR])/2 );
}
int32_t system_convert_corexy_to_y_axis_steps(int32_t *steps)
{
    return( (steps[A_MOTOR] - steps[B_MOTOR])/2 );
}
#endif

```

// Checks and reports if target array exceeds machine travel limits.

```

uint8_t system_check_travel_limits(float *target)
{
    uint8_t idx;
    for (idx=0; idx<N_AXIS; idx++) {
#ifdef HOMING_FORCE_SET_ORIGIN
        // When homing forced set origin is enabled, soft limits checks need to account for directionality.
        // NOTE: max_travel is stored as negative
        if (bit_istrue(settings.homing_dir_mask, bit(idx))) {
            if (target[idx] < 0 || target[idx] > -settings.max_travel[idx]) { return(true); }
        } else {
            if (target[idx] > 0 || target[idx] < settings.max_travel[idx]) { return(true); }
        }
    }
    #else
        // NOTE: max_travel is stored as negative
        if (target[idx] > 0 || target[idx] < settings.max_travel[idx]) { return(true); }
    #endif
}
return(false);
}

```

// Special handlers for setting and clearing Grbl's real-time execution flags.

```
void system_set_exec_state_flag(uint8_t mask) {  
    uint8_t sreg = SREG;  
    cli();  
    sys_rt_exec_state |= (mask);  
    SREG = sreg;  
}
```

```
void system_clear_exec_state_flag(uint8_t mask) {  
    uint8_t sreg = SREG;  
    cli();  
    sys_rt_exec_state &= ~(mask);  
    SREG = sreg;  
}
```

```
void system_set_exec_alarm(uint8_t code) {  
    uint8_t sreg = SREG;  
    cli();  
    sys_rt_exec_alarm = code;  
    SREG = sreg;  
}
```

```
void system_clear_exec_alarm() {  
    uint8_t sreg = SREG;  
    cli();  
    sys_rt_exec_alarm = 0;  
    SREG = sreg;  
}
```

```
void system_set_exec_motion_override_flag(uint8_t mask) {  
    uint8_t sreg = SREG;  
    cli();  
    sys_rt_exec_motion_override |= (mask);  
    SREG = sreg;  
}
```

```
void system_set_exec_accessory_override_flag(uint8_t mask) {  
    uint8_t sreg = SREG;  
    cli();  
    sys_rt_exec_accessory_override |= (mask);  
    SREG = sreg;  
}
```

```
void system_clear_exec_motion_overrides() {  
    uint8_t sreg = SREG;  
    cli();  
    sys_rt_exec_motion_override = 0;  
    SREG = sreg;  
}
```

```
void system_clear_exec_accessory_overrides() {
```

```

uint8_t sreg = SREG;
cli();
sys_rt_exec_accessory_override = 0;
SREG = sreg;
}

```

File: system.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

/*

system.h - Header for system level commands and real-time processes
Part of Grbl

Copyright (c) 2014-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

Grbl is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```

#ifndef system_h
#define system_h

```

```

#include "grbl.h"

```

```

// Define system executor bit map. Used internally by realtime protocol as realtime command flags,
// which notifies the main program to execute the specified realtime command asynchronously.
// NOTE: The system executor uses an unsigned 8-bit volatile variable (8 flag limit.) The default
// flags are always false, so the realtime protocol only needs to check for a non-zero value to
// know when there is a realtime command to execute.

```

```

#define EXEC_STATUS_REPORT bit(0) // bitmask 00000001
#define EXEC_CYCLE_START   bit(1) // bitmask 00000010
#define EXEC_CYCLE_STOP    bit(2) // bitmask 00000100
#define EXEC_FEED_HOLD     bit(3) // bitmask 00001000
#define EXEC_RESET         bit(4) // bitmask 00010000
#define EXEC_SAFETY_DOOR   bit(5) // bitmask 00100000
#define EXEC_MOTION_CANCEL bit(6) // bitmask 01000000
#define EXEC_SLEEP         bit(7) // bitmask 10000000

```

```

// Alarm executor codes. Valid values (1-255). Zero is reserved.

```

```

#define EXEC_ALARM_HARD_LIMIT      1
#define EXEC_ALARM_SOFT_LIMIT      2
#define EXEC_ALARM_ABORT_CYCLE     3
#define EXEC_ALARM_PROBE_FAIL_INITIAL 4
#define EXEC_ALARM_PROBE_FAIL_CONTACT 5
#define EXEC_ALARM_HOMING_FAIL_RESET 6
#define EXEC_ALARM_HOMING_FAIL_DOOR 7
#define EXEC_ALARM_HOMING_FAIL_PULLOFF 8
#define EXEC_ALARM_HOMING_FAIL_APPROACH 9
#define EXEC_ALARM_HOMING_FAIL_DUAL_APPROACH 10

```

```

// Override bit maps. Realtime bitflags to control feed, rapid, spindle, and coolant overrides.
// Spindle/coolant and feed/rapids are separated into two controlling flag variables.

```

```

#define EXEC_FEED_OVR_RESET      bit(0)
#define EXEC_FEED_OVR_COARSE_PLUS bit(1)
#define EXEC_FEED_OVR_COARSE_MINUS bit(2)
#define EXEC_FEED_OVR_FINE_PLUS  bit(3)
#define EXEC_FEED_OVR_FINE_MINUS bit(4)
#define EXEC_RAPID_OVR_RESET     bit(5)
#define EXEC_RAPID_OVR_MEDIUM    bit(6)
#define EXEC_RAPID_OVR_LOW       bit(7)
// #define EXEC_RAPID_OVR_EXTRA_LOW bit(*) // *NOT SUPPORTED*

```

```

#define EXEC_SPINDLE_OVR_RESET    bit(0)
#define EXEC_SPINDLE_OVR_COARSE_PLUS bit(1)
#define EXEC_SPINDLE_OVR_COARSE_MINUS bit(2)
#define EXEC_SPINDLE_OVR_FINE_PLUS  bit(3)
#define EXEC_SPINDLE_OVR_FINE_MINUS bit(4)
#define EXEC_SPINDLE_OVR_STOP      bit(5)
#define EXEC_COOLANT_FLOOD_OVR_TOGGLE bit(6)
#define EXEC_COOLANT_MIST_OVR_TOGGLE bit(7)

```

```

// Define system state bit map. The state variable primarily tracks the individual functions
// of Grbl to manage each without overlapping. It is also used as a messaging flag for
// critical events.

```

```

#define STATE_IDLE      0    // Must be zero. No flags.
#define STATE_ALARM     bit(0) // In alarm state. Locks out all g-code processes. Allows settings access.
#define STATE_CHECK_MODE bit(1) // G-code check mode. Locks out planner and motion only.
#define STATE_HOMING     bit(2) // Performing homing cycle
#define STATE_CYCLE      bit(3) // Cycle is running or motions are being executed.
#define STATE_HOLD       bit(4) // Active feed hold
#define STATE_JOG        bit(5) // Jogging mode.
#define STATE_SAFETY_DOOR bit(6) // Safety door is ajar. Feed holds and de-energizes system.
#define STATE_SLEEP      bit(7) // Sleep state.

```

```

// Define system suspend flags. Used in various ways to manage suspend states and procedures.

```

```

#define SUSPEND_DISABLE      0    // Must be zero.
#define SUSPEND_HOLD_COMPLETE bit(0) // Indicates initial feed hold is complete.
#define SUSPEND_RESTART_RETRACT bit(1) // Flag to indicate a retract from a restore parking motion.
#define SUSPEND_RETRACT_COMPLETE bit(2) // (Safety door only) Indicates retraction and de-energizing is complete.
#define SUSPEND_INITIATE_RESTORE bit(3) // (Safety door only) Flag to initiate resume procedures from a cycle start.

```

```

#define SUSPEND_RESTORE_COMPLETE bit(4) // (Safety door only) Indicates ready to resume normal operation.
#define SUSPEND_SAFETY_DOOR_AJAR bit(5) // Tracks safety door state for resuming.
#define SUSPEND_MOTION_CANCEL bit(6) // Indicates a canceled resume motion. Currently used by probing routine.
#define SUSPEND_JOG_CANCEL bit(7) // Indicates a jog cancel in process and to reset buffers when complete.

// Define step segment generator state flags.
#define STEP_CONTROL_NORMAL_OP 0 // Must be zero.
#define STEP_CONTROL_END_MOTION bit(0)
#define STEP_CONTROL_EXECUTE_HOLD bit(1)
#define STEP_CONTROL_EXECUTE_SYS_MOTION bit(2)
#define STEP_CONTROL_UPDATE_SPINDLE_PWM bit(3)

// Define control pin index for Grbl internal use. Pin maps may change, but these values don't.
#ifndef ENABLE_SAFETY_DOOR_INPUT_PIN
#define N_CONTROL_PIN 4
#define CONTROL_PIN_INDEX_SAFETY_DOOR bit(0)
#define CONTROL_PIN_INDEX_RESET bit(1)
#define CONTROL_PIN_INDEX_FEED_HOLD bit(2)
#define CONTROL_PIN_INDEX_CYCLE_START bit(3)
#else
#define N_CONTROL_PIN 3
#define CONTROL_PIN_INDEX_RESET bit(0)
#define CONTROL_PIN_INDEX_FEED_HOLD bit(1)
#define CONTROL_PIN_INDEX_CYCLE_START bit(2)
#endif

// Define spindle stop override control states.
#define SPINDLE_STOP_OVR_DISABLED 0 // Must be zero.
#define SPINDLE_STOP_OVR_ENABLED bit(0)
#define SPINDLE_STOP_OVR_INITIATE bit(1)
#define SPINDLE_STOP_OVR_RESTORE bit(2)
#define SPINDLE_STOP_OVR_RESTORE_CYCLE bit(3)

// Define global system variables
typedef struct {
  uint8_t state; // Tracks the current system state of Grbl.
  uint8_t abort; // System abort flag. Forces exit back to main loop for reset.
  uint8_t suspend; // System suspend bitflag variable that manages holds, cancels, and safety door.
  uint8_t soft_limit; // Tracks soft limit errors for the state machine. (boolean)
  uint8_t step_control; // Governs the step segment generator depending on system state.
  uint8_t probe_succeeded; // Tracks if last probing cycle was successful.
  uint8_t homing_axis_lock; // Locks axes when limits engage. Used as an axis motion mask in the stepper ISR.
#ifdef ENABLE_DUAL_AXIS
  uint8_t homing_axis_lock_dual;
#endif
  uint8_t f_override; // Feed rate override value in percent
  uint8_t r_override; // Rapids override value in percent
  uint8_t spindle_speed_ovr; // Spindle speed value in percent
  uint8_t spindle_stop_ovr; // Tracks spindle stop override states
  uint8_t report_ovr_counter; // Tracks when to add override data to status reports.

```

```

uint8_t report_wco_counter; // Tracks when to add work coordinate offset data to status reports.
#ifdef ENABLE_PARKING_OVERRIDE_CONTROL
    uint8_t override_ctrl; // Tracks override control states.
#endif
#ifdef VARIABLE_SPINDLE
    float spindle_speed;
#endif
} system_t;
extern system_t sys;

```

```

// NOTE: These position variables may need to be declared as volatiles, if problems arise.
extern int32_t sys_position[N_AXIS]; // Real-time machine (aka home) position vector in steps.
extern int32_t sys_probe_position[N_AXIS]; // Last probe position in machine coordinates and steps.

```

```

extern volatile uint8_t sys_probe_state; // Probing state value. Used to coordinate the probing cycle with stepper ISR.
extern volatile uint8_t sys_rt_exec_state; // Global realtime executor bitflag variable for state management. See EXEC bitm
extern volatile uint8_t sys_rt_exec_alarm; // Global realtime executor bitflag variable for setting various alarms.
extern volatile uint8_t sys_rt_exec_motion_override; // Global realtime executor bitflag variable for motion-based overrides.
extern volatile uint8_t sys_rt_exec_accessory_override; // Global realtime executor bitflag variable for spindle/coolant overri

```

```

#ifdef DEBUG
    #define EXEC_DEBUG_REPORT bit(0)
    extern volatile uint8_t sys_rt_exec_debug;
#endif

```

```

// Initialize the serial protocol
void system_init();

```

```

// Returns bitfield of control pin states, organized by CONTROL_PIN_INDEX. (1=triggered, 0=not triggered).
uint8_t system_control_get_state();

```

```

// Returns if safety door is open or closed, based on pin state.
uint8_t system_check_safety_door_ajar();

```

```

// Executes an internal system command, defined as a string starting with a '$'
uint8_t system_execute_line(char *line);

```

```

// Execute the startup script lines stored in EEPROM upon initialization
void system_execute_startup(char *line);

```

```

void system_flag_wco_change();

```

```

// Returns machine position of axis 'idx'. Must be sent a 'step' array.
float system_convert_axis_steps_to_mpos(int32_t *steps, uint8_t idx);

```

```

// Updates a machine 'position' array based on the 'step' array sent.
void system_convert_array_steps_to_mpos(float *position, int32_t *steps);

```

```

// CoreXY calculation only. Returns x or y-axis "steps" based on CoreXY motor steps.
#ifdef COREXY

```

```

int32_t system_convert_corexy_to_x_axis_steps(int32_t *steps);
int32_t system_convert_corexy_to_y_axis_steps(int32_t *steps);
#endif

// Checks and reports if target array exceeds machine travel limits.
uint8_t system_check_travel_limits(float *target);

// Special handlers for setting and clearing Grbl's real-time execution flags.
void system_set_exec_state_flag(uint8_t mask);
void system_clear_exec_state_flag(uint8_t mask);
void system_set_exec_alarm(uint8_t code);
void system_clear_exec_alarm();
void system_set_exec_motion_override_flag(uint8_t mask);
void system_set_exec_accessory_override_flag(uint8_t mask);
void system_clear_exec_motion_overrides();
void system_clear_exec_accessory_overrides();

#endif

```

File: config.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
  config.h - compile time configuration
  Part of Grbl

  Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC
  Copyright (c) 2009-2011 Simen Svale Skogsrud

  Grbl is free software: you can redistribute it and/or modify
  it under the terms of the GNU General Public License as published by
  the Free Software Foundation, either version 3 of the License, or
  (at your option) any later version.

  Grbl is distributed in the hope that it will be useful,
  but WITHOUT ANY WARRANTY; without even the implied warranty of
  MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
  GNU General Public License for more details.

  You should have received a copy of the GNU General Public License
  along with Grbl. If not, see <http://www.gnu.org/licenses/>.

  */

// This file contains compile-time configurations for Grbl's internal system. For the most part,
// users will not need to directly modify these, but they are here for specific needs, i.e.
// performance tuning or adjusting to non-typical machines.

// IMPORTANT: Any changes here requires a full re-compiling of the source code to propagate them.

```

```

#ifndef config_h
#define config_h
#include "grbl.h" // For Arduino IDE compatibility.

// Define CPU pin map and default settings.
// NOTE: OEMs can avoid the need to maintain/update the defaults.h and cpu_map.h files and use only
// one configuration file by placing their specific defaults and pin map at the bottom of this file.
// If doing so, simply comment out these two defines and see instructions below.
#define DEFAULTS_GENERIC
#define CPU_MAP_ATMEGA328P // Arduino Uno CPU

// Serial baud rate
// #define BAUD_RATE 230400
#define BAUD_RATE 115200

// Define realtime command special characters. These characters are 'picked-off' directly from the
// serial read data stream and are not passed to the grbl line execution parser. Select characters
// that do not and must not exist in the streamed g-code program. ASCII control characters may be
// used, if they are available per user setup. Also, extended ASCII codes (>127), which are never in
// g-code programs, maybe selected for interface programs.
// NOTE: If changed, manually update help message in report.c.

#define CMD_RESET 0x18 // ctrl-x.
#define CMD_STATUS_REPORT '?'
#define CMD_CYCLE_START '~'
#define CMD_FEED_HOLD '!'

// NOTE: All override realtime commands must be in the extended ASCII character set, starting
// at character value 128 (0x80) and up to 255 (0xFF). If the normal set of realtime commands,
// such as status reports, feed hold, reset, and cycle start, are moved to the extended set
// space, serial.c's RX ISR will need to be modified to accomodate the change.
// #define CMD_RESET 0x80
// #define CMD_STATUS_REPORT 0x81
// #define CMD_CYCLE_START 0x82
// #define CMD_FEED_HOLD 0x83
#define CMD_SAFETY_DOOR 0x84
#define CMD_JOG_CANCEL 0x85
#define CMD_DEBUG_REPORT 0x86 // Only when DEBUG enabled, sends debug report in '{}' braces.
#define CMD_FEED_OVR_RESET 0x90 // Restores feed override value to 100%.
#define CMD_FEED_OVR_COARSE_PLUS 0x91
#define CMD_FEED_OVR_COARSE_MINUS 0x92
#define CMD_FEED_OVR_FINE_PLUS 0x93
#define CMD_FEED_OVR_FINE_MINUS 0x94
#define CMD_RAPID_OVR_RESET 0x95 // Restores rapid override value to 100%.
#define CMD_RAPID_OVR_MEDIUM 0x96
#define CMD_RAPID_OVR_LOW 0x97
// #define CMD_RAPID_OVR_EXTRA_LOW 0x98 // *NOT SUPPORTED*
#define CMD_SPINDLE_OVR_RESET 0x99 // Restores spindle override value to 100%.
#define CMD_SPINDLE_OVR_COARSE_PLUS 0x9A

```

```
#define CMD_SPINDLE_OVR_COARSE_MINUS 0x9B
#define CMD_SPINDLE_OVR_FINE_PLUS 0x9C
#define CMD_SPINDLE_OVR_FINE_MINUS 0x9D
#define CMD_SPINDLE_OVR_STOP 0x9E
#define CMD_COOLANT_FLOOD_OVR_TOGGLE 0xA0
#define CMD_COOLANT_MIST_OVR_TOGGLE 0xA1
```

```
// If homing is enabled, homing init lock sets Grbl into an alarm state upon power up. This forces
// the user to perform the homing cycle (or override the locks) before doing anything else. This is
// mainly a safety feature to remind the user to home, since position is unknown to Grbl.
```

```
#define HOMING_INIT_LOCK // Comment to disable
```

```
// Define the homing cycle patterns with bitmasks. The homing cycle first performs a search mode
// to quickly engage the limit switches, followed by a slower locate mode, and finished by a short
// pull-off motion to disengage the limit switches. The following HOMING_CYCLE_x defines are executed
// in order starting with suffix 0 and completes the homing routine for the specified-axes only. If
// an axis is omitted from the defines, it will not home, nor will the system update its position.
// Meaning that this allows for users with non-standard cartesian machines, such as a lathe (x then z,
```

```
// with no y), to configure the homing cycle behavior to their needs.
// NOTE: The homing cycle is designed to allow sharing of limit pins, if the axes are not in the same
// cycle, but this requires some pin settings changes in cpu_map.h file. For example, the default homing
// cycle can share the Z limit pin with either X or Y limit pins, since they are on different cycles.
```

```
// By sharing a pin, this frees up a precious IO pin for other purposes. In theory, all axes limit pins
// may be reduced to one pin, if all axes are homed with separate cycles, or vice versa, all three axes
// on separate pin, but homed in one cycle. Also, it should be noted that the function of hard limits
// will not be affected by pin sharing.
```

```
// NOTE: Defaults are set for a traditional 3-axis CNC machine. Z-axis first to clear, followed by X & Y.
```

```
#define HOMING_CYCLE_0 (1<<Z_AXIS) // REQUIRED: First move Z to clear workspace.
```

```
#define HOMING_CYCLE_1 ((1<<X_AXIS)|(1<<Y_AXIS)) // OPTIONAL: Then move X,Y at the same time.
```

```
// #define HOMING_CYCLE_2 // OPTIONAL: Uncomment and add axes mask to enable
```

```
// NOTE: The following are two examples to setup homing for 2-axis machines.
```

```
// #define HOMING_CYCLE_0 ((1<<X_AXIS)|(1<<Y_AXIS)) // NOT COMPATIBLE WITH COREXY: Homes both X-Y in one
```

```
// #define HOMING_CYCLE_0 (1<<X_AXIS) // COREXY COMPATIBLE: First home X
```

```
// #define HOMING_CYCLE_1 (1<<Y_AXIS) // COREXY COMPATIBLE: Then home Y
```

```
// Number of homing cycles performed after when the machine initially jogs to limit switches.
```

```
// This help in preventing overshoot and should improve repeatability. This value should be one or
// greater.
```

```
#define N_HOMING_LOCATE_CYCLE 1 // Integer (1-128)
```

```
// Enables single axis homing commands. $HX, $HY, and $HZ for X, Y, and Z-axis homing. The full homing
// cycle is still invoked by the $H command. This is disabled by default. It's here only to address
```

```
// users that need to switch between a two-axis and three-axis machine. This is actually very rare.
```

```
// If you have a two-axis machine, DON'T USE THIS. Instead, just alter the homing cycle for two-axes.
```

```
// #define HOMING_SINGLE_AXIS_COMMANDS // Default disabled. Uncomment to enable.
```

```
// After homing, Grbl will set by default the entire machine space into negative space, as is typical
```

```
// for professional CNC machines, regardless of where the limit switches are located. Uncomment this
```

```
// define to force Grbl to always set the machine origin at the homed location despite switch orientation.
```

```

// #define HOMING_FORCE_SET_ORIGIN // Uncomment to enable.

// Number of blocks Grbl executes upon startup. These blocks are stored in EEPROM, where the size
// and addresses are defined in settings.h. With the current settings, up to 2 startup blocks may
// be stored and executed in order. These startup blocks would typically be used to set the g-code
// parser state depending on user preferences.
#define N_STARTUP_LINE 2 // Integer (1-2)

// Number of floating decimal points printed by Grbl for certain value types. These settings are
// determined by realistic and commonly observed values in CNC machines. For example, position
// values cannot be less than 0.001mm or 0.0001in, because machines can not be physically more
// precise this. So, there is likely no need to change these, but you can if you need to here.
// NOTE: Must be an integer value from 0 to ~4. More than 4 may exhibit round-off errors.
#define N_DECIMAL_COORDVALUE_INCH 4 // Coordinate or position value in inches
#define N_DECIMAL_COORDVALUE_MM 3 // Coordinate or position value in mm
#define N_DECIMAL_RATEVALUE_INCH 1 // Rate or velocity value in in/min
#define N_DECIMAL_RATEVALUE_MM 0 // Rate or velocity value in mm/min
#define N_DECIMAL_SETTINGVALUE 3 // Decimals for floating point setting values
#define N_DECIMAL_RPMVALUE 0 // RPM value in rotations per min.

// If your machine has two limits switches wired in parallel to one axis, you will need to enable
// this feature. Since the two switches are sharing a single pin, there is no way for Grbl to tell
// which one is enabled. This option only effects homing, where if a limit is engaged, Grbl will
// alarm out and force the user to manually disengage the limit switch. Otherwise, if you have one
// limit switch for each axis, don't enable this option. By keeping it disabled, you can perform a
// homing cycle while on the limit switch and not have to move the machine off of it.
// #define LIMITS_TWO_SWITCHES_ON_AXES

// Allows GRBL to track and report gcode line numbers. Enabling this means that the planning buffer
// goes from 16 to 15 to make room for the additional line number data in the plan_block_t struct
// #define USE_LINE_NUMBERS // Disabled by default. Uncomment to enable.

// Upon a successful probe cycle, this option provides immediately feedback of the probe coordinates
// through an automatically generated message. If disabled, users can still access the last probe
// coordinates through Grbl '$#' print parameters.
#define MESSAGE_PROBE_COORDINATES // Enabled by default. Comment to disable.

// Enables a second coolant control pin via the mist coolant g-code command M7 on the Arduino Uno
// analog pin 4. Only use this option if you require a second coolant control pin.
// NOTE: The M8 flood coolant control pin on analog pin 3 will still be functional regardless.
// #define ENABLE_M7 // Disabled by default. Uncomment to enable.

// This option causes the feed hold input to act as a safety door switch. A safety door, when triggered,
// immediately forces a feed hold and then safely de-energizes the machine. Resuming is blocked until
// the safety door is re-engaged. When it is, Grbl will re-energize the machine and then resume on the
// previous tool path, as if nothing happened.
// #define ENABLE_SAFETY_DOOR_INPUT_PIN // Default disabled. Uncomment to enable.

// After the safety door switch has been toggled and restored, this setting sets the power-up delay
// between restoring the spindle and coolant and resuming the cycle.
#define SAFETY_DOOR_SPINDLE_DELAY 4.0 // Float (seconds)

```

```

#define SAFETY_DOOR_COOLANT_DELAY 1.0 // Float (seconds)

// Enable CoreXY kinematics. Use ONLY with CoreXY machines.
// IMPORTANT: If homing is enabled, you must reconfigure the homing cycle #defines above to
// #define HOMING_CYCLE_0 (1<<X_AXIS) and #define HOMING_CYCLE_1 (1<<Y_AXIS)
// NOTE: This configuration option alters the motion of the X and Y axes to principle of operation
// defined at (http://corexy.com/theory.html). Motors are assumed to positioned and wired exactly as
// described, if not, motions may move in strange directions. Grbl requires the CoreXY A and B motors
// have the same steps per mm internally.
// #define COREXY // Default disabled. Uncomment to enable.

// Inverts pin logic of the control command pins based on a mask. This essentially means you can use
// normally-closed switches on the specified pins, rather than the default normally-open switches.
// NOTE: The top option will mask and invert all control pins. The bottom option is an example of
// inverting only two control pins, the safety door and reset. See cpu_map.h for other bit definitions.
// #define INVERT_CONTROL_PIN_MASK CONTROL_MASK // Default disabled. Uncomment to disable.
// #define INVERT_CONTROL_PIN_MASK ((1<<CONTROL_SAFETY_DOOR_BIT)|(1<<CONTROL_RESET_BIT)) // Default disabled. Uncomment to enable.

// Inverts select limit pin states based on the following mask. This effects all limit pin functions,
// such as hard limits and homing. However, this is different from overall invert limits setting.
// This build option will invert only the limit pins defined here, and then the invert limits setting
// will be applied to all of them. This is useful when a user has a mixed set of limit pins with both
// normally-open(NO) and normally-closed(NC) switches installed on their machine.
// NOTE: PLEASE DO NOT USE THIS, unless you have a situation that needs it.
// #define INVERT_LIMIT_PIN_MASK ((1<<X_LIMIT_BIT)|(1<<Y_LIMIT_BIT)) // Default disabled. Uncomment to enable.

// Inverts the spindle enable pin from low-disabled/high-enabled to low-enabled/high-disabled. Useful
// for some pre-built electronic boards.
// NOTE: If VARIABLE_SPINDLE is enabled(default), this option has no effect as the PWM output and
// spindle enable are combined to one pin. If you need both this option and spindle speed PWM,
// uncomment the config option USE_SPINDLE_DIR_AS_ENABLE_PIN below.
// #define INVERT_SPINDLE_ENABLE_PIN // Default disabled. Uncomment to enable.

// Inverts the selected coolant pin from low-disabled/high-enabled to low-enabled/high-disabled. Useful
// for some pre-built electronic boards.
// #define INVERT_COOLANT_FLOOD_PIN // Default disabled. Uncomment to enable.
// #define INVERT_COOLANT_MIST_PIN // Default disabled. Note: Enable M7 mist coolant in config.h

// When Grbl powers-cycles or is hard reset with the Arduino reset button, Grbl boots up with no ALARM
// by default. This is to make it as simple as possible for new users to start using Grbl. When homing
// is enabled and a user has installed limit switches, Grbl will boot up in an ALARM state to indicate
// Grbl doesn't know its position and to force the user to home before proceeding. This option forces
// Grbl to always initialize into an ALARM state regardless of homing or not. This option is more for
// OEMs and LinuxCNC users that would like this power-cycle behavior.
// #define FORCE_INITIALIZATION_ALARM // Default disabled. Uncomment to enable.

// At power-up or a reset, Grbl will check the limit switch states to ensure they are not active
// before initialization. If it detects a problem and the hard limits setting is enabled, Grbl will
// simply message the user to check the limits and enter an alarm state, rather than idle. Grbl will
// not throw an alarm message.
#define CHECK_LIMITS_AT_INIT

```

```
// -----
// ADVANCED CONFIGURATION OPTIONS:

// Enables code for debugging purposes. Not for general use and always in constant flux.
// #define DEBUG // Uncomment to enable. Default disabled.

// Configure rapid, feed, and spindle override settings. These values define the max and min
// allowable override values and the coarse and fine increments per command received. Please
// note the allowable values in the descriptions following each define.
#define DEFAULT_FEED_OVERRIDE      100 // 100%. Don't change this value.
#define MAX_FEED_RATE_OVERRIDE     200 // Percent of programmed feed rate (100-255). Usually 120% or 200%
#define MIN_FEED_RATE_OVERRIDE     10 // Percent of programmed feed rate (1-100). Usually 50% or 1%
#define FEED_OVERRIDE_COARSE_INCREMENT  10 // (1-99). Usually 10%.
#define FEED_OVERRIDE_FINE_INCREMENT   1 // (1-99). Usually 1%.

#define DEFAULT_RAPID_OVERRIDE 100 // 100%. Don't change this value.
#define RAPID_OVERRIDE_MEDIUM  50 // Percent of rapid (1-99). Usually 50%.
#define RAPID_OVERRIDE_LOW     25 // Percent of rapid (1-99). Usually 25%.
// #define RAPID_OVERRIDE_EXTRA_LOW 5 // *NOT SUPPORTED* Percent of rapid (1-99). Usually 5%.

#define DEFAULT_SPINDLE_SPEED_OVERRIDE 100 // 100%. Don't change this value.
#define MAX_SPINDLE_SPEED_OVERRIDE     200 // Percent of programmed spindle speed (100-255). Usually 200%.
#define MIN_SPINDLE_SPEED_OVERRIDE     10 // Percent of programmed spindle speed (1-100). Usually 10%.
#define SPINDLE_OVERRIDE_COARSE_INCREMENT 10 // (1-99). Usually 10%.
#define SPINDLE_OVERRIDE_FINE_INCREMENT   1 // (1-99). Usually 1%.

// When a M2 or M30 program end command is executed, most g-code states are restored to their defaults.
// This compile-time option includes the restoring of the feed, rapid, and spindle speed override values
// to their default values at program end.
#define RESTORE_OVERRIDES_AFTER_PROGRAM_END // Default enabled. Comment to disable.

// The status report change for Grbl v1.1 and after also removed the ability to disable/enable most data
// fields from the report. This caused issues for GUI developers, who've had to manage several scenarios
// and configurations. The increased efficiency of the new reporting style allows for all data fields to
// be sent without potential performance issues.
// NOTE: The options below are here only provide a way to disable certain data fields if a unique
// situation demands it, but be aware GUIs may depend on this data. If disabled, it may not be compatible.
#define REPORT_FIELD_BUFFER_STATE // Default enabled. Comment to disable.
#define REPORT_FIELD_PIN_STATE // Default enabled. Comment to disable.
#define REPORT_FIELD_CURRENT_FEED_SPEED // Default enabled. Comment to disable.
#define REPORT_FIELD_WORK_COORD_OFFSET // Default enabled. Comment to disable.
#define REPORT_FIELD_OVERRIDES // Default enabled. Comment to disable.
#define REPORT_FIELD_LINE_NUMBERS // Default enabled. Comment to disable.

// Some status report data isn't necessary for realtime, only intermittently, because the values don't
// change often. The following macros configures how many times a status report needs to be called before
// the associated data is refreshed and included in the status report. However, if one of these value
// changes, Grbl will automatically include this data in the next status report, regardless of what the
// count is at the time. This helps reduce the communication overhead involved with high frequency reporting
// and aggressive streaming. There is also a busy and an idle refresh count, which sets up Grbl to send
```

```

// refreshes more often when its not doing anything important. With a good GUI, this data doesn't need
// to be refreshed very often, on the order of a several seconds.
// NOTE: WCO refresh must be 2 or greater. OVR refresh must be 1 or greater.
#define REPORT_OVR_REFRESH_BUSY_COUNT 20 // (1-255)
#define REPORT_OVR_REFRESH_IDLE_COUNT 10 // (1-255) Must be less than or equal to the busy count
#define REPORT_WCO_REFRESH_BUSY_COUNT 30 // (2-255)
#define REPORT_WCO_REFRESH_IDLE_COUNT 10 // (2-255) Must be less than or equal to the busy count

// The temporal resolution of the acceleration management subsystem. A higher number gives smoother
// acceleration, particularly noticeable on machines that run at very high feedrates, but may negatively
// impact performance. The correct value for this parameter is machine dependent, so it's advised to
// set this only as high as needed. Approximate successful values can widely range from 50 to 200 or more.
// NOTE: Changing this value also changes the execution time of a segment in the step segment buffer.
// When increasing this value, this stores less overall time in the segment buffer and vice versa. Make
// certain the step segment buffer is increased/decreased to account for these changes.
#define ACCELERATION_TICKS_PER_SECOND 100

// Adaptive Multi-Axis Step Smoothing (AMASS) is an advanced feature that does what its name implies,
// smoothing the stepping of multi-axis motions. This feature smooths motion particularly at low step
// frequencies below 10kHz, where the aliasing between axes of multi-axis motions can cause audible
// noise and shake your machine. At even lower step frequencies, AMASS adapts and provides even better
// step smoothing. See stepper.c for more details on the AMASS system works.
#define ADAPTIVE_MULTI_AXIS_STEP_SMOOTHING // Default enabled. Comment to disable.

// Sets the maximum step rate allowed to be written as a Grbl setting. This option enables an error
// check in the settings module to prevent settings values that will exceed this limitation. The maximum
// step rate is strictly limited by the CPU speed and will change if something other than an AVR running
// at 16MHz is used.
// NOTE: For now disabled, will enable if flash space permits.
// #define MAX_STEP_RATE_HZ 30000 // Hz

// By default, Grbl sets all input pins to normal-high operation with their internal pull-up resistors
// enabled. This simplifies the wiring for users by requiring only a switch connected to ground,
// although its recommended that users take the extra step of wiring in low-pass filter to reduce
// electrical noise detected by the pin. If the user inverts the pin in Grbl settings, this just flips
// which high or low reading indicates an active signal. In normal operation, this means the user
// needs to connect a normal-open switch, but if inverted, this means the user should connect a
// normal-closed switch.
// The following options disable the internal pull-up resistors, sets the pins to a normal-low
// operation, and switches must be now connect to Vcc instead of ground. This also flips the meaning
// of the invert pin Grbl setting, where an inverted setting now means the user should connect a
// normal-open switch and vice versa.
// NOTE: All pins associated with the feature are disabled, i.e. XYZ limit pins, not individual axes.
// WARNING: When the pull-ups are disabled, this requires additional wiring with pull-down resistors!
// #define DISABLE_LIMIT_PIN_PULL_UP
// #define DISABLE_PROBE_PIN_PULL_UP
// #define DISABLE_CONTROL_PIN_PULL_UP

// Sets which axis the tool length offset is applied. Assumes the spindle is always parallel with
// the selected axis with the tool oriented toward the negative direction. In other words, a positive
// tool length offset value is subtracted from the current location.

```

```

#define TOOL_LENGTH_OFFSET_AXIS Z_AXIS // Default z-axis. Valid values are X_AXIS, Y_AXIS, or Z_AXIS.

// Enables variable spindle output voltage for different RPM values. On the Arduino Uno, the spindle
// enable pin will output 5V for maximum RPM with 256 intermediate levels and 0V when disabled.
// NOTE: IMPORTANT for Arduino Unos! When enabled, the Z-limit pin D11 and spindle enable pin D12 switch!
// The hardware PWM output on pin D11 is required for variable spindle output voltages.
#define VARIABLE_SPINDLE // Default enabled. Comment to disable.

// Used by variable spindle output only. This forces the PWM output to a minimum duty cycle when enabled.
// The PWM pin will still read 0V when the spindle is disabled. Most users will not need this option, but
// it may be useful in certain scenarios. This minimum PWM settings coincides with the spindle rpm minimum
// setting, like rpm max to max PWM. This is handy if you need a larger voltage difference between 0V disabled
// and the voltage set by the minimum PWM for minimum rpm. This difference is 0.02V per PWM value. So, when
// minimum PWM is at 1, only 0.02 volts separate enabled and disabled. At PWM 5, this would be 0.1V. Keep
// in mind that you will begin to lose PWM resolution with increased minimum PWM values, since you have less
// and less range over the total 255 PWM levels to signal different spindle speeds.
// NOTE: Compute duty cycle at the minimum PWM by this equation: (% duty cycle)=(SPINDLE_PWM_MIN_VALUE/255)*100
// #define SPINDLE_PWM_MIN_VALUE 5 // Default disabled. Uncomment to enable. Must be greater than zero. Integer (1-255)

// By default on a 328p(Uno), Grbl combines the variable spindle PWM and the enable into one pin to help
// preserve I/O pins. For certain setups, these may need to be separate pins. This configure option uses
// the spindle direction pin(D13) as a separate spindle enable pin along with spindle speed PWM on pin D11.
// NOTE: This configure option only works with VARIABLE_SPINDLE enabled and a 328p processor (Uno).
// NOTE: Without a direction pin, M4 will not have a pin output to indicate a difference with M3.
// NOTE: BEWARE! The Arduino bootloader toggles the D13 pin when it powers up. If you flash Grbl with
// a programmer (you can use a spare Arduino as "Arduino as ISP". Search the web on how to wire this.),
// this D13 LED toggling should go away. We haven't tested this though. Please report how it goes!
// #define USE_SPINDLE_DIR_AS_ENABLE_PIN // Default disabled. Uncomment to enable.

// Alters the behavior of the spindle enable pin with the USE_SPINDLE_DIR_AS_ENABLE_PIN option . By default,
// Grbl will not disable the enable pin if spindle speed is zero and M3/4 is active, but still sets the PWM
// output to zero. This allows the users to know if the spindle is active and use it as an additional control
// input. However, in some use cases, user may want the enable pin to disable with a zero spindle speed and
// re-enable when spindle speed is greater than zero. This option does that.
// NOTE: Requires USE_SPINDLE_DIR_AS_ENABLE_PIN to be enabled.
// #define SPINDLE_ENABLE_OFF_WITH_ZERO_SPEED // Default disabled. Uncomment to enable.

// With this enabled, Grbl sends back an echo of the line it has received, which has been pre-parsed (spaces
// removed, capitalized letters, no comments) and is to be immediately executed by Grbl. Echoes will not be
// sent upon a line buffer overflow, but should for all normal lines sent to Grbl. For example, if a user
// sendss the line 'g1 x1.032 y2.45 (test comment)', Grbl will echo back in the form '[echo: G1X1.032Y2.45]'.
// NOTE: Only use this for debugging purposes!! When echoing, this takes up valuable resources and can effect
// performance. If absolutely needed for normal operation, the serial write buffer should be greatly increased
// to help minimize transmission waiting within the serial write protocol.
// #define REPORT_ECHO_LINE_RECEIVED // Default disabled. Uncomment to enable.

// Minimum planner junction speed. Sets the default minimum junction speed the planner plans to at
// every buffer block junction, except for starting from rest and end of the buffer, which are always
// zero. This value controls how fast the machine moves through junctions with no regard for acceleration
// limits or angle between neighboring block line move directions. This is useful for machines that can't
// tolerate the tool dwelling for a split second, i.e. 3d printers or laser cutters. If used, this value

```

```

// should not be much greater than zero or to the minimum value necessary for the machine to work.
#define MINIMUM_JUNCTION_SPEED 0.0 // (mm/min)

// Sets the minimum feed rate the planner will allow. Any value below it will be set to this minimum
// value. This also ensures that a planned motion always completes and accounts for any floating-point
// round-off errors. Although not recommended, a lower value than 1.0 mm/min will likely work in smaller
// machines, perhaps to 0.1mm/min, but your success may vary based on multiple factors.
#define MINIMUM_FEED_RATE 1.0 // (mm/min)

// Number of arc generation iterations by small angle approximation before exact arc trajectory
// correction with expensive sin() and cos() calculations. This parameter maybe decreased if there
// are issues with the accuracy of the arc generations, or increased if arc execution is getting
// bogged down by too many trig calculations.
#define N_ARC_CORRECTION 12 // Integer (1-255)

// The arc G2/3 g-code standard is problematic by definition. Radius-based arcs have horrible numerical
// errors when arc at semi-circles( $\pi$ ) or full-circles( $2\pi$ ). Offset-based arcs are much more accurate
// but still have a problem when arcs are full-circles ( $2\pi$ ). This define accounts for the floating
// point issues when offset-based arcs are commanded as full circles, but get interpreted as extremely
// small arcs with around machine epsilon ( $1.2e-7$ rad) due to numerical round-off and precision issues.
// This define value sets the machine epsilon cutoff to determine if the arc is a full-circle or not.
// NOTE: Be very careful when adjusting this value. It should always be greater than  $1.2e-7$  but not too
// much greater than this. The default setting should capture most, if not all, full arc error situations.
#define ARC_ANGULAR_TRAVEL_EPSILON 5E-7 // Float (radians)

// Time delay increments performed during a dwell. The default value is set at 50ms, which provides
// a maximum time delay of roughly 55 minutes, more than enough for most any application. Increasing
// this delay will increase the maximum dwell time linearly, but also reduces the responsiveness of
// run-time command executions, like status reports, since these are performed between each dwell
// time step. Also, keep in mind that the Arduino delay timer is not very accurate for long delays.
#define DWELL_TIME_STEP 50 // Integer (1-255) (milliseconds)

// Creates a delay between the direction pin setting and corresponding step pulse by creating
// another interrupt (Timer2 compare) to manage it. The main Grbl interrupt (Timer1 compare)
// sets the direction pins, and does not immediately set the stepper pins, as it would in
// normal operation. The Timer2 compare fires next to set the stepper pins after the step
// pulse delay time, and Timer2 overflow will complete the step pulse, except now delayed
// by the step pulse time plus the step pulse delay. (Thanks langwadt for the idea!)
// NOTE: Uncomment to enable. The recommended delay must be  $> 3\mu s$ , and, when added with the
// user-supplied step pulse time, the total time must not exceed  $127\mu s$ . Reported successful
// values for certain setups have ranged from 5 to  $20\mu s$ .
// #define STEP_PULSE_DELAY 10 // Step pulse delay in microseconds. Default disabled.

// The number of linear motions in the planner buffer to be planned at any give time. The vast
// majority of RAM that Grbl uses is based on this buffer size. Only increase if there is extra
// available RAM, like when re-compiling for a Mega2560. Or decrease if the Arduino begins to
// crash due to the lack of available RAM or if the CPU is having trouble keeping up with planning
// new incoming motions as they are executed.
// #define BLOCK_BUFFER_SIZE 16 // Uncomment to override default in planner.h.

// Governs the size of the intermediary step segment buffer between the step execution algorithm

```

```
// and the planner blocks. Each segment is set of steps executed at a constant velocity over a
// fixed time defined by ACCELERATION_TICKS_PER_SECOND. They are computed such that the planner
// block velocity profile is traced exactly. The size of this buffer governs how much step
// execution lead time there is for other Grbl processes have to compute and do their thing
// before having to come back and refill this buffer, currently at ~50msec of step moves.
// #define SEGMENT_BUFFER_SIZE 6 // Uncomment to override default in stepper.h.
```

```
// Line buffer size from the serial input stream to be executed. Also, governs the size of
// each of the startup blocks, as they are each stored as a string of this size. Make sure
// to account for the available EEPROM at the defined memory address in settings.h and for
// the number of desired startup blocks.
// NOTE: 80 characters is not a problem except for extreme cases, but the line buffer size
// can be too small and g-code blocks can get truncated. Officially, the g-code standards
// support up to 256 characters. In future versions, this default will be increased, when
// we know how much extra memory space we can re-invest into this.
// #define LINE_BUFFER_SIZE 80 // Uncomment to override default in protocol.h
```

```
// Serial send and receive buffer size. The receive buffer is often used as another streaming
// buffer to store incoming blocks to be processed by Grbl when its ready. Most streaming
// interfaces will character count and track each block send to each block response. So,
// increase the receive buffer if a deeper receive buffer is needed for streaming and available
// memory allows. The send buffer primarily handles messages in Grbl. Only increase if large
// messages are sent and Grbl begins to stall, waiting to send the rest of the message.
// NOTE: Grbl generates an average status report in about 0.5msec, but the serial TX stream at
// 115200 baud will take 5 msec to transmit a typical 55 character report. Worst case reports are
// around 90-100 characters. As long as the serial TX buffer doesn't get continually maxed, Grbl
// will continue operating efficiently. Size the TX buffer around the size of a worst-case report.
// #define RX_BUFFER_SIZE 128 // (1-254) Uncomment to override defaults in serial.h
// #define TX_BUFFER_SIZE 100 // (1-254)
```

```
// A simple software debouncing feature for hard limit switches. When enabled, the interrupt
// monitoring the hard limit switch pins will enable the Arduino's watchdog timer to re-check
// the limit pin state after a delay of about 32msec. This can help with CNC machines with
// problematic false triggering of their hard limit switches, but it WILL NOT fix issues with
// electrical interference on the signal cables from external sources. It's recommended to first
// use shielded signal cables with their shielding connected to ground (old USB/computer cables
// work well and are cheap to find) and wire in a low-pass circuit into each limit pin.
// #define ENABLE_SOFTWARE_DEBOUNCE // Default disabled. Uncomment to enable.
```

```
// Configures the position after a probing cycle during Grbl's check mode. Disabled sets
// the position to the probe target, when enabled sets the position to the start position.
// #define SET_CHECK_MODE_PROBE_TO_START // Default disabled. Uncomment to enable.
```

```
// Force Grbl to check the state of the hard limit switches when the processor detects a pin
// change inside the hard limit ISR routine. By default, Grbl will trigger the hard limits
// alarm upon any pin change, since bouncing switches can cause a state check like this to
// misread the pin. When hard limits are triggered, they should be 100% reliable, which is the
// reason that this option is disabled by default. Only if your system/electronics can guarantee
// that the switches don't bounce, we recommend enabling this option. This will help prevent
// triggering a hard limit when the machine disengages from the switch.
// NOTE: This option has no effect if SOFTWARE_DEBOUNCE is enabled.
```

```

// #define HARD_LIMIT_FORCE_STATE_CHECK // Default disabled. Uncomment to enable.

// Adjusts homing cycle search and locate scalars. These are the multipliers used by Grbl's
// homing cycle to ensure the limit switches are engaged and cleared through each phase of
// the cycle. The search phase uses the axes max-travel setting times the SEARCH_SCALAR to
// determine distance to look for the limit switch. Once found, the locate phase begins and
// uses the homing pull-off distance setting times the LOCATE_SCALAR to pull-off and re-engage
// the limit switch.
// NOTE: Both of these values must be greater than 1.0 to ensure proper function.
// #define HOMING_AXIS_SEARCH_SCALAR 1.5 // Uncomment to override defaults in limits.c.
// #define HOMING_AXIS_LOCATE_SCALAR 10.0 // Uncomment to override defaults in limits.c.

// Enable the '$RST=*', '$RST=$', and '$RST=#' eeprom restore commands. There are cases where
// these commands may be undesirable. Simply comment the desired macro to disable it.
// NOTE: See SETTINGS_RESTORE_ALL macro for customizing the '$RST=*' command.
#define ENABLE_RESTORE_EEPROM_WIPE_ALL // '$RST=*' Default enabled. Comment to disable.
#define ENABLE_RESTORE_EEPROM_DEFAULT_SETTINGS // '$RST=$' Default enabled. Comment to disable.
#define ENABLE_RESTORE_EEPROM_CLEAR_PARAMETERS // '$RST=#' Default enabled. Comment to disable.

// Defines the EEPROM data restored upon a settings version change and '$RST=*' command. Whenever the
// the settings or other EEPROM data structure changes between Grbl versions, Grbl will automatically
// wipe and restore the EEPROM. This macro controls what data is wiped and restored. This is useful
// particularly for OEMs that need to retain certain data. For example, the BUILD_INFO string can be
// written into the Arduino EEPROM via a separate .INO sketch to contain product data. Altering this
// macro to not restore the build info EEPROM will ensure this data is retained after firmware upgrades.
// NOTE: Uncomment to override defaults in settings.h
// #define SETTINGS_RESTORE_ALL (SETTINGS_RESTORE_DEFAULTS | SETTINGS_RESTORE_PARAMETERS | SE

// Enable the '$I=(string)' build info write command. If disabled, any existing build info data must
// be placed into EEPROM via external means with a valid checksum value. This macro option is useful
// to prevent this data from being over-written by a user, when used to store OEM product data.
// NOTE: If disabled and to ensure Grbl can never alter the build info line, you'll also need to enable
// the SETTING_RESTORE_ALL macro above and remove SETTINGS_RESTORE_BUILD_INFO from the mask.
// NOTE: See the included grblWrite_BuildInfo.ino example file to write this string separately.
#define ENABLE_BUILD_INFO_WRITE_COMMAND // '$I=' Default enabled. Comment to disable.

// AVR processors require all interrupts to be disabled during an EEPROM write. This includes both
// the stepper ISRs and serial comm ISRs. In the event of a long EEPROM write, this ISR pause can
// cause active stepping to lose position and serial receive data to be lost. This configuration
// option forces the planner buffer to completely empty whenever the EEPROM is written to prevent
// any chance of lost steps.
// However, this doesn't prevent issues with lost serial RX data during an EEPROM write, especially
// if a GUI is preemptively filling up the serial RX buffer simultaneously. It's highly advised for
// GUIs to flag these gcodes (G10,G28.1,G30.1) to always wait for an 'ok' after a block containing
// one of these commands before sending more data to eliminate this issue.
// NOTE: Most EEPROM write commands are implicitly blocked during a job (all '$' commands). However,
// coordinate set g-code commands (G10,G28/30.1) are not, since they are part of an active streaming
// job. At this time, this option only forces a planner buffer sync with these g-code commands.
#define FORCE_BUFFER_SYNC_DURING_EEPROM_WRITE // Default enabled. Comment to disable.

// In Grbl v0.9 and prior, there is an old outstanding bug where the `WPos:` work position reported

```

```

// may not correlate to what is executing, because `WPos:` is based on the g-code parser state, which
// can be several motions behind. This option forces the planner buffer to empty, sync, and stop
// motion whenever there is a command that alters the work coordinate offsets `G10,G43.1,G92,G54-59`.
// This is the simplest way to ensure `WPos:` is always correct. Fortunately, it's exceedingly rare
// that any of these commands are used need continuous motions through them.
#define FORCE_BUFFER_SYNC_DURING_WCO_CHANGE // Default enabled. Comment to disable.

// By default, Grbl disables feed rate overrides for all G38.x probe cycle commands. Although this
// may be different than some pro-class machine control, it's arguable that it should be this way.
// Most probe sensors produce different levels of error that is dependent on rate of speed. By
// keeping probing cycles to their programmed feed rates, the probe sensor should be a lot more
// repeatable. If needed, you can disable this behavior by uncommenting the define below.
// #define ALLOW_FEED_OVERRIDE_DURING_PROBE_CYCLES // Default disabled. Uncomment to enable.

// Enables and configures parking motion methods upon a safety door state. Primarily for OEMs
// that desire this feature for their integrated machines. At the moment, Grbl assumes that
// the parking motion only involves one axis, although the parking implementation was written
// to be easily refactored for any number of motions on different axes by altering the parking
// source code. At this time, Grbl only supports parking one axis (typically the Z-axis) that
// moves in the positive direction upon retracting and negative direction upon restoring position.
// The motion executes with a slow pull-out retraction motion, power-down, and a fast park.
// Restoring to the resume position follows these set motions in reverse: fast restore to
// pull-out position, power-up with a time-out, and plunge back to the original position at the
// slower pull-out rate.
// NOTE: Still a work-in-progress. Machine coordinates must be in all negative space and
// does not work with HOMING_FORCE_SET_ORIGIN enabled. Parking motion also moves only in
// positive direction.
// #define PARKING_ENABLE // Default disabled. Uncomment to enable

// Configure options for the parking motion, if enabled.
#define PARKING_AXIS Z_AXIS // Define which axis that performs the parking motion
#define PARKING_TARGET -5.0 // Parking axis target. In mm, as machine coordinate [-max_travel,0].
#define PARKING_RATE 500.0 // Parking fast rate after pull-out in mm/min.
#define PARKING_PULLOUT_RATE 100.0 // Pull-out/plunge slow feed rate in mm/min.
#define PARKING_PULLOUT_INCREMENT 5.0 // Spindle pull-out and plunge distance in mm. Incremental distance.
// Must be positive value or equal to zero.

// Enables a special set of M-code commands that enables and disables the parking motion.
// These are controlled by `M56`, `M56 P1`, or `M56 Px` to enable and `M56 P0` to disable.
// The command is modal and will be set after a planner sync. Since it is g-code, it is
// executed in sync with g-code commands. It is not a real-time command.
// NOTE: PARKING_ENABLE is required. By default, M56 is active upon initialization. Use
// DEACTIVATE_PARKING_UPON_INIT to set M56 P0 as the power-up default.
// #define ENABLE_PARKING_OVERRIDE_CONTROL // Default disabled. Uncomment to enable
// #define DEACTIVATE_PARKING_UPON_INIT // Default disabled. Uncomment to enable.

// This option will automatically disable the laser during a feed hold by invoking a spindle stop
// override immediately after coming to a stop. However, this also means that the laser still may
// be reenabled by disabling the spindle stop override, if needed. This is purely a safety feature
// to ensure the laser doesn't inadvertently remain powered while at a stop and cause a fire.
#define DISABLE_LASER_DURING_HOLD // Default enabled. Comment to disable.

```

```

// This feature alters the spindle PWM/speed to a nonlinear output with a simple piecewise linear
// curve. Useful for spindles that don't produce the right RPM from Grbl's standard spindle PWM
// linear model. Requires a solution by the 'fit_nonlinear_spindle.py' script in the /doc/script
// folder of the repo. See file comments on how to gather spindle data and run the script to
// generate a solution.
// #define ENABLE_PIECEWISE_LINEAR_SPINDLE // Default disabled. Uncomment to enable.

// N_PIECES, RPM_MAX, RPM_MIN, RPM_POINTxx, and RPM_LINE_XX constants are all set and given by
// the 'fit_nonlinear_spindle.py' script solution. Used only when ENABLE_PIECEWISE_LINEAR_SPINDLE
// is enabled. Make sure the constant values are exactly the same as the script solution.
// NOTE: When N_PIECES < 4, unused RPM_LINE and RPM_POINT defines are not required and omitted.
#define N_PIECES 4 // Integer (1-4). Number of piecewise lines used in script solution.
#define RPM_MAX 11686.4 // Max RPM of model. $30 > RPM_MAX will be limited to RPM_MAX.
#define RPM_MIN 202.5 // Min RPM of model. $31 < RPM_MIN will be limited to RPM_MIN.
#define RPM_POINT12 6145.4 // Used N_PIECES >=2. Junction point between lines 1 and 2.
#define RPM_POINT23 9627.8 // Used N_PIECES >=3. Junction point between lines 2 and 3.
#define RPM_POINT34 10813.9 // Used N_PIECES = 4. Junction point between lines 3 and 4.
#define RPM_LINE_A1 3.197101e-03 // Used N_PIECES >=1. A and B constants of line 1.
#define RPM_LINE_B1 -3.526076e-1
#define RPM_LINE_A2 1.722950e-2 // Used N_PIECES >=2. A and B constants of line 2.
#define RPM_LINE_B2 8.588176e+01
#define RPM_LINE_A3 5.901518e-02 // Used N_PIECES >=3. A and B constants of line 3.
#define RPM_LINE_B3 4.881851e+02
#define RPM_LINE_A4 1.203413e-01 // Used N_PIECES = 4. A and B constants of line 4.
#define RPM_LINE_B4 1.151360e+03

/* -----
This optional dual axis feature is primarily for the homing cycle to locate two sides of
a dual-motor gantry independently, i.e. self-squaring. This requires an additional limit
switch for the cloned motor. To self square, both limit switches on the cloned axis must
be physically positioned to trigger when the gantry is square. Highly recommend keeping
the motors always enabled to ensure the gantry stays square with the $1=255 setting.

For Grbl on the Arduino Uno, the cloned axis limit switch must be shared with and
wired with z-axis limit pin due to the lack of available pins. The homing cycle must home
the z-axis and cloned axis in different cycles, which is already the default config.

The dual axis feature works by cloning an axis step output onto another pair of step
and direction pins. The step pulse and direction of the cloned motor can be set
independently of the main axis motor. However to save precious flash and memory, this
dual axis feature must share the same settings (step/mm, max speed, acceleration) as the
parent motor. This is NOT a feature for an independent fourth axis. Only a motor clone.

WARNING: Make sure to test the directions of your dual axis motors! They must be setup
to move the same direction BEFORE running your first homing cycle or any long motion!
Motors moving in opposite directions can cause serious damage to your machine! Use this
dual axis feature at your own risk.
*/
// NOTE: This feature requires approximately 400 bytes of flash. Certain configurations can
// run out of flash to fit on an Arduino 328p/Uno. Only X and Y axes are supported. Variable

```

```

// spindle/laser mode IS supported, but only for one config option. Core XY, spindle direction
// pin, and M7 mist coolant are disabled/not supported.
// #define ENABLE_DUAL_AXIS // Default disabled. Uncomment to enable.

// Select the one axis to mirror another motor. Only X and Y axis is supported at this time.
#define DUAL_AXIS_SELECT X_AXIS // Must be either X_AXIS or Y_AXIS

// To prevent the homing cycle from racking the dual axis, when one limit triggers before the
// other due to switch failure or noise, the homing cycle will automatically abort if the second
// motor's limit switch does not trigger within the three distance parameters defined below.
// Axis length percent will automatically compute a fail distance as a percentage of the max
// travel of the other non-dual axis, i.e. if dual axis select is X_AXIS at 5.0%, then the fail
// distance will be computed as 5.0% of y-axis max travel. Fail distance max and min are the
// limits of how far or little a valid fail distance is.
#define DUAL_AXIS_HOMING_FAIL_AXIS_LENGTH_PERCENT 5.0 // Float (percent)
#define DUAL_AXIS_HOMING_FAIL_DISTANCE_MAX 25.0 // Float (mm)
#define DUAL_AXIS_HOMING_FAIL_DISTANCE_MIN 2.5 // Float (mm)

// Dual axis pin configuration currently supports two shields. Uncomment the shield you want,
// and comment out the other one(s).
// NOTE: Protoneer CNC Shield v3.51 has A.STP and A.DIR wired to pins A4 and A3 respectively.
// The variable spindle (i.e. laser mode) build option works and may be enabled or disabled.
// Coolant pin A3 is moved to D13, replacing spindle direction.
#define DUAL_AXIS_CONFIG_PROTONEER_V3_51 // Uncomment to select. Comment other configs.

// NOTE: Arduino CNC Shield Clone (Originally Protoneer v3.0) has A.STP and A.DIR wired to
// D12 and D13, respectively. With the limit pins and stepper enable pin on this same port,
// the spindle enable pin had to be moved and spindle direction pin deleted. The spindle
// enable pin now resides on A3, replacing coolant enable. Coolant enable is bumped over to
// pin A4. Spindle enable is used far more and this pin setup helps facilitate users to
// integrate this feature without arguably too much work.
// Variable spindle (i.e. laser mode) does NOT work with this shield as configured. While
// variable spindle technically can work with this shield, it requires too many changes for
// most user setups to accomodate. It would best be implemented by sharing all limit switches
// on pins D9/D10 (as [X1,Z]/[X2,Y] or [X,Y2]/[Y1,Z]), home each axis independently, and
// updating lots of code to ensure everything is running correctly.
// #define DUAL_AXIS_CONFIG_CNC_SHIELD_CLONE // Uncomment to select. Comment other configs.

/* -----
OEM Single File Configuration Option

Instructions: Paste the cpu_map and default setting definitions below without an enclosing
#ifdef. Comment out the CPU_MAP_xxx and DEFAULT_xxx defines at the top of this file, and
the compiler will ignore the contents of defaults.h and cpu_map.h and use the definitions
below.
*/

// Paste CPU_MAP definitions here.

// Paste default settings definitions here.

```

```
#endif
```

File: coolant_control.c

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```
/*
```

```
coolant_control.c - coolant control methods  
Part of Grbl
```

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

```
*/
```

```
#include "grbl.h"
```

```
void coolant_init()
```

```
{  
  COOLANT_FLOOD_DDR |= (1 << COOLANT_FLOOD_BIT); // Configure as output pin  
#ifdef ENABLE_M7  
  COOLANT_MIST_DDR |= (1 << COOLANT_MIST_BIT);  
#endif  
  coolant_stop();  
}
```

```
// Returns current coolant output state. Overrides may alter it from programmed state.
```

```
uint8_t coolant_get_state()
```

```
{  
  uint8_t cl_state = COOLANT_STATE_DISABLE;  
#ifdef INVERT_COOLANT_FLOOD_PIN  
  if (bit_isfalse(COOLANT_FLOOD_PORT,(1 << COOLANT_FLOOD_BIT))) {  
#else  
  if (bit_istrue(COOLANT_FLOOD_PORT,(1 << COOLANT_FLOOD_BIT))) {  
#endif
```

```

    cl_state |= COOLANT_STATE_FLOOD;
}
#ifdef ENABLE_M7
    #ifdef INVERT_COOLANT_MIST_PIN
        if (bit_isfalse(COOLANT_MIST_PORT,(1 << COOLANT_MIST_BIT))) {
    #else
        if (bit_istrue(COOLANT_MIST_PORT,(1 << COOLANT_MIST_BIT))) {
    #endif
        cl_state |= COOLANT_STATE_MIST;
    }
#endif
return(cl_state);
}

```

// Directly called by coolant_init(), coolant_set_state(), and mc_reset(), which can be at
// an interrupt-level. No report flag set, but only called by routines that don't need it.

```

void coolant_stop()
{
    #ifdef INVERT_COOLANT_FLOOD_PIN
        COOLANT_FLOOD_PORT |= (1 << COOLANT_FLOOD_BIT);
    #else
        COOLANT_FLOOD_PORT &= ~(1 << COOLANT_FLOOD_BIT);
    #endif
    #ifdef ENABLE_M7
        #ifdef INVERT_COOLANT_MIST_PIN
            COOLANT_MIST_PORT |= (1 << COOLANT_MIST_BIT);
        #else
            COOLANT_MIST_PORT &= ~(1 << COOLANT_MIST_BIT);
        #endif
    #endif
}

```

// Main program only. Immediately sets flood coolant running state and also mist coolant,
// if enabled. Also sets a flag to report an update to a coolant state.

// Called by coolant toggle override, parking restore, parking retract, sleep mode, g-code
// parser program end, and g-code parser coolant_sync().

```

void coolant_set_state(uint8_t mode)
{
    if (sys.abort) { return; } // Block during abort.

```

```

    if (mode & COOLANT_FLOOD_ENABLE) {
        #ifdef INVERT_COOLANT_FLOOD_PIN
            COOLANT_FLOOD_PORT &= ~(1 << COOLANT_FLOOD_BIT);
        #else
            COOLANT_FLOOD_PORT |= (1 << COOLANT_FLOOD_BIT);
        #endif
    } else {
        #ifdef INVERT_COOLANT_FLOOD_PIN
            COOLANT_FLOOD_PORT |= (1 << COOLANT_FLOOD_BIT);

```

```

    ##else
    ###COOLANT_FLOOD_PORT &= ~(1 << COOLANT_FLOOD_BIT);
    ##endif
  }

  #ifdef ENABLE_M7
  ##if (mode & COOLANT_MIST_ENABLE) {
  ####ifdef INVERT_COOLANT_MIST_PIN
  ####COOLANT_MIST_PORT &= ~(1 << COOLANT_MIST_BIT);
  ####else
  ####COOLANT_MIST_PORT |= (1 << COOLANT_MIST_BIT);
  ####endif
  ##} else {
  ####ifdef INVERT_COOLANT_MIST_PIN
  ####COOLANT_MIST_PORT |= (1 << COOLANT_MIST_BIT);
  ####else
  ####COOLANT_MIST_PORT &= ~(1 << COOLANT_MIST_BIT);
  ####endif
  ##}
  #endif
  #
  sys.report_ovr_counter = 0; // Set to report change immediately
}

```

```

// G-code parser entry-point for setting coolant state. Forces a planner buffer sync and bails
// if an abort or check-mode is active.
void coolant_sync(uint8_t mode)
{
  if (sys.state == STATE_CHECK_MODE) { return; }
  protocol_buffer_synchronize(); // Ensure coolant turns on when specified in program.
  coolant_set_state(mode);
}

```

File: coolant_control.h

Directory: C:/Users/sense/Downloads/grbl-master/grbl

```

/*
  coolant_control.h - spindle control methods
  Part of Grbl

```

Copyright (c) 2012-2016 Sungeun K. Jeon for Gnea Research LLC

Grbl is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Grbl is distributed in the hope that it will be useful,

but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with Grbl. If not, see <<http://www.gnu.org/licenses/>>.

*/

```
#ifndef coolant_control_h
#define coolant_control_h
```

```
#define COOLANT_NO_SYNC    false
#define COOLANT_FORCE_SYNC true
```

```
#define COOLANT_STATE_DISABLE 0 // Must be zero
#define COOLANT_STATE_FLOOD   PL_COND_FLAG_COOLANT_FLOOD
#define COOLANT_STATE_MIST    PL_COND_FLAG_COOLANT_MIST
```

```
// Initializes coolant control pins.
void coolant_init();
```

```
// Returns current coolant output state. Overrides may alter it from programmed state.
uint8_t coolant_get_state();
```

```
// Immediately disables coolant pins.
void coolant_stop();
```

```
// Sets the coolant pins according to state specified.
void coolant_set_state(uint8_t mode);
```

```
// G-code parser entry-point for setting coolant states. Checks for and executes additional conditions.
void coolant_sync(uint8_t mode);
```

```
#endif
```